

AWS Certified GenAI Developer

Banco de preguntas de práctica

97 preguntas — pregunta y respuestas / solución y explicación

A retail company has a generative AI (GenAI) product recommendation application that uses Amazon Bedrock. The application suggests products to customers based on browsing history and demographics. The company needs to implement fairness evaluation across multiple demographic groups to detect and measure bias in recommendations between two prompt approaches. The company wants to collect and monitor fairness metrics in real time. The company must receive an alert if the fairness metrics show a discrepancy of more than 15% between demographic groups. The company must receive weekly reports that compare the performance of the two prompt approaches. Which solution will meet these requirements with the LEAST custom development effort?

- A Configure an Amazon CloudWatch dashboard to display default metrics from Amazon Bedrock API calls. Create custom metrics based on model outputs. Set up Amazon EventBridge rules to invoke AWS lambda functions that perform post-processing analysis on model responses and publish custom fairness metrics.
- B Create the two prompt variants in Amazon Bedrock Prompt Management. Use Amazon Bedrock Flows to deploy the prompt variants with defined traffic allocation. Configure Amazon Bedrock guardrails that have content filters to monitor demographic fairness. Set up Amazon CloudWatch alarms on the GuardrailContentSource dimension that use InvocationsIntervened metrics to detect recommendation discrepancy threshold violations.
- C Set up Amazon SageMaker Clarify to analyze model outputs. Publish fairness metrics to Amazon CloudWatch. Create CloudWatch composite alarms that combine SageMaker Clarify bias metrics with Amazon Bedrock latency metrics to provide a comprehensive fairness evaluation dashboard.
- D Create an Amazon Bedrock model evaluation job to compare fairness between the two prompt variants. Enable model invocation logging in Amazon CloudWatch. Set up CloudWatch alarms for InvocationsIntervened metrics with a dimension for each demographic group.

Respuesta correcta: **C**

- A Configure an Amazon CloudWatch dashboard to display default metrics from Amazon Bedrock API calls. Create custom metrics based on model outputs. Set up Amazon EventBridge rules to invoke AWS lambda functions that perform post-processing analysis on model responses and publish custom fairness metrics.
- B Create the two prompt variants in Amazon Bedrock Prompt Management. Use Amazon Bedrock Flows to deploy the prompt variants with defined traffic allocation. Configure Amazon Bedrock guardrails that have content filters to monitor demographic fairness. Set up Amazon CloudWatch alarms on the GuardrailContentSource dimension that use InvocationsIntervened metrics to detect recommendation discrepancy threshold violations.
- C** Set up Amazon SageMaker Clarify to analyze model outputs. Publish fairness metrics to Amazon CloudWatch. Create CloudWatch composite alarms that combine SageMaker Clarify bias metrics with Amazon Bedrock latency metrics to provide a comprehensive fairness evaluation dashboard. ✓
- D Create an Amazon Bedrock model evaluation job to compare fairness between the two prompt variants. Enable model invocation logging in Amazon CloudWatch. Set up CloudWatch alarms for InvocationsIntervened metrics with a dimension for each demographic group.

Explicación

Opción A: construir un pipeline de post-procesamiento a medida con reglas de Amazon EventBridge y funciones de AWS Lambda para calcular métricas de equidad personalizadas exige diseñar, codificar y mantener toda la lógica de detección de sesgo desde cero. Esto contradice directamente el requisito de "mínimo esfuerzo de desarrollo personalizado", ya que existe un servicio de AWS gestionado (SageMaker Clarify) diseñado específicamente para esta tarea.

Opción B: los filtros de contenido (content filters) de Amazon Bedrock Guardrails están diseñados para detectar categorías de contenido dañino (odio, insultos, contenido sexual, violencia, ataques de prompt/jailbreak), no para medir equidad estadística ni disparidad de resultados entre grupos demográficos. Según la documentación de CloudWatch para Guardrails, las dimensiones nativas disponibles son Operation, GuardrailContentSource, GuardrailPolicyType y GuardrailArn; no existe una dimensión por grupo demográfico. La métrica InvocationsIntervened cuenta cuántas invocaciones fueron intervenidas por un guardrail (por ejemplo, bloqueadas o modificadas), no calcula ninguna discrepancia de recomendación entre segmentos de población. Por tanto, esta solución no puede detectar ni alertar sobre un umbral de disparidad del 15% entre grupos.

Opción C (Correcta): Amazon SageMaker Clarify es el servicio de AWS diseñado específicamente para el análisis de sesgo (bias) y equidad (fairness). Proporciona un conjunto de métricas de sesgo pre-entrenamiento y post-entrenamiento (once métricas post-entrenamiento, como Accuracy Difference, Disparate Impact, etc.) que cuantifican distintas nociones de equidad comparando resultados entre "facets" o grupos demográficos favorecidos y desfavorecidos, y esta capacidad se extiende también a la evaluación de modelos de lenguaje/generativos (foundation model evaluations). Además, los trabajos de monitorización de deriva de sesgo (bias drift monitoring) de Clarify publican estas métricas automáticamente en Amazon CloudWatch (namespaces como aws/sagemaker/endpoints/bias-metrics), incluyendo dimensiones Autom8 y FacetValue que identifican el grupo demográfico analizado. Esto permite crear alarmas de CloudWatch (incluidas alarmas compuestas que combinen estas métricas de sesgo con métricas de latencia de Bedrock) para alertar ante una discrepancia superior al 15%, así como dashboards y reportes periódicos (semanales) comparando las dos variantes de prompt, todo ello con un esfuerzo de desarrollo mínimo al apoyarse en un servicio gestionado de AWS.

Opción D: los trabajos de evaluación de modelos (model evaluation jobs) de Amazon Bedrock —tanto automáticos como basados en humanos o en "modelo como juez"— están orientados a comparar la calidad de las respuestas de los modelos (precisión, robustez, toxicidad, etc.) frente a conjuntos de datos de prompts, pero no incluyen de forma nativa métricas de equidad segmentadas por grupo demográfico. Adicionalmente, InvocationsIntervened es una métrica de Amazon Bedrock Guardrails (no de los trabajos de evaluación de modelos) y sus dimensiones nativas en CloudWatch no incluyen una dimensión "por grupo demográfico", por lo que no es apta para medir sesgo entre segmentos de población ni para generar los informes comparativos semanales solicitados.

Referencias:

<https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-measure-post-training-bias.html>

<https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-model-monitor-bias-drift-cw.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-guardrails-cw-metrics.html>

A finance company is developing an AI assistant to help clients plan investments and manage their portfolios. The company identifies several high-risk conversation patterns such as requests for specific stock recommendations or guaranteed returns. High-risk conversation patterns could lead to regulatory violations if the company cannot implement appropriate controls. The company must ensure that the AI assistant does not provide inappropriate financial advice, generate content about competitors, or make claims that are not factually grounded in the company's approved financial guidance. The company wants to use Amazon Bedrock Guardrails to implement a solution. Which combination of steps will meet these requirements? (Choose three.)

Selecciona 3 respuestas

- A Add the high-risk conversation patterns to a denied topics guardrail.
- B Configure a content filter guardrail to filter prompts that contain the high-risk conversation patterns.
- C Configure a content filter guardrail to filter prompts that contain competitor names.
- D Add the names of competitors as custom word filters. Set the input and output actions to block.
- E Set a low grounding score threshold.
- F Set a high grounding score threshold.

Respuesta correcta: A, D, F

- ☒ A Add the high-risk conversation patterns to a denied topics guardrail. ✓
- ☐ B Configure a content filter guardrail to filter prompts that contain the high-risk conversation patterns.
- ☐ C Configure a content filter guardrail to filter prompts that contain competitor names.
- ☒ D Add the names of competitors as custom word filters. Set the input and output actions to block. ✓
- ☐ E Set a low grounding score threshold.
- ☒ F Set a high grounding score threshold. ✓

Explicación

Opción A (Correcta): Amazon Bedrock Guardrails permite definir "denied topics" (temas denegados), una política que describe semánticamente un conjunto de temas no deseados (por ejemplo, "recomendaciones específicas de acciones" o "promesas de retornos garantizados"). Cuando el modelo detecta que una entrada o una respuesta se ajusta a ese tema, la bloquea. Este es exactamente el mecanismo diseñado para bloquear los patrones de conversación de alto riesgo descritos por la empresa financiera.

Opción B: los content filters de Bedrock Guardrails funcionan sobre categorías predefinidas de contenido dañino (odio, insultos, contenido sexual, violencia, mal uso/misconduct y ataques de prompt), con umbrales de intensidad configurables. No están diseñados para capturar patrones de negocio específicos como "solicitud de consejo de inversión concreto"; ese tipo de intención se detecta mejor con un "denied topic", por lo que un content filter no es la herramienta adecuada aquí.

Opción C: igual que en el caso anterior, un content filter no sirve para bloquear nombres propios o listas específicas de palabras como los nombres de la competencia, ya que trabaja sobre categorías de daño, no sobre coincidencias léxicas exactas de términos concretos.

Opción D (Correcta): la documentación de AWS indica explícitamente que los "word filters" (filtros de palabras) de Bedrock Guardrails permiten bloquear palabras y frases mediante coincidencia exacta tanto en las entradas (prompts) como en las respuestas del modelo, y menciona de forma literal que se pueden usar para bloquear "contenido con nombres de competidores o de productos". Se puede añadir una lista de palabras personalizadas (custom word filter) con hasta 10.000 elementos y configurar las acciones de entrada y salida en modo de bloqueo, cumpliendo el requisito de no generar contenido sobre competidores.

Opción E: un umbral bajo (low) de contextual grounding check dejaría pasar respuestas con una puntuación baja de fundamentación (grounding) o relevancia, es decir, permitiría respuestas poco ancladas en la fuente de referencia aprobada; esto es lo contrario de lo que la empresa necesita para evitar afirmaciones no respaldadas por su guía financiera.

Opción F (Correcta): el "contextual grounding check" de Bedrock Guardrails evalúa cada respuesta del modelo frente a una fuente de referencia (grounding source) y la consulta del usuario, generando puntuaciones de "grounding" (precisión factual respecto a la fuente) y "relevance" (relevancia respecto a la consulta), con valores configurables entre 0 y 0.99. Cuanto más alto se configure el umbral, mayor es la probabilidad de bloquear contenido no fundamentado o irrelevante (alucinaciones), por lo que un umbral alto es precisamente lo que se necesita para garantizar que las respuestas se ciñan a la guía financiera aprobada por la empresa.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-denied-topics.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-word-filters.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>

A company has deployed an AI assistant as a React application that uses AWS Amplify, an AWS AppSync GraphQL API, and Amazon Bedrock Knowledge Bases. The application uses the GraphQL API to call the Amazon Bedrock RetrieveAndGenerate API for knowledge base interactions. The company configures an AWS Lambda resolver to use the RequestResponse invocation type. Application users report frequent timeouts and slow response times. Users report these problems more frequently for complex questions that require longer processing. The company needs a solution to fix these performance issues and enhance the user experience. Which solution will meet these requirements?

- A Use AWS Amplify AI Kit to implement streaming responses from the GraphQL API and to optimize client-side rendering.
- B Increase the timeout value of the Lambda resolver. Implement retry logic with exponential backoff.
- C Update the application to send an API request to an Amazon SQS queue. Update the AWS AppSync resolver to poll and process the queue.
- D Change the RetrieveAndGenerate API to the InvokeModelWithResponseStream API. Update the application to use an Amazon API Gateway WebSocket API to support the streaming response.

Respuesta correcta: A

- ☒ A Use AWS Amplify AI Kit to implement streaming responses from the GraphQL API and to optimize client-side rendering. 
- ☐ B Increase the timeout value of the Lambda resolver. Implement retry logic with exponential backoff.
- ☐ C Update the application to send an API request to an Amazon SQS queue. Update the AWS AppSync resolver to poll and process the queue.
- ☐ D Change the RetrieveAndGenerate API to the InvokeModelWithResponseStream API. Update the application to use an Amazon API Gateway WebSocket API to support the streaming response.

Explicación

Opción A (Correcta): AWS Amplify AI Kit (parte de Amplify Gen 2, paquete @aws-amplify/ui-react-ai) está diseñado específicamente para aplicaciones React que consumen modelos de Amazon Bedrock a través de una API de AWS AppSync. Sus "conversation routes" utilizan las suscripciones GraphQL en tiempo real de AppSync (basadas en WebSocket) para entregar la respuesta del modelo de forma incremental (token a token) al cliente, en lugar de esperar a que el backend genere la respuesta completa. Esto resuelve directamente el problema descrito: el resolver Lambda invocado de forma síncrona (RequestResponse) puede seguir haciendo la llamada a RetrieveAndGenerate en segundo plano, pero el usuario empieza a ver contenido de inmediato gracias al streaming vía suscripción, eliminando la sensación de timeout/lentitud en preguntas complejas, sin necesidad de rediseñar la arquitectura AppSync/Amplify ya existente.

Opción B: aumentar el valor de timeout del resolver Lambda y añadir reintentos con backoff exponencial no ataca la causa raíz del problema. El tipo de invocación RequestResponse (el valor por defecto en los resolvers Lambda de AppSync, según la documentación oficial) es síncrono: AppSync espera a que Lambda termine de ejecutarse y devuelva la respuesta completa antes de responder al cliente. Subir el timeout solo pospone el corte, y los reintentos con backoff pueden incluso duplicar llamadas costosas a RetrieveAndGenerate sin mejorar la experiencia percibida por el usuario, que sigue viendo una pantalla en blanco durante todo el procesamiento.

Opción C: enviar la petición a una cola de Amazon SQS y hacer que el resolver de AppSync haga polling sobre la cola añade latencia adicional (por los intervalos de sondeo) y complejidad de orquestación, sin resolver el problema de fondo: la respuesta seguiría entregándose de una sola vez al finalizar todo el procesamiento, en lugar de mostrarse progresivamente. Además, esta arquitectura no está pensada para respuestas en tiempo real y requeriría un mecanismo adicional (por ejemplo, otra suscripción) para notificar al cliente, complejidad que Amplify AI Kit ya resuelve de forma nativa.

Opción D: es cierto que InvokeModelWithResponseStream permite obtener una respuesta en streaming de un modelo fundacional en Amazon Bedrock, pero esa API invoca el modelo de forma directa y no incluye la lógica de recuperación (retrieval) contra la base de conocimiento que sí ofrece RetrieveAndGenerate. De hecho, Amazon Bedrock Knowledge Bases expone una API dedicada para este caso, RetrieveAndGenerateStream, que sí combina recuperación y generación en modo streaming; sustituirla por InvokeModelWithResponseStream obligaría a reimplementar manualmente la orquestación de recuperación y aumento de contexto (RAG) que hoy gestiona Bedrock. Además, esta opción exige reemplazar por completo la capa AppSync/GraphQL por una API WebSocket de Amazon API Gateway, lo que implica rediseñar toda la arquitectura del backend, un esfuerzo mucho mayor que aprovechar el soporte de streaming ya integrado en Amplify AI Kit sobre la pila AppSync existente.

Referencias:

https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerateStream.html

<https://docs.aws.amazon.com/appsync/latest/devguide/resolver-mapping-template-reference-lambda.html>

https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerate.html

An ecommerce company operates a global product recommendation system that needs to switch between multiple foundation models (FM) in Amazon Bedrock based on regulations, cost optimization, and performance requirements. The company must apply custom controls based on proprietary business logic, including dynamic cost thresholds, AWS Region-specific compliance rules, and real-time A/B testing across multiple FMs. The system must be able to switch between FMs without deploying new code. The system must route user requests based on complex rules including user tier, transaction value, regulatory zone, and real-time cost metrics that change hourly and require immediate propagation across thousands of concurrent requests. Which solution will meet these requirements?

- A Deploy an AWS Lambda function that uses environment variables to store routing rules and Amazon Bedrock FM IDs. Use the Lambda console to update the environment variables when business requirements change. Configure an Amazon API Gateway REST API to read request parameters to make routing decisions.
- B Deploy Amazon API Gateway REST API request transformation templates to implement routing logic based on request attributes. Store Amazon Bedrock FM endpoints as REST API stage variables. Update the variables when the system switches between models.
- C Configure an AWS Lambda function to fetch routing configurations from the AWS AppConfig Agent for each user request. Run business logic in the Lambda function to select the appropriate FM for each request. Expose the FM through a single Amazon API Gateway REST API endpoint.
- D Use AWS Lambda authorizers for an Amazon API Gateway REST API to evaluate routing rules that are stored in AWS AppConfig. Return authorization contexts based on business logic. Route requests to model-specific Lambda functions for each Amazon Bedrock FM.

Respuesta correcta: C

- A Deploy an AWS Lambda function that uses environment variables to store routing rules and Amazon Bedrock FM IDs. Use the Lambda console to update the environment variables when business requirements change. Configure an Amazon API Gateway REST API to read request parameters to make routing decisions.
- B Deploy Amazon API Gateway REST API request transformation templates to implement routing logic based on request attributes. Store Amazon Bedrock FM endpoints as REST API stage variables. Update the variables when the system switches between models.
- C** Configure an AWS Lambda function to fetch routing configurations from the AWS AppConfig Agent for each user request. Run business logic in the Lambda function to select the appropriate FM for each request. Expose the FM through a single Amazon API Gateway REST API endpoint. ✓
- D Use AWS Lambda authorizers for an Amazon API Gateway REST API to evaluate routing rules that are stored in AWS AppConfig. Return authorization contexts based on business logic. Route requests to model-specific Lambda functions for each Amazon Bedrock FM.

Explicación

Opción A: almacenar las reglas de enrutamiento en variables de entorno de una función Lambda obliga a publicar una nueva versión de la función (o al menos una actualización de configuración) cada vez que cambian los umbrales de coste, las reglas de cumplimiento por región o los parámetros de las pruebas A/B. Esto no permite propagación inmediata a miles de solicitudes concurrentes ni una gestión centralizada de la configuración, y no está pensado para cambios que varían cada hora.

Opción B: las variables de stage de Amazon API Gateway son un mecanismo de configuración estático pensado para valores que cambian con poca frecuencia (por ejemplo, el ARN de un backend). Actualizarlas requiere desplegar de nuevo la etapa de la API, y las plantillas de transformación de solicitudes (VTL) no están diseñadas para evaluar lógica de negocio compleja (nivel de usuario, valor de transacción, zona regulatoria, métricas de coste en tiempo real). No cumple el requisito de reglas dinámicas que cambian cada hora ni de lógica de negocio propietaria.

Opción C (Correcta): AWS AppConfig (una capacidad de AWS Systems Manager) está diseñado precisamente para separar la configuración dinámica del código de la aplicación. El AWS AppConfig Agent Lambda extension se añade como una capa (layer) a la función Lambda y actúa como un proceso complementario: la función Lambda consulta la configuración a través de un endpoint HTTP local (localhost:2772), y la extensión mantiene una caché local de los datos de configuración, gestiona los tokens de sesión necesarios y comprueba periódicamente en segundo plano si hay actualizaciones en AWS AppConfig, actualizando la caché local cuando detecta cambios. Esto permite que la lógica de negocio (umbrales de coste, reglas de cumplimiento por región, asignación de tráfico para pruebas A/B, selección del FM de Amazon Bedrock) se ejecute dentro de la función Lambda leyendo una configuración que se puede actualizar sin desplegar código nuevo, con baja latencia añadida gracias a la caché local, y exponiendo un único endpoint de Amazon API Gateway hacia los clientes. Es la solución recomendada por AWS para exactamente este patrón: feature flags y datos de configuración dinámicos consumidos por Lambda sin necesidad de redepliegue.

Opción D: usar autorizadores Lambda de API Gateway para evaluar las reglas de enrutamiento y devolver un contexto de autorización, enrutando después a una función Lambda distinta por cada FM de Bedrock, añade complejidad arquitectónica innecesaria (múltiples funciones Lambda que mantener y sincronizar) y mezcla responsabilidades de autorización con lógica de enrutamiento de negocio. No aporta ninguna ventaja frente a centralizar la lógica y la configuración dinámica (vía AppConfig) en una única función Lambda detrás de un solo endpoint.

Referencias:

<https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-integration-lambda-extensions.html>

<https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-integration-lambda-extensions-how-it-works.html>

A company is developing an internal generative AI (GenAI) assistant that uses Amazon Bedrock to summarize corporate documents for multiple business units. The GenAI assistant must generate responses in a consistent format that includes a document summary, classification of business risks, and terms that are flagged for review. The GenAI assistant must adapt the tone of responses for each user's business unit, such as legal, human resources, or finance. The GenAI assistant must block hate speech, inappropriate topics, and sensitive information such as personal health information. The company needs a solution to centrally manage prompt variants across business units and teams. The company wants to minimize ongoing orchestration efforts and maintenance for post-processing logic. The company also wants to have the ability to adjust content moderation criteria for the GenAI assistant over time. Which solution will meet these requirements with the LEAST maintenance overhead?

- A Use Amazon Bedrock Prompt Management to configure reusable templates and business unit-specific prompt variants. Apply Amazon Bedrock guardrails that have category filters and sensitive term lists to block prohibited content.
- B Use Amazon Bedrock Prompt Management to define base templates. Enforce business unit-specific tone by using system prompt variables. Configure Amazon Bedrock guardrails to apply audience-based threshold tuning. Manage the guardrails by using an internal administration API.
- C Use Amazon Bedrock with business unit-based instruction injection in API calls. Store response formatting rules in Amazon DynamoDB. Use AWS Step functions to validate responses. Use Amazon Comprehend to apply content filters after the GenAI assistant generates responses.
- D Use Amazon Bedrock with custom prompt templates that are stored in Amazon DynamoDB. Create one AWS Lambda function to select business unit-specific prompts. Create a second Lambda function to call Amazon Comprehend to filter prohibited content from responses.

Respuesta correcta: A

- A** Use Amazon Bedrock Prompt Management to configure reusable templates and business unit-specific prompt variants. Apply Amazon Bedrock guardrails that have category filters and sensitive term lists to block prohibited content. ✓
- B** Use Amazon Bedrock Prompt Management to define base templates. Enforce business unit-specific tone by using system prompt variables. Configure Amazon Bedrock guardrails to apply audience-based threshold tuning. Manage the guardrails by using an internal administration API.
- C** Use Amazon Bedrock with business unit-based instruction injection in API calls. Store response formatting rules in Amazon DynamoDB. Use AWS Step functions to validate responses. Use Amazon Comprehend to apply content filters after the GenAI assistant generates responses.
- D** Use Amazon Bedrock with custom prompt templates that are stored in Amazon DynamoDB. Create one AWS Lambda function to select business unit-specific prompts. Create a second Lambda function to call Amazon Comprehend to filter prohibited content from responses.

Explicación

Opción A (Correcta): Amazon Bedrock Prompt Management permite crear, guardar y reutilizar plantillas de prompt con variables y variantes (alternative configurations of a prompt) de forma nativa y versionada, lo que permite gestionar de forma centralizada las variantes específicas de cada unidad de negocio (tono legal, RR. HH., finanzas) sin construir infraestructura propia. Amazon Bedrock Guardrails complementa esto aportando content filters por categoría predefinida (Hate, Insults, Sexual, Violence, Misconduct, Prompt Attack) para bloquear odio y temas inapropiados, junto con word filters (listas de términos personalizados) y sensitive information filters (para PII, incluida información de salud). Todo esto se gestiona de forma totalmente administrada por AWS, sin necesidad de post-procesamiento propio, y los criterios de moderación se pueden ajustar en cualquier momento editando la configuración del guardrail, cumpliendo el requisito de mínimo mantenimiento.

Opción B: aunque también usa Prompt Management (correcto para las plantillas), introduce una "API de administración interna" para gestionar los guardrails y un mecanismo de "audience-based threshold tuning" que no son funcionalidades nativas de Bedrock Guardrails documentadas; construir y mantener una API propia para administrar los guardrails añade justo la sobrecarga de mantenimiento que la empresa quiere evitar, cuando la consola/API de Guardrails ya permite ajustar fuerza de filtros y listas de términos de forma nativa.

Opción C: inyectar instrucciones por unidad de negocio directamente en las llamadas a la API, almacenar reglas de formato en DynamoDB, validar respuestas con AWS Step Functions y filtrar contenido después de generarlo con Amazon Comprehend implica diseñar, orquestar y mantener una canalización de post-procesamiento completamente personalizada. Esto contradice explícitamente el objetivo de minimizar la orquestación y el mantenimiento de lógica de post-procesamiento, y Comprehend no ofrece los controles de denied topics, word filters o content filters por categoría que sí ofrece Guardrails de forma gestionada.

Opción D: almacenar las plantillas en DynamoDB y usar dos funciones Lambda (una para seleccionar el prompt según la unidad de negocio y otra para invocar Amazon Comprehend y filtrar contenido prohibido) obliga a desarrollar y mantener manualmente capacidades que Prompt Management (gestión centralizada de plantillas/variantes) y Guardrails (moderación de contenido configurable) ya proporcionan como servicio gestionado, aumentando la superficie de código y el esfuerzo operativo continuo.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>

A financial services company is building a customer support application that retrieves relevant financial regulation documents from a database based on semantic similarities to user queries. The application must integrate with Amazon Bedrock to generate responses. The application must be able to search documents that are in English, Spanish, and Portuguese. The application must filter documents by metadata such as publication date, regulatory agency, and document type. The database stores approximately 10 million document embeddings. To minimize operational overhead, the company wants a solution that minimizes management and maintenance effort. The application must provide low-latency responses for real-time customer interactions. Which solution will meet these requirements?

- A Use Amazon OpenSearch Serverless to provide vector search capabilities and metadata filtering. Connect to Amazon Bedrock Knowledge Bases to enable Retrieval Augmented Generation (RAG) capabilities that use an Anthropic Claude foundation model (FM).
- B Deploy an Amazon Aurora PostgreSQL database with the pgvector extension. Define tables to store embeddings and metadata. Use SQL queries to perform similarity searches. Send retrieved documents to Amazon Bedrock to generate responses.
- C Use Amazon S3 Vectors to configure a vector index and non-filterable metadata fields. Integrate S3 Vectors with Amazon Bedrock to enable Retrieval Augmented Generation (RAG) capabilities.
- D Set up an Amazon Neptune Analytics graph database. Configure a vector index that has appropriate dimensionality to store document embeddings. Use Amazon Bedrock to perform graph-based retrieval and to generate responses.

Respuesta correcta: A

- A** Use Amazon OpenSearch Serverless to provide vector search capabilities and metadata filtering. Connect to Amazon Bedrock Knowledge Bases to enable Retrieval Augmented Generation (RAG) capabilities that use an Anthropic Claude foundation model (FM). 
- B** Deploy an Amazon Aurora PostgreSQL database with the pgvector extension. Define tables to store embeddings and metadata. Use SQL queries to perform similarity searches. Send retrieved documents to Amazon Bedrock to generate responses.
- C** Use Amazon S3 Vectors to configure a vector index and non-filterable metadata fields. Integrate S3 Vectors with Amazon Bedrock to enable Retrieval Augmented Generation (RAG) capabilities.
- D** Set up an Amazon Neptune Analytics graph database. Configure a vector index that has appropriate dimensionality to store document embeddings. Use Amazon Bedrock to perform graph-based retrieval and to generate responses.

Explicación

Opción A (Correcta): Amazon Bedrock Knowledge Bases es el servicio totalmente gestionado de AWS para implementar RAG: se encarga de trocear los documentos, generar los embeddings (con modelos que soportan múltiples idiomas, incluidos inglés, español y portugués), indexarlos en el almacén vectorial elegido y orquestar la recuperación más la generación con un modelo de Amazon Bedrock como Anthropic Claude. Al usar Amazon OpenSearch Serverless como almacén vectorial, AWS gestiona automáticamente el aprovisionamiento, el escalado y el mantenimiento de la infraestructura (sin servidores ni clústeres que administrar), lo que la hace idónea para una colección de ~10 millones de embeddings con baja latencia. Además, OpenSearch Serverless soporta filtrado por campos de metadatos (fecha de publicación, agencia reguladora, tipo de documento) de forma nativa dentro de la consulta vectorial, cumpliendo así todos los requisitos (búsqueda semántica multilingüe, filtrado por metadatos, baja latencia y mínimo esfuerzo operativo) con la combinación más directa y recomendada por AWS para este patrón.

Opción B: Aurora PostgreSQL con la extensión pgvector es una opción válida técnicamente para almacenar embeddings y hacer búsquedas de similitud mediante SQL, y Bedrock Knowledge Bases también la soporta como almacén vectorial. Sin embargo, requiere que el cliente aprovisione y administre la instancia/clúster de base de datos (parcheo, escalado, alta disponibilidad), y para habilitar el filtrado por metadatos en Aurora es necesario crear y mantener manualmente columnas específicas para cada campo de metadatos, lo que añade carga operativa y de diseño de esquema frente a una opción serverless nativa. Esto contradice el requisito de minimizar el esfuerzo de gestión y mantenimiento.

Opción C: Amazon S3 Vectors permite dos tipos de metadatos: "filterable metadata" (filtrable por defecto en toda consulta de similitud, salvo que se indique lo contrario) y "non-filterable metadata" (metadatos que se almacenan y se pueden devolver junto con los resultados, pero que explícitamente NO pueden usarse como filtro de la consulta, y una vez marcados como no filtrables no se pueden cambiar). La opción C describe configurar explícitamente los campos de metadatos como "non-filterable", lo cual impediría precisamente filtrar por fecha de publicación, agencia reguladora y tipo de documento, incumpliendo un requisito explícito del enunciado. Si se hubieran configurado como "filterable" (el comportamiento por defecto), S3 Vectors sí podría integrarse con Bedrock Knowledge Bases para RAG, pero tal como está planteada la opción no satisface el caso de uso.

Opción D: Amazon Neptune Analytics puede usarse como almacén vectorial para Amazon Bedrock Knowledge Bases, principalmente en el patrón "GraphRAG", que combina embeddings vectoriales con relaciones de grafo entre entidades para mejorar la recuperación cuando existen conexiones semánticas complejas entre documentos. Para este caso de uso, que es una búsqueda semántica directa sobre grandes volúmenes de documentos con filtrado simple por metadatos (no relaciones entre entidades), configurar y mantener un grafo con Neptune Analytics añade una complejidad de modelado y operación innecesaria frente a un almacén vectorial serverless dedicado como OpenSearch Serverless.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-test-config.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html>

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors-metadata-filtering.html>

A medical company is building a generative AI (GenAI) application that uses RAG to provide evidence-based medical information. The application uses Amazon OpenSearch Service to retrieve vector embeddings. Users report that searches frequently miss results that contain exact medical terms and acronyms and return too many semantically similar but irrelevant documents. The company needs to improve retrieval quality and maintain low end user latency, even as the document collection grows to millions of documents. Which solution will meet these requirements with the LEAST operational overhead?

- A Configure hybrid search by combining vector similarity with keyword matching to improve semantic understanding and exact term and acronym matching.
- B Increase the dimensions of the vector embeddings from 384 to 1536. Use a post-processing AWS Lambda function to filter out irrelevant results after retrieval.
- C Replace OpenSearch Service with Amazon Kendra. Use query expansion to handle medical acronyms and terminology variants during pre-processing.
- D Implement a two-stage retrieval architecture in which initial vector search results are re-ranked by an ML model that is hosted on Amazon SageMaker AI.

Respuesta correcta: A

- A** Configure hybrid search by combining vector similarity with keyword matching to improve semantic understanding and exact term and acronym matching. ✓
- B** Increase the dimensions of the vector embeddings from 384 to 1536. Use a post-processing AWS Lambda function to filter out irrelevant results after retrieval.
- C** Replace OpenSearch Service with Amazon Kendra. Use query expansion to handle medical acronyms and terminology variants during pre-processing.
- D** Implement a two-stage retrieval architecture in which initial vector search results are re-ranked by an ML model that is hosted on Amazon SageMaker AI.

Explicación

Opción A (Correcta): la búsqueda híbrida ("hybrid search") en Amazon OpenSearch Service combina, dentro de una única consulta, la puntuación léxica basada en palabras clave (algoritmo BM25) con la puntuación semántica de la búsqueda vectorial (neural/k-NN), usando un pipeline de búsqueda que normaliza y combina ambos scores (por ejemplo con normalización min_max o l2 y combinación por media aritmética, geométrica o armónica). Esto permite capturar tanto el significado semántico de la consulta como coincidencias exactas de términos y acrónimos médicos (que la búsqueda puramente vectorial tiende a "diluir" semánticamente), sin necesidad de desplegar componentes adicionales, infraestructura nueva ni modelos extra: se configura mediante un search pipeline y una consulta de tipo "hybrid" sobre el mismo dominio/clúster de OpenSearch ya existente, por lo que escala igual de bien a millones de documentos y mantiene baja latencia con la mínima sobrecarga operativa.

Opción B: aumentar la dimensionalidad de los embeddings (de 384 a 1536) no resuelve el problema de fondo, que es que la búsqueda puramente vectorial no realiza coincidencia léxica exacta de términos y acrónimos; embeddings de mayor dimensión pueden incluso aumentar el coste de cómputo y almacenamiento sin garantizar mejor recall de términos exactos. Además, añadir una función AWS Lambda de post-procesamiento para filtrar resultados introduce latencia adicional en el camino crítico de la consulta y un componente nuevo que hay que desarrollar, desplegar y mantener, lo que incrementa la sobrecarga operativa en vez de reducirla.

Opción C: sustituir Amazon OpenSearch Service por Amazon Kendra implica migrar toda la arquitectura de recuperación (reindexar el contenido, adaptar la integración con la aplicación RAG, revisar costes y límites del nuevo servicio) en lugar de simplemente activar una capacidad de búsqueda híbrida ya disponible en el motor que la empresa ya opera. Aunque Kendra ofrece expansión de consultas y comprensión de lenguaje natural, el cambio de servicio completo supone un esfuerzo de migración y una carga operativa muy superiores a la alternativa de la opción A.

Opción D: implementar una arquitectura de recuperación en dos etapas, donde los resultados iniciales de la búsqueda vectorial se re-ordenan ("re-rank") mediante un modelo alojado en un endpoint de Amazon SageMaker AI, añade una etapa de inferencia adicional y un endpoint que hay que aprovisionar, escalar, monitorizar y mantener. Esto incrementa tanto la latencia extremo a extremo (una llamada de inferencia adicional por consulta) como la complejidad y el coste operativo, frente a la solución nativa de búsqueda híbrida de OpenSearch que resuelve el mismo problema de relevancia dentro del propio motor de búsqueda.

Referencias:

- <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/serverless-configure-neural-search.html>
- <https://docs.aws.amazon.com/prescriptive-guidance/latest/migration-solr-opensearch/operational-architecture.html>

A company runs a generative AI (GenAI)-powered summarization application in an application AWS account that uses Amazon Bedrock. The application architecture includes an Amazon API Gateway REST API that forwards requests to AWS Lambda functions that are attached to private VPC subnets. The application summarizes sensitive customer records that the company stores in a governed data lake in a centralized data storage account. The company has enabled Amazon S3, Amazon Athena, and AWS Glue in the data storage account. The company must ensure that calls that the application makes to Amazon Bedrock use only private connectivity between the company's application VPC and Amazon Bedrock. The company's data lake must provide fine-grained column-level access across the company's AWS accounts. Which solution will meet these requirements?

- A In the application account, create interface VPC endpoints for Amazon Bedrock runtimes. Run Lambda functions in private subnets. Use IAM conditions on inference and data-plane policies to allow calls only to approved endpoints and roles. In the data storage account, use AWS Lake Formation LF-tag-based access control to create table and column-level cross-account grants.
- B Run Lambda functions in private subnets. Configure a NAT gateway to provide access to Amazon Bedrock and the data lake. Use S3 bucket policies and ACLs to manage permissions. Export AWS CloudTrail logs to Amazon S3 to perform weekly reviews.
- C Create a gateway endpoint only for Amazon S3 in the application account. Invoke Amazon Bedrock through public endpoints. Use database-level grants in AWS Lake Formation to manage data access. Stream AWS CloudTrail logs to Amazon CloudWatch Logs. Do not set up metric filters or alarms.
- D Use VPC endpoints to provide access to Amazon Bedrock and Amazon S3 in the application account. Use only IAM path-based policies to manage data lake access. Send AWS CloudTrail logs to Amazon CloudWatch Logs. Periodically create dashboards and allow public fallback for cross-Region reads to reduce setup time.

Respuesta correcta: A

- A** In the application account, create interface VPC endpoints for Amazon Bedrock runtimes. Run Lambda functions in private subnets. Use IAM conditions on inference and data-plane policies to allow calls only to approved endpoints and roles. In the data storage account, use AWS Lake Formation LF-tag-based access control to create table and column-level cross-account grants. ✓
- B** Run Lambda functions in private subnets. Configure a NAT gateway to provide access to Amazon Bedrock and the data lake. Use S3 bucket policies and ACLs to manage permissions. Export AWS CloudTrail logs to Amazon S3 to perform weekly reviews.
- C** Create a gateway endpoint only for Amazon S3 in the application account. Invoke Amazon Bedrock through public endpoints. Use database-level grants in AWS Lake Formation to manage data access. Stream AWS CloudTrail logs to Amazon CloudWatch Logs. Do not set up metric filters or alarms.
- D** Use VPC endpoints to provide access to Amazon Bedrock and Amazon S3 in the application account. Use only IAM path-based policies to manage data lake access. Send AWS CloudTrail logs to Amazon CloudWatch Logs. Periodically create dashboards and allow public fallback for cross-Region reads to reduce setup time.

Explicación

Opción A (Correcta): Los interface VPC endpoints de Amazon Bedrock, basados en AWS PrivateLink, permiten crear una conexión privada entre la VPC de la aplicación y Amazon Bedrock sin necesidad de gateway de Internet, dispositivo NAT, VPN ni Direct Connect; las instancias/funciones Lambda en la VPC no necesitan direcciones IP públicas para acceder al servicio, y AWS documenta endpoints específicos para el plano de control (bedrock) y el plano de datos/runtime (bedrock-runtime), entre otros. Añadir condiciones IAM en las políticas de los endpoints e inferencia (por ejemplo restringiendo por `aws:sourceVpce` o por rol) permite limitar las llamadas solo a los endpoints y roles aprobados, cumpliendo el requisito de conectividad exclusivamente privada. En la cuenta de almacenamiento de datos, AWS Lake Formation con tag-based access control (LF-TBAC) permite crear LF-Tags y asignarlas a bases de datos, tablas y columnas del Data Catalog, y luego conceder permisos a cuentas externas (cross-account) basados en esas etiquetas, lo que proporciona el control de acceso detallado a nivel de columna que exige el escenario, sin necesidad de crear un grant independiente por cada combinación de principal y recurso.

Opción B: Un NAT gateway únicamente traduce direcciones para tráfico saliente hacia Internet; enviar las llamadas a Amazon Bedrock y al data lake a través de un NAT gateway implica que el tráfico sale por Internet en lugar de mantenerse dentro de la red privada de AWS, lo que incumple el requisito de conectividad exclusivamente privada. Además, los bucket policies y las ACL de S3 solo controlan el acceso a nivel de objeto/bucket, no ofrecen el control de acceso a nivel de columna que se necesita, y una revisión semanal de CloudTrail es un control detectivo, no preventivo.

Opción C: Invocar Amazon Bedrock a través de endpoints públicos contradice directamente el requisito de que las llamadas usen solo conectividad privada entre la VPC y Bedrock, incluso si se crea un gateway endpoint de S3 (que de por sí solo cubre S3, no Bedrock). Los permisos a nivel de base de datos en AWS Lake Formation son demasiado amplios: no permiten restringir el acceso a columnas específicas, que es justamente lo que exige el enunciado. Tampoco configurar métricas ni alarmas deja sin visibilidad operativa proactiva.

Opción D: Aunque se usen VPC endpoints para Bedrock y S3 en la cuenta de la aplicación, apoyarse únicamente en políticas IAM basadas en rutas (path-based) para gestionar el acceso al data lake no proporciona el control de acceso a nivel de columna que ofrece Lake Formation mediante LF-Tags; las políticas IAM basadas en recursos de S3 no distinguen entre columnas de una tabla en Athena/Glue. Además, permitir un "fallback" público para lecturas entre regiones anula el requisito de conectividad exclusivamente privada al introducir una ruta pública.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html>

<https://docs.aws.amazon.com/lake-formation/latest/dg/cross-account-TBAC.html>

<https://docs.aws.amazon.com/lake-formation/latest/dg/TBAC-granting-tags.html>

A media company must use Amazon Bedrock to implement a robust governance process for AI-generated content. The company needs to manage hundreds of prompt templates. Multiple teams use the templates across multiple AWS Regions to generate content. The solution must provide version control with approval workflows that include notifications for pending reviews. The solution must also provide detailed audit trails that document prompt activities and consistent prompt parameterization to enforce quality standards. Which solution will meet these requirements?

- A Configure Amazon Bedrock Studio prompt templates. Use Amazon CloudWatch to create dashboards that display prompt usage metrics. Store the approval status of content in Amazon DynamoDB. Use AWS Lambda functions to enforce approvals.
- B Use Amazon Bedrock Prompt Management to implement version control. Configure AWS CloudTrail for audit logging. Use IAM policies to control approval permissions. Create parameterized prompt templates by specifying variables.
- C Use AWS Step Functions to create an approval workflow. Store prompts as documents in Amazon S3. Use tags to implement version control. Use Amazon EventBridge to send notifications.
- D Deploy Amazon SageMaker Canvas with prompt templates that are stored in Amazon S3. Use AWS CloudFormation to implement version control. Use AWS Config to enforce approval policies.

Respuesta correcta: **B**

- A Configure Amazon Bedrock Studio prompt templates. Use Amazon CloudWatch to create dashboards that display prompt usage metrics. Store the approval status of content in Amazon DynamoDB. Use AWS Lambda functions to enforce approvals.
- B Use Amazon Bedrock Prompt Management to implement version control. Configure AWS CloudTrail for audit logging. Use IAM policies to control approval permissions. Create parameterized prompt templates by specifying variables.** ✓
- C Use AWS Step Functions to create an approval workflow. Store prompts as documents in Amazon S3. Use tags to implement version control. Use Amazon EventBridge to send notifications.
- D Deploy Amazon SageMaker Canvas with prompt templates that are stored in Amazon S3. Use AWS CloudFormation to implement version control. Use AWS Config to enforce approval policies.

Explicación

Opción A: Amazon Bedrock Studio (actualmente integrado en Amazon SageMaker Unified Studio) está pensado para colaborar en la construcción de aplicaciones de IA generativa, no para gestionar el ciclo de vida de plantillas de prompts a escala empresarial. Usar CloudWatch solo aporta métricas de uso, no versionado; y almacenar el estado de aprobación en DynamoDB con funciones Lambda para "forzarlo" implica diseñar y mantener un sistema de aprobación totalmente a medida, sin apoyarse en ninguna capacidad nativa de Bedrock para prompts.

Opción B (Correcta): Amazon Bedrock Prompt Management permite crear, editar y guardar prompts reutilizables, incluir variables (placeholders del tipo {{variable}}) para parametrizar las plantillas y adaptarlas a distintos casos de uso, y crear versiones inmutables de cada prompt (con capacidad de comparar versiones y desplegar una versión concreta en producción), lo que cubre el requisito de parametrización consistente y control de versiones a través de múltiples equipos y Regiones. El acceso a las operaciones de Prompt Management (crear, modificar, ver, eliminar prompts y versiones) se controla mediante políticas de IAM basadas en identidad, lo que permite restringir quién puede crear o publicar una versión, aproximando un control de permisos de aprobación. Además, AWS CloudTrail registra automáticamente como eventos de gestión las llamadas a la API de Bedrock (incluidas las de Prompt Management, como CreatePrompt, CreatePromptVersion, UpdatePrompt, etc.), proporcionando el rastro de auditoría detallado que exige el enunciado. Es la combinación de servicios nativos que mejor cubre los requisitos sin necesidad de desarrollo adicional significativo, aunque cabe matizar que el propio Prompt Management no incluye una funcionalidad nativa de "notificaciones automáticas de revisión pendiente" tipo flujo de aprobación; ese aspecto concreto se cubriría apoyándose en el control de permisos de IAM (p. ej., que solo un rol de "aprobador" pueda crear una nueva versión) más que en una notificación push nativa del servicio.

Opción C: Construir el flujo con AWS Step Functions, almacenar los prompts como documentos en Amazon S3, usar etiquetas para emular versionado y Amazon EventBridge para notificaciones sí cubriría el requisito de notificaciones de forma más directa que Prompt Management, pero obliga a reconstruir desde cero toda la capa de versionado, parametrización y trazabilidad de prompts que Bedrock Prompt Management ya ofrece de manera nativa e integrada con el resto de Bedrock (modelos, flows, etc.); además, usar etiquetas de S3 como mecanismo de versionado es frágil y no proporciona un historial de versiones inmutable ni comparación entre versiones como sí hace Prompt Management.

Opción D: Amazon SageMaker Canvas es una herramienta de no-code orientada a la creación y uso de modelos de machine learning (predicción, clasificación, etc.), no a la gestión de plantillas de prompts de Amazon Bedrock. Usar AWS CloudFormation para "versionar" y AWS Config para "aprobar" tampoco corresponde al propósito de esos servicios: CloudFormation gestiona el ciclo de vida de infraestructura como código y AWS Config evalúa el cumplimiento de configuración de recursos, ninguno de los dos está diseñado para flujos de aprobación de contenido generado o de plantillas de prompts.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management-prereq.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html>

A company is developing a customer support application that uses Amazon Bedrock foundation models (FMs) to provide real-time AI assistance to the company's employees. The application must display AI-generated responses character by character as the responses are generated. The application needs to support thousands of concurrent users with minimal latency. The responses typically take 15 to 45 seconds to finish. Which solution will meet these requirements?

- A Configure an Amazon API Gateway WebSocket API with an AWS Lambda integration. Configure the WebSocket API to invoke the Amazon Bedrock `InvokeModelWithResponseStream` API and stream partial responses through WebSocket connections.
- B Configure an Amazon API Gateway REST API with an AWS Lambda integration. Configure the REST API to invoke the Amazon Bedrock standard `InvokeModel` API and implement frontend client-side polling every 100 ms for complete response chunks.
- C Implement direct frontend client connections to Amazon Bedrock by using IAM user credentials and the `InvokeModelWithResponseStream` API without any intermediate gateway or proxy layer.
- D Configure an Amazon API Gateway HTTP API with an AWS Lambda integration. Configure the HTTP API to cache complete responses in an Amazon DynamoDB table and serve the responses through multiple paginated GET requests to frontend clients.

Respuesta correcta: A

- A** Configure an Amazon API Gateway WebSocket API with an AWS Lambda integration. Configure the WebSocket API to invoke the Amazon Bedrock InvokeModelWithResponseStream API and stream partial responses through WebSocket connections. ✓
- B** Configure an Amazon API Gateway REST API with an AWS Lambda integration. Configure the REST API to invoke the Amazon Bedrock standard InvokeModel API and implement frontend client-side polling every 100 ms for complete response chunks.
- C** Implement direct frontend client connections to Amazon Bedrock by using IAM user credentials and the InvokeModelWithResponseStream API without any intermediate gateway or proxy layer.
- D** Configure an Amazon API Gateway HTTP API with an AWS Lambda integration. Configure the HTTP API to cache complete responses in an Amazon DynamoDB table and serve the responses through multiple paginated GET requests to frontend clients.

Explicación

Opción A (Correcta): Amazon API Gateway ofrece un tipo de API WebSocket que mantiene una conexión persistente y bidireccional entre el cliente y el backend, a diferencia de las APIs REST/HTTP basadas en solicitud-respuesta única. Al integrar esta WebSocket API con una función AWS Lambda que invoca la operación InvokeModelWithResponseStream de Amazon Bedrock Runtime, el modelo fundacional devuelve la respuesta en fragmentos (chunks) a medida que se van generando, en lugar de esperar a la respuesta completa. La función Lambda puede reenviar cada fragmento al cliente a través de la conexión WebSocket (por ejemplo, usando la API de administración de conexiones de API Gateway para "postear" a la conexión activa), logrando así el efecto de texto apareciendo carácter a carácter. Este patrón es el recomendado por AWS para aplicaciones de chat e IA generativa en tiempo real, ya que las conexiones WebSocket son ligeras, permiten miles de conexiones concurrentes y ofrecen baja latencia percibida incluso cuando la generación completa de la respuesta tarda entre 15 y 45 segundos, porque el usuario empieza a ver contenido casi de inmediato en vez de esperar el resultado final.

Opción B: una REST API de API Gateway con InvokeModel (la versión no-streaming) debe esperar a que Bedrock genere la respuesta completa antes de devolver cualquier dato, por lo que no existe forma de mostrar texto incrementalmente. Además, implementar polling desde el cliente cada 100 ms multiplica innecesariamente el número de solicitudes (miles de usuarios concurrentes generarían una carga masiva e ineficiente sobre la API y el backend), y sigue sin resolver el problema de fondo: no hay fragmentos parciales que consultar hasta que el modelo termine, por lo que el polling no logra el efecto de streaming real solicitado.

Opción C: conectar el frontend directamente a Amazon Bedrock usando credenciales de usuario IAM, sin ninguna capa intermedia (gateway o proxy), es una práctica de seguridad desaconsejada por AWS, ya que expondría credenciales de larga duración en el cliente (navegador/app), con el riesgo de robo o uso indebido. Los patrones de referencia de AWS recomiendan siempre pasar por un backend (Lambda, contenedor, etc.) con credenciales temporales o roles de IAM, nunca invocar servicios de AWS directamente desde el frontend con credenciales de usuario.

Opción D: cachear la respuesta completa en una tabla de Amazon DynamoDB y exponerla mediante múltiples solicitudes GET paginadas no constituye streaming en tiempo real: la respuesta debe generarse y almacenarse por completo en DynamoDB antes de que el cliente pueda empezar a consultarla, lo que anula el requisito de mostrar el texto a medida que se genera. Además, añade latencia y complejidad adicional (escritura en DynamoDB y múltiples solicitudes HTTP de lectura) sin ningún beneficio frente al streaming nativo que ofrece InvokeModelWithResponseStream.

Referencias:

- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_InvokeModelWithResponseStream.html
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>
- <https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/chat-use-case-1.html>

A company is using Amazon Bedrock to design an application to help researchers apply for grants. The application is based on an Amazon Nova Pro foundation model (FM). The application contains four required inputs and must provide responses in a consistent text format. The company wants to receive a notification in Amazon Bedrock if a response contains bullying language. However, the company does not want to block all flagged responses. The company creates an Amazon Bedrock flow that takes an input prompt and sends it to the Amazon Nova Pro FM. The Amazon Nova Pro FM provides a response. Which additional steps must the company take to meet these requirements? (Choose two.)

Selecciona 2 respuestas

- A Use Amazon Bedrock Prompt Management to specify the required inputs as variables. Select an Amazon Nova Pro FM. Specify the output format for the response. Add the prompt to the prompts node of the flow.
- B Create an Amazon Bedrock guardrail that applies the hate content filter. Set the filter response to block. Add the guardrail to the prompts node of the flow.
- C Create an Amazon Bedrock prompt router. Specify an Amazon Nova Pro FM. Add the required inputs as variables to the input node of the flow. Add the prompt router to the prompts node. Add the output format to the output node.
- D Create an Amazon Bedrock guardrail that applies the insults content filter. Set the filter response to detect. Add the guardrail to the prompts node of the flow.
- E Create an Amazon Bedrock application inference profile that specifies an Amazon Nova Pro FM. Specify the output format for the response in the description. Include a tag for each of the input variables. Add the profile to the prompts node of the flow.

Respuesta correcta: A, D

- A** Use Amazon Bedrock Prompt Management to specify the required inputs as variables. Select an Amazon Nova Pro FM. Specify the output format for the response. Add the prompt to the prompts node of the flow. ✓
- B** Create an Amazon Bedrock guardrail that applies the hate content filter. Set the filter response to block. Add the guardrail to the prompts node of the flow.
- C** Create an Amazon Bedrock prompt router. Specify an Amazon Nova Pro FM. Add the required inputs as variables to the input node of the flow. Add the prompt router to the prompts node. Add the output format to the output node.
- D** Create an Amazon Bedrock guardrail that applies the insults content filter. Set the filter response to detect. Add the guardrail to the prompts node of the flow. ✓
- E** Create an Amazon Bedrock application inference profile that specifies an Amazon Nova Pro FM. Specify the output format for the response in the description. Include a tag for each of the input variables. Add the profile to the prompts node of the flow.

Explicación

Opción A (Correcta): Amazon Bedrock Prompt Management permite crear un prompt reutilizable en el que las cuatro entradas requeridas se definen como "variables" (placeholders que se rellenan al invocar el modelo), se selecciona el FM de Amazon Nova Pro y se especifica el formato de salida deseado en las instrucciones del prompt para obtener respuestas consistentes. Ese prompt gestionado se añade después al nodo de prompts ("Prompt node") del flujo de Amazon Bedrock, tal como describe la documentación de Prompt Management y de los nodos de flujo.

Opción B: el filtro de contenido "Hate" (odio) de Amazon Bedrock Guardrails cubre contenido que discrimina, insulta o deshumaniza a una persona o grupo por motivos de identidad (raza, género, religión, etc.), no el lenguaje de acoso o bullying en general, por lo que no es la categoría adecuada. Además, configurar la acción de filtrado en "Block" bloquearía automáticamente cualquier respuesta marcada, lo que contradice el requisito explícito de la empresa de no bloquear todas las respuestas señaladas.

Opción C: un "prompt router" de Amazon Bedrock (enrutamiento inteligente de prompts) es un mecanismo que dirige dinámicamente cada solicitud hacia distintos modelos fundacionales dentro de la misma familia según criterios de rendimiento y coste, con un modelo de reserva ("fallback") configurado; no sirve para declarar variables de entrada de un prompt ni para fijar el formato de la respuesta, por lo que no cumple lo que pide el enunciado.

Opción D (Correcta): dentro de las categorías de content filters de Amazon Bedrock Guardrails, "Insults" se define explícitamente como lenguaje humillante, burlón o denigrante, y la propia documentación aclara que "este tipo de lenguaje también se etiqueta como bullying". Al configurar la acción de la guardrail como "Detect (no action)" en lugar de "Block", el guardrail identifica y registra el contenido dañino (permitiendo su seguimiento/alerta) pero deja pasar la respuesta al usuario sin bloquearla, que es exactamente lo que la empresa necesita. Esta guardrail se asocia al nodo de prompts del flujo.

Opción E: un "application inference profile" de Amazon Bedrock es un recurso pensado para el seguimiento y la atribución de costes (permite asignar etiquetas de asignación de costes personalizadas a las invocaciones de un modelo concreto), no para definir variables de entrada de un prompt ni para especificar el formato de la respuesta del modelo, por lo que no resuelve el requisito planteado.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html>

A healthcare company is using Amazon Bedrock to build a Retrieval Augmented Generation (RAG) application that helps practitioners make clinical decisions. The application must achieve high accuracy for patient information retrievals, identify hallucinations in generated content, and reduce human review costs. Which solution will meet these requirements?

- A Use Amazon Comprehend to analyze and classify RAG responses and to extract medical entities and relationships. Use AWS Step Functions to orchestrate automated evaluations. Configure Amazon CloudWatch metrics to track entity recognition confidence scores. Configure CloudWatch to send an alert when accuracy falls below specified thresholds.
- B Implement automated large language model (LLM)-based evaluations that use a specialized model that is fine-tuned for medical content to assess all responses. Deploy AWS Lambda functions to parallelize evaluations. Publish results to Amazon CloudWatch metrics that track relevance and factual accuracy.
- C Configure Amazon CloudWatch Synthetics to generate test queries that have known answers on a regular schedule, and track model success rates. Set up dashboards that compare synthetic test results against expected outcomes.
- D Deploy a hybrid evaluation system that uses an automated LLM-as-a-judge evaluation to initially screen responses and targeted human reviews for edge cases. Use Amazon SageMaker Feature Store to maintain evaluation datasets. Use a built-in Amazon Bedrock evaluation to track retrieval precision and hallucination rates.

Respuesta correcta: **D**

- A Use Amazon Comprehend to analyze and classify RAG responses and to extract medical entities and relationships. Use AWS Step Functions to orchestrate automated evaluations. Configure Amazon CloudWatch metrics to track entity recognition confidence scores. Configure CloudWatch to send an alert when accuracy falls below specified thresholds.
- B Implement automated large language model (LLM)-based evaluations that use a specialized model that is fine-tuned for medical content to assess all responses. Deploy AWS Lambda functions to parallelize evaluations. Publish results to Amazon CloudWatch metrics that track relevance and factual accuracy.
- C Configure Amazon CloudWatch Synthetics to generate test queries that have known answers on a regular schedule, and track model success rates. Set up dashboards that compare synthetic test results against expected outcomes.
- D** Deploy a hybrid evaluation system that uses an automated LLM-as-a-judge evaluation to initially screen responses and targeted human reviews for edge cases. Use Amazon SageMaker Feature Store to maintain evaluation datasets. Use a built-in Amazon Bedrock evaluation to track retrieval precision and hallucination rates. ✓

Explicación

Opción A: Amazon Comprehend está orientado a tareas de PLN como extracción de entidades médicas, clasificación de texto y análisis de sentimiento, pero no está diseñado para evaluar la fidelidad de una respuesta generada por un LLM frente al contenido recuperado. Una puntuación de confianza en el reconocimiento de entidades mide la certeza del modelo de NER, no si la respuesta del sistema RAG contiene alucinaciones o si la información recuperada es precisa respecto a la consulta clínica.

Opción B: someter el 100% de las respuestas únicamente a un LLM evaluador especializado, sin ningún mecanismo de revisión humana para los casos límite o de alto riesgo, no resuelve el requisito de "reducir los costes de revisión humana" de forma responsable en un dominio clínico: en ausencia de una capa de revisión humana dirigida, cualquier error sistemático del LLM evaluador (por ejemplo, en escenarios ambiguos o poco frecuentes) pasaría sin control adicional, lo cual es inaceptable cuando las respuestas apoyan decisiones clínicas.

Opción C: Amazon CloudWatch Synthetics ejecuta "canarios" (scripts programados) pensados para monitorizar la disponibilidad y el rendimiento de endpoints y flujos de usuario (por ejemplo, tiempos de respuesta de una API), no para evaluar la calidad semántica, la precisión factual o la presencia de alucinaciones en respuestas de texto abierto generadas por un sistema RAG. Comparar resultados frente a "respuestas conocidas" de forma programada no captura matices de lenguaje natural ni fidelidad al contexto recuperado.

Opción D (Correcta): Amazon Bedrock ofrece evaluaciones nativas (Bedrock Evaluations) que incluyen evaluaciones automáticas mediante "LLM-as-a-judge" (un segundo modelo actúa como juez, puntuando las respuestas y aportando una explicación), evaluaciones con trabajadores humanos, y evaluaciones específicas de RAG con métricas integradas. Para trabajos de evaluación RAG de tipo "retrieve-only" existen métricas como Context Relevance y Context Coverage, que miden precisión y cobertura de la recuperación; para trabajos "retrieve-and-generate" existe la métrica Faithfulness, que mide explícitamente "qué tan bien las respuestas evitan la alucinación con respecto a los textos recuperados". Un diseño híbrido que use el LLM-as-a-judge para cribar automáticamente el grueso de las respuestas y reserve la revisión humana solo para los casos límite reduce sustancialmente el coste de revisión manual sin sacrificar la fiabilidad, mientras que las métricas nativas de Bedrock permiten cuantificar de forma continua la precisión de recuperación y la tasa de alucinaciones, cumpliendo los tres requisitos del enunciado (alta precisión en la recuperación, detección de alucinaciones y reducción de costes de revisión humana).

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-evaluation-metrics.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-judge.html>

Company configures a landing zone in AWS Control Tower. The company handles sensitive data that must remain within the European Union. The company must use only the eu-central-1 Region. The company uses SCPs to enforce data residency policies. GenAI developers at the company are assigned IAM roles that have full permissions for Amazon Bedrock. The company must ensure that GenAI developers can use the Amazon Nova Pro model through Amazon Bedrock only by using cross-Region inference (CRI) and only in eu-central-1. The company enables model access for the GenAI developer IAM roles in Amazon Bedrock. However, when a GenAI developer attempts to invoke the model through the Amazon Bedrock Chat/Text playground, the GenAI developer receives the following error. User: arn:aws:sts::123456789012:assumed-role/AssumedDevRole/DevUserName Action: bedrock:InvokeModelWithResponseStream On resource(s): arn:aws:bedrock:eu-west-3::foundation-model/amazon.nova-pro-v1:0 Context: a service control policy explicitly denies the action The company needs a solution to resolve the error. The solution must retain the company's existing governance controls and must provide precise access control. The solution must comply with the company's existing data residency policies. Which combination of solutions will meet these requirements? (Choose two.)

Selecciona 2 respuestas

- A Add an AdministratorAccess policy to the GenAI developer IAM role.
- B Extend the existing SCPs to enable CRI for the eu.amazon.nova-pro-v1:0 inference profile.
- C Enable Amazon Bedrock model access for Amazon Nova Pro in the eu-west-3 Region.
- D Validate that the GenAI developer IAM roles have permissions to invoke Amazon Nova Pro through the eu.amazon.nova-pro.v1:0 inference profile on all European Union AWS Regions that can serve the model.
- E Extend the existing SCP to enable CRI for the eu.* inference profile.

Respuesta correcta: B, D

- A Add an AdministratorAccess policy to the GenAI developer IAM role.
- B Extend the existing SCPs to enable CRI for the eu.amazon.nova-pro-v1:0 inference profile. ✓**
- C Enable Amazon Bedrock model access for Amazon Nova Pro in the eu-west-3 Region.
- D Validate that the GenAI developer IAM roles have permissions to invoke Amazon Nova Pro through the eu.amazon.nova-pro.v1:0 inference profile on all European Union AWS Regions that can serve the model. ✓**
- E Extend the existing SCP to enable CRI for the eu.* inference profile.

Explicación

Opción A: otorgar la política AdministratorAccess al rol de IAM del desarrollador elimina cualquier control de acceso granular y viola el requisito de la empresa de mantener un control de acceso preciso ("least privilege"). Además no resuelve la causa raíz del error, que es una denegación explícita a nivel de SCP, no una carencia de permisos IAM.

Opción B (Correcta): el error "a service control policy explicitly denies the action" indica que la SCP de la organización está bloqueando el acceso a regiones de la UE distintas de eu-central-1 (en este caso eu-west-3). El inference profile geográfico "eu.amazon.nova-pro-v1:0" (Geographic cross-Region inference) enruta las solicitudes entre varias regiones de destino dentro de la UE (por ejemplo eu-central-1, eu-west-1, eu-west-3, eu-north-1, según disponibilidad del modelo), no solo eu-central-1. Según la documentación oficial de AWS sobre los requisitos de SCP para Geographic cross-Region inference, si la SCP restringe el acceso únicamente a eu-central-1, la inferencia cross-Region fallará al intentar enrutar a las demás regiones de destino del perfil. Por tanto, la solución correcta es ampliar la SCP existente para permitir explícitamente el perfil de inferencia "eu.amazon.nova-pro-v1:0" (y las regiones de destino que dicho perfil utiliza), lo cual sigue cumpliendo la política de residencia de datos porque todas esas regiones de destino permanecen dentro de la UE, y mantiene un control preciso al autorizar un perfil concreto en lugar de un acceso amplio.

Opción C: habilitar el acceso al modelo Amazon Nova Pro en eu-west-3 no soluciona el problema. El error no se origina por falta de "model access" habilitado en Bedrock, sino por una denegación explícita de la SCP sobre la acción "bedrock:InvokeModelWithResponseStream" en ese recurso regional; mientras la SCP siga bloqueando esa región de destino del perfil geográfico, la inferencia seguirá fallando independientemente de si el acceso al modelo está habilitado.

Opción D (Correcta): según la documentación oficial de AWS sobre los requisitos de política de IAM para Geographic cross-Region inference, para invocar un modelo mediante un perfil de inferencia geográfico es necesario conceder permisos de invocación (por ejemplo "bedrock:InvokeModel"/"bedrock:InvokeModelWithResponseStream") tanto sobre el ARN del propio inference profile como sobre el ARN del foundation model subyacente en la región de origen y en todas las regiones de destino que ese perfil pueda utilizar (puede reforzarse con la condición "bedrock:InferenceProfileArn" para mayor precisión). Por ello, hay que validar que los roles de IAM de los desarrolladores GenAI tengan permiso de invocación sobre "amazon.nova-pro-v1:0" en todas las regiones de la UE que el perfil "eu.amazon.nova-pro-v1:0" pueda usar como destino, no solo en eu-central-1.

Opción E: ampliar la SCP para permitir de forma genérica cualquier perfil con prefijo "eu.*" es una autorización excesivamente amplia. Permitiría el uso de cualquier modelo con inference profile geográfico de la UE (no solo Nova Pro), lo que contradice el requisito de la empresa de un control de acceso preciso; la solución correcta es autorizar específicamente el perfil "eu.amazon.nova-pro-v1:0".

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-support.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-pro.html>

A financial services company is developing a customer service AI assistant by using Amazon Bedrock. The AI assistant must not discuss investment advice with users. The AI assistant must block harmful content, mask personally identifiable information (PII), and maintain audit trails for compliance reporting. The AI assistant must apply content filtering to both user inputs and model responses based on content sensitivity. The company requires an Amazon Bedrock guardrail configuration that will effectively enforce policies with minimal false positives. The solution must provide multiple handling strategies for multiple types of sensitive content. Which solution will meet these requirements?

- A Configure a single guardrail and set content filters to high for all categories. Set up denied topics for investment advice and include sample phrases to block. Set up sensitive information filters that apply the block action for all PII entities. Apply the guardrail to all model inference calls.
- B Configure multiple guardrails by using tiered policies. Create one guardrail and set content filters to high. Configure the guardrail to block PII for public interactions. Configure a second guardrail and set content filters to medium. Configure the second guardrail to mask PII for internal use. Configure multiple topic-specific guardrails to block investment advice and set up contextual grounding checks.
- C Configure a guardrail and set content filters to medium for harmful content. Set up denied topics for investment advice and include clear definitions and sample phrases to block. Configure sensitive information filters to mask PII in responses and to block financial information in inputs. Enable both input and output evaluations that use custom blocked messages for audits.
- D Create a separate guardrail for each use case. Create one guardrail that applies a harmful content filter. Create a guardrail to apply topic filters for investment advice. Create a guardrail to apply sensitive information filters to block PII. Use AWS Step Functions to chain the guardrails together sequentially. Use conditional logic based on content classification.

Respuesta correcta: **C**

- A** Configure a single guardrail and set content filters to high for all categories. Set up denied topics for investment advice and include sample phrases to block. Set up sensitive information filters that apply the block action for all PII entities. Apply the guardrail to all model inference calls.
- B** Configure multiple guardrails by using tiered policies. Create one guardrail and set content filters to high. Configure the guardrail to block PII for public interactions. Configure a second guardrail and set content filters to medium. Configure the second guardrail to mask PII for internal use. Configure multiple topic-specific guardrails to block investment advice and set up contextual grounding checks.
- C** Configure a guardrail and set content filters to medium for harmful content. Set up denied topics for investment advice and include clear definitions and sample phrases to block. Configure sensitive information filters to mask PII in responses and to block financial information in inputs. Enable both input and output evaluations that use custom blocked messages for audits. ✓
- D** Create a separate guardrail for each use case. Create one guardrail that applies a harmful content filter. Create a guardrail to apply topic filters for investment advice. Create a guardrail to apply sensitive information filters to block PII. Use AWS Step Functions to chain the guardrails together sequentially. Use conditional logic based on content classification.

Explicación

Opción A: fijar los content filters al nivel "High" para todas las categorías representa la configuración más estricta posible: según la documentación de Amazon Bedrock Guardrails, con el nivel "High" se bloquea contenido clasificado con confianza HIGH, MEDIUM y LOW, dejando pasar únicamente el contenido con confianza NONE. Esto maximiza la tasa de falsos positivos, contradiciendo el requisito explícito del enunciado de minimizarlos. Además, bloquear (en vez de enmascarar) toda la PII no ofrece múltiples estrategias de tratamiento según el tipo de contenido.

Opción B: usar varias guardrails organizadas por niveles ("tiered") multiplica la complejidad operativa (crear, versionar, mantener y aplicar correctamente cada guardrail según el contexto de la interacción) cuando Amazon Bedrock Guardrails permite combinar filtros de contenido, temas denegados, filtros de información sensible y comprobaciones de fundamentación contextual (contextual grounding) dentro de una única guardrail. No aporta ninguna ventaja funcional frente a una sola guardrail bien configurada y añade puntos de fallo en la gestión.

Opción C (Correcta): configurar los content filters en nivel "Medium" para contenido dañino es la opción intermedia entre minimizar contenido dañino y minimizar falsos positivos: según la documentación oficial, con "Medium" se bloquea el contenido clasificado como dañino con confianza HIGH y MEDIUM, mientras que el contenido con confianza NONE o LOW se permite; esto reduce los falsos positivos frente al nivel "High" (que también bloquea confianza LOW) sin dejar de filtrar contenido claramente dañino. Los denied topics, al incluir un nombre, una definición precisa (hasta 200 caracteres) y hasta cinco frases de ejemplo, permiten a la guardrail detectar semánticamente conversaciones sobre asesoramiento de inversión con mayor precisión (AWS recomienda definiciones claras y sin ambigüedad para mejorar la exactitud de la detección). Los sensitive information filters de Amazon Bedrock Guardrails admiten configurar acciones distintas para PII: la API/CloudFormation expone tanto una acción general (Action: BLOCK/ANONYMIZE/NONE) como acciones específicas por dirección del tráfico (InputAction y OutputAction), lo que permite exactamente el patrón descrito en la opción: enmascarar (ANONYMIZE) la PII en las respuestas del modelo y bloquear (BLOCK) información financiera sensible en las entradas del usuario, aplicando distintas estrategias de manejo para distintos tipos y direcciones de contenido sensible dentro de la misma guardrail. Finalmente, habilitar la evaluación tanto de input como de output con mensajes de bloqueo personalizados (blocked messaging) proporciona la trazabilidad necesaria para los informes de cumplimiento y auditoría. Todo esto se logra con una única guardrail, cumpliendo el requisito de una solución que aplique políticas eficazmente con mínimos falsos positivos y múltiples estrategias de tratamiento.

Opción D: crear una guardrail separada por cada tipo de control (contenido dañino, temas denegados, información sensible) y orquestarlas secuencialmente con AWS Step Functions y lógica condicional introduce una complejidad de orquestación innecesaria. Amazon Bedrock Guardrails está diseñado para que todas estas políticas (content filters, denied topics, sensitive information filters, contextual grounding, word filters) se configuren y apliquen conjuntamente dentro de un único recurso de guardrail en cada llamada de inferencia, sin necesidad de servicios adicionales de orquestación.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters-overview.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-denied-topics.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-bedrock-guardrail-piientityconfig.html>

An ecommerce company is developing a generative AI (GenAI) solution that uses Amazon Bedrock with Anthropic Claude to recommend products to customers. Customers report that some of the recommended products are not available for sale on the website or are not relevant to the customer. Customers also report that the solutions takes a long time to generate some recommendations. The company investigates the issues and finds that most interactions between customers and the product recommendation solution are unique. The company confirms that the solutions recommends products that are not in the company's product catalog. The company must resolve these issues. Which solution will meet this requirement?

- A Increase grounding within Amazon Bedrock Guardrails. Enable Automated Reasoning checks. Set up provisioned throughput.
- B Use prompt engineering to restrict the model responses to relevant products. Use streaming techniques such as the `InvokeModelWithResponseStream` action to reduce perceived latency for the customers.
- C Create an Amazon Bedrock knowledge base. Implement Retrieval Augmented Generation (RAG). Set the `PerformanceConfigLatency` parameter to optimized.
- D Store product catalog data in Amazon OpenSearch Service. Validate the model's product recommendations against the product catalog. Use Amazon DynamoDB to implement response caching.

Respuesta correcta: **C**

- A Increase grounding within Amazon Bedrock Guardrails. Enable Automated Reasoning checks. Set up provisioned throughput.
- B Use prompt engineering to restrict the model responses to relevant products. Use streaming techniques such as the `InvokeModelWithResponseStream` action to reduce perceived latency for the customers.
- C Create an Amazon Bedrock knowledge base. Implement Retrieval Augmented Generation (RAG). Set the `PerformanceConfigLatency` parameter to optimized.**
- D Store product catalog data in Amazon OpenSearch Service. Validate the model's product recommendations against the product catalog. Use Amazon DynamoDB to implement response caching.

Explicación

Opción A: aumentar el "grounding" en Amazon Bedrock Guardrails y habilitar Automated Reasoning checks sirve para verificar afirmaciones lógicas o matemáticas frente a reglas formales definidas por el usuario, pero no ancla las recomendaciones en el catálogo real de productos, por lo que el modelo podría seguir sugiriendo artículos inexistentes; además, configurar Provisioned Throughput solo garantiza capacidad de inferencia reservada a un costo fijo, pero no ataca la causa raíz de la lentitud si el problema es la complejidad del prompt o la falta de recuperación eficiente de datos. Esta opción no resuelve ninguno de los dos problemas de forma directa.

Opción B: la ingeniería de prompts por sí sola no garantiza que el modelo se limite estrictamente al catálogo real, ya que sin una fuente de datos autorizada y verificable el modelo puede seguir alucinando productos que no existen a la venta; usar `InvokeModelWithResponseStream` mejora la latencia percibida por el usuario (el texto empieza a mostrarse antes), pero no reduce el tiempo real de generación ni corrige la relevancia o exactitud de las recomendaciones.

Opción C (Correcta): crear una base de conocimiento (Knowledge Base) de Amazon Bedrock e implementar Retrieval Augmented Generation (RAG) permite que el modelo recupere fragmentos reales del catálogo de productos (almacenados como embeddings en un índice vectorial) y genere las recomendaciones a partir de ese contexto recuperado, en lugar de basarse únicamente en lo aprendido durante el entrenamiento; esto ancla ("grounds") las respuestas en datos verídicos y evita que el modelo invente productos inexistentes o irrelevantes, sin necesidad de reentrenar el modelo. Adicionalmente, configurar el parámetro de rendimiento de latencia (`performanceConfig` con `latency` establecido en "optimized") activa la inferencia optimizada para latencia disponible para determinados modelos en Amazon Bedrock, entre ellos Anthropic Claude 3.5 Haiku, reduciendo el tiempo de respuesta de la inferencia sin sacrificar precisión y sin requerir configuración adicional. Esta combinación resuelve directamente ambos problemas reportados: recomendaciones fuera de catálogo/irrelevantes y tiempos de generación elevados.

Opción D: almacenar el catálogo en Amazon OpenSearch Service y validar las recomendaciones del modelo contra el catálogo después de generarlas añade una capa de validación posterior (post-procesamiento) que requiere más desarrollo y mantenimiento que anclar la generación directamente en los datos reales mediante RAG, y no evita que el modelo genere inicialmente sugerencias incorrectas (solo las filtra después). Además, el enunciado indica que la mayoría de las interacciones entre los clientes y la solución son únicas, lo que limita fuertemente la efectividad de una caché de respuestas en Amazon DynamoDB, ya que habría muy pocas coincidencias de caché y el problema de latencia persistiría en la mayoría de los casos.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-it-works.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_InvokeModel.html

A company is using AWS Lambda and REST APIs to build a reasoning agent to automate support workflows. The system must preserve memory across interactions, share the relevant agent state, and support event-driven invocation and synchronous invocation. The system must also enforce access control and session-based permissions. Which combination of steps provides the MOST scalable solution? (Choose two.)

Selecciona 2 respuestas

- A Use Amazon Bedrock AgentCore to manage memory and session-aware reasoning. Deploy the agent with built-in identity support, event handling, and observability.
- B Register the Lambda functions and the REST APIs as actions by using Amazon API Gateway and Amazon EventBridge. Enable Amazon Bedrock AgentCore to invoke the Lambda functions and the REST APIs without custom orchestration code.
- C Use Amazon Bedrock Agents for reasoning and conversation management. Use AWS Step Functions and Amazon SQS queues for orchestration. Store the agent state in Amazon DynamoDB to maintain memory between steps.
- D Deploy the reasoning logic as a container on Amazon ECS behind Amazon API Gateway. Use Amazon Aurora to store memory data and identity data.
- E Build a custom RAG pipeline by using Amazon Kendra and Amazon Bedrock. Use AWS Lambda to orchestrate tool invocations. Store the agent state in Amazon S3.

Respuesta correcta: A, B

- A** Use Amazon Bedrock AgentCore to manage memory and session-aware reasoning. Deploy the agent with built-in identity support, event handling, and observability. ✓
- B** Register the Lambda functions and the REST APIs as actions by using Amazon API Gateway and Amazon EventBridge. Enable Amazon Bedrock AgentCore to invoke the Lambda functions and the REST APIs without custom orchestration code. ✓
- C** Use Amazon Bedrock Agents for reasoning and conversation management. Use AWS Step Functions and Amazon SQS queues for orchestration. Store the agent state in Amazon DynamoDB to maintain memory between steps.
- D** Deploy the reasoning logic as a container on Amazon ECS behind Amazon API Gateway. Use Amazon Aurora to store memory data and identity data.
- E** Build a custom RAG pipeline by using Amazon Kendra and Amazon Bedrock. Use AWS Lambda to orchestrate tool invocations. Store the agent state in Amazon S3.

Explicación

Opción A (Correcta): Amazon Bedrock AgentCore es la plataforma agéntica gestionada de AWS diseñada exactamente para este escenario: incluye el servicio Memory (memoria a corto y largo plazo, compartible entre agentes y persistente entre sesiones), el servicio Identity (gestión de identidad, autenticación y autorización compatible con proveedores externos como Cognito, Okta o Entra ID) y el servicio Runtime, que ofrece aislamiento de sesión mediante microVMs dedicadas, soporte para invocación síncrona (API InvokeAgentRuntime) y para procesamiento asíncrono/en segundo plano (event-driven), además de Observability integrada. Esto evita construir manualmente el manejo de estado, permisos y eventos.

Opción B (Correcta): Amazon Bedrock AgentCore Gateway permite convertir APIs existentes (incluyendo REST APIs expuestas mediante Amazon API Gateway) y funciones AWS Lambda en herramientas compatibles con Model Context Protocol (MCP) con solo unas pocas líneas de código, sin necesidad de escribir código de orquestación personalizado. Gateway centraliza el descubrimiento de herramientas, la autenticación de entrada y salida, y la traducción de protocolos, mientras que Amazon EventBridge puede usarse para disparar la invocación basada en eventos de esas funciones y APIs. Esta combinación reduce semanas de desarrollo de integración a una configuración declarativa.

Opción C: usar Amazon Bedrock Agents junto con AWS Step Functions, colas Amazon SQS y Amazon DynamoDB para el estado es una arquitectura válida pero exige diseñar, desplegar y mantener manualmente toda la orquestación, la persistencia de memoria y el control de sesión que AgentCore ya ofrece de forma nativa y gestionada, por lo que no es la opción más escalable ni la que menos esfuerzo operativo requiere.

Opción D: desplegar la lógica de razonamiento como contenedor en Amazon ECS detrás de Amazon API Gateway, usando Amazon Aurora para memoria e identidad, obliga a gestionar infraestructura de cómputo, escalado, parches y una base de datos relacional propia para cubrir capacidades (memoria de sesión, identidad, observabilidad) que AgentCore proporciona como servicio administrado, además de no simplificar el control de acceso basado en sesión.

Opción E: construir un pipeline RAG personalizado con Amazon Kendra y Amazon Bedrock, orquestando las invocaciones de herramientas manualmente con AWS Lambda y almacenando el estado del agente en Amazon S3, implica desarrollar desde cero la gestión de memoria, el control de sesión y el cumplimiento de permisos, careciendo de las garantías de aislamiento, identidad y observabilidad que ofrece AgentCore, por lo que no es la solución más escalable.

Referencias:

<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html>

<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway.html>

<https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-sessions.html>

A financial services company is developing a Retrieval Augmented Generation (RAG) application to help investment analysts query complex financial relationships across multiple investment vehicles, market sectors, and regulatory environments. The dataset contains highly interconnected entities that have multi-hop relationships. The analysts must be able to examine the relationships holistically to provide accurate investment guidance. The application must deliver comprehensive answers that capture indirect relationships between financial entities. The application must produce responses in less than 3 seconds. Which solution will meet these requirements with the LEAST operational overhead?

- A Use Amazon Bedrock Knowledge Bases with Graph RAG and Amazon Neptune Analytics to store the financial data. Analyze the multi-hop relationships between entities and automatically identify related information across documents.
- B Use Amazon Bedrock Knowledge Bases and an Amazon OpenSearch Service vector store to implement custom relationship identification logic that uses AWS Lambda functions to query multiple vector embeddings in sequence.
- C Use an Amazon OpenSearch Serverless vector database with k-nearest neighbor (k-NN) searches. Implement manual relationship mapping in an application layer that runs in an Amazon EC2 Auto Scaling group.
- D Use Amazon DynamoDB to store financial data in a custom indexing system. Use an AWS Lambda function to query relevant records based on input questions. Use Amazon SageMaker AI to generate responses.

Respuesta correcta: A

- A** Use Amazon Bedrock Knowledge Bases with Graph RAG and Amazon Neptune Analytics to store the financial data. Analyze the multi-hop relationships between entities and automatically identify related information across documents. ✓
- B** Use Amazon Bedrock Knowledge Bases and an Amazon OpenSearch Service vector store to implement custom relationship identification logic that uses AWS Lambda functions to query multiple vector embeddings in sequence.
- C** Use an Amazon OpenSearch Serverless vector database with k-nearest neighbor (k-NN) searches. Implement manual relationship mapping in an application layer that runs in an Amazon EC2 Auto Scaling group.
- D** Use Amazon DynamoDB to store financial data in a custom indexing system. Use an AWS Lambda function to query relevant records based on input questions. Use Amazon SageMaker AI to generate responses.

Explicación

Opción A (Correcta): Amazon Bedrock Knowledge Bases ofrece GraphRAG como una función totalmente gestionada que combina modelado de grafos con IA generativa, usando Amazon Neptune Analytics como almacén subyacente. Según la documentación oficial de AWS, GraphRAG identifica y utiliza automáticamente las relaciones entre entidades y elementos estructurales (como títulos de sección) a través de múltiples documentos ingeridos en la base de conocimiento, sin necesidad de configurar lógica de consulta personalizada. Su funcionamiento consiste en: (1) realizar una búsqueda vectorial inicial para encontrar los nodos de grafo relevantes, (2) recuperar los nodos o fragmentos de chunk relacionados vinculados a esos resultados, y (3) expandir el contexto recorriendo el grafo para obtener respuestas más completas y contextualmente precisas, especialmente cuando la información requiere conectarse a través de múltiples pasos lógicos (relaciones multi-hop). Esto es exactamente lo que necesita la empresa de servicios financieros para analizar relaciones indirectas entre vehículos de inversión, sectores de mercado y entornos regulatorios. Al ser un servicio gestionado (Amazon Bedrock Knowledge Bases + Neptune Analytics), no requiere que el equipo administre infraestructura de cómputo, escale servidores ni implemente lógica de recorrido de grafos manualmente, lo que minimiza la sobrecarga operativa en comparación con las demás opciones, cumpliendo también con el requisito de baja latencia.

Opción B: usar Amazon Bedrock Knowledge Bases con un almacén vectorial en Amazon OpenSearch Service y funciones AWS Lambda para consultar múltiples embeddings vectoriales de forma secuencial obligaría al equipo a desarrollar y mantener lógica personalizada de identificación de relaciones. Esta solución no modela relaciones estructuradas entre entidades como un grafo nativo, por lo que capturar relaciones multi-hop requeriría múltiples llamadas secuenciales, incrementando la latencia y la complejidad de desarrollo, además de la carga operativa de mantener las funciones Lambda.

Opción C: una base de datos vectorial en Amazon OpenSearch Serverless con búsquedas k-nearest neighbor (k-NN) captura similitud semántica entre embeddings, pero no representa relaciones estructuradas ni permite recorridos multi-hop de forma nativa. Implementar el mapeo de relaciones manualmente en una capa de aplicación ejecutada en un grupo de Auto Scaling de Amazon EC2 exige desarrollo, pruebas y mantenimiento continuos de la infraestructura y de la lógica de negocio, lo que representa una sobrecarga operativa considerablemente mayor.

Opción D: almacenar los datos financieros en Amazon DynamoDB con un sistema de indexación personalizado, usar una función AWS Lambda para consultar los registros relevantes según las preguntas de entrada, y usar Amazon SageMaker AI para generar las respuestas no ofrece capacidades nativas de recuperación semántica (RAG) ni de modelado de relaciones entre entidades. Esta arquitectura requeriría construir desde cero toda la lógica de indexación, búsqueda y razonamiento sobre relaciones, lo cual implica el mayor esfuerzo de desarrollo y mantenimiento entre todas las opciones.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-build-graphs.html>, <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-build-graphs-build.html>, <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-neptune.html>

A healthcare company uses Amazon Bedrock to deploy an application that generates summaries of clinical documents. The application experiences inconsistent response quality with occasional factual hallucinations. Monthly costs exceed the company's projections by 40%. A GenAI developer must implement a near real-time monitoring solution to detect hallucinations, identify abnormal token consumption, and provide early warnings of cost anomalies. The solution must require minimal custom development work and maintenance overhead. Which solution will meet these requirements?

- A Configure Amazon CloudWatch alarms to monitor InputTokenCount and OutputTokenCount metrics to detect anomalies. Store model invocation logs in an Amazon S3 bucket. Use AWS Glue and Amazon Athena to identify potential hallucinations.
- B Run Amazon Bedrock evaluation jobs that use LLM-based judgments to detect hallucinations. Configure Amazon CloudWatch to track token usage. Create an AWS Lambda function to process CloudWatch metrics. Configure the Lambda function to send usage pattern notifications.
- C Configure Amazon Bedrock to store model invocation logs in an Amazon S3 bucket. Enable text output logging. Configure Amazon Bedrock guardrails to run contextual grounding checks to detect hallucinations. Create Amazon CloudWatch anomaly detection alarms for token usage metrics.
- D Use AWS CloudTrail to log all Amazon Bedrock API calls. Create a custom dashboard in Amazon QuickSight to visualize token usage patterns. Use Amazon SageMaker Model Monitor to detect quality drift in generated summaries.

Respuesta correcta: C

- A Configure Amazon CloudWatch alarms to monitor InputTokenCount and OutputTokenCount metrics to detect anomalies. Store model invocation logs in an Amazon S3 bucket. Use AWS Glue and Amazon Athena to identify potential hallucinations.
- B Run Amazon Bedrock evaluation jobs that use LLM-based judgments to detect hallucinations. Configure Amazon CloudWatch to track token usage. Create an AWS Lambda function to process CloudWatch metrics. Configure the Lambda function to send usage pattern notifications.
- C Configure Amazon Bedrock to store model invocation logs in an Amazon S3 bucket. Enable text output logging. Configure Amazon Bedrock guardrails to run contextual grounding checks to detect hallucinations. Create Amazon CloudWatch anomaly detection alarms for token usage metrics.**
- D Use AWS CloudTrail to log all Amazon Bedrock API calls. Create a custom dashboard in Amazon QuickSight to visualize token usage patterns. Use Amazon SageMaker Model Monitor to detect quality drift in generated summaries.

Explicación

Opción A: configurar alarmas de Amazon CloudWatch sobre las métricas InputTokenCount y OutputTokenCount solo permite detectar variaciones de volumen de tokens mediante umbrales estáticos definidos manualmente, pero no dice nada sobre la fidelidad factual de las respuestas. Además, analizar los logs de invocación almacenados en S3 con AWS Glue y Amazon Athena para "inferir" alucinaciones exige construir y mantener un pipeline analítico propio (crawlers, ETL, consultas SQL ad hoc), lo que contradice el requisito de mínimo desarrollo y mantenimiento.

Opción B: los trabajos de evaluación de Amazon Bedrock (Model Evaluation) que usan un modelo como juez (LLM-as-a-judge) sí pueden detectar hallazgos de calidad, incluyendo alucinaciones, pero se ejecutan como trabajos por lotes sobre un conjunto de prompts/respuestas, no de forma continua ni casi en tiempo real sobre cada invocación en producción. Añadir una función AWS Lambda personalizada para procesar métricas de CloudWatch y generar notificaciones de patrones de uso también introduce desarrollo y mantenimiento adicionales que la solución debería evitar.

Opción C (Correcta): Amazon Bedrock puede registrar los logs de invocación de modelos (incluyendo el texto de entrada y salida) en un bucket de Amazon S3 simplemente habilitando el registro de invocaciones del modelo, sin necesidad de código adicional. Las comprobaciones de fundamentación contextual (contextual grounding checks) de Amazon Bedrock Guardrails evalúan, en el momento de cada invocación (a través de las API Invoke, Converse o ApplyGuardrail), si la respuesta del modelo está "grounded" (fundamentada factualmente) en la fuente de referencia proporcionada y si es "relevant" respecto a la consulta del usuario; a cada respuesta se le asignan puntuaciones de confianza de grounding y relevance, y se pueden configurar umbrales para bloquear automáticamente las respuestas que se consideren alucinadas, todo ello de forma prácticamente inmediata y sin necesidad de desarrollo personalizado. Por su parte, las alarmas de detección de anomalías de Amazon CloudWatch (CloudWatch Anomaly Detection) aplicadas a las métricas de consumo de tokens (por ejemplo InputTokenCount/ OutputTokenCount o los agregados de tokens de Bedrock) utilizan modelos estadísticos para aprender el patrón normal de uso y generar alertas cuando el comportamiento se desvía de ese patrón, evitando tener que definir y mantener umbrales estáticos manualmente. La combinación de ambos mecanismos nativos de Bedrock y CloudWatch cumple el objetivo de monitorización casi en tiempo real de alucinaciones y de anomalías de consumo/coste con mínimo esfuerzo de desarrollo y mantenimiento.

Opción D: AWS CloudTrail registra las llamadas a la API de Bedrock (quién invocó qué operación y cuándo), pero no captura el contenido de las respuestas generadas ni puede usarse para detectar alucinaciones. Un dashboard personalizado en Amazon QuickSight para visualizar patrones de uso de tokens requiere trabajo de desarrollo y mantenimiento continuo (modelado de datos, actualización de visualizaciones). Además, Amazon SageMaker Model Monitor está diseñado para detectar drift de datos/calidad en modelos desplegados en endpoints de SageMaker, no para evaluar la fidelidad factual de resúmenes generados por modelos de Amazon Bedrock, por lo que no es la herramienta adecuada para este caso de uso.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>

<https://docs.aws.amazon.com/prescriptive-guidance/latest/gen-ai-inference-architecture-and-best-practices-on-aws/amazon-bedrock.html>

A company is building a generative AI (GenAI) application that produces content based on a variety of internal and external data sources. The company wants to ensure that the generated output is fully traceable. The application must support data source registration and enable metadata tagging to attribute content to its original source. The application must also maintain audit logs of data access and usage throughout the pipeline. Which solution will meet these requirements?

- A Use AWS Lake Formation to catalog data sources and control access. Apply metadata tags directly in Amazon S3. Use AWS CloudTrail to monitor API activity.
- B Use AWS Glue Data Catalog to register and tag data sources. Use Amazon CloudWatch Logs to monitor access patterns and application behavior.
- C Store data in Amazon S3 and use object tagging for attribution. Use AWS Glue Data Catalog to manage schema information. Use AWS CloudTrail to log access to S3 buckets.
- D Use AWS Glue Data Catalog to register all data sources. Apply metadata tags to attribute data sources. Use AWS CloudTrail to log access and activity across services.

Respuesta correcta: **D**

- A Use AWS Lake Formation to catalog data sources and control access. Apply metadata tags directly in Amazon S3. Use AWS CloudTrail to monitor API activity.
- B Use AWS Glue Data Catalog to register and tag data sources. Use Amazon CloudWatch Logs to monitor access patterns and application behavior.
- C Store data in Amazon S3 and use object tagging for attribution. Use AWS Glue Data Catalog to manage schema information. Use AWS CloudTrail to log access to S3 buckets.
- D Use AWS Glue Data Catalog to register all data sources. Apply metadata tags to attribute data sources. Use AWS CloudTrail to log access and activity across services.**

Explicación

Opción A: AWS Lake Formation se apoya internamente en el AWS Glue Data Catalog para gestionar los metadatos y aplicar control de acceso granular (incluidos LF-Tags), pero la opción describe aplicar las etiquetas directamente sobre los objetos de Amazon S3 en lugar de sobre el catálogo centralizado. Esto rompe la atribución uniforme, ya que las etiquetas de S3 son propiedades del objeto individual y no quedan integradas en el repositorio de metadatos unificado que usa el resto de servicios analíticos (Athena, EMR, Redshift, etc.), dificultando trazar el origen de cada dato de forma consistente en toda la canalización.

Opción B: Amazon CloudWatch Logs sirve para monitorizar logs de aplicación y métricas operativas, pero no constituye un registro de auditoría de llamadas a la API con el nivel de detalle (identidad del llamador, hora del evento, IP de origen, recurso afectado, parámetros de la solicitud) que exige un requisito de trazabilidad completa. Ese tipo de auditoría de actividad y acceso es precisamente la función de AWS CloudTrail, no de CloudWatch Logs.

Opción C: aplicar etiquetas a nivel de objeto de S3 para la atribución es, de nuevo, menos centralizado que registrar y etiquetar las fuentes directamente en el AWS Glue Data Catalog (que expone una API TagResource para asociar etiquetas a bases de datos, tablas y otros recursos del catálogo). Además, limitar CloudTrail a "acceso a buckets de S3" es más estrecho que loguear la actividad y el acceso across servicios, que es el requisito planteado (la solución debe cubrir toda la canalización, no solo S3).

Opción D (Correcta): el AWS Glue Data Catalog es el repositorio de metadatos unificado de AWS para registrar y catalogar múltiples fuentes de datos (se integra de forma nativa con Amazon Athena, Amazon EMR, Amazon Redshift, RDS y aplicaciones compatibles con Apache Hive Metastore), y admite añadir etiquetas (tags) a sus recursos mediante la operación TagResource, lo que permite atribuir cada fuente de datos a su origen de forma centralizada. Combinado con AWS CloudTrail, que registra de forma nativa la actividad de la API (identidad del usuario, hora del evento, origen del evento, IP de origen, parámetros de la solicitud) a través de los distintos servicios de AWS involucrados en la canalización, se obtiene un registro de auditoría end-to-end y una trazabilidad completa del contenido generado, con un esfuerzo de integración mínimo.

Referencias:

<https://docs.aws.amazon.com/athena/latest/ug/data-sources-glue.html>

<https://docs.aws.amazon.com/glue/latest/dg/aws-glue-api-tags.html>

<https://docs.aws.amazon.com/whitepapers/latest/build-secure-enterprise-ml-platform/audit-trail-management.html>

A financial services company needs to build a document analysis system that uses Amazon Bedrock to process quarterly reports. The system must analyze financial data, perform sentiment analysis, and validate compliance across batches of reports. Each batch contains 5 reports. Each report requires multiple foundation model (FM) calls. The solution must finish the analysis within 10 seconds for each batch. Current sequential processing takes 45 seconds for each batch. Which solution will meet these requirements?

- A Use AWS Lambda functions with provisioned concurrency to process each analysis type sequentially. Configure the Lambda function timeouts to 10 seconds. Configure automatic retries with exponential backoff.
- B Use AWS Step Functions with a Parallel state to invoke separate AWS Lambda functions for each analysis type simultaneously. Configure Amazon Bedrock client timeouts. Use Amazon CloudWatch metrics to track execution time and model inference latency.
- C Create an Amazon SQS queue to buffer analysis requests. Deploy multiple AWS Lambda functions with reserved concurrency. Configure each Lambda function to process different aspects of each report sequentially and then combine the results.
- D Deploy an Amazon ECS cluster that runs containers that process each report sequentially. Use a load balancer to distribute batch workloads. Configure an auto-scaling policy based on CPU utilization to handle demand fluctuations.

Respuesta correcta: **B**

- A Use AWS Lambda functions with provisioned concurrency to process each analysis type sequentially. Configure the Lambda function timeouts to 10 seconds. Configure automatic retries with exponential backoff.
- B Use AWS Step Functions with a Parallel state to invoke separate AWS Lambda functions for each analysis type simultaneously. Configure Amazon Bedrock client timeouts. Use Amazon CloudWatch metrics to track execution time and model inference latency.** ✓
- C Create an Amazon SQS queue to buffer analysis requests. Deploy multiple AWS Lambda functions with reserved concurrency. Configure each Lambda function to process different aspects of each report sequentially and then combine the results.
- D Deploy an Amazon ECS cluster that runs containers that process each report sequentially. Use a load balancer to distribute batch workloads. Configure an auto-scaling policy based on CPU utilization to handle demand fluctuations.

Explicación

Opción A: la concurrencia aprovisionada en AWS Lambda elimina la latencia de arranque en frío (cold start), pero la propuesta indica que cada tipo de análisis se procesa "sequentially" (de forma secuencial) dentro de la función. Según la documentación de AWS sobre Lambda y Step Functions, el procesamiento secuencial mantiene la suma de todas las latencias de las llamadas al modelo, por lo que no resuelve el cuello de botella real (45 segundos) y no permite bajar a los 10 segundos requeridos, independientemente de cómo se gestione el arranque en frío.

Opción B (Correcta): AWS Step Functions ofrece de forma nativa un estado Parallel diseñado específicamente para "ejecutar múltiples ramas del flujo de trabajo simultáneamente" ("executes multiple branches of your workflow simultaneously"), tal y como confirma la documentación oficial de AWS Lambda sobre orquestación con Step Functions. En este escenario, cada tipo de análisis (datos financieros, sentimiento, cumplimiento) se puede implementar como una rama independiente dentro del estado Parallel, cada una invocando su propia función Lambda que a su vez llama a Amazon Bedrock. Al ejecutarse todas las ramas en paralelo en lugar de en serie, el tiempo total del lote deja de ser la suma de las latencias individuales y pasa a ser aproximadamente el tiempo de la rama más lenta, lo que permite reducir el procesamiento de 45 segundos a menos de 10 segundos por lote. Además, es posible configurar timeouts en el cliente de Amazon Bedrock (por ejemplo mediante el parámetro `read_timeout` de `botocore.config.Config` al crear el cliente `bedrock-runtime`) para controlar cuánto tiempo se espera la respuesta de cada llamada al modelo, y usar métricas de Amazon CloudWatch para monitorizar la duración de ejecución de cada Lambda y la latencia de inferencia de cada llamada a Bedrock, detectando cuellos de botella y validando que se cumple el SLA de 10 segundos.

Opción C: usar Amazon SQS para almacenar en búfer las solicitudes junto con Lambda de concurrencia reservada mejora la resiliencia y evita sobrecargar el servicio, pero la propia descripción indica que cada Lambda "procesa diferentes aspectos de cada informe de forma secuencial y luego combina los resultados" ("process different aspects of each report sequentially and then combine the results"). Esto significa que no existe paralelismo real entre los distintos tipos de análisis de un mismo informe, por lo que la latencia acumulada seguiría siendo demasiado alta para cumplir el límite de 10 segundos.

Opción D: desplegar un clúster de Amazon ECS con balanceador de carga y auto-scaling basado en CPU permite escalar horizontalmente el número de informes procesados en paralelo, pero la propia opción especifica que los contenedores "procesan cada informe de forma secuencial" ("run containers that process each report sequentially"). Es decir, dentro de cada informe las distintas llamadas al modelo fundacional siguen ejecutándose una tras otra, por lo que no se elimina el cuello de procesamiento en serie que impide bajar de 45 a 10 segundos por lote, además de introducir mayor complejidad operativa que Step Functions para este caso de uso.

Referencias: <https://docs.aws.amazon.com/lambda/latest/dg/with-step-functions.html> ; <https://docs.aws.amazon.com/step-functions/latest/dg/state-parallel.html> ; <https://docs.aws.amazon.com/nova/latest/nova2-userguide/troubleshooting.html>

A company is using Amazon Bedrock to build a customer-facing AI assistant to handle sensitive customer inquiries. The company must use defense-in-depth safety controls to block sophisticated prompt injection attacks. The company must keep audit logs of all safety interventions. The AI assistant must have cross-Region failover capabilities. Which solution will meet these requirements?

- A Configure Amazon Bedrock guardrails to use content filters to protect against prompt injection attacks. Set the content filters to high. Use a guardrail profile to implement cross-Region guardrail inference. Use Amazon CloudWatch Logs with custom metrics to capture detailed guardrail intervention events.
- B Configure Amazon Bedrock guardrails to use content filters to protect against prompt injection attacks. Set the content filters to high. Use AWS WAF to block suspicious inputs. Use AWS CloudTrail to log API calls for audits.
- C Deploy Amazon Comprehend custom classification to detect prompt injection attacks. Use Amazon API Gateway to validate requests. Use Amazon CloudWatch Logs with custom metrics to capture detailed intervention events.
- D Configure Amazon Bedrock guardrails to use custom content filters to protect against harmful content. Set the content filters to high. Use word filters to protect against known attack patterns. Configure cross-Region guardrail replication to provide failover capabilities. Store logs in AWS CloudTrail for compliance auditing.

Respuesta correcta: A

- A** Configure Amazon Bedrock guardrails to use content filters to protect against prompt injection attacks. Set the content filters to high. Use a guardrail profile to implement cross-Region guardrail inference. Use Amazon CloudWatch Logs with custom metrics to capture detailed guardrail intervention events. ✓
- B** Configure Amazon Bedrock guardrails to use content filters to protect against prompt injection attacks. Set the content filters to high. Use AWS WAF to block suspicious inputs. Use AWS CloudTrail to log API calls for audits.
- C** Deploy Amazon Comprehend custom classification to detect prompt injection attacks. Use Amazon API Gateway to validate requests. Use Amazon CloudWatch Logs with custom metrics to capture detailed intervention events.
- D** Configure Amazon Bedrock guardrails to use custom content filters to protect against harmful content. Set the content filters to high. Use word filters to protect against known attack patterns. Configure cross-Region guardrail replication to provide failover capabilities. Store logs in AWS CloudTrail for compliance auditing.

Explicación

Opción A (Correcta): los content filters de Amazon Bedrock Guardrails incluyen una categoría dedicada a la detección de "prompt attacks" (jailbreaks, prompt injection y prompt leakage) que evalúa semánticamente las entradas del usuario y las salidas del modelo; configurados en intensidad alta ("high"), maximizan el bloqueo de intentos de manipulación sofisticados, aportando así defensa en profundidad. El guardrail profile es el mecanismo específico de Bedrock Guardrails para habilitar la inferencia cross-Region: al asociarlo al guardrail, las solicitudes de evaluación de la política se enrutan automáticamente entre Regiones de AWS dentro de la misma geografía, lo que aporta resiliencia y continuidad si una Región no está disponible, sin coste adicional. Para la auditoría, Amazon Bedrock Guardrails publica métricas en el namespace de CloudWatch (por ejemplo, `InvocationsIntervened`), que se pueden desglosar con dimensiones como `Operation` o `GuardrailContentSource`; con métricas personalizadas y alarmas sobre estos datos se obtiene un registro detallado de cada intervención de seguridad, cumpliendo el requisito de logs de auditoría.

Opción B: AWS WAF opera a nivel de red y de aplicación web (filtrando por reglas HTTP, patrones de tráfico, IP, etc.) y no está diseñado para analizar semánticamente el contenido de un prompt dirigido a un modelo de lenguaje, por lo que no detecta ataques de prompt injection sofisticados de la misma forma que el filtro de contenido de Guardrails. Además, AWS CloudTrail registra las llamadas a la API (quién invocó qué operación y cuándo), pero no captura el detalle semántico de qué intervención de seguridad se aplicó dentro de una invocación de Guardrails, por lo que no cumple igual de bien el requisito de auditoría detallada de intervenciones.

Opción C: Amazon Comprehend con clasificación personalizada obligaría a entrenar, etiquetar datos y mantener un modelo de clasificación propio para detectar ataques de prompt, en lugar de aprovechar la funcionalidad de detección de "prompt attacks" ya integrada y gestionada por Amazon Bedrock Guardrails; además, ni Comprehend ni API Gateway ofrecen por sí solos la capacidad de failover cross-Region de guardrails que exige el enunciado.

Opción D: no existe en Amazon Bedrock Guardrails una funcionalidad llamada "replicación cross-Region de guardrails"; el mecanismo real y documentado es el guardrail profile para inferencia cross-Region descrito en la opción A. Además, basar la defensa contra ataques de prompt únicamente en word filters (listas de palabras con coincidencia exacta) es mucho menos robusto frente a ataques sofisticados que el filtro de contenido dedicado a "prompt attacks", que analiza el significado y la intención del texto, no solo palabras clave. Por último, AWS CloudTrail no ofrece el nivel de detalle sobre intervenciones de guardrails que sí proporcionan las métricas de CloudWatch.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-prompt-attack.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-cross-region.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-guardrails-cw-metrics.html>

A company is designing a canary deployment strategy for a payment processing API. The system must support automated gradual traffic shifting between multiple Amazon Bedrock models based on real-time inference metrics, historical traffic patterns, and service health. The solution must be able to gradually increase traffic to new model versions. The system must increase traffic if metrics remain healthy and decrease traffic if the performance degrades below acceptable thresholds. The company needs to comprehensively monitor inference latency and error rates during the deployment phase. The company must also be able to halt deployments and revert to a previous model version without any manual intervention. Which solution will meet these requirements?

- A Use Amazon Bedrock with provisioned throughput to host the versions of the model. Configure an Amazon EventBridge rule to invoke an AWS Step Functions workflow when a new model version is released. Configure the workflow to shift traffic in stages, wait for a specified time period, and invoke an AWS Lambda function to check Amazon CloudWatch performance metrics. Configure the workflow to increase traffic if the metrics meet thresholds and to trigger a traffic rollback if performance metrics fall below thresholds.
- B Use AWS Lambda functions to invoke various Amazon Bedrock model versions. Use an Amazon API Gateway HTTP API with stage variables and weighted routing to shift traffic gradually to new model versions. Use Amazon CloudWatch to monitor performance metrics. Use external logic to adjust traffic between model versions and to roll back if performance falls below thresholds.
- C Use Amazon SageMaker AI endpoint variants to represent multiple Amazon Bedrock model versions. Use variant weights to shift traffic. Use Amazon CloudWatch to monitor performance metrics. Use SageMaker Model Monitor to trigger AWS Lambda functions to roll back a model deployment if performance drops below a specified threshold. Configure an Amazon EventBridge rule to roll back model deployments if an anomaly is detected.
- D Use Amazon OpenSearch Service to track inference logs. Configure OpenSearch Service to invoke an AWS Systems Manager Automation runbook to update Amazon Bedrock model endpoints to shift traffic based on the inference logs.

Respuesta correcta: A

- A** Use Amazon Bedrock with provisioned throughput to host the versions of the model. Configure an Amazon EventBridge rule to invoke an AWS Step Functions workflow when a new model version is released. Configure the workflow to shift traffic in stages, wait for a specified time period, and invoke an AWS Lambda function to check Amazon CloudWatch performance metrics. Configure the workflow to increase traffic if the metrics meet thresholds and to trigger a traffic rollback if performance metrics fall below thresholds. ✓
- B** Use AWS Lambda functions to invoke various Amazon Bedrock model versions. Use an Amazon API Gateway HTTP API with stage variables and weighted routing to shift traffic gradually to new model versions. Use Amazon CloudWatch to monitor performance metrics. Use external logic to adjust traffic between model versions and to roll back if performance falls below thresholds.
- C** Use Amazon SageMaker AI endpoint variants to represent multiple Amazon Bedrock model versions. Use variant weights to shift traffic. Use Amazon CloudWatch to monitor performance metrics. Use SageMaker Model Monitor to trigger AWS Lambda functions to roll back a model deployment if performance drops below a specified threshold. Configure an Amazon EventBridge rule to roll back model deployments if an anomaly is detected.
- D** Use Amazon OpenSearch Service to track inference logs. Configure OpenSearch Service to invoke an AWS Systems Manager Automation runbook to update Amazon Bedrock model endpoints to shift traffic based on the inference logs.

Explicación

Opción A (Correcta): Amazon Bedrock ofrece Provisioned Throughput, un modo de aprovisionamiento que reserva capacidad de inferencia dedicada (Model Units) para un modelo base o personalizado, lo que permite garantizar capacidad estable para las distintas versiones del modelo durante la transición del despliegue canario. Como Bedrock no ofrece de forma nativa un mecanismo de enrutamiento ponderado entre versiones de modelo (a diferencia de, por ejemplo, los alias con routing configuration de Step Functions o los endpoint variants de SageMaker), hay que construir la orquestación con AWS Step Functions: una regla de Amazon EventBridge dispara el flujo de trabajo cuando se publica una nueva versión del modelo, el flujo desplaza el tráfico en fases, espera un intervalo definido entre cada fase e invoca una función AWS Lambda que consulta las métricas de Amazon CloudWatch (latencia de inferencia y tasa de error). Con esa información, Step Functions decide de forma automática si aumenta el porcentaje de tráfico hacia la nueva versión (si las métricas están dentro de umbral) o si revierte el tráfico a la versión anterior (si las métricas se degradan), sin intervención manual. Este patrón de despliegue gradual con verificación de alarmas entre fases y rollback automático es precisamente el que Step Functions soporta de forma nativa para el "gradual deployment" de versiones (aplicable también a la propia máquina de estados), y es el enfoque recomendado para construir canary deployments personalizados sobre servicios como Bedrock que no exponen un mecanismo de traffic-shifting propio.

Opción B: aunque Amazon API Gateway sí soporta despliegues canary nativos mediante "canary release deployment" con variables de stage y distribución ponderada del tráfico, aquí la propuesta delega la decisión de aumentar/disminuir tráfico y el rollback a "lógica externa" no especificada. Eso obliga a diseñar, implementar y mantener manualmente el motor de decisión (evaluación de métricas, temporización de fases, rollback), en lugar de apoyarse en un orquestador con estado, reintentos y manejo de errores integrado como Step Functions, que es el servicio pensado para este tipo de flujos de aprobación/decisión automatizados.

Opción C: los "endpoint variants" y Amazon SageMaker AI Model Monitor son funcionalidades de los endpoints de inferencia de SageMaker (donde un mismo endpoint puede repartir tráfico entre varias "production variants" de modelos alojados en SageMaker). Los modelos de Amazon Bedrock no se despliegan como variantes de un endpoint de SageMaker: se invocan a través de la API de Bedrock (bajo demanda o mediante Provisioned Throughput/Custom Model Deployment), por lo que no existe una forma soportada de "representar" versiones de modelos de Bedrock como variantes de un endpoint de SageMaker. Esta opción mezcla dos servicios de forma técnicamente inconsistente.

Opción D: Amazon OpenSearch Service está diseñado para indexación, búsqueda y análisis de logs, no para orquestar despliegues ni desencadenar automatizaciones de cambio de tráfico. Además, Amazon Bedrock no tiene "endpoints" configurables mediante runbooks de AWS Systems Manager Automation para desplazar tráfico entre versiones de modelo; esta

combinación no corresponde a ningún flujo soportado por estos servicios para gestionar un despliegue canario de modelos de Bedrock.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>

<https://docs.aws.amazon.com/step-functions/latest/dg/version-rolling-deployment.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/manage-customized-models.html>

A financial services company uses an AI application to process financial documents by using Amazon Bedrock. During business hours, the application handles approximately 10,000 requests each hour, which requires consistent throughput. The company uses the `CreateProvisionedModelThroughput` API to purchase provisioned throughput. Amazon CloudWatch metrics show that the provisioned capacity is unused while on-demand requests are being throttled. The company finds the following code in the application: `python response = bedrock_runtime.invoke_model(modelId="anthropic.claude-v2", body=json.dumps(payload))` The company needs the application to use the provisioned throughput and to resolve the throttling issues. Which solution will meet these requirements?

- A Increase the number of model units (MUs) in the provisioned throughput configuration.
- B Replace the model ID parameter with the ARN of the provisioned model that the `CreateProvisionedModelThroughput` API returns.
- C Add exponential backoff retry logic to handle throttling exceptions during peak hours.
- D Modify the application to use the `InvokeModelWithResponseStream` API instead of the `InvokeModel` API.

Respuesta correcta: **B**

- A Increase the number of model units (MUs) in the provisioned throughput configuration.
- B Replace the model ID parameter with the ARN of the provisioned model that the CreateProvisionedModelThroughput API returns.** ✓
- C Add exponential backoff retry logic to handle throttling exceptions during peak hours.
- D Modify the application to use the InvokeModelWithResponseStream API instead of the InvokeModel API.

Explicación

Opción A: aumentar el número de model units (MUs) en la configuración de Provisioned Throughput incrementa la capacidad reservada disponible, pero no resuelve el problema descrito. La causa raíz es que el código de la aplicación sigue invocando el modelo mediante ``modelId="anthropic.claude-v2"`` (el identificador del modelo base on-demand), por lo que las solicitudes nunca llegan a la capacidad aprovisionada, sin importar cuántas MUs se compren; esta seguiría apareciendo como no utilizada en CloudWatch.

Opción B (Correcta): según la documentación oficial de Amazon Bedrock, para ejecutar inferencia usando una Provisioned Throughput hay que enviar una solicitud `InvokeModel`, `InvokeModelWithResponseStream`, `Converse` o `ConverseStream` especificando el ARN del modelo aprovisionado (provisioned model ARN) como parámetro ``modelId``, en lugar del ID del modelo base. Ese ARN es precisamente el que devuelve la API `CreateProvisionedModelThroughput` (campo ``provisionedModelArn``) al crear la capacidad reservada. Como el código actual usa el ID del modelo base, todo el tráfico se enruta a la capacidad on-demand compartida, lo que explica exactamente los síntomas observados: la capacidad aprovisionada permanece sin uso en las métricas de CloudWatch mientras las solicitudes on-demand sufren throttling en las horas pico. Sustituir el ``modelId`` por el ARN del modelo aprovisionado dirige el tráfico a la capacidad dedicada y resuelve el throttling sin necesidad de cambios adicionales.

Opción C: añadir lógica de reintento con backoff exponencial puede mitigar temporalmente los errores de throttling (por ejemplo, absorbiendo picos puntuales), pero no soluciona la causa raíz: la capacidad aprovisionada que la empresa ya paga sigue sin utilizarse porque las solicitudes nunca se dirigen a ella. Es una solución paliativa, no correctiva.

Opción D: cambiar de la API `InvokeModel` a `InvokeModelWithResponseStream` solo afecta a la forma en que se recibe la respuesta (en streaming en lugar de en un único bloque), pero no cambia el recurso de destino de la invocación. De hecho, la propia documentación de AWS indica que `InvokeModelWithResponseStream` también requiere especificar el ARN del modelo aprovisionado como ``modelId`` para usar Provisioned Throughput; sin ese cambio de ``modelId``, el problema de enrutamiento a la capacidad aprovisionada persistiría igualmente.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-thru-use.html> ; https://docs.aws.amazon.com/bedrock/latest/APIReference/API_CreateProvisionedModelThroughput.html

A company is building an AI advisory application by using Amazon Bedrock. The application will provide recommendations to customers. The company needs the application to explain its reasoning process and cite specific sources for data. The application must retrieve information from company data sources and show step-by-step reasoning for recommendations. The application must also link data claims to source documents and maintain response latency under 3 seconds. Which solution will meet these requirements with the LEAST operational overhead?

- A Use Amazon Bedrock Knowledge Bases with source attribution enabled. Use the Anthropic Claude Messages API with RAG to set high-relevance thresholds for source documents. Store reasoning and citations in Amazon S3 for auditing purposes.
- B Use Amazon Bedrock with Anthropic Claude models and extended thinking. Configure a 4,000-token thinking budget. Store reasoning traces and citations in Amazon DynamoDB for auditing purposes.
- C Configure Amazon SageMaker AI with a custom Anthropic Claude model. Use the model's reasoning parameter and AWS Lambda to process responses. Add source citations from a separate Amazon RDS database.
- D Use Amazon Bedrock with Anthropic Claude models and chain-of-thought reasoning. Configure custom retrieval tracking with the Amazon Bedrock Knowledge Bases API. Use Amazon CloudWatch to monitor response latency metrics.

Respuesta correcta: A

- A** Use Amazon Bedrock Knowledge Bases with source attribution enabled. Use the Anthropic Claude Messages API with RAG to set high-relevance thresholds for source documents. Store reasoning and citations in Amazon S3 for auditing purposes. ✓
- B** Use Amazon Bedrock with Anthropic Claude models and extended thinking. Configure a 4,000-token thinking budget. Store reasoning traces and citations in Amazon DynamoDB for auditing purposes.
- C** Configure Amazon SageMaker AI with a custom Anthropic Claude model. Use the model's reasoning parameter and AWS Lambda to process responses. Add source citations from a separate Amazon RDS database.
- D** Use Amazon Bedrock with Anthropic Claude models and chain-of-thought reasoning. Configure custom retrieval tracking with the Amazon Bedrock Knowledge Bases API. Use Amazon CloudWatch to monitor response latency metrics.

Explicación

Opción A (Correcta): Amazon Bedrock Knowledge Bases incorpora de forma nativa la atribución de fuentes (citations) a través de la operación RetrieveAndGenerate: cada respuesta generada incluye referencias (citas/footnotes) que enlazan las afirmaciones del texto con los fragmentos (chunks) concretos de los documentos de origen de los que proceden, sin necesidad de construir un mecanismo de citación propio. Además, es posible ajustar la configuración de recuperación (número de resultados, tipo de búsqueda, reranking) para priorizar los fragmentos más relevantes como fuente de las respuestas. Usar el modelo Anthropic Claude a través de la Messages API con este patrón de RAG gestionado y almacenar el razonamiento y las citas en Amazon S3 para fines de auditoría permite cumplir el requisito de explicabilidad y trazabilidad documental con la mínima carga operativa posible, ya que Bedrock Knowledge Bases gestiona la ingesta, indexación, recuperación y generación de citas de forma totalmente administrada, lo cual también favorece cumplir con el requisito de latencia baja al no añadir componentes adicionales de orquestación.

Opción B: la función de "extended thinking" (razonamiento extendido) de los modelos Claude en Amazon Bedrock expone una traza del proceso de razonamiento interno del modelo, pero esta traza no está diseñada como mecanismo de atribución de fuentes documentales verificables ni se vincula automáticamente a documentos de la base de conocimiento; para enlazar esa traza de pensamiento con las fuentes de datos de la empresa habría que desarrollar lógica adicional. Además, un presupuesto de pensamiento de 4.000 tokens puede incrementar la latencia de la respuesta, dificultando cumplir el requisito de menos de 3 segundos, y usar DynamoDB para las trazas de razonamiento y citas añade una pieza de infraestructura adicional a gestionar.

Opción C: desplegar un modelo Claude personalizado en Amazon SageMaker AI en lugar de usar Amazon Bedrock, junto con AWS Lambda para procesar las respuestas y una base de datos Amazon RDS separada para las citas, implica construir manualmente toda la lógica de recuperación, generación y citación (RAG "casero"), lo que representa la mayor carga operativa de todas las opciones: gestión de infraestructura de SageMaker, del pipeline de Lambda y de la base de datos relacional para las fuentes.

Opción D: usar Amazon Bedrock con Claude y chain-of-thought reasoning junto con un seguimiento de recuperación "personalizado" (custom retrieval tracking) sobre la API de Knowledge Bases duplica una funcionalidad de citación que Knowledge Bases ya ofrece de forma nativa mediante RetrieveAndGenerate, añadiendo complejidad y desarrollo innecesarios. Usar CloudWatch para monitorizar la latencia es una buena práctica de observabilidad, pero no resuelve el requisito de citación con el mínimo esfuerzo operativo, ya que sigue exigiendo construir el propio sistema de trazabilidad en lugar de aprovechar el que ya integra el servicio.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-retrieve-generate.html> ; https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerate.html ; <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html>

A financial services company uses multiple foundation models (FMs) through Amazon Bedrock for its generative AI (GenAI) applications. To comply with a new regulation for GenAI use with sensitive financial data, the company needs a token management solution. The token management solution must proactively alert when applications approach model-specific token limits. The solution must also process more than 5,000 requests each minute and maintain token usage metrics to allocate costs across business units. Which solution will meet these requirements?

- A Develop model-specific tokenizers in an AWS Lambda function. Configure the Lambda function to estimate token usage before sending requests to Amazon Bedrock. Configure the Lambda function to publish metrics to Amazon CloudWatch and trigger alarms when requests approach thresholds. Store detailed token usage in Amazon DynamoDB to report costs.
- B Implement Amazon Bedrock Guardrails with token quota policies. Capture metrics on rejected requests. Configure Amazon EventBridge rules to trigger notifications based on Amazon Bedrock Guardrails metrics. Use Amazon CloudWatch dashboards to visualize token usage trends across models.
- C Deploy an Amazon SQS dead-letter queue for failed requests. Configure an AWS Lambda function to analyze token-related failures. Use Amazon CloudWatch Logs Insights to generate reports on token usage patterns based on error logs from Amazon Bedrock API responses.
- D Use Amazon API Gateway to create a proxy for all Amazon Bedrock API calls. Configure request throttling based on custom usage plans with predefined token quotas. Configure API Gateway to reject requests that will exceed token limits.

Respuesta correcta: A

- A** Develop model-specific tokenizers in an AWS Lambda function. Configure the Lambda function to estimate token usage before sending requests to Amazon Bedrock. Configure the Lambda function to publish metrics to Amazon CloudWatch and trigger alarms when requests approach thresholds. Store detailed token usage in Amazon DynamoDB to report costs. ✓
- B** Implement Amazon Bedrock Guardrails with token quota policies. Capture metrics on rejected requests. Configure Amazon EventBridge rules to trigger notifications based on Amazon Bedrock Guardrails metrics. Use Amazon CloudWatch dashboards to visualize token usage trends across models.
- C** Deploy an Amazon SQS dead-letter queue for failed requests. Configure an AWS Lambda function to analyze token-related failures. Use Amazon CloudWatch Logs Insights to generate reports on token usage patterns based on error logs from Amazon Bedrock API responses.
- D** Use Amazon API Gateway to create a proxy for all Amazon Bedrock API calls. Configure request throttling based on custom usage plans with predefined token quotas. Configure API Gateway to reject requests that will exceed token limits.

Explicación

Opción A (Correcta): desarrollar tokenizadores específicos por modelo dentro de una función AWS Lambda permite estimar el número de tokens de cada solicitud antes de enviarla a Amazon Bedrock (el conteo de tokens es específico de cada modelo, ya que cada uno usa una estrategia de tokenización distinta). Esa misma función puede publicar el uso como métricas personalizadas en Amazon CloudWatch y disparar alarmas cuando el consumo se acerca a los límites de tokens por minuto/día de cada modelo, cumpliendo el requisito de alertas proactivas. Lambda escala de forma automática y puede sostener un volumen de más de 5.000 solicitudes por minuto, y almacenar el detalle de uso de tokens en Amazon DynamoDB permite generar informes de coste desglosados por unidad de negocio. Esta combinación es la única de las cuatro que satisface simultáneamente los tres requisitos: alerta proactiva antes de alcanzar el límite, alto rendimiento y atribución de costes por unidad de negocio.

Opción B: según la documentación oficial de Amazon Bedrock Guardrails, un guardrail se compone de políticas de filtros de contenido, temas denegados, filtros de información sensible, filtros de palabras y filtros de contenido de imagen; no existe ningún tipo de "política de cuota de tokens". Los guardrails evalúan y filtran el contenido de las solicitudes y respuestas (seguridad y cumplimiento), pero no miden, limitan ni alertan sobre el consumo de tokens, por lo que esta opción describe una capacidad que Guardrails no ofrece.

Opción C: usar una cola de mensajes fallidos (dead-letter queue) de Amazon SQS junto con el análisis de errores en Amazon CloudWatch Logs Insights es un enfoque puramente reactivo: solo se generan datos después de que una solicitud ya ha fallado por exceder límites de tokens. Esto no permite "alertar proactivamente cuando las aplicaciones se acercan a los límites", que es un requisito explícito del enunciado, ya que la detección ocurre después del incumplimiento, no antes.

Opción D: los planes de uso (usage plans) de Amazon API Gateway controlan el número de solicitudes o la tasa de llamadas (throttling), pero no inspeccionan el contenido de la petición para calcular cuántos tokens consumirá en un modelo concreto de Bedrock; cada modelo tokeniza de forma distinta, algo que API Gateway no puede evaluar. Además, un usage plan no ofrece de forma nativa el desglose de costes por unidad de negocio basado en tokens que exige el enunciado.

Nota adicional: actualmente Amazon Bedrock también ofrece una API nativa, CountTokens, que permite estimar el número de tokens de una solicitud (InvokeModel o Converse) antes de ejecutarla, sin coste adicional, como alternativa a construir tokenizadores propios. Esto no invalida la opción A —que sigue siendo una arquitectura válida y es la única que cumple los tres requisitos— pero es una capacidad más moderna a tener en cuenta si se actualiza la pregunta.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-how.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/count-tokens.html> ; https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_CountTokens.html

A retail company is developing a customer service application that must process 10,000 daily queries about products, orders, and warranties. The application must be able to respond to queries about 50,000 product documents that are updated every day. The application must integrate with an order management API to check the status of orders and to help process returns. The application must maintain context throughout multi-turn interactions with customers. The company must collect complete audit trails for application responses. Which solution will meet these requirements with the LEAST operational overhead?

- A Deploy a fine-tuned Amazon Bedrock Anthropic Claude model for each product category. Create AWS Lambda functions to connect each model to the order management API. Store conversation history in Amazon DynamoDB.
- B Create a custom model that uses continued pre-training on Amazon Bedrock to handle all product documentation. Set up an Amazon API Gateway REST API that uses AWS Lambda functions to connect the model to the order management API.
- C Use Amazon SageMaker AI with containers to deploy models. Use Amazon Kendra to search product documents. Use AWS Step Functions to orchestrate calls to the order management API.
- D Use an Amazon Bedrock agent with action groups to integrate with the order management API. Associate an Amazon Bedrock knowledge base with the agent to search product documentation by using Retrieval Augmentation Generation (RAG). Enable trace events to capture audit trails.

Respuesta correcta: **D**

- A Deploy a fine-tuned Amazon Bedrock Anthropic Claude model for each product category. Create AWS Lambda functions to connect each model to the order management API. Store conversation history in Amazon DynamoDB.
- B Create a custom model that uses continued pre-training on Amazon Bedrock to handle all product documentation. Set up an Amazon API Gateway REST API that uses AWS Lambda functions to connect the model to the order management API.
- C Use Amazon SageMaker AI with containers to deploy models. Use Amazon Kendra to search product documents. Use AWS Step Functions to orchestrate calls to the order management API.
- D Use an Amazon Bedrock agent with action groups to integrate with the order management API. Associate an Amazon Bedrock knowledge base with the agent to search product documentation by using Retrieval Augmentation Generation (RAG). Enable trace events to capture audit trails.**

Explicación

Opción A: ajustar (fine-tune) un modelo de Anthropic Claude en Amazon Bedrock para cada categoría de producto obliga a entrenar, versionar y mantener múltiples modelos personalizados, además de reentrenarlos cada vez que cambien los 50.000 documentos de producto (que se actualizan a diario). También requiere construir manualmente, con funciones Lambda, la integración con la API de gestión de pedidos y gestionar a mano el historial de conversación en Amazon DynamoDB. Esto implica una carga operativa alta, muy alejada de la orquestación nativa que ofrecen los agentes de Bedrock.

Opción B: el pre-entrenamiento continuo (continued pre-training) en Amazon Bedrock es un proceso de personalización de modelos pensado para inyectar conocimiento de dominio de forma duradera en los pesos del modelo, no para exponer contenido que cambia constantemente. Con 50.000 documentos actualizados cada día, habría que reentrenar el modelo base con una frecuencia inviable en tiempo y coste; además, tampoco resuelve por sí sola la integración con la API de pedidos, que igualmente requeriría una capa adicional de Lambda y API Gateway construida y operada manualmente.

Opción C: desplegar modelos en contenedores propios con Amazon SageMaker AI implica aprovisionar, escalar y mantener la infraestructura de inferencia (instancias, contenedores, actualizaciones), en contraste con un servicio totalmente gestionado. Usar Amazon Kendra para la búsqueda de documentos añade otro servicio a administrar en paralelo a los modelos, y orquestar las llamadas a la API de pedidos con AWS Step Functions exige diseñar y mantener manualmente las máquinas de estado de orquestación, en lugar de apoyarse en la orquestación de agentes ya integrada en Bedrock. El conjunto es funcionalmente viable pero con una sobrecarga operativa notablemente mayor.

Opción D (Correcta): un agente de Amazon Bedrock, según la documentación oficial de AWS, gestiona automáticamente el ciclo de vida de una petición en tres fases (pre-procesamiento, orquestación y post-procesamiento). En la fase de orquestación, el agente usa un modelo fundacional para razonar sobre la entrada del usuario, decide si debe invocar una acción de un action group (que conecta con una función Lambda para llamar a la API de gestión de pedidos y procesar devoluciones) o consultar una base de conocimiento (knowledge base) asociada que implementa Retrieval Augmented Generation (RAG) sobre los 50.000 documentos de producto; estas bases de conocimiento admiten sincronización incremental de las fuentes de datos, por lo que solo se reprocesan los documentos añadidos, modificados o eliminados desde la última sincronización, algo idóneo para contenido que se actualiza a diario sin gestionar infraestructura de indexación propia. El propio servicio conserva el historial de la sesión (conversation history) entre llamadas a InvokeAgent, manteniendo el contexto en interacciones multi-turno sin necesidad de una base de datos externa gestionada por el desarrollador. Además, al habilitar el trace en tiempo de ejecución, cada respuesta del agente va acompañada de eventos de traza (trace events) que documentan el razonamiento (rationale), las acciones invocadas, las consultas a la base de conocimiento y las observaciones en cada paso, proporcionando un rastro de auditoría completo del comportamiento del agente. Toda esta arquitectura es un servicio gestionado por AWS, por lo que es la solución que cumple los requisitos con la menor carga operativa.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/trace-events.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-sync-ingest.html>

An ecommerce company is using Amazon Bedrock to build a generative AI (GenAI) application. The application uses AWS Step Functions to orchestrate a multi-agent workflow to produce detailed product descriptions. The workflow consists of three sequential states: a description generator, a technical specifications validator, and a brand voice consistency checker. Each state produces intermediate reasoning traces and outputs that are passed to the next state. The application uses an Amazon S3 bucket for process storage and to store outputs. During testing, the company discovers that outputs between Step Functions states frequently exceed the 256 KB quota and cause workflow failures. A GenAI Developer needs to revise the application architecture to efficiently handle the Step Functions 256 KB quota and maintain workflow observability. The revised architecture must preserve the existing multi-agent reasoning and acting (ReAct) pattern. Which solution will meet these requirements with the LEAST operational overhead?

- A Store intermediate outputs in Amazon DynamoDB. Pass only references between states. Create a Map state that retrieves the complete data from DynamoDB when required for each agent's processing step.
- B Configure an Amazon Bedrock integration to use the S3 bucket URI in the input parameter for large outputs. Use the ResultPath field and the ResultSelector field to route S3 references between the agent steps while maintaining the sequential validation workflow.
- C Use AWS Lambda functions to compress outputs to less than 256 KB before each agent state. Configure each agent task to decompress the outputs before processing and to compress results before passing them to the next state.
- D Configure a separate Step Functions state machine to handle each agent's processing. Use Amazon EventBridge to coordinate the execution flow between state machines. Use S3 references for the outputs as event data.

Respuesta correcta: **B**

- A Store intermediate outputs in Amazon DynamoDB. Pass only references between states. Create a Map state that retrieves the complete data from DynamoDB when required for each agent's processing step.
- B Configure an Amazon Bedrock integration to use the S3 bucket URI in the input parameter for large outputs. Use the ResultPath field and the ResultSelector field to route S3 references between the agent steps while maintaining the sequential validation workflow.** ✓
- C Use AWS Lambda functions to compress outputs to less than 256 KB before each agent state. Configure each agent task to decompress the outputs before processing and to compress results before passing them to the next state.
- D Configure a separate Step Functions state machine to handle each agent's processing. Use Amazon EventBridge to coordinate the execution flow between state machines. Use S3 references for the outputs as event data.

Explicación

Opción A: almacenar las salidas intermedias en Amazon DynamoDB y usar un estado Map para recuperarlas introduce un servicio de almacenamiento adicional que no aprovecha el bucket de S3 que la aplicación ya usa para el "process storage". Además, un estado Map está pensado para iterar sobre colecciones de elementos (por ejemplo, procesar varios elementos en paralelo o en lote), no para recuperar un único valor de referencia en un flujo estrictamente secuencial de tres pasos; esto añadiría complejidad operativa (tablas, IAM, capacidad de lectura/escritura) sin necesidad real.

Opción B (Correcta): la documentación oficial de AWS Step Functions confirma que, cuando el payload de un estado supera el límite de 256 KiB, la práctica recomendada es almacenar los datos grandes en Amazon S3 y pasar solo una referencia entre estados, en lugar de la carga completa ("Using Amazon S3 ARNs instead of passing large payloads in Step Functions"). Más específicamente, la integración optimizada de Step Functions con Amazon Bedrock (acción "arn:aws:states:::bedrock:invokeModel") admite de forma nativa los campos opcionales "Input" y "Output", cada uno con una propiedad "S3Uri": "Input" permite indicar que los datos de entrada (Body) se lean desde un objeto de S3, y "Output" permite que la respuesta del modelo se escriba en S3, devolviendo en el resultado del estado solo una referencia a esa ubicación. La propia documentación indica textualmente que si el payload de "Body" supera 256 KiB, se recomienda usar "Input" (S3Uri) en su lugar. Sobre esa referencia (pequeña, dentro del límite), se pueden usar los campos del Amazon States Language "ResultSelector" (para filtrar/transformar el resultado de la tarea antes de combinarlo, por ejemplo quedándose solo con el URI de S3 devuelto) y "ResultPath" (para especificar en qué parte del JSON de salida del estado se inserta ese resultado, combinándolo o reemplazando la entrada original). Esto permite mantener el flujo secuencial ReAct entre el generador de descripciones, el validador técnico y el verificador de tono de marca, preservando la observabilidad de Step Functions (Step Functions sigue viendo y registrando cada transición, solo que los datos voluminosos residen en S3) con el mínimo esfuerzo operativo, ya que no requiere Lambdas adicionales de compresión, tablas nuevas ni máquinas de estado adicionales.

Opción C: comprimir y descomprimir las salidas con funciones Lambda antes y después de cada estado añade latencia, complejidad de mantenimiento (lógica de compresión/descompresión duplicada en cada paso) y no resuelve el problema de fondo: para descripciones de producto muy detalladas, con trazas de razonamiento intermedias, el contenido comprimido puede seguir superando 256 KiB, por lo que no es una solución robusta ni de mínimo esfuerzo operativo.

Opción D: dividir el flujo en varias máquinas de estado independientes coordinadas por Amazon EventBridge rompe la orquestación nativa y secuencial de Step Functions, dificulta el seguimiento de una única ejecución de extremo a extremo (la observabilidad se fragmenta entre varias máquinas de estado y el bus de eventos) y añade una capa de coordinación (reglas de EventBridge, gestión de eventos, posibles reintentos) que aumenta significativamente el overhead operativo frente a simplemente enrutar referencias de S3 dentro de la misma máquina de estado.

Referencias: <https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html> ; <https://docs.aws.amazon.com/step-functions/latest/dg/sfn-best-practices.html> ; <https://docs.aws.amazon.com/step-functions/latest/dg/input-output-resultpath.html>

A company provides a service that helps users from around the world discover new restaurants. The service has 50 million monthly active users. The company wants to implement a semantic search solution across a database that contains 20 million restaurants and 200 million reviews. The company currently stores the data in a PostgreSQL database. The solution must support complex natural language queries and return results for at least 95% of queries within 500 ms. The solution must maintain data freshness for restaurant details that update hourly. The solution must also scale cost-effectively during peak usage periods. Which solution will meet these requirements with the LEAST development effort?

- A Migrate the restaurant data to Amazon OpenSearch Service. Implement keyword-based search rules that use custom analyzers and relevance tuning to find restaurants based on attributes such as cuisine type, feature, and location. Create Amazon API Gateway HTTP API endpoints to transform user queries into structured search parameters.
- B Migrate the restaurant data to Amazon OpenSearch Service. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant descriptions, reviews, and menu items. When users submit natural language queries, convert the queries to embeddings by using the same FM. Perform k-nearest neighbors (k-NN) searches to find semantically similar results.
- C Keep the restaurant data in PostgreSQL and implement a pgvector extension. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant data. Store the vector embeddings directly in PostgreSQL. Create an AWS Lambda function to convert natural language queries to vector representations by using the same FM. Configure the Lambda function to perform similarity searches within the database.
- D Migrate restaurant data to an Amazon Bedrock knowledge base by using a custom ingestion pipeline. Configure the knowledge base to automatically generate embeddings from restaurant information. Use the Amazon Bedrock Retrieve API with built-in vector search capabilities to query the knowledge base directly by using natural language input.

Respuesta correcta: **B**

- A Migrate the restaurant data to Amazon OpenSearch Service. Implement keyword-based search rules that use custom analyzers and relevance tuning to find restaurants based on attributes such as cuisine type, feature, and location. Create Amazon API Gateway HTTP API endpoints to transform user queries into structured search parameters.
- B Migrate the restaurant data to Amazon OpenSearch Service. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant descriptions, reviews, and menu items. When users submit natural language queries, convert the queries to embeddings by using the same FM. Perform k-nearest neighbors (k-NN) searches to find semantically similar results.**
- C Keep the restaurant data in PostgreSQL and implement a pgvector extension. Use a foundation model (FM) in Amazon Bedrock to generate vector embeddings from restaurant data. Store the vector embeddings directly in PostgreSQL. Create an AWS Lambda function to convert natural language queries to vector representations by using the same FM. Configure the Lambda function to perform similarity searches within the database.
- D Migrate restaurant data to an Amazon Bedrock knowledge base by using a custom ingestion pipeline. Configure the knowledge base to automatically generate embeddings from restaurant information. Use the Amazon Bedrock Retrieve API with built-in vector search capabilities to query the knowledge base directly by using natural language input.

Explicación

Opción A: implementar reglas de búsqueda basadas en palabras clave, aunque se apoyen en analizadores personalizados y ajuste de relevancia, sigue siendo búsqueda léxica (coincidencia de términos), no búsqueda semántica. Este enfoque no captura el significado ni la intención de consultas complejas en lenguaje natural (por ejemplo, "un sitio romántico y tranquilo para cenar cerca del mar"), por lo que no cumple el requisito central del caso de uso.

Opción B (Correcta): según la documentación oficial de Amazon OpenSearch Service, el servicio ofrece capacidades nativas de "vector search" que convierten texto, imágenes u otro contenido en embeddings de alta dimensión y permiten comparar la consulta contra esos vectores mediante búsquedas k-NN (k-nearest neighbors) y k-NN aproximado (HNSW) para encontrar los resultados semánticamente más similares. OpenSearch se integra de forma nativa con Amazon Bedrock para generar esos embeddings a partir de un modelo de fundación (FM), y AWS documenta explícitamente "semantic search" y "RAG (Retrieval Augmented Generation)" como casos de uso soportados. Con 20 millones de restaurantes y 200 millones de reseñas, migrar los datos a OpenSearch Service, generar embeddings con un FM de Bedrock a partir de descripciones, reseñas y menús, y convertir las consultas del usuario a vectores con ese mismo FM para ejecutar búsquedas k-NN es la arquitectura de referencia de AWS para este escenario: un motor de búsqueda distribuido y gestionado que escala horizontalmente para cumplir la latencia p95 de 500 ms, se adapta de forma coste-eficiente a picos de carga (incluyendo opciones serverless), y admite actualizaciones frecuentes de los datos (freshness horaria) mediante reindexado o ingesta incremental. Todo esto se logra combinando dos servicios totalmente gestionados (OpenSearch Service + Bedrock), lo que representa el menor esfuerzo de desarrollo entre las opciones.

Opción C: usar la extensión pgvector en PostgreSQL y una función Lambda que genere embeddings al vuelo para cada consulta es una solución válida a menor escala, pero PostgreSQL (incluso con pgvector) no está diseñado como motor de búsqueda distribuido de alto rendimiento; escalar índices de similitud vectorial sobre cientos de millones de filas para 50 millones de usuarios activos mensuales, manteniendo el percentil 95 por debajo de 500 ms y un escalado coste-eficiente en picos, exige un esfuerzo de ajuste operativo (sharding, réplicas, tuning de índices HNSW/IVFFlat) considerablemente mayor que aprovechar un motor de búsqueda vectorial gestionado y ya optimizado para ese fin, como OpenSearch.

Opción D: las bases de conocimiento (knowledge bases) de Amazon Bedrock están diseñadas principalmente para flujos RAG a partir de documentos no estructurados (o, en su variante más reciente, mediante conexión directa a almacenes de datos estructurados usando un motor de consultas como Amazon Redshift, sin necesidad de una canalización de ingesta a medida). En este escenario, migrar los datos mediante una "canalización de ingesta personalizada" (custom ingestion pipeline) desde PostgreSQL implica desarrollo específico adicional para transformar y sincronizar los datos, lo que supone más esfuerzo que aprovechar directamente OpenSearch Service junto con el modelo de embeddings de Bedrock, que es un patrón de integración ya soportado de forma nativa.

Referencias:

<https://docs.aws.amazon.com/opensearch-service/latest/developerguide/vector-search.html>

<https://docs.aws.amazon.com/opensearch-service/latest/developerguide/semantic-search.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-supported.html>

A medical company uses Amazon Bedrock to power a clinical documentation summarization system. The system produces inconsistent summaries when handling complex clinical documents. The system performed well on simple clinical documents. The company needs a solution that diagnoses inconsistencies, compares prompt performance against established metrics, and maintains historical records of prompt versions. Which solution will meet these requirements?

- A Create multiple prompt variants by using Prompt management in Amazon Bedrock. Manually test the prompts with simple clinical documents. Deploy the highest performing version by using the Amazon Bedrock console.
- B Implement version control for prompts in a code repository with a test suite that contains complex clinical documents and quantifiable evaluation metrics. Use an automated testing framework to compare prompt versions and document performance patterns.
- C Deploy each new prompt version to separate Amazon Bedrock API endpoints. Split production traffic between the endpoints. Configure Amazon CloudWatch to capture response metrics and user feedback for automatic version selection.
- D Create a custom prompt evaluation flow in Amazon Bedrock Flows that applies the same clinical document inputs to different prompt variants. Use Amazon Comprehend Medical to analyze and score the factual accuracy of each version.

Respuesta correcta: **B**

- A Create multiple prompt variants by using Prompt management in Amazon Bedrock. Manually test the prompts with simple clinical documents. Deploy the highest performing version by using the Amazon Bedrock console.
- B Implement version control for prompts in a code repository with a test suite that contains complex clinical documents and quantifiable evaluation metrics. Use an automated testing framework to compare prompt versions and document performance patterns.**
- C Deploy each new prompt version to separate Amazon Bedrock API endpoints. Split production traffic between the endpoints. Configure Amazon CloudWatch to capture response metrics and user feedback for automatic version selection.
- D Create a custom prompt evaluation flow in Amazon Bedrock Flows that applies the same clinical document inputs to different prompt variants. Use Amazon Comprehend Medical to analyze and score the factual accuracy of each version.

Explicación

Opción A: Prompt management de Amazon Bedrock sí ofrece versionado nativo de prompts, pero tal como se plantea la opción, las pruebas se realizan manualmente y solo con documentos clínicos simples, es decir, precisamente con los casos donde el sistema ya funciona bien. Esto no permite diagnosticar por qué falla con documentos complejos, no introduce métricas de evaluación establecidas y cuantificables, y desplegar "la versión de mejor rendimiento" desde la consola sin una suite de pruebas sistemática no constituye un proceso de comparación objetivo ni reproducible.

Opción B (Correcta): mantener el control de versiones de los prompts en un repositorio de código, junto con una suite de pruebas que incluya específicamente documentos clínicos complejos (los casos problemáticos) y métricas de evaluación cuantificables, y usar un framework de pruebas automatizado para comparar versiones y documentar patrones de rendimiento, cubre exactamente los tres requisitos del enunciado: (1) diagnostica las inconsistencias al ejecutar sistemáticamente los prompts contra los documentos complejos que causan el problema; (2) compara el rendimiento frente a métricas establecidas y cuantificables en lugar de una revisión manual subjetiva; y (3) mantiene un historial reproducible de cada versión de prompt mediante control de versiones. Esta práctica coincide con la guía de AWS Well-Architected (Agentic AI Lens, AGENTOPS02-BP01), que recomienda tratar los prompts con la misma disciplina que el código de aplicación: control de versiones, revisión, pruebas automatizadas, evaluación con criterios de calidad medibles y capacidad de rollback, típicamente implementado combinando Amazon Bedrock Prompt Management (versionado) con Amazon Bedrock Evaluations (puntuaciones cuantificables sobre un dataset estandarizado). La opción B refleja fielmente ese enfoque de "disciplina de nivel código" aplicado a prompts.

Opción C: dividir el tráfico de producción entre distintos endpoints y basarse en CloudWatch y feedback de usuario para la selección automática de versión expone a los pacientes/usuarios a versiones no validadas de prompts sin haber realizado antes un diagnóstico controlado ni una comparación con métricas cuantificables; es un enfoque de A/B testing en producción, no de diagnóstico y evaluación previa.

Opción D: Amazon Comprehend Medical está diseñado para extraer y estructurar entidades clínicas (medicamentos, diagnósticos, procedimientos) a partir de texto médico, no para evaluar de forma general la exactitud factual o la calidad de un resumen generado por un LLM, ni para gestionar el versionado de prompts. Aunque Amazon Bedrock Flows permite orquestar la aplicación de distintas variantes de prompt a las mismas entradas, combinarlo con Comprehend Medical no proporciona ni métricas de evaluación de resumen establecidas ni un historial de versiones adecuado para este caso de uso.

Referencias:

<https://docs.aws.amazon.com/wellarchitected/latest/agentic-ai-lens/agentops02-bp01.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management-version-compare.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-built-in-metrics.html>

A company uses Amazon Bedrock to generate technical content for customers. The company has recently experienced a surge in hallucination outputs when the company's model generates summaries of long technical documents. The model outputs include inaccurate or fabricated details. The company's current solution uses a large foundation model (FM) with a basic one-shot prompt that includes the full document in a single input. The company needs a solution that will reduce hallucinations and meet factual accuracy goals. The solution must process more than 1,000 documents each hour and deliver summaries within 3 seconds for each document. Which combination of solutions will meet these requirements? (Choose two.)

Selecciona 2 respuestas

- A Implement zero-shot chain-of-thought (CoT) instructions that require step-by-step reasoning with explicit fact verification before the model generates each summary.
- B Use Retrieval Augmented Generation (RAG) with an Amazon Bedrock knowledge base. Apply semantic chunking and tuned embeddings to ground summaries in source content.
- C Configure Amazon Bedrock guardrails to block any generated output that matches patterns that are associated with hallucinated content.
- D Increase the temperature parameter in Amazon Bedrock.
- E Prompt the Amazon Bedrock model to summarize each full document in one pass.

Respuesta correcta: A, B

- ☒ **A** Implement zero-shot chain-of-thought (CoT) instructions that require step-by-step reasoning with explicit fact verification before the model generates each summary. ☒
- ☒ **B** Use Retrieval Augmented Generation (RAG) with an Amazon Bedrock knowledge base. Apply semantic chunking and tuned embeddings to ground summaries in source content. ☒
- ☐ **C** Configure Amazon Bedrock guardrails to block any generated output that matches patterns that are associated with hallucinated content.
- ☐ **D** Increase the temperature parameter in Amazon Bedrock.
- ☐ **E** Prompt the Amazon Bedrock model to summarize each full document in one pass.

Explicación

Opción A (Correcta): exigir instrucciones de cadena de pensamiento (chain-of-thought, CoT) de tipo zero-shot que obliguen al modelo a razonar paso a paso y a verificar explícitamente los hechos antes de emitir cada resumen es una técnica de prompt engineering reconocida por AWS como mitigación de alucinaciones ("Model hallucinations and output integrity"), ya que fuerza al modelo a contrastar internamente sus afirmaciones contra el contenido de origen antes de producir la salida final, en lugar de generar directamente una respuesta sin ese paso intermedio de comprobación.

Opción B (Correcta): usar Retrieval Augmented Generation (RAG) con una base de conocimiento (Knowledge Base) de Amazon Bedrock, aplicando chunking semántico (SEMANTIC chunking strategy) y embeddings ajustados, ancla los resúmenes en los fragmentos de origen realmente relevantes recuperados para cada consulta, en vez de depender de que el modelo procese el documento completo de una sola vez. Esto reduce las fabricaciones porque el modelo solo genera contenido a partir del contexto recuperado, y además ayuda a cumplir el requisito de latencia y volumen (más de 1000 documentos/hora, menos de 3 segundos por documento), ya que se opera sobre fragmentos más pequeños en lugar de sobre el documento íntegro.

Opción C: es incorrecta tal como está formulada. Amazon Bedrock Guardrails no dispone de un filtro que bloquee genéricamente "patrones asociados a contenido alucinado". Los content filters de Guardrails detectan categorías de contenido dañino predefinidas (odio, violencia, etc.), no alucinaciones. El mecanismo real de Amazon Bedrock para detectar hallucinations es el contextual grounding check, que calcula puntuaciones de confianza de "grounding" (si la respuesta está fundamentada en la fuente proporcionada) y de "relevance" (si responde a la consulta del usuario) comparando la respuesta contra una fuente de referencia (grounding source) y una query, bloqueando la respuesta si la puntuación cae por debajo de un umbral configurable (0–0.99). Esta opción describe un mecanismo distinto (bloqueo por patrones) al que realmente existe en el servicio, por lo que no es la respuesta correcta.

Opción D: aumentar el parámetro de temperature incrementa la aleatoriedad y la creatividad en la generación de texto, lo que tiende a aumentar, no a reducir, la probabilidad de que el modelo introduzca información fabricada o inconsistente con la fuente. Es contraria al objetivo de precisión factual que busca la empresa.

Opción E: seguir resumiendo el documento completo en una sola pasada con un prompt básico es exactamente el enfoque actual descrito en el enunciado, que es la causa del problema de alucinaciones. Mantenerlo no introduce ninguna mejora ni ayuda a cumplir los requisitos de precisión, y tampoco resuelve el reto de latencia/escala al forzar el procesamiento de documentos largos completos en cada llamada.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-customize-ingestion.html>

<https://docs.aws.amazon.com/prescriptive-guidance/latest/strategy-data-considerations-gen-ai/security.html>

A company has a recommendation system. The system's applications run on Amazon EC2 instances. The applications make API calls to Amazon Bedrock foundation models (FMs) to analyze customer behavior and generate personalized product recommendations. The system is experiencing intermittent issues. Some recommendations do not match customer preferences. The company needs an observability solution to monitor operational metrics and detect patterns of operational performance degradation compared to established baselines. The solution must also generate alerts with correlation data within 10 minutes when FM behavior deviates from expected patterns. Which solution will meet these requirements?

- A Configure Amazon CloudWatch Container Insights for the application infrastructure. Set up CloudWatch alarms for latency thresholds. Add custom metrics for token counts by using the CloudWatch embedded metric format. Create CloudWatch dashboards to visualize the data.
- B Implement AWS X-Ray to trace requests through the application components. Enable CloudWatch Logs Insights for error pattern detection. Set up AWS CloudTrail to monitor all API calls to Amazon Bedrock. Create custom dashboards in Amazon QuickSight.
- C Enable Amazon CloudWatch Application Insights for the application resources. Create custom metrics for recommendation quality, token usage, and response latency by using the CloudWatch embedded metric format with dimensions for request types and user segments. Configure CloudWatch anomaly detection on the model metrics. Establish log pattern analysis by using CloudWatch Logs Insights.
- D Use Amazon OpenSearch Service with the Observability plugin. Ingest model metrics and logs by using Amazon Kinesis. Create custom Piped Processing Language (PPL) queries to analyze model behavior patterns. Establish operational dashboards to visualize anomalies in real time.

Respuesta correcta: **C**

- A Configure Amazon CloudWatch Container Insights for the application infrastructure. Set up CloudWatch alarms for latency thresholds. Add custom metrics for token counts by using the CloudWatch embedded metric format. Create CloudWatch dashboards to visualize the data.
- B Implement AWS X-Ray to trace requests through the application components. Enable CloudWatch Logs Insights for error pattern detection. Set up AWS CloudTrail to monitor all API calls to Amazon Bedrock. Create custom dashboards in Amazon QuickSight.
- C** Enable Amazon CloudWatch Application Insights for the application resources. Create custom metrics for recommendation quality, token usage, and response latency by using the CloudWatch embedded metric format with dimensions for request types and user segments. Configure CloudWatch anomaly detection on the model metrics. Establish log pattern analysis by using CloudWatch Logs Insights. ✓
- D Use Amazon OpenSearch Service with the Observability plugin. Ingest model metrics and logs by using Amazon Kinesis. Create custom Piped Processing Language (PPL) queries to analyze model behavior patterns. Establish operational dashboards to visualize anomalies in real time.

Explicación

Opción A: Amazon CloudWatch Container Insights recopila métricas de infraestructura de contenedores (CPU, memoria, red, disco) para clústeres ECS/EKS/Kubernetes en EC2. No analiza la calidad de las recomendaciones del modelo ni establece líneas base de comportamiento del FM, por lo que no cumple el requisito de detectar desviaciones respecto a un patrón operativo esperado.

Opción B: AWS X-Ray traza la latencia y el flujo de solicitudes entre componentes de la aplicación, y AWS CloudTrail audita quién y cuándo invocó la API de Amazon Bedrock (control plane / data plane), pero ninguno de los dos establece una línea base estadística de métricas de negocio (calidad de recomendación, uso de tokens) ni genera de forma nativa alertas correlacionadas cuando el comportamiento se desvía de lo esperado; requeriría construir esa lógica de correlación manualmente en QuickSight, lo que no cumple el requisito de detección automática en minutos.

Opción C (Correcta): Amazon CloudWatch Application Insights escanea los recursos de la aplicación (incluidas instancias EC2) y monitoriza continuamente métricas y logs para detectar y correlacionar anomalías y errores; agrupa estas "observaciones" relacionadas en "problemas", genera automáticamente paneles (dashboards) que muestran las anomalías de métricas y los errores de log correlacionados junto con posibles causas raíz, y puede enviar notificaciones (por ejemplo, vía Amazon SNS o CloudWatch Events) cuando se detecta un problema. Al añadir métricas personalizadas en formato EMF (Embedded Metric Format) para calidad de recomendación, uso de tokens y latencia de respuesta —con dimensiones por tipo de solicitud y segmento de usuario—, estas métricas personalizadas quedan disponibles para que CloudWatch Anomaly Detection aprenda una línea base estadística por métrica (basada en patrones históricos, incluyendo estacionalidad) y dispare alarmas cuando el valor real se sale de la banda esperada. Combinado con CloudWatch Logs Insights para el análisis de patrones de error en los logs, esta solución integra correlación de métricas, detección de anomalías respecto a baselines establecidos y generación de alertas en cuestión de minutos, cumpliendo todos los requisitos del enunciado.

Opción D: Amazon OpenSearch Service con el plugin de Observability y consultas PPL (Piped Processing Language) es una solución potente de análisis de logs y trazas, pero no está integrado de forma nativa con las métricas de Amazon Bedrock ni con CloudWatch; requeriría un pipeline de ingesta adicional (Kinesis) y desarrollo de consultas y paneles a medida, lo que añade complejidad operativa y latencia innecesarias frente a una solución nativa como CloudWatch Application Insights.

Referencias:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/appinsights-what-is.html>

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Embedded_Metric_Format.html

<https://docs.aws.amazon.com/prescriptive-guidance/latest/implementing-logging-monitoring-cloudwatch/cloudwatch-search-analysis.html>

An enterprise application uses an Amazon Bedrock foundation model (FM) to process and analyze 50 to 200 pages of technical documents. Users are experiencing inconsistent responses and receiving truncated outputs when processing documents that exceed the FM's context window limits. Which solution will resolve this problem?

- A Configure fixed-size chunking at 4,000 tokens for each chunk with 20% overlap. Use application-level logic to link multiple chunks sequentially until the FM's maximum context window of 200,000 tokens is reached before making inference calls.
- B Use hierarchical chunking with parent chunks of 8,000 tokens and child chunks of 2,000 tokens. Use Amazon Bedrock Knowledge Bases built-in retrieval to automatically select relevant parent chunks based on query context. Configure overlap tokens to maintain semantic continuity.
- C Use semantic chunking with a breakpoint percentile threshold of 95% and a buffer size of 3 sentences. Use the Amazon Bedrock RetrieveAndGenerate API call to dynamically select the most relevant chunks based on embedding similarity scores.
- D Create a pre-processing AWS Lambda function that analyzes document token count by using the FM's tokenizer. Configure the lambda function to split documents into equal segments that fit within 80% of the context window. Configure the Lambda function to process each segment independently before aggregating the results.

Respuesta correcta: **C**

- A Configure fixed-size chunking at 4,000 tokens for each chunk with 20% overlap. Use application-level logic to link multiple chunks sequentially until the FM's maximum context window of 200,000 tokens is reached before making inference calls.
- B Use hierarchical chunking with parent chunks of 8,000 tokens and child chunks of 2,000 tokens. Use Amazon Bedrock Knowledge Bases built-in retrieval to automatically select relevant parent chunks based on query context. Configure overlap tokens to maintain semantic continuity.
- C** Use semantic chunking with a breakpoint percentile threshold of 95% and a buffer size of 3 sentences. Use the Amazon Bedrock RetrieveAndGenerate API call to dynamically select the most relevant chunks based on embedding similarity scores. ✓
- D Create a pre-processing AWS Lambda function that analyzes document token count by using the FM's tokenizer. Configure the lambda function to split documents into equal segments that fit within 80% of the context window. Configure the Lambda function to process each segment independently before aggregating the results.

Explicación

Opción A: el chunking de tamaño fijo (4,000 tokens por fragmento con 20% de solapamiento) enlazado secuencialmente hasta alcanzar los 200,000 tokens de ventana de contexto no resuelve el problema de fondo: sigue enviando al modelo bloques de texto completos sin priorizar qué información es realmente relevante para la consulta del usuario, por lo que documentos muy extensos seguirán generando respuestas inconsistentes o truncadas al acercarse al límite de la ventana de contexto. Además, el chunking de tamaño fijo puede cortar el contenido en puntos arbitrarios que rompen el significado semántico.

Opción B: el chunking jerárquico (parent chunks de 8,000 tokens y child chunks de 2,000 tokens) permite mantener contexto entre fragmentos padre e hijo, y Amazon Bedrock Knowledge Bases sí puede sustituir automáticamente los child chunks recuperados por su parent chunk correspondiente en la respuesta final. Sin embargo, no divide el texto según límites semánticos reales (solo por tamaño en dos niveles), por lo que no garantiza que los fragmentos recuperados sean los más relevantes desde el punto de vista del significado, y el uso de parent chunks grandes (8,000 tokens) puede seguir empujando la ventana de contexto del modelo hacia el límite cuando se combinan varios de ellos.

Opción C (Correcta): el chunking semántico (SEMANTIC) en Amazon Bedrock Knowledge Bases divide los documentos en fragmentos que priorizan el significado semántico sobre la estructura sintáctica, usando un umbral de percentil de ruptura (breakpointPercentileThreshold) y un tamaño de buffer en oraciones (bufferSize) para decidir dónde cortar el texto sin romper ideas completas. Esto evita que un fragmento mezcle contenido no relacionado o corte una idea a la mitad, lo cual reduce las respuestas inconsistentes. Combinado con la API RetrieveAndGenerate de Amazon Bedrock Knowledge Bases, el sistema realiza una búsqueda vectorial (configurable mediante KnowledgeBaseVectorSearchConfiguration) que recupera únicamente los fragmentos más relevantes por similitud de embeddings para la consulta del usuario, en lugar de enviar el documento completo al modelo. De esta forma se evita superar la ventana de contexto de la FM y se reducen tanto las salidas truncadas como las respuestas inconsistentes al procesar documentos de 50 a 200 páginas.

Opción D: crear una función AWS Lambda de preprocesamiento que calcule el conteo de tokens con el tokenizador de la FM y divida el documento en segmentos de igual tamaño (80% de la ventana de contexto) es una solución completamente manual que no agrupa el contenido por significado semántico. Procesar cada segmento de forma independiente y luego agregar los resultados puede producir respuestas inconsistentes porque cada llamada de inferencia carece del contexto completo del documento, y no existe ningún mecanismo de recuperación que priorice la información más relevante para la consulta.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-customize-ingestion.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-retrieve-generate.html>

A company is developing a generative AI (GenAI) application that analyzes customer service calls in real-time and generates suggested responses for human customer service agents. The application must process 500,000 concurrent calls during peak hours with less than 200 ms end-to-end latency for each suggestion. The company uses existing architecture to transcribe customer call audio streams. The application must not exceed a pre-defined monthly compute budget and must maintain auto scaling capabilities. Which solution will meet these requirements?

- A Deploy a large, complex reasoning model on Amazon Bedrock. Purchase provisioned throughput and optimize for batch processing.
- B Deploy a low-latency, real-time optimized model on Amazon Bedrock. Purchase provisioned throughput and set up automatic scaling policies.
- C Deploy a large language model (LLM) on an Amazon SageMaker AI real-time endpoint that uses dedicated GPU instances.
- D Deploy a mid-sized language model on an Amazon SageMaker AI serverless endpoint that is optimized for batch processing.

Respuesta correcta: **B**

- A Deploy a large, complex reasoning model on Amazon Bedrock. Purchase provisioned throughput and optimize for batch processing.
- B Deploy a low-latency, real-time optimized model on Amazon Bedrock. Purchase provisioned throughput and set up automatic scaling policies. ✓**
- C Deploy a large language model (LLM) on an Amazon SageMaker AI real-time endpoint that uses dedicated GPU instances.
- D Deploy a mid-sized language model on an Amazon SageMaker AI serverless endpoint that is optimized for batch processing.

Explicación

Opción A: un modelo grande orientado a razonamiento complejo, combinado con una optimización para procesamiento por lotes (batch), no está pensado para cumplir una latencia inferior a 200 ms por respuesta. El procesamiento por lotes agrupa solicitudes para maximizar el rendimiento (throughput) agregado, no para minimizar la latencia individual de cada sugerencia en un escenario de generación en tiempo real, por lo que no cumple el requisito principal del enunciado.

Opción B (Correcta): Amazon Bedrock ofrece "Latency Optimized Inference" (inferencia optimizada para latencia), una capacidad que reduce el tiempo de respuesta del modelo sin sacrificar la precisión, adecuada para generar sugerencias en vivo durante llamadas de atención al cliente. El Provisioned Throughput de Bedrock permite reservar capacidad de cómputo dedicada (Model Units) mediante un compromiso de duración (sin compromiso, 1 mes o 6 meses) con un coste horario fijo y predecible, lo que ayuda a controlar el presupuesto mensual y a sostener un volumen alto de llamadas concurrentes. Combinado con políticas de auto scaling a nivel de aplicación (por ejemplo, para la capa de orquestación/Lambda que invoca el modelo), esta es la opción que mejor encaja de las cuatro con los requisitos de latencia, escala y presupuesto del enunciado.

Opción C: un endpoint en tiempo real de Amazon SageMaker AI con instancias GPU dedicadas sí admite auto scaling nativo mediante Application Auto Scaling y puede alcanzar baja latencia, pero obliga a la empresa a alojar y gestionar directamente el modelo (selección y dimensionamiento de instancias, actualizaciones, políticas de escalado, parcheo), en lugar de aprovechar un servicio de inferencia totalmente gestionado como Bedrock. Además, esta opción no incorpora una capacidad equivalente a la inferencia optimizada para latencia de Bedrock, lo que aumenta el riesgo y la complejidad operativa frente a la opción B.

Opción D: un endpoint sin servidor (serverless) de SageMaker AI está pensado para tráfico intermitente e impredecible y escala automáticamente los recursos, pero introduce latencia de arranque en frío (cold start, reflejada en la métrica OverheadLatency de SageMaker) que fácilmente puede superar el umbral de 200 ms, en particular para modelos de lenguaje. Además, SageMaker Serverless Inference no es una modalidad "optimizada para procesamiento por lotes": el procesamiento por lotes en SageMaker se realiza mediante Batch Transform, un mecanismo distinto orientado a inferencia asíncrona y no interactiva. Por ambos motivos, esta opción no es apta para el caso de uso en tiempo real descrito.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>

<https://docs.aws.amazon.com/sagemaker/latest/dg/serverless-endpoints.html>

An ecommerce company is building an internal platform to develop generative AI applications by using Amazon Bedrock foundation models (FMs). Developers need to select models based on evaluations that are aligned to ecommerce use cases. The platform must display accuracy metrics for text generation and summarization in dashboards. The company has custom ecommerce datasets to use as standardized evaluation inputs. Which combination of steps will meet these requirements with the LEAST operational overhead? (Choose two.)

Selecciona 2 respuestas

- A Import the datasets to an Amazon S3 bucket. Provide appropriate IAM permissions and cross-origin resource sharing (CORS) permissions to give the evaluation jobs access to the datasets.
- B Import the datasets to an Amazon S3 bucket. Provide appropriate IAM permissions and a VPC endpoint configuration to give the evaluation jobs access to the datasets.
- C Configure an AWS Lambda function to create model evaluation jobs on a schedule in the Amazon Bedrock console. Provide the URI of the S3 bucket that contains the datasets as an input. Configure the evaluation jobs to measure the real world knowledge (RWK) score for text generation and BERT Score for summarization. Configure a second Lambda function to check the status of the jobs and publish custom logs to Amazon CloudWatch. Create a custom Amazon CloudWatch Logs Insights dashboard.
- D Use Amazon SageMaker Clarify on a schedule to create model evaluation jobs. Use open source frameworks to create and run standardized evaluations. Publish results to Amazon CloudWatch namespaces. Use the word error rate score for text generation and toxicity for summarization as metrics for accuracy. Configure an AWS Lambda function to check the status of the jobs and publish custom logs to CloudWatch. Create a custom Amazon CloudWatch Logs Insights dashboard.
- E Run an Amazon SageMaker AI notebook job on a schedule by using the fmevals or ragas framework to run evaluations that use the datasets in the S3 bucket. Write Python code in the notebook that makes direct InvokeModel API calls to the FMs and processes their responses for evaluation. Publish job status and results to Amazon CloudWatch Logs to measure the real world knowledge (RWK) score for text generation and toxicity for summarization as metrics for accuracy. Create a custom CloudWatch Logs Insights dashboard.

Respuesta correcta: A, C

- A** Import the datasets to an Amazon S3 bucket. Provide appropriate IAM permissions and cross-origin resource sharing (CORS) permissions to give the evaluation jobs access to the datasets. ✓
- B** Import the datasets to an Amazon S3 bucket. Provide appropriate IAM permissions and a VPC endpoint configuration to give the evaluation jobs access to the datasets.
- C** Configure an AWS Lambda function to create model evaluation jobs on a schedule in the Amazon Bedrock console. Provide the URI of the S3 bucket that contains the datasets as an input. Configure the evaluation jobs to measure the real world knowledge (RWK) score for text generation and BERT Score for summarization. Configure a second Lambda function to check the status of the jobs and publish custom logs to Amazon CloudWatch. Create a custom Amazon CloudWatch Logs Insights dashboard. ✓
- D** Use Amazon SageMaker Clarify on a schedule to create model evaluation jobs. Use open source frameworks to create and run standardized evaluations. Publish results to Amazon CloudWatch namespaces. Use the word error rate score for text generation and toxicity for summarization as metrics for accuracy. Configure an AWS Lambda function to check the status of the jobs and publish custom logs to CloudWatch. Create a custom Amazon CloudWatch Logs Insights dashboard.
- E** Run an Amazon SageMaker AI notebook job on a schedule by using the fmevals or ragas framework to run evaluations that use the datasets in the S3 bucket. Write Python code in the notebook that makes direct InvokeModel API calls to the FMs and processes their responses for evaluation. Publish job status and results to Amazon CloudWatch Logs to measure the real world knowledge (RWK) score for text generation and toxicity for summarization as metrics for accuracy. Create a custom CloudWatch Logs Insights dashboard.

Explicación

Opción A (Correcta): para que un trabajo de evaluación de modelos de Amazon Bedrock pueda leer y escribir datos, el bucket de Amazon S3 debe estar en la misma región de AWS que el trabajo, y el rol de servicio de Bedrock necesita permisos de IAM adecuados sobre ese bucket (lectura de los datasets de entrada y escritura de los resultados). Es importante matizar, según la documentación oficial de AWS, que la configuración de Cross-Origin Resource Sharing (CORS) en el bucket de salida es un requisito exclusivo de los trabajos de evaluación basados en revisión humana (para que el portal de anotadores pueda mostrar los prompts y las respuestas del modelo en el navegador); los trabajos de evaluación automática, como el que necesita esta empresa, no requieren configuración CORS. Aun así, entre las opciones disponibles, A representa el enfoque de acceso a datos con menor sobrecarga operativa (solo IAM sobre S3), frente a la alternativa de la opción B.

Opción B: un endpoint de VPC no es necesario para que el servicio de evaluación de modelos de Bedrock acceda a un bucket de S3 estándar; solo sería necesario si el tráfico debiera mantenerse dentro de una VPC sin salida a Internet, algo que el enunciado no exige. Añadir un endpoint de VPC (Gateway o Interface) implica configuración de red adicional, puntos de enrutamiento y mantenimiento continuo, lo que incrementa innecesariamente la sobrecarga operativa frente a la opción A.

Opción C (Correcta): Amazon Bedrock permite crear trabajos de evaluación automática de modelos (sin intervención humana) por consola, API o SDK, usando datasets propios almacenados en S3 como entrada. Según la documentación oficial, para la tarea de "General text generation" la métrica de precisión (Accuracy) se calcula como el score de Real World Knowledge (RWK), y para la tarea de "Text summarization" la métrica de precisión se calcula como BERTScore; esto coincide exactamente con lo descrito en la opción. Automatizar la creación periódica de estos trabajos con una función Lambda, monitorizar su estado con una segunda Lambda que publique logs personalizados en Amazon CloudWatch, y visualizar los resultados en un panel de CloudWatch Logs Insights, es una arquitectura totalmente gestionada (serverless) que aprovecha las capacidades nativas de evaluación de Bedrock, ofreciendo así la mínima sobrecarga operativa.

Opción D: Amazon SageMaker Clarify no es el mecanismo nativo de evaluación de modelos generativos de Amazon Bedrock (aunque SageMaker Clarify puede evaluar modelos de Bedrock mediante el framework FMEval, requiere más configuración y mantenimiento que los trabajos de evaluación integrados de Bedrock). Además, las métricas propuestas en esta opción son incorrectas para el objetivo del enunciado: "word error rate" es la métrica asociada a la tarea de robustez en generación de texto (no a precisión/accuracy) y "toxicity" es una métrica de toxicidad, no de precisión para resumen; la métrica de precisión correcta para resumen es BERTScore. Esta combinación suma fragmentación, métricas mal alineadas y complejidad operativa innecesarias.

Opción E: ejecutar un notebook job de SageMaker AI que invoque directamente la API InvokeModel de Bedrock y use frameworks de código abierto (fmeval o ragas) obliga a construir y mantener manualmente toda la lógica de orquestación, cálculo de métricas y manejo de errores, en lugar de usar los trabajos de evaluación nativos y totalmente gestionados de Bedrock. Esto implica un esfuerzo de desarrollo, prueba y mantenimiento considerablemente mayor, además de que, igual que en la opción D, "real world knowledge (RWK)" sí es la métrica correcta de precisión para generación de texto, pero "toxicity" no es una métrica de precisión para resumen (la métrica de precisión correcta es BERTScore), lo que añade además una imprecisión técnica a la sobrecarga operativa de este enfoque.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-type-automatic.html>, <https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-tasks-general-text.html>, <https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-tasks-text-summary.html>

An elevator service company has developed an AI assistant application by using Amazon Bedrock. The application generates elevator maintenance recommendations to support the company's elevator technicians. The company uses Amazon Kinesis Data Streams to collect the elevator sensor data. New regulatory rules require that a human technician must review all AI-generated recommendations. The company needs to establish human oversight workflows to review and approve AI recommendations. The company must store all human technician review decisions for audit purposes. Which solution will meet these requirements?

- A Create a custom approval workflow by using AWS Lambda functions and Amazon SQS queues for human review of AI recommendations. Store all review decisions in Amazon DynamoDB for audit purposes.
- B Create an AWS Step Functions workflow that has a human approval step that uses the `waitForTaskToken` API to pause execution. After a human technician completes a review, use an AWS Lambda function to call the `SendTaskSuccess` API that has the approval decision. Store all review decisions in Amazon DynamoDB.
- C Create an AWS Glue workflow that has a human approval step. After the human technician review, integrate the application with an AWS Lambda function that calls the `SendTaskSuccess` API. Store all human technician review decisions in Amazon DynamoDB.
- D Configure Amazon EventBridge rules with custom event patterns to route AI recommendations to human technicians for review. Create AWS Glue jobs to process human technician approval queues. Use Amazon ElastiCache to cache all human technician review decisions.

Respuesta correcta: **B**

- A Create a custom approval workflow by using AWS Lambda functions and Amazon SQS queues for human review of AI recommendations. Store all review decisions in Amazon DynamoDB for audit purposes.
- B Create an AWS Step Functions workflow that has a human approval step that uses the `waitForTaskToken` API to pause execution. After a human technician completes a review, use an AWS Lambda function to call the `SendTaskSuccess` API that has the approval decision. Store all review decisions in Amazon DynamoDB.**
- C Create an AWS Glue workflow that has a human approval step. After the human technician review, integrate the application with an AWS Lambda function that calls the `SendTaskSuccess` API. Store all human technician review decisions in Amazon DynamoDB.
- D Configure Amazon EventBridge rules with custom event patterns to route AI recommendations to human technicians for review. Create AWS Glue jobs to process human technician approval queues. Use Amazon ElastiCache to cache all human technician review decisions.

Explicación

Opción A: Construir un flujo de aprobación personalizado con funciones AWS Lambda y colas Amazon SQS es técnicamente posible, pero requiere implementar manualmente toda la lógica de sondeo (polling), el seguimiento del estado de la aprobación y el reintento ante fallos. AWS no ofrece esto como un patrón nativo de orquestación con pausa, por lo que añade complejidad operativa y de mantenimiento innecesaria frente a una solución ya soportada de forma nativa por el servicio de orquestación.

Opción B (Correcta): AWS Step Functions admite de forma nativa el patrón de integración de servicio "Wait for a Callback with the Task Token" (identificado por el sufijo `.waitForTaskToken` en el recurso de la tarea). Cuando el flujo de trabajo llega a este paso, Step Functions genera un token de tarea único, lo entrega al recurso integrado (por ejemplo, una función Lambda que notifica al técnico, vía Amazon SNS o similar) y pausa la ejecución del state machine indefinidamente (hasta el tiempo límite configurado) sin consumir cómputo mientras espera. Cuando el técnico humano completa la revisión, una función Lambda invoca la API `SendTaskSuccess` (o `SendTaskFailure` en caso de rechazo) pasando el token de tarea junto con la decisión, lo que reanuda automáticamente la ejecución del flujo en el punto exacto donde se pausó. Este es precisamente el patrón documentado por AWS en el tutorial oficial "Deploying a workflow that waits for human approval in Step Functions", que despliega un ejemplo completo con Lambda, SNS y Step Functions. Almacenar cada decisión de revisión en Amazon DynamoDB cumple el requisito regulatorio de contar con un registro de auditoría duradero y consultable de todas las aprobaciones/rechazos humanos.

Opción C: AWS Glue es un servicio de integración y transformación de datos (ETL) orientado a trabajos por lotes; no está diseñado como orquestador de flujos de trabajo de aprobación y no ofrece un mecanismo nativo de pausa a la espera de una decisión humana. La mención de que un Glue workflow "llama a `SendTaskSuccess`" no corresponde a ninguna integración real documentada por AWS: esa API es específica de Step Functions y de sus integraciones de callback, no de AWS Glue.

Opción D: Amazon EventBridge permite enrutar eventos según patrones, pero no tiene capacidad de pausar y reanudar una ejecución en espera de una decisión externa; una vez publicado el evento, EventBridge no mantiene un "estado de espera". Usar AWS Glue para procesar colas de aprobación tampoco resuelve el problema de orquestación con pausa (ver opción C), y Amazon ElastiCache es una caché en memoria pensada para datos efímeros y de alto rendimiento, no un almacén duradero y auditable como el que exige la normativa para conservar decisiones de revisión humana.

Referencias:

- <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-human-approval.html>
- <https://docs.aws.amazon.com/lambda/latest/dg/with-step-functions.html>
- <https://docs.aws.amazon.com/step-functions/latest/dg/connect-lambda.html>

A bank is building a generative AI (GenAI) application that uses Amazon Bedrock to assess loan applications by using scanned financial documents. The application must extract structured data from the documents. The application must redact personally identifiable information (PII) before inference. The application must use foundation models (FMs) to generate approvals. The application must route low-confidence document extraction results to human reviewers who are within the same AWS Region as the loan applicant. The company must ensure that the application complies with strict Regional data residency and auditability requirements. The application must be able to scale to handle 25,000 applications each day and provide 99.9% availability. Which combination of solutions will meet these requirements? (Choose three.)

Selecciona 3 respuestas

- A Deploy Amazon Textract and Amazon Augmented AI (Amazon A2I) within the same Region to extract relevant data from the scanned documents. Route low-confidence pages to human reviewers.
- B Use AWS Lambda functions to detect and redact PII from submitted documents before inference. Apply Amazon Bedrock guardrails to prevent inappropriate or unauthorized content in model outputs. Configure Region-specific IAM roles to enforce data residency requirements and to control access to the extracted data.
- C Use Amazon Kendra and Amazon OpenSearch Service to extract field level values semantically from the uploaded documents before inference.
- D Store uploaded documents in Amazon S3 and apply object metadata. Configure IAM policies to store original documents within the same Region as each applicant. Enable object tagging for future audits.
- E Use AWS Glue Data Quality to validate the structured document data. Use AWS Step Functions to orchestrate a review workflow that includes a prompt engineering step that transforms validated data into optimized prompts before invoking Amazon Bedrock to assess loan applications.
- F Use Amazon SageMaker Clarify to generate fairness and bias reports based on model scoring decisions that Amazon Bedrock makes.

Respuesta correcta: A, B, E

- A** Deploy Amazon Textract and Amazon Augmented AI (Amazon A2I) within the same Region to extract relevant data from the scanned documents. Route low-confidence pages to human reviewers. ✓
- B** Use AWS Lambda functions to detect and redact PII from submitted documents before inference. Apply Amazon Bedrock guardrails to prevent inappropriate or unauthorized content in model outputs. Configure Region-specific IAM roles to enforce data residency requirements and to control access to the extracted data. ✓
- C** Use Amazon Kendra and Amazon OpenSearch Service to extract field level values semantically from the uploaded documents before inference.
- D** Store uploaded documents in Amazon S3 and apply object metadata. Configure IAM policies to store original documents within the same Region as each applicant. Enable object tagging for future audits.
- E** Use AWS Glue Data Quality to validate the structured document data. Use AWS Step Functions to orchestrate a review workflow that includes a prompt engineering step that transforms validated data into optimized prompts before invoking Amazon Bedrock to assess loan applications. ✓
- F** Use Amazon SageMaker Clarify to generate fairness and bias reports based on model scoring decisions that Amazon Bedrock makes.

Explicación

Opción A (Correcta): Amazon Textract extrae datos estructurados (campos, tablas, elementos de selección) de documentos escaneados mediante operaciones como `AnalyzeDocument`. Amazon Augmented AI (A2I) se integra de forma nativa con Textract mediante condiciones de activación (por ejemplo, `ImportantFormKeyConfidenceCheck`) que enrutan automáticamente a revisores humanos las páginas o campos cuya confianza de extracción esté por debajo de un umbral definido. La documentación de AWS indica explícitamente que, al usar A2I con Textract, los recursos de A2I deben crearse en la misma Región de AWS en la que se invoca `AnalyzeDocument`, lo cual satisface directamente el requisito de mantener a los revisores humanos dentro de la misma Región que el solicitante del préstamo.

Opción B (Correcta): funciones AWS Lambda pueden invocar Amazon Comprehend (mediante `StartPiiEntitiesDetectionJob` o la integración con S3 Object Lambda) para detectar y redactar entidades de información de identificación personal (PII) en los documentos antes de que estos se envíen a inferencia. Los guardrails de Amazon Bedrock incorporan una política de información sensible (`Sensitive Information Policy`) que puede detectar y enmascarar o bloquear PII, además de filtrar contenido dañino o intentos de prompt injection en las entradas y salidas del modelo, lo que ayuda a prevenir contenido inapropiado o no autorizado en las respuestas generadas. Combinado con roles de IAM configurados con condiciones específicas por Región (por ejemplo, restringiendo acciones a una Región concreta) y con el resto de la arquitectura Regional, se contribuye a cumplir los requisitos de residencia de datos y de control de acceso a los datos extraídos.

Opción C: Amazon Kendra (búsqueda empresarial semántica) y Amazon OpenSearch Service (indexación y búsqueda de texto completo/vectorial) están orientados a la búsqueda y recuperación de información sobre contenido ya indexado, no a la extracción estructurada campo a campo de formularios y documentos escaneados. Esa capacidad de extracción estructurada (texto, tablas, pares clave-valor) es la que proporciona específicamente Amazon Textract, por lo que Kendra y OpenSearch no cubren el flujo de extracción y revisión humana que exige el enunciado.

Opción D: almacenar los documentos originales en Amazon S3 con metadatos de objeto, políticas de IAM para mantener los documentos en la misma Región que cada solicitante y etiquetado de objetos (object tagging) es una buena práctica complementaria para auditoría y residencia de datos de los documentos en reposo, pero por sí sola no realiza la extracción de datos estructurados, ni la redacción de PII, ni la generación de decisiones de aprobación mediante modelos fundacionales; no cubre los requisitos funcionales centrales del caso de uso.

Opción E (Correcta): AWS Glue Data Quality permite definir y evaluar reglas de calidad sobre los datos estructurados ya extraídos (por ejemplo, completitud o formato de los campos del préstamo), mientras que AWS Step Functions orquesta el flujo de trabajo completo: valida los datos, ejecuta el paso de transformación a prompts optimizados y finalmente invoca Amazon Bedrock (mediante la integración nativa de Step Functions con la API de Bedrock) para generar la evaluación/aprobación de la solicitud de préstamo, aportando además trazabilidad y manejo de errores en cada paso, lo que favorece la auditabilidad exigida.

Opción F: Amazon SageMaker Clarify está diseñado para generar informes de sesgo (bias) y explicabilidad sobre modelos de machine learning tradicionales entrenados con datos tabulares y atributos protegidos definidos, típicamente en el contexto de SageMaker; no es la herramienta pensada para auditar decisiones generativas de texto producidas por Amazon Bedrock en un flujo documental como este, por lo que introduce complejidad adicional sin cubrir ningún requisito explícito del enunciado.

Referencias:

<https://docs.aws.amazon.com/sagemaker/latest/dg/a2i-textract-task-type.html>

<https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>

A software company is using Amazon Q Business to build an AI assistant that allows employees to access company information and personal information by using natural language prompts. The company stores this information in an Amazon S3 bucket. Each department in the company has a dedicated prefix in the S3 bucket. Each object name includes the S3 prefix of the department that it belongs to. Each department can belong to only a single group in AWS IAM Identity Center. Each employee belongs to a single department. The company configures Amazon Q Business to access data stored in an S3 bucket as a data source. The company needs to ensure that the AI assistant respects access controls based on the user's IAM Identity Center group membership. Which solution will meet this requirement with the LEAST operational overhead?

- A Create a JSON file named `acl.json` in each department folder. In each file, create access control entries that specify the IAM Identity Center group that should have access to that department's data. Indicate the location of the JSON file in the Access Control section of the data source settings.
- B Create a single JSON file named `acl.json` at the top level of the S3 bucket. Add access control entries that map each department's S3 prefix to its corresponding IAM Identity Center group. Indicate the location of the JSON file in the Access Control section of the data source settings.
- C For each IAM Identity Center group, create a separate permissions set that denies access to all prefixes in the S3 bucket. Add a `StringNotEquals` condition key to the permissions set for each group that specifies the department each group is associated with. Attach the permissions sets to the Identity Center groups.
- D Create a metadata file named `metadata.json` at the top level of the S3 bucket. Add an `AccessControlList` object to the file that specifies the S3 path of each department's prefix. Specify the IAM Identity Center group that should have access to each department's prefix. Reference the file location in the data source metadata settings.

Respuesta correcta: **B**

- A Create a JSON file named `acl.json` in each department folder. In each file, create access control entries that specify the IAM Identity Center group that should have access to that department's data. Indicate the location of the JSON file in the Access Control section of the data source settings.
- B Create a single JSON file named `acl.json` at the top level of the S3 bucket. Add access control entries that map each department's S3 prefix to its corresponding IAM Identity Center group. Indicate the location of the JSON file in the Access Control section of the data source settings. ✓**
- C For each IAM Identity Center group, create a separate permissions set that denies access to all prefixes in the S3 bucket. Add a `StringNotEquals` condition key to the permissions set for each group that specifies the department each group is associated with. Attach the permissions sets to the Identity Center groups.
- D Create a metadata file named `metadata.json` at the top level of the S3 bucket. Add an `AccessControlList` object to the file that specifies the S3 path of each department's prefix. Specify the IAM Identity Center group that should have access to each department's prefix. Reference the file location in the data source metadata settings.

Explicación

Opción A: crear un archivo `acl.json` distinto en cada carpeta de departamento no es el mecanismo soportado por el conector de S3 de Amazon Q Business. La documentación oficial especifica que el archivo de configuración de ACL es un único archivo JSON (referenciado mediante el parámetro `aclConfigurationFilePath`) que contiene un array con todas las entradas de control de acceso para todos los prefijos del bucket; multiplicar archivos por departamento añade complejidad de mantenimiento sin ningún beneficio funcional.

Opción B (Correcta): el conector de S3 de Amazon Q Business admite un archivo de configuración de control de acceso (comúnmente llamado `acl.json`) que se indica mediante el parámetro `aclConfigurationFilePath` en la configuración del origen de datos (o en la sección "Access control" de la consola). Ese archivo contiene una estructura JSON tipo array, donde cada elemento define un "keyPrefix" (por ejemplo, `s3://BUCKETNAME/prefix1/"`) junto con una lista de "aclEntries", cada una con "Name" (el nombre del usuario o grupo), "Type" (USER o GROUP) y "Access" (ALLOW o DENY). Al usar `Type=GROUP` con el nombre del grupo de IAM Identity Center correspondiente a cada departamento, un único archivo colocado en el mismo bucket (por ejemplo en la raíz) puede mapear todos los prefijos de departamento a sus respectivos grupos, cubriendo el requisito con el mínimo esfuerzo operativo. Nota importante de la documentación: cualquier prefijo que NO aparezca en el archivo ACL queda accesible para TODOS los usuarios, algo a tener en cuenta al diseñar el archivo.

Opción C: crear permission sets de IAM Identity Center con una condición `StringNotEquals` por prefijo, y asociarlos a los grupos, opera a nivel de gestión de acceso de AWS (IAM/SSO), no a nivel de filtrado de resultados dentro del índice de Amazon Q Business. Amazon Q Business necesita conocer la relación documento-grupo mediante su propio mecanismo de ACL (el archivo `aclConfigurationFilePath`) para poder filtrar las respuestas que devuelve el asistente según el usuario que pregunta; los permission sets de Identity Center no proporcionan esa información al conector ni son el método soportado.

Opción D: no existe en Amazon Q Business un archivo "metadata.json" con un objeto "AccessControlList" configurado en la "configuración de metadatos" del origen de datos. Los archivos de metadatos de documentos en el conector de S3 se usan para mapear atributos/campos personalizados de cada documento (document attributes), no para definir control de acceso; el mecanismo real y documentado para ACL es el archivo JSON referenciado a través de `aclConfigurationFilePath`, con la estructura `keyPrefix/aclEntries` descrita en la opción B.

Referencias:

<https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/s3-user-management.html>

<https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/s3-console.html>

<https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/s3-v2-api.html>

A healthcare company is using Amazon Bedrock to build a system to help practitioners make clinical decisions. The system must provide treatment recommendations to physicians based only on approved medical documentation and must cite specific sources. The system must not hallucinate or produce factually incorrect information. Which solution will meet these requirements with the LEAST operational overhead?

- A Integrate Amazon Bedrock with Amazon Kendra to retrieve approved documents. Implement custom post-processing to compare generated responses against source documents and to include citations.
- B Deploy an Amazon Bedrock knowledge base and connect it to approved clinical source documents. Use the Amazon Bedrock RetrieveAndGenerate API to return citations from the knowledge base.
- C Use Amazon Bedrock and Amazon Comprehend Medical to extract medical entities. Implement verification logic against a medical terminology database.
- D Use an Amazon Bedrock knowledge base with Retrieve API calls and InvokeModel API calls to retrieve approved clinical source documents. Implement verification logic to compare against retrieved sources and to cite sources.

Respuesta correcta: **B**

- A Integrate Amazon Bedrock with Amazon Kendra to retrieve approved documents. Implement custom post-processing to compare generated responses against source documents and to include citations.
- B Deploy an Amazon Bedrock knowledge base and connect it to approved clinical source documents. Use the Amazon Bedrock RetrieveAndGenerate API to return citations from the knowledge base.** ✓
- C Use Amazon Bedrock and Amazon Comprehend Medical to extract medical entities. Implement verification logic against a medical terminology database.
- D Use an Amazon Bedrock knowledge base with Retrieve API calls and InvokeModel API calls to retrieve approved clinical source documents. Implement verification logic to compare against retrieved sources and to cite sources.

Explicación

Opción A: integrar Amazon Bedrock con Amazon Kendra para recuperar los documentos aprobados y luego implementar post-procesamiento personalizado (comparación de la respuesta generada contra las fuentes y generación manual de citas) obliga a construir y mantener lógica adicional fuera de los servicios gestionados, lo que aumenta significativamente la sobrecarga operativa frente a una solución nativa.

Opción B (Correcta): al crear una base de conocimiento (knowledge base) de Amazon Bedrock conectada a los documentos clínicos aprobados y usar la API RetrieveAndGenerate, la recuperación de los fragmentos relevantes, el aumento del prompt con ese contexto y la generación de la respuesta se ejecutan en una única llamada gestionada por el servicio. Según la documentación oficial de AWS, "the responses are returned with citations to the original source data", es decir, RetrieveAndGenerate devuelve automáticamente las citas hacia las fuentes originales sin necesidad de lógica adicional. Esto reduce el riesgo de alucinación (las respuestas se fundamentan en los documentos recuperados) y minimiza la sobrecarga operativa, ya que no es necesario orquestar por separado la recuperación, la construcción del prompt ni la extracción de citas.

Opción C: Amazon Comprehend Medical permite extraer entidades médicas (medicamentos, condiciones, dosis, etc.) de texto no estructurado, pero no realiza recuperación aumentada de documentos ni genera citas de fuentes por sí mismo. Implementar verificación adicional contra una base de terminología médica requeriría desarrollo y mantenimiento propios, aumentando el esfuerzo operativo sin resolver el requisito de citación.

Opción D: usar la API Retrieve (que solo devuelve los fragmentos relevantes de la base de conocimiento) junto con InvokeModel (para generar la respuesta) por separado obliga al equipo a implementar manualmente la lógica de verificación de las respuestas contra las fuentes recuperadas y la generación de las citas, funcionalidad que la API RetrieveAndGenerate ya proporciona de forma nativa en una sola llamada. Por tanto, esta opción implica más esfuerzo de desarrollo que la opción B para lograr el mismo resultado.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-retrieve-generate.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerate.html

A financial services company is developing a real-time generative AI (GenAI) assistant to support human call center agents. The GenAI assistant must transcribe live customer speech, analyze context, and provide incremental suggestions to call center agents while a customer is still speaking. To preserve responsiveness, the GenAI assistant must maintain end-to-end latency under 1 second from speech to initial response display. The architecture must use only managed AWS services and must support bidirectional streaming to ensure that call center agents receive updates in real time. Which solution will meet these requirements?

- A Use the Amazon Transcribe streaming API to transcribe calls. Pass the text to Amazon Comprehend to perform sentiment analysis. Feed the results to Anthropic Claude on Amazon Bedrock by using the InvokeModel API. Store results in Amazon DynamoDB. Use a WebSocket API to display the results.
- B Use Amazon Transcribe streaming with partial results enabled to deliver fragments of transcribed text before customers finish speaking. Forward text fragments to Amazon Bedrock by using the InvokeModelWithResponseStream API. Stream responses to call center agents through an Amazon API Gateway WebSocket API.
- C Use Amazon Transcribe batch processing to convert calls to text. Pass complete transcripts to Anthropic Claude on Amazon Bedrock by using the ConverseStream API. Return responses through an Amazon Lex chatbot interface that call center agents can access from their work computers.
- D Use the Amazon Transcribe streaming API with an AWS Lambda function to transcribe each audio segment. Configure the Lambda function to call the Amazon Titan Embeddings model on Amazon Bedrock by using the InvokeModel API. Configure the Lambda function to publish results to an Amazon SNS topic. Subscribe the call center agents to the SNS topic.

Respuesta correcta: **B**

- A Use the Amazon Transcribe streaming API to transcribe calls. Pass the text to Amazon Comprehend to perform sentiment analysis. Feed the results to Anthropic Claude on Amazon Bedrock by using the InvokeModel API. Store results in Amazon DynamoDB. Use a WebSocket API to display the results.
- B Use Amazon Transcribe streaming with partial results enabled to deliver fragments of transcribed text before customers finish speaking. Forward text fragments to Amazon Bedrock by using the InvokeModelWithResponseStream API. Stream responses to call center agents through an Amazon API Gateway WebSocket API.** ✓
- C Use Amazon Transcribe batch processing to convert calls to text. Pass complete transcripts to Anthropic Claude on Amazon Bedrock by using the ConverseStream API. Return responses through an Amazon Lex chatbot interface that call center agents can access from their work computers.
- D Use the Amazon Transcribe streaming API with an AWS Lambda function to transcribe each audio segment. Configure the Lambda function to call the Amazon Titan Embeddings model on Amazon Bedrock by using the InvokeModel API. Configure the Lambda function to publish results to an Amazon SNS topic. Subscribe the call center agents to the SNS topic.

Explicación

Opción A: aunque usa la API de streaming de Amazon Transcribe, introduce un paso adicional con Amazon Comprehend para análisis de sentimiento que no se solicita en el requisito y añade latencia innecesaria a la cadena. Además, usa InvokeModel (respuesta única, no incremental) en lugar de una API de streaming de Bedrock, por lo que el modelo no puede emitir sugerencias parciales mientras el cliente sigue hablando. Combinar esto con persistencia en DynamoDB antes de mostrar resultados añade más retardo, dificultando cumplir el objetivo de menos de 1 segundo de latencia extremo a extremo.

Opción B (Correcta): Amazon Transcribe, en su API de streaming, genera resultados parciales (partial results) de forma continua a medida que llega el audio, y el marcador "IsPartial" indica si un segmento aún puede cambiar. Activar la estabilización de resultados parciales (partial-result stabilization) permite fijar las últimas palabras estables de cada fragmento antes de que el hablante termine el segmento completo, reduciendo la latencia percibida a costa de una pequeña pérdida de precisión, lo cual es idóneo para casos de baja latencia como este. Reenviar esos fragmentos de texto a Amazon Bedrock mediante la operación InvokeModelWithResponseStream permite que el modelo (por ejemplo, Anthropic Claude) genere y transmita su respuesta de forma incremental, token a token, en lugar de esperar a la respuesta completa. Finalmente, una API WebSocket de Amazon API Gateway proporciona una conexión persistente y bidireccional totalmente gestionada, ideal para empujar (push) esas actualizaciones incrementales hacia la interfaz del agente del centro de llamadas en tiempo real. Esta combinación de servicios totalmente gestionados (Transcribe streaming + Bedrock streaming + WebSocket API) es la arquitectura recomendada por AWS para minimizar la latencia extremo a extremo en escenarios de asistencia conversacional en vivo.

Opción C: el procesamiento por lotes (batch) de Amazon Transcribe requiere que la llamada completa finalice antes de generar la transcripción, lo que es incompatible con el requisito de sugerencias incrementales mientras el cliente aún está hablando y con la latencia menor a 1 segundo. Usar ConverseStream sobre transcripciones completas no soluciona el problema de fondo, ya que el cuello de botella está en la fase de transcripción por lotes. Además, un chatbot de Amazon Lex no es el canal adecuado para mostrar sugerencias en tiempo real a un agente humano durante una llamada activa.

Opción D: invocar el modelo de embeddings de Amazon Titan (Titan Embeddings) no es apropiado, ya que este modelo genera representaciones vectoriales de texto para búsqueda semántica o RAG, no respuestas conversacionales o sugerencias en lenguaje natural para el agente. Además, Amazon SNS es un servicio de mensajería de publicación/suscripción diseñado para notificaciones asíncronas de baja frecuencia, no para streaming bidireccional de baja latencia hacia una interfaz de usuario en vivo.

Referencias:

- <https://docs.aws.amazon.com/transcribe/latest/dg/streaming-partial-results.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_InvokeModelWithResponseStream.html
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>

A media company is launching a platform that allows thousands of users every hour to upload images and text content. The platform uses Amazon Bedrock to process the uploaded content to generate creative compositions. The company needs a solution to ensure that the platform does not process or produce inappropriate content. The platform must not expose personally identifiable information (PII) in the compositions. The solution must integrate with the company's existing Amazon S3 storage workflow. Which solution will meet these requirements with the LEAST infrastructure management overhead?

- A Enable the Enhanced Monitoring tool. Use an Amazon CloudWatch alarm to filter traffic to the platform. Use Amazon Comprehend PII detection to pre-process the data. Create a CloudWatch alarm to monitor for Amazon Comprehend PII detection events. Create an AWS Step Functions workflow that includes an Amazon Rekognition image moderation step.
- B Use an Amazon API Gateway HTTP API with request validation templates to screen content before storing the uploaded content in Amazon S3. Use Amazon SageMaker AI to build custom content moderation models that process content before sending the processed content to Amazon Bedrock.
- C Create an Amazon Cognito user pool that uses pre-authentication AWS Lambda functions to run content moderation checks. Use Amazon Textract to filter text content and Amazon Rekognition to filter image content before allowing users to upload content to the platform.
- D Create an AWS Step Functions workflow that uses built-in Amazon Bedrock guardrails to filter content. Use Amazon Comprehend PII detection to pre-process the content. Use Amazon Rekognition image moderation.

Respuesta correcta: **D**

- A Enable the Enhanced Monitoring tool. Use an Amazon CloudWatch alarm to filter traffic to the platform. Use Amazon Comprehend PII detection to pre-process the data. Create a CloudWatch alarm to monitor for Amazon Comprehend PII detection events. Create an AWS Step Functions workflow that includes an Amazon Rekognition image moderation step.
- B Use an Amazon API Gateway HTTP API with request validation templates to screen content before storing the uploaded content in Amazon S3. Use Amazon SageMaker AI to build custom content moderation models that process content before sending the processed content to Amazon Bedrock.
- C Create an Amazon Cognito user pool that uses pre-authentication AWS Lambda functions to run content moderation checks. Use Amazon Textract to filter text content and Amazon Rekognition to filter image content before allowing users to upload content to the platform.
- D Create an AWS Step Functions workflow that uses built-in Amazon Bedrock guardrails to filter content. Use Amazon Comprehend PII detection to pre-process the content. Use Amazon Rekognition image moderation.**

Explicación

Opción A: combinar "Enhanced Monitoring" con una alarma de Amazon CloudWatch para "filtrar tráfico" no es un mecanismo real de moderación de contenido. CloudWatch genera alarmas basadas en métricas y puede notificar o disparar una acción, pero no inspecciona ni bloquea el contenido de imágenes o texto antes de que se procese; por tanto no cumple el requisito de impedir que se procese o produzca contenido inapropiado.

Opción B: usar Amazon API Gateway con plantillas de validación de solicitudes solo valida la estructura/formato de la petición HTTP, no el contenido semántico de imágenes o texto, por lo que no detecta contenido inapropiado ni PII. Además, construir modelos de moderación personalizados con Amazon SageMaker AI exige entrenar, desplegar, escalar y mantener los propios modelos y la infraestructura de inferencia, lo que aumenta significativamente la carga operativa frente a usar servicios ya gestionados como Rekognition o Comprehend.

Opción C: Amazon Cognito con funciones Lambda de pre-autenticación gestiona la autenticación e identidad de los usuarios, no el contenido que suben; ese hook se ejecuta en el flujo de login, no sobre cada imagen o texto cargado. Además, Amazon Textract (según la documentación de AWS, es un servicio de extracción de texto e datos de documentos, OCR/formularios/tablas) no es un servicio de moderación de contenido dañino, por lo que no cumple el requisito de filtrar texto inapropiado.

Opción D (Correcta): según la documentación oficial de Amazon Bedrock, Amazon Bedrock Guardrails incluye filtros de contenido nativos que detectan y bloquean contenido dañino en texto e imágenes (content filters) y políticas de información sensible que detectan y enmascaran o bloquean PII (sensitive information / PII filters), pudiéndose invocar de forma independiente mediante la API ApplyGuardrail sin necesidad de invocar el modelo. Amazon Comprehend ofrece la operación DetectPiiEntities para detectar entidades de información personal identificable en texto. Amazon Rekognition ofrece DetectModerationLabels para detectar contenido inapropiado o no seguro en imágenes. Orquestar estos tres servicios gestionados con AWS Step Functions permite construir un flujo de moderación completo (texto e imagen, PII y contenido dañino) sin aprovisionar, entrenar ni administrar infraestructura propia, cumpliendo el requisito de mínima gestión de infraestructura y de integración con el flujo de trabajo existente en Amazon S3.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html>
<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>
<https://docs.aws.amazon.com/rekognition/latest/dg/procedure-moderate-images.html>

A company has set up Amazon Q Developer Pro licenses for all developers at the company. The company maintains a list of approved resources that developers must use when developing applications. The approved resources include internal libraries, proprietary algorithmic techniques, and sample code with approved styling. A new team of developers is using Amazon Q Developer to develop a new Java-based application. The company must ensure that the new developer team uses the company's approved resources. The company does not want to make project-level modifications. Which solution will meet these requirements?

- A Create a Git repository that contains all of the approved internal libraries, algorithms, and code samples. Include this Git repository in the application project locally as part of the workspace. Ensure that the developers use the `@workspace` context to retrieve suggestions from the Git repository.
- B In the project root folder, create a folder named `.amazonq/rules`. Add the approved internal libraries, algorithms, and code samples to the folder.
- C Create a folder in the application project named `rules`. Store the guidelines and code in the folder for Amazon Q Developer to reference product code suggestions.
- D Create an Amazon Q Developer customization that includes the approved data sources. Ensure that the developers use the customization to develop the application.

Respuesta correcta: **D**

- A Create a Git repository that contains all of the approved internal libraries, algorithms, and code samples. Include this Git repository in the application project locally as part of the workspace. Ensure that the developers use the @workspace context to retrieve suggestions from the Git repository.
- B In the project root folder, create a folder named .amazonq/rules. Add the approved internal libraries, algorithms, and code samples to the folder.
- C Create a folder in the application project named rules. Store the guidelines and code in the folder for Amazon Q Developer to reference product code suggestions.
- D Create an Amazon Q Developer customization that includes the approved data sources. Ensure that the developers use the customization to develop the application.** ✓

Explicación

Opción A: crear un repositorio Git con las bibliotecas, algoritmos y ejemplos aprobados e incluirlo localmente en el workspace del proyecto para usarlo mediante el contexto @workspace obliga a añadir y mantener manualmente ese repositorio en cada proyecto de cada desarrollador. Esto es exactamente el tipo de modificación a nivel de proyecto que la empresa quiere evitar, además de no ser un mecanismo soportado ni gestionado de forma centralizada por Amazon Q Developer.

Opción B: la carpeta .amazonq/rules, ubicada en la raíz del proyecto (project-root/.amazonq/rules), almacena "project rules" en archivos Markdown que Amazon Q Developer usa automáticamente como contexto cuando un desarrollador chatea con Amazon Q dentro de ese proyecto concreto (por ejemplo, para exigir el uso de type hints en Python o comentarios Javadoc en Java). Es una funcionalidad de reglas de chat por proyecto, no un repositorio de código fuente para generar sugerencias in-line, y por definición requiere crear y mantener esta carpeta en cada proyecto, lo cual constituye una configuración a nivel de proyecto y no la solución centralizada que pide el enunciado.

Opción C: crear una carpeta "rules" arbitraria dentro del propio proyecto no corresponde a ninguna funcionalidad reconocida de Amazon Q Developer (el nombre y la ubicación correctos, si se usara ese mecanismo, serían .amazonq/rules, no "rules"), y en cualquier caso también implicaría una modificación a nivel de proyecto que la empresa descarta explícitamente.

Opción D (Correcta): la funcionalidad de "customizations" (personalizaciones) de Amazon Q Developer permite a los administradores conectar de forma centralizada, mediante Amazon S3 o AWS CodeConnections (antes AWS CodeStar Connections), los repositorios de código propios de la empresa (bibliotecas internas, algoritmos propietarios y ejemplos de código con el estilo aprobado). Amazon Q Developer utiliza después ese repositorio personalizado, con la seguridad correspondiente habilitada, para generar sugerencias de código in-line alineadas con los patrones y estándares de la organización. Esta configuración se gestiona una sola vez a nivel de cuenta/organización y se aplica a todos los desarrolladores y proyectos que usen la personalización, sin necesidad de tocar la configuración de cada proyecto individualmente, que es justo lo que requiere el escenario.

Referencias:

<https://docs.aws.amazon.com/prescriptive-guidance/latest/best-practices-code-generation/advanced-capabilities.html>

<https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/context-project-rules.html>

An ecommerce company is using Amazon Bedrock to build a customer service AI assistant. The AI assistant needs to process over 50,000 customer inquiries every day. The AI assistant occasionally experiences traffic spikes of up to 150,000 inquiries every day during promotional events. Analysis shows that 40% of inquiries follow similar patterns that share the same context. A GenAI developer must design a solution that will ensure low latency and consistent performance for the AI assistant during traffic spikes. Which solution will meet these requirements MOST cost-effectively?

- A Configure latency-optimized inference by setting the latency parameter to optimized in the performance configuration of the request to Amazon Bedrock. Use prompt caching to handle the repetitive inquiries.
- B Purchase provisioned throughput and model units (MUs) that are sized to handle peak traffic loads. Use Amazon ElastiCache (Redis OSS) to cache repetitive inquiries.
- C Use Amazon Bedrock Agents and custom knowledge bases to pre-process customer inquiries. Configure cross-Region inference to distribute traffic.
- D Use AWS Lambda functions to pre-process requests by using a custom prompt routing mechanism. Use Amazon DynamoDB as a caching layer to handle frequently asked questions.

Respuesta correcta: A

- A** Configure latency-optimized inference by setting the latency parameter to optimized in the performance configuration of the request to Amazon Bedrock. Use prompt caching to handle the repetitive inquiries. ✓
- B** Purchase provisioned throughput and model units (MUs) that are sized to handle peak traffic loads. Use Amazon ElastiCache (Redis OSS) to cache repetitive inquiries.
- C** Use Amazon Bedrock Agents and custom knowledge bases to pre-process customer inquiries. Configure cross-Region inference to distribute traffic.
- D** Use AWS Lambda functions to pre-process requests by using a custom prompt routing mechanism. Use Amazon DynamoDB as a caching layer to handle frequently asked questions.

Explicación

Opción A (Correcta): configurar el parámetro "Latency" en "optimized" dentro de performanceConfig al invocar la API de Amazon Bedrock activa la inferencia optimizada para latencia (Latency Optimized Inference), una función que ofrece tiempos de respuesta más rápidos sin necesidad de configuración adicional ni de reentrenar/afinar el modelo, y sin tener que aprovisionar capacidad fija: se paga solo por el uso, y si se agota la cuota de latencia optimizada la solicitud simplemente cae a inferencia estándar (facturada a tarifa estándar) en lugar de fallar. Combinado con prompt caching -que reduce la latencia de respuesta y el coste de los tokens de entrada al reutilizar automáticamente (caché implícita) o mediante puntos de control explícitos (caché explícita) los prefijos de prompt que se repiten entre solicitudes-, esta solución aprovecha directamente que el 40% de las consultas comparten el mismo contexto, reduciendo coste y latencia en esos casos y absorbiendo los picos de tráfico de forma elástica sin sobreaprovisionar capacidad.

Opción B: comprar Provisioned Throughput y Model Units (MUs) dimensionados para el pico de 150,000 solicitudes diarias obliga a pagar de forma constante (por hora/mes) por una capacidad fija que solo se necesita durante los picos puntuales de eventos promocionales, quedando infrautilizada el resto del tiempo; añadir Amazon ElastiCache (Redis OSS) como capa de caché resuelve parcialmente la repetición de consultas, pero implica desplegar, operar y pagar por infraestructura adicional (nodos de caché) que prompt caching ya ofrece de forma nativa e integrada en Bedrock, por lo que en conjunto resulta menos rentable.

Opción C: Amazon Bedrock Agents y las bases de conocimiento personalizadas están orientados a orquestar tareas y enriquecer respuestas con recuperación de información (RAG), no a reducir la latencia de solicitudes repetidas ni a optimizar el coste por reutilización de contexto. La inferencia entre regiones (cross-Region inference) distribuye la carga entre regiones para mejorar disponibilidad y rendimiento durante picos, pero no aprovecha el patrón de contexto compartido del 40% de las consultas ni reduce directamente el coste por token, por lo que no es la opción más rentable.

Opción D: construir con AWS Lambda un enrutador de prompts personalizado y usar Amazon DynamoDB como capa de caché de preguntas frecuentes implica diseñar, desarrollar, probar y mantener lógica propia (invocaciones de Lambda, esquema de DynamoDB, lógica de invalidación de caché) que sustituye una capacidad ya integrada de forma nativa en Bedrock (prompt caching), añadiendo complejidad operativa y coste de desarrollo sin ofrecer ventajas de latencia frente a la solución nativa.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html>

A legal research company has a Retrieval Augmented Generation (RAG) application that uses Amazon Bedrock and Amazon OpenSearch Service. The application stores 768-dimensional vector embeddings for 15 million legal documents, including statutes, court rulings, and case summaries. The company's current chunking strategy segments text into fixed-length blocks of 500 tokens. The current chunking strategy often splits contextually linked information such as legal arguments, court opinions, or statute references across separate chunks. Researchers report that generated outputs frequently omit key context or cite outdated legal information. Recent application logs show a 40% increase in response times. The p95 latency metric exceeds 2 seconds. The company expects storage needs for the application to grow from 90 GB to 360 GB within a year. The company needs a solution to improve retrieval relevance and system performance at scale. Which solution will meet these requirements?

- A Increase the embedding vector dimensionality from 768 to 4,096 without changing the existing chunking or pre-processing strategy.
- B Replace dynamic retrieval with static, pre-written summaries that are stored in Amazon S3. Use Amazon CloudFront to serve the summaries to reduce compute demand and improve predictability.
- C Update the chunking strategy to use semantic boundaries such as complete legal arguments, clauses, or sections rather than fixed token limits. Regenerate vector embeddings to align with the new chunk structure.
- D Migrate from OpenSearch Service to Amazon DynamoDB. Implement keyword-based indexes to enable faster lookups for legal concepts.

Respuesta correcta: **C**

- A Increase the embedding vector dimensionality from 768 to 4,096 without changing the existing chunking or pre-processing strategy.
- B Replace dynamic retrieval with static, pre-written summaries that are stored in Amazon S3. Use Amazon CloudFront to serve the summaries to reduce compute demand and improve predictability.
- C Update the chunking strategy to use semantic boundaries such as complete legal arguments, clauses, or sections rather than fixed token limits. Regenerate vector embeddings to align with the new chunk structure.**
- D Migrate from OpenSearch Service to Amazon DynamoDB. Implement keyword-based indexes to enable faster lookups for legal concepts.

Explicación

Opción A: aumentar la dimensionalidad del embedding de 768 a 4.096 sin cambiar la estrategia de chunking no ataca la causa raíz del problema. El chunking de tamaño fijo (500 tokens) sigue fragmentando argumentos legales, cláusulas y referencias a estatutos en trozos separados, por lo que el modelo seguirá recuperando contexto incompleto independientemente de cuántas dimensiones tenga el vector. Además, vectores de mayor dimensionalidad incrementan significativamente el almacenamiento y el coste computacional de indexación/búsqueda en OpenSearch, agravando el problema de latencia ($p95 > 2s$) y el crecimiento previsto de almacenamiento (90 GB a 360 GB), en vez de mitigarlo.

Opción B: sustituir la recuperación dinámica por resúmenes estáticos precalculados almacenados en Amazon S3 y servidos vía Amazon CloudFront elimina la esencia de un sistema RAG, que es recuperar y fundamentar las respuestas en los documentos originales y vigentes. Esto no soluciona la fragmentación semántica del contexto (el problema de fondo) y, al depender de resúmenes fijos, introduce el riesgo de que la información legal citada quede desactualizada según cambien los estatutos o las resoluciones judiciales, justo el síntoma que los investigadores ya reportan.

Opción C (Correcta): actualizar la estrategia de chunking para usar límites semánticos (argumentos legales completos, cláusulas o secciones) en lugar de límites de tokens fijos, y regenerar los embeddings vectoriales alineados con la nueva estructura de chunks, es la solución respaldada por la documentación oficial de Amazon Bedrock Knowledge Bases. Bedrock ofrece explícitamente una estrategia de "semantic chunking" (chunking semántico) diseñada para dividir el texto en fragmentos significativos en función del contenido semántico en lugar de la estructura sintáctica, mejorando la precisión de recuperación (parámetros configurables como `maxTokens`, `bufferSize` y `breakpointPercentileThreshold`). Al preservar unidades de contexto coherentes —como un argumento legal completo o una cláusula—, se reduce la omisión de contexto clave y la citación de información desactualizada. La documentación también aclara que la estrategia de chunking no puede modificarse una vez conectada la fuente de datos, lo que implica que aplicar un nuevo chunking requiere reconfigurar la ingesta y regenerar los embeddings, exactamente lo que propone esta opción. Esto también ayuda a controlar el crecimiento de almacenamiento y la latencia, ya que chunks semánticamente coherentes y bien delimitados suelen reducir la necesidad de recuperar múltiples fragmentos redundantes o fragmentados para reconstruir el contexto.

Opción D: migrar de Amazon OpenSearch Service a Amazon DynamoDB con índices basados en palabras clave abandona la búsqueda vectorial por similitud semántica, que es la base de la precisión de recuperación en una arquitectura RAG. DynamoDB no está diseñado para búsquedas de similitud vectorial a gran escala como OpenSearch (que sí soporta motores de vectores como k-NN/OpenSearch Vector Engine), y una búsqueda basada en palabras clave no resuelve el problema de que la información relevante esté fragmentada entre varios chunks; de hecho, degradaría aún más la relevancia semántica de los resultados en un dominio tan dependiente de matices lingüísticos como el legal.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-customize-ingestion.html>

A company is developing a generative AI (GenAI)-powered customer support application that uses Amazon Bedrock foundation models (FMs). The application must maintain conversational context across multiple interactions with the same user. The application must run clarification workflows to handle ambiguous user queries. The company must store encrypted records of each user conversation to use for personalization. The application must be able to handle thousands of concurrent users while responding to each user quickly. Which solution will meet these requirements?

- A Use an AWS Step Functions Express workflow to orchestrate conversation flow. Invoke AWS Lambda functions to run clarification logic. Store conversation history in Amazon RDS and use session IDs as the primary key.
- B Use an AWS Step Functions Standard workflow to orchestrate clarification workflows. Include Wait for a Callback patterns to manage the workflows. Store conversation history in Amazon DynamoDB. Purchase on-demand capacity and configure server-side encryption.
- C Deploy the application by using an Amazon API Gateway REST API to route user requests to an AWS Lambda function to update and retrieve conversation context. Store conversation history in Amazon S3 and configure server-side encryption. Save each interaction as a separate JSON file.
- D Use AWS Lambda functions to call Amazon Bedrock inference APIs. Use Amazon SQS queues to orchestrate clarification steps. Store conversation history in an Amazon ElastiCache (Redis OSS) cluster. Configure encryption at rest.

Respuesta correcta: **B**

- A Use an AWS Step Functions Express workflow to orchestrate conversation flow. Invoke AWS Lambda functions to run clarification logic. Store conversation history in Amazon RDS and use session IDs as the primary key.
- B Use an AWS Step Functions Standard workflow to orchestrate clarification workflows. Include Wait for a Callback patterns to manage the workflows. Store conversation history in Amazon DynamoDB. Purchase on-demand capacity and configure server-side encryption. ✓**
- C Deploy the application by using an Amazon API Gateway REST API to route user requests to an AWS Lambda function to update and retrieve conversation context. Store conversation history in Amazon S3 and configure server-side encryption. Save each interaction as a separate JSON file.
- D Use AWS Lambda functions to call Amazon Bedrock inference APIs. Use Amazon SQS queues to orchestrate clarification steps. Store conversation history in an Amazon ElastiCache (Redis OSS) cluster. Configure encryption at rest.

Explicación

Opción A: un flujo de trabajo Express de AWS Step Functions está diseñado para ejecuciones de alto volumen y corta duración (hasta 5 minutos) con semántica "at-least-once", sin historial de ejecución completo ni capacidad de pausarse de forma duradera; por tanto no admite el patrón "Wait for a Callback with Task Token" necesario para suspender la conversación mientras se espera una aclaración del usuario. Además, usar Amazon RDS como almacén de historial de conversación obliga a gestionar el escalado, las conexiones y el particionado de una base de datos relacional para miles de usuarios concurrentes, lo que añade complejidad operativa innecesaria frente a una base de datos NoSQL gestionada.

Opción B (Correcta): un flujo de trabajo Standard de Step Functions ofrece ejecuciones duraderas, de larga duración y con semántica "exactly-once", e incluye de forma nativa el patrón de integración "Wait for a Callback with Task Token" (`.waitForTaskToken`), que pausa la máquina de estados hasta que un proceso externo (por ejemplo, la respuesta de aclaración del usuario) devuelve el token de tarea con éxito o error; esto encaja exactamente con el requisito de ejecutar flujos de aclaración ante consultas ambiguas. Amazon DynamoDB con capacidad bajo demanda (on-demand) escala automáticamente el rendimiento de lectura/escritura según la carga real sin necesidad de aprovisionar capacidad, lo que permite atender miles de usuarios concurrentes con baja latencia, y el cifrado en el servidor (SSE) protege en reposo el historial de conversación almacenado para fines de personalización, cumpliendo el requisito de mantener registros cifrados.

Opción C: usar Amazon API Gateway y AWS Lambda para actualizar y recuperar el contexto es viable como capa de acceso, pero no proporciona ningún mecanismo de orquestación con estado para ejecutar flujos de aclaración (no hay forma nativa de pausar y reanudar una interacción esperando una respuesta del usuario). Además, guardar cada interacción como un archivo JSON independiente en Amazon S3 dificulta actualizar y consultar de forma eficiente y en tiempo real el contexto conversacional acumulado, ya que S3 no está optimizado para lecturas/escrituras de baja latencia sobre registros individuales que cambian con frecuencia.

Opción D: Amazon SQS es un servicio de colas de mensajes y no ofrece de forma nativa un mecanismo de orquestación de flujo de trabajo con estados, pausas condicionadas ni el patrón de callback que Step Functions proporciona para gestionar aclaraciones; construir esa lógica sobre SQS exigiría desarrollo adicional considerable. Además, Amazon ElastiCache (Redis OSS) es una caché en memoria pensada para acceso de baja latencia a datos volátiles, no un almacén de datos duradero y persistente adecuado como fuente principal del historial de conversación con fines de personalización a largo plazo, aunque soporte cifrado en reposo.

Referencias:

<https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html>

<https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/step-functions-workflows.html>

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/on-demand-capacity-mode.html>

A financial services company needs to pre-process unstructured data such as customer transcripts, financial reports, and documentation. The company stores the unstructured data in Amazon S3 to support an Amazon Bedrock application. The company must validate data quality, create auditable metadata, monitor data metrics, and customize text chunking to optimize foundation model (FM) performance. Which solution will meet these requirements with the LEAST development effort?

- A Use Amazon SageMaker Data Wrangler to create a data flow. Configure Amazon CloudWatch metrics and alarms to monitor data quality. Use a custom AWS Lambda function to pre-process the data. Load processed data into Amazon Bedrock.
- B Set up an AWS Glue crawler to catalog data sources. Create AWS Glue ETL jobs to run custom transformation scripts. Use AWS Glue Data Quality to validate and monitor data quality. Load processed data into Amazon Bedrock.
- C Use Amazon Comprehend to extract entities. Create an AWS Lambda function to chunk text. Run Amazon Athena to query and validate data quality. Load processed data into Amazon Bedrock.
- D Create an AWS Step Functions workflow to orchestrate data pre-processing tasks. Run custom code on Amazon EC2 instances to process the data. Use Amazon SageMaker Model Monitor to monitor data quality. Load processed data into Amazon Bedrock.

Respuesta correcta: **B**

- A Use Amazon SageMaker Data Wrangler to create a data flow. Configure Amazon CloudWatch metrics and alarms to monitor data quality. Use a custom AWS Lambda function to pre-process the data. Load processed data into Amazon Bedrock.
- B Set up an AWS Glue crawler to catalog data sources. Create AWS Glue ETL jobs to run custom transformation scripts. Use AWS Glue Data Quality to validate and monitor data quality. Load processed data into Amazon Bedrock. ✓**
- C Use Amazon Comprehend to extract entities. Create an AWS Lambda function to chunk text. Run Amazon Athena to query and validate data quality. Load processed data into Amazon Bedrock.
- D Create an AWS Step Functions workflow to orchestrate data pre-processing tasks. Run custom code on Amazon EC2 instances to process the data. Use Amazon SageMaker Model Monitor to monitor data quality. Load processed data into Amazon Bedrock.

Explicación

Opción A: Amazon SageMaker Data Wrangler está pensado para la preparación y transformación de datos tabulares dentro de flujos de trabajo de machine learning (feature engineering), no para el procesamiento de texto no estructurado con reglas de calidad declarativas. Además, la solución exigiría desarrollar una función Lambda personalizada adicional solo para el pre-procesamiento/chunking, y configurar manualmente alarmas de CloudWatch para simular el monitoreo de calidad, lo que aumenta el esfuerzo de desarrollo frente a un servicio con validación de calidad nativa.

Opción B (Correcta): un rastreador (crawler) de AWS Glue cataloga automáticamente los datos de origen en el AWS Glue Data Catalog, generando metadatos auditables (esquema, particiones, linaje) sin esfuerzo manual. Los trabajos de AWS Glue ETL permiten ejecutar scripts de transformación personalizados, incluida la lógica de chunking de texto necesaria para optimizar el rendimiento del modelo fundacional. AWS Glue Data Quality, integrado de forma nativa en el mismo servicio, usa el lenguaje declarativo DQDL (Data Quality Definition Language, basado en el framework open-source Deequ) para definir y evaluar más de 25 reglas de calidad listas para usar, generar una puntuación de calidad de datos, detectar anomalías mediante ML y aislar los registros problemáticos. Todo esto se integra en una sola canalización gestionada y sin servidores, minimizando el esfuerzo de desarrollo antes de cargar los datos procesados en Amazon Bedrock.

Opción C: Amazon Comprehend permite extraer entidades de texto no estructurado, pero no ofrece un mecanismo declarativo de reglas de calidad de datos ni genera metadatos auditables por sí mismo. Habría que construir manualmente la lógica de chunking (vía Lambda) y de validación de calidad (vía consultas Athena), lo que implica un desarrollo considerablemente mayor que usar Glue Data Quality de forma nativa.

Opción D: orquestar el pre-procesamiento con AWS Step Functions ejecutando código personalizado en instancias EC2 obliga a aprovisionar, parchear y gestionar servidores, elevando notablemente el esfuerzo operativo. Además, Amazon SageMaker Model Monitor está diseñado para supervisar la deriva de datos y el rendimiento de modelos de ML ya desplegados en producción, no para validar la calidad de datos de origen no estructurados antes de su ingesta en Amazon Bedrock.

Referencias:

<https://docs.aws.amazon.com/glue/latest/dg/glue-data-quality.html>

<https://docs.aws.amazon.com/glue/latest/dg/dqdl.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-customize-ingestion.html>

A company uses Amazon Bedrock to build a Retrieval Augmented Generation (RAG) system. The RAG system uses an Amazon Bedrock knowledge base that is based on an Amazon S3 bucket as the data source for emergency news video content. The system retrieves transcripts, archived reports, and related documents from the S3 bucket. The RAG system uses state-of-the-art embedding models and a high-performing retrieval setup. However, users report slow responses and irrelevant results, which cause decreased user satisfaction. The company notices that vector searches are evaluating too many documents across too many content types and over long periods of time. The company determines that the underlying models will not benefit from additional fine tuning. The company must improve retrieval accuracy by applying smarter constraints. The company wants a solution that requires minimal changes to the existing architecture. Which solution will meet these requirements?

- A Enhance embeddings by using a domain-adapted model that is specifically trained on emergency news content for improved vector similarity.
- B Migrate to Amazon OpenSearch Service. Use vector fields and metadata filters to define the scope of results retrieval.
- C Enable metadata-aware filtering within the Amazon Bedrock knowledge base by indexing S3 object metadata.
- D Migrate to an Amazon Q Business index to perform structured metadata filtering and document categorization during retrieval.

Respuesta correcta: **C**

- A Enhance embeddings by using a domain-adapted model that is specifically trained on emergency news content for improved vector similarity.
- B Migrate to Amazon OpenSearch Service. Use vector fields and metadata filters to define the scope of results retrieval.
- C Enable metadata-aware filtering within the Amazon Bedrock knowledge base by indexing S3 object metadata.** ✓
- D Migrate to an Amazon Q Business index to perform structured metadata filtering and document categorization during retrieval.

Explicación

Opción A: entrenar o adaptar un modelo de embeddings específico para el dominio de noticias de emergencia es, en esencia, un proceso de ajuste (fine-tuning) del modelo que el enunciado descarta explícitamente al indicar que los modelos subyacentes no se beneficiarán de ajustes adicionales. Además, esto obligaría a regenerar los embeddings y reindexar todo el contenido existente, lo que supone un esfuerzo considerable y no resuelve directamente el problema de que la búsqueda vectorial evalúa demasiados documentos de tipos y periodos irrelevantes.

Opción B: migrar de la base de conocimiento de Amazon Bedrock a Amazon OpenSearch Service para usar campos vectoriales y filtros de metadatos de forma nativa implicaría sustituir el almacén vectorial subyacente, reconfigurar la ingesta de datos y adaptar la integración con Bedrock, lo que representa un cambio arquitectónico mayor y contradice el requisito explícito de minimizar los cambios sobre la arquitectura existente.

Opción C (Correcta): las bases de conocimiento (Knowledge Bases) de Amazon Bedrock con un origen de datos en Amazon S3 permiten adjuntar metadatos personalizados a cada documento mediante archivos complementarios ".metadata.json" (por ejemplo, "informe.pdf.metadata.json") que definen atributos como tipo de contenido, fecha de creación, autor u otras propiedades del objeto S3. Una vez indexados estos atributos, se pueden aplicar filtros de metadatos (mediante el parámetro "retrievalConfiguration" en las operaciones Retrieve/RetrieveAndGenerate, o de forma manual/implícita en la consola) para acotar el alcance de la búsqueda vectorial a un subconjunto relevante de documentos (por tipo de contenido, rango de fechas, etc.) antes de calcular la similitud semántica. Esto reduce el número de documentos irrelevantes evaluados, mejora la precisión y la latencia de la recuperación, y no requiere migrar el almacén vectorial ni reentrenar los modelos de embeddings, cumpliendo así el requisito de cambios mínimos sobre la arquitectura actual.

Opción D: migrar a un índice de Amazon Q Business implicaría reemplazar por completo la base de conocimiento de Bedrock por otro servicio con su propio modelo de datos, conectores y flujo de generación de respuestas, lo que supone cambios arquitectónicos sustanciales muy superiores a los que el enunciado busca evitar.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-ds-s3.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-test-config.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-neptune.html>

A financial services company is creating a Retrieval Augmented Generation (RAG) application that uses Amazon Bedrock to generate summaries of market activities. The application relies on a vector database that stores a small proprietary dataset that has a low index count. The application must perform similarity searches. The Amazon Bedrock model's responses must maximize accuracy and maintain high performance. The company needs to configure the vector database and integrate it with the application. Which solution will meet these requirements?

- A Launch an Amazon MemoryDB cluster and configure the index by using the Flat algorithm. Configure a horizontal scaling policy based on performance metrics.
- B Launch an Amazon MemoryDB cluster and configure the index by using the Hierarchical Navigable Small World (HNSW) algorithm. Configure a vertical policy based on performance metrics.
- C Launch an Amazon Aurora PostgreSQL cluster and configure the index by using the Inverted File with Flat Compression (IVFFlat) algorithm. Configure the instance class to scale to a larger size when the load increases.
- D Launch an Amazon DocumentDB cluster that has an Inverted File with Flat Compression (IVFFlat) index and a high probe value. Configure connections to the cluster as a replica set. Distribute reads to replica instances.

Respuesta correcta: A

- ☒ A Launch an Amazon MemoryDB cluster and configure the index by using the Flat algorithm. Configure a horizontal scaling policy based on performance metrics. ✓
- ☐ B Launch an Amazon MemoryDB cluster and configure the index by using the Hierarchical Navigable Small World (HNSW) algorithm. Configure a vertical policy based on performance metrics.
- ☐ C Launch an Amazon Aurora PostgreSQL cluster and configure the index by using the Inverted File with Flat Compression (IVFFlat) algorithm. Configure the instance class to scale to a larger size when the load increases.
- ☐ D Launch an Amazon DocumentDB cluster that has an Inverted File with Flat Compression (IVFFlat) index and a high probe value. Configure connections to the cluster as a replica set. Distribute reads to replica instances.

Explicación

Opción A (Correcta): según la documentación oficial de Amazon MemoryDB sobre búsqueda vectorial, el algoritmo Flat realiza un procesamiento de fuerza bruta (lineal) sobre cada vector del índice, devolviendo resultados exactos dentro de los límites de precisión del cálculo de distancia (k-NN exacto, sin aproximación). AWS indica explícitamente que, debido a este procesamiento lineal, los tiempos de ejecución pueden ser muy altos en índices grandes, pero para un dataset propietario pequeño con un recuento de índice bajo —como el del enunciado— Flat es la opción recomendada porque maximiza la precisión sin el coste de rendimiento que tendría en un índice de gran tamaño. Al ser MemoryDB una base de datos en memoria, además se obtiene una latencia muy baja. MemoryDB admite resharding en línea (online horizontal scaling), que permite añadir o quitar shards sin downtime, por lo que configurar una política de escalado horizontal basada en métricas de rendimiento es una forma válida y recomendada de ajustar la capacidad del clúster según la carga.

Opción B: HNSW (Hierarchical Navigable Small World) es, según la misma documentación de AWS, una alternativa que ofrece una aproximación de la respuesta correcta (ANN, approximate nearest neighbor) a cambio de tiempos de ejecución sustancialmente menores; está pensado para escalar bien en índices grandes, pero sacrifica algo de precisión (recall) frente al resultado exacto. Para un dataset pequeño con bajo recuento de índices, donde el enunciado exige maximizar la precisión, HNSW no es la mejor opción porque introduce aproximación innecesaria. Además, una política de escalado vertical (cambiar el tamaño de la instancia) es menos flexible y más disruptiva que el escalado horizontal para ajustar capacidad dinámicamente.

Opción C: Amazon Aurora PostgreSQL con la extensión pgvector y el índice IVFFlat (Inverted File with Flat Compression) es también un método de búsqueda aproximada: divide el espacio vectorial en particiones ("listas") y solo compara contra un subconjunto de ellas, lo que introduce pérdida de precisión respecto a una búsqueda exacta, y requiere ajustar el número de listas y sondas para balancear precisión y rendimiento. Aurora es una base de datos relacional en disco, por lo que no ofrece el mismo perfil de latencia en memoria que MemoryDB, y escalar la clase de instancia (escalado vertical) es menos ágil que el escalado horizontal para atender picos de carga.

Opción D: Amazon DocumentDB con un índice IVFFlat y un valor de "probe" alto es igualmente una técnica de aproximación (ANN), no una búsqueda exacta; aumentar el valor de probe mejora el recall pero incrementa el coste computacional sin llegar a igualar la exactitud de un algoritmo de fuerza bruta como Flat. Distribuir las lecturas entre instancias de réplica mejora el rendimiento de lectura, pero no resuelve el requisito principal del enunciado de maximizar la precisión en un dataset pequeño, por lo que esta opción no es la más adecuada.

Referencias:

<https://docs.aws.amazon.com/memorydb/latest/devguide/vector-search-overview.html>

<https://docs.aws.amazon.com/memorydb/latest/devguide/scaling-cluster.html>

<https://docs.aws.amazon.com/documentdb/latest/devguide/vector-search.html>

A GenAI developer is building a Retrieval Augmented Generation (RAG)-based customer support application that uses Amazon Bedrock foundation models (FMs). The application needs to process 50 GB of historical customer conversations that are stored in an Amazon S3 bucket as JSON files. The application must use the processed data as its retrieval corpus. The application's data processing workflow must extract relevant data from customer support documents, remove customer personally identifiable information (PII), and generate embeddings for vector storage. The processing workflow must be cost-effective and must finish within 4 hours. Which solution will meet these requirements with the LEAST operational overhead?

- A Use AWS Lambda and Amazon Comprehend to process files in parallel, remove PII, and call Amazon Bedrock APIs to generate vectors. Configure Lambda concurrency limits and memory settings to optimize throughput.
- B Create an AWS Glue ETL job to run PII detection scripts on the data. Use Amazon SageMaker Processing to run the HuggingFaceProcessor to generate embeddings by using a pre-trained model. Store the embeddings in Amazon OpenSearch Service.
- C Deploy an Amazon EMR cluster that runs Apache Spark with user-defined functions (UDFs) that call Amazon Comprehend to detect PII. Use Amazon Bedrock APIs to generate vectors. Store outputs in Amazon Aurora PostgreSQL with the pgvector extension.
- D Implement a data processing pipeline that uses AWS Step Functions to orchestrate a workload that uses Amazon Comprehend to detect PII and Amazon Bedrock to generate embeddings. Directly integrate the workflow with Amazon OpenSearch Serverless to store vectors and provide similarity search capabilities.

Respuesta correcta: **D**

- A Use AWS Lambda and Amazon Comprehend to process files in parallel, remove PII, and call Amazon Bedrock APIs to generate vectors. Configure Lambda concurrency limits and memory settings to optimize throughput.
- B Create an AWS Glue ETL job to run PII detection scripts on the data. Use Amazon SageMaker Processing to run the HuggingFaceProcessor to generate embeddings by using a pre-trained model. Store the embeddings in Amazon OpenSearch Service.
- C Deploy an Amazon EMR cluster that runs Apache Spark with user-defined functions (UDFs) that call Amazon Comprehend to detect PII. Use Amazon Bedrock APIs to generate vectors. Store outputs in Amazon Aurora PostgreSQL with the pgvector extension.
- D** Implement a data processing pipeline that uses AWS Step Functions to orchestrate a workload that uses Amazon Comprehend to detect PII and Amazon Bedrock to generate embeddings. Directly integrate the workflow with Amazon OpenSearch Serverless to store vectors and provide similarity search capabilities. ✓

Explicación

Opción A: usar AWS Lambda junto con Amazon Comprehend para procesar los archivos en paralelo es técnicamente posible, pero Lambda tiene un límite de tiempo de ejecución de 15 minutos por invocación y límites de memoria/almacenamiento efímero, por lo que procesar 50 GB de JSON en 4 horas obligaría a diseñar manualmente el troceado de archivos, la gestión de la concurrencia, los reintentos ante fallos parciales y el ensamblaje de resultados. Esa orquestación manual aumenta la sobrecarga operativa frente a una solución que orqueste el flujo de forma nativa.

Opción B: esta alternativa obliga a coordinar dos cargas de procesamiento por lotes distintas: un job de AWS Glue ETL para la detección de PII y, por separado, un job de Amazon SageMaker Processing (HuggingFaceProcessor) para generar los embeddings con un modelo preentrenado. Cada uno tiene su propio aprovisionamiento de recursos de cómputo, tiempos de arranque y monitorización independientes, lo que supone más componentes que gestionar y sincronizar que una orquestación serverless única, incrementando la sobrecarga operativa.

Opción C: desplegar un clúster de Amazon EMR con Apache Spark exige aprovisionar, dimensionar, parchear y monitorizar la infraestructura del clúster, además de mantener las funciones definidas por el usuario (UDFs) que invocan Amazon Comprehend. Esta gestión de infraestructura persistente va directamente en contra del requisito de mínima sobrecarga operativa, incluso si el trabajo pudiera completarse dentro de las 4 horas.

Opción D (Correcta): AWS Step Functions permite orquestar todo el flujo sin servidores que administrar. Step Functions ofrece integraciones optimizadas con Amazon Bedrock (por ejemplo, la API InvokeModel mediante el recurso `arn:aws:states:::bedrock:invokeModel`) para generar los embeddings de los documentos, e integraciones con el AWS SDK que permiten invocar directamente miles de acciones de API de más de 200 servicios de AWS —incluyendo Amazon Comprehend (por ejemplo, `DetectPiiEntities`)— para eliminar la información de identificación personal. Adicionalmente, mediante la tarea HTTP de Step Functions (recurso `arn:aws:states:::http:invoke`) el flujo puede llamar directamente al endpoint de la colección de Amazon OpenSearch Serverless para indexar los vectores y habilitar la búsqueda por similitud, sin necesidad de aprovisionar ni gestionar clústeres o servidores adicionales. Esta combinación cumple el objetivo de coste y el plazo de 4 horas con la menor sobrecarga operativa de las cuatro opciones planteadas.

Referencias:

<https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html>

<https://docs.aws.amazon.com/step-functions/latest/dg/integrate-services.html>

<https://docs.aws.amazon.com/step-functions/latest/dg/call-https-apis.html>

A financial services company is developing a generative AI (GenAI) application that serves both premium customers and standard customers. The application uses AWS Lambda functions behind an Amazon API Gateway REST API to process requests. The company needs to dynamically switch between AI models based on which customer tier each user belongs to. The company also wants to perform A/B testing for new features without redeploying code. The company needs to validate model parameters like temperature and maximum token limits before applying changes. Which solution will meet these requirements with the LEAST operational overhead?

- A Create an AWS Systems Manager Parameter Store parameters for each configuration. Use Lambda functions to poll for parameter updates. Use Amazon EventBridge events to trigger redeployments when configurations change.
- B Store model configurations in Amazon DynamoDB tables. Optimize access patterns to retrieve configurations according to customer tier. Configure Lambda functions to query DynamoDB at the beginning of each request to determine which model to use.
- C Use AWS AppConfig to manage model configurations. Use feature flags to perform A/B testing. Define JSON schema validation rules for model parameters. Configure Lambda functions to retrieve configurations by using the AWS AppConfig Agent.
- D Create an Amazon ElastiCache (Redis OSS) cluster to store model configurations. Set short TTL values. Run custom validation logic in Lambda functions. Use Amazon CloudWatch metrics to monitor configuration usage.

Respuesta correcta: C

- A Create an AWS Systems Manager Parameter Store parameters for each configuration. Use Lambda functions to poll for parameter updates. Use Amazon EventBridge events to trigger redeployments when configurations change.
- B Store model configurations in Amazon DynamoDB tables. Optimize access patterns to retrieve configurations according to customer tier. Configure Lambda functions to query DynamoDB at the beginning of each request to determine which model to use.
- C Use AWS AppConfig to manage model configurations. Use feature flags to perform A/B testing. Define JSON schema validation rules for model parameters. Configure Lambda functions to retrieve configurations by using the AWS AppConfig Agent.**
- D Create an Amazon ElastiCache (Redis OSS) cluster to store model configurations. Set short TTL values. Run custom validation logic in Lambda functions. Use Amazon CloudWatch metrics to monitor configuration usage.

Explicación

Opción A: AWS Systems Manager Parameter Store permite almacenar valores de configuración, pero no ofrece de forma nativa capacidades de feature flags para pruebas A/B ni validación de esquema de los parámetros del modelo (como temperature o límites de tokens). Además, la propuesta de que los Lambdas sondeen (polling) los cambios y que Amazon EventBridge dispare redeployments contradice directamente el requisito de la empresa de cambiar configuraciones sin necesidad de redespigar código, y añade complejidad operativa innecesaria (polling, orquestación de eventos, pipelines de despliegue).

Opción B: almacenar configuraciones de modelo en Amazon DynamoDB y optimizar los patrones de acceso por nivel de cliente es una solución viable para el enrutamiento dinámico, pero DynamoDB no ofrece de forma nativa feature flags para pruebas A/B ni validación de esquema de parámetros antes de aplicar los cambios. Toda esa lógica (validación, gestión de variantes de prueba, invalidación de caché) tendría que construirse y mantenerse manualmente, lo que incrementa la sobrecarga operativa frente a un servicio administrado diseñado para ese propósito.

Opción C (Correcta): AWS AppConfig es un servicio administrado creado específicamente para gestionar configuraciones de aplicación de forma dinámica y segura. Ofrece feature flags nativos que permiten realizar pruebas A/B y activar/desactivar funcionalidades sin redespigar código, y soporta validadores de tipo JSON Schema (y también validadores personalizados basados en funciones Lambda) que verifican la validez sintáctica y semántica de los parámetros del modelo (como temperature y los límites máximos de tokens) antes de que la configuración se despliegue, evitando así cambios incorrectos en producción. Además, el AWS AppConfig Agent (como extensión de Lambda) permite que las funciones Lambda recuperen la configuración desde una caché local en memoria a través de un endpoint HTTP local (localhost:2772), reduciendo las llamadas al servicio de AppConfig y la latencia, sin necesidad de redeploy cuando cambia la configuración. Esta combinación de capacidades nativas (feature flags, validación de esquema, distribución segura y gradual de configuración, y acceso en caché local desde Lambda) cubre todos los requisitos del enunciado —enrutamiento dinámico por nivel de cliente, pruebas A/B sin redeploy, y validación de parámetros del modelo— con la menor sobrecarga operativa posible, ya que evita construir manualmente lógica de validación, feature flags o mecanismos de despliegue.

Opción D: Amazon ElastiCache (Redis OSS) con TTLs cortos permite almacenar configuraciones con una expiración automática, pero requiere implementar manualmente toda la lógica de validación de parámetros (no hay validadores nativos de esquema) y no proporciona capacidades nativas de feature flags o pruebas A/B. También introduce la sobrecarga operativa adicional de aprovisionar, dimensionar y mantener un clúster de caché, lo que va en contra del objetivo de minimizar la carga operativa.

Referencias:

- <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-creating-configuration-and-profile-validators.html>
- <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-type-reference-feature-flags.html>
- <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-integration-lambda-extensions.html>

A company is using Amazon Bedrock and Anthropic Claude 3 Haiku to develop an AI assistant. The AI assistant normally processes 10,000 requests each hour but experiences surges of up to 30,000 requests each hour during peak usage periods. The AI assistant must respond within 2 seconds while operating across multiple AWS Regions. The company observes that during peak usage periods, the AI assistant experiences throughput bottlenecks that cause increased latency and occasional request timeouts. The company must resolve the performance issues. Which solution will meet this requirement?

- A Purchase provisioned throughput and sufficient model units (MUs) in a single Region. Configure the application to retry failed requests with exponential backoff.
- B Implement token batching to reduce API overhead. Use cross-Region inference profiles to automatically distribute traffic across available Regions.
- C Set up auto scaling AWS Lambda functions in each Region. Implement client-side round-robin request distribution. Purchase one model unit (MU) of provisioned throughput as a backup.
- D Implement batch inference for all requests by using Amazon S3 buckets across multiple Regions. Use Amazon SQS to set up an asynchronous retrieval process.

Respuesta correcta: **B**

- A Purchase provisioned throughput and sufficient model units (MUs) in a single Region. Configure the application to retry failed requests with exponential backoff.
- B Implement token batching to reduce API overhead. Use cross-Region inference profiles to automatically distribute traffic across available Regions.** ✓
- C Set up auto scaling AWS Lambda functions in each Region. Implement client-side round-robin request distribution. Purchase one model unit (MU) of provisioned throughput as a backup.
- D Implement batch inference for all requests by using Amazon S3 buckets across multiple Regions. Use Amazon SQS to set up an asynchronous retrieval process.

Explicación

Opción A: comprar Provisioned Throughput con suficientes model units (MUs) en una única región garantiza capacidad reservada y predecible, pero según la documentación de Amazon Bedrock los inference profiles (y por tanto el enrutamiento entre regiones) no son compatibles con Provisioned Throughput, y al concentrar toda la capacidad en una sola región no se resuelve el requisito de operar across multiple AWS Regions ni se gana la flexibilidad para absorber picos de 3x el tráfico normal (de 10,000 a 30,000 solicitudes/hora). Reintentar con exponential backoff solo retrasa las solicitudes fallidas, no aumenta la capacidad disponible ni evita los timeouts durante el pico.

Opción B (Correcta): los cross-Region inference profiles de Amazon Bedrock son una funcionalidad gestionada que enruta automáticamente las solicitudes de inferencia hacia otras regiones de AWS Commercial dentro del profile (geográfico o global) cuando la región de origen se satura, sin necesidad de aprovisionar capacidad fija por región ni gestionar el enrutamiento manualmente; el tráfico entre regiones viaja siempre por la red interna de AWS (cifrado, sin salir a internet público) y no añade coste adicional de enrutamiento. La documentación de AWS indica explícitamente que el uso de cross-Region inference es la opción recomendada cuando la disponibilidad y el throughput priman sobre el coste ("Mission-critical uptime → Cross-Region Inference"), precisamente el escenario descrito (picos que provocan cuellos de botella, aumento de latencia y timeouts).

Complementar esto con batching de solicitudes/tokens para reducir el número de llamadas y la sobrecarga de la API ayuda a mantener el requisito de respuesta en menos de 2 segundos incluso durante los picos de 30,000 solicitudes/hora.

Opción C: montar funciones Lambda con auto scaling propio en cada región y una distribución round-robin del lado del cliente reconstruye manualmente una capacidad de enrutamiento multi-región que Amazon Bedrock ya ofrece de forma nativa y gestionada mediante los inference profiles, añadiendo complejidad operativa innecesaria. Además, una sola model unit (MU) de Provisioned Throughput como respaldo es claramente insuficiente para absorber un pico de hasta 30,000 solicitudes por hora, por lo que el cuello de botella persistiría.

Opción D: la inferencia por lotes (batch inference) en Amazon Bedrock está diseñada para procesamiento asíncrono y no interactivo (trabajos de hasta 10,000 registros con una ventana de procesamiento de hasta 24 horas), pensada para casos como generación de datos de entrenamiento o moderación de contenido en background, no para solicitudes en tiempo real. Usar S3 y SQS para orquestar este flujo asíncrono no puede cumplir un requisito de respuesta interactiva en menos de 2 segundos por solicitud.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/capacity-limits-cost-optimization.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-support.html>

A company uses Amazon Bedrock to develop an AI assistant to provide customer support. Analysis shows that 40% of customer queries use varied phrasing or wording to ask the same questions. The company wants a solution to reduce redundant model calls. The solution must ensure that semantically equivalent questions receive consistent answers. The solution must ensure low latency. Which solution will meet these requirements?

- A Deploy an Amazon DynamoDB Accelerator (DAX) cluster as an in-memory cache. Specify the query text as the partition key and the model response text as the sort key. Query the cache by using a filter expression with the LIKE operator.
- B Use Amazon Bedrock to generate embeddings from customer queries. Use Amazon MemoryDB for Redis to store hash sets of vector embeddings and model responses. Use a RANGE query to find similar queries and their responses.
- C Deploy Amazon OpenSearch Service that has k-nearest neighbor (k-NN) capabilities to store query-response text pairs. Use an approximate k-NN technique to find similar queries and their responses.
- D Create a caching solution by using Amazon DynamoDB to create a global secondary index on the normalized query text. Apply stemming to incoming queries. Query the index of cached customer queries.

Respuesta correcta: **C**

- A Deploy an Amazon DynamoDB Accelerator (DAX) cluster as an in-memory cache. Specify the query text as the partition key and the model response text as the sort key. Query the cache by using a filter expression with the LIKE operator.
- B Use Amazon Bedrock to generate embeddings from customer queries. Use Amazon MemoryDB for Valkey to store hash sets of vector embeddings and model responses. Use a RANGE query to find similar queries and their responses.
- C** Deploy Amazon OpenSearch Service that has k-nearest neighbor (k-NN) capabilities to store query-response text pairs. Use an approximate k-NN technique to find similar queries and their responses. ✓
- D Create a caching solution by using Amazon DynamoDB to create a global secondary index on the normalized query text. Apply stemming to incoming queries. Query the index of cached customer queries.

Explicación

Opción A: Amazon DynamoDB Accelerator (DAX) es una caché en memoria que acelera las lecturas de DynamoDB mediante coincidencias exactas de clave (partición/orden) o consultas con operadores de comparación/filtrado sobre el texto almacenado. Usar el texto de la consulta como clave de partición y aplicar un filtro con el operador LIKE solo permite detectar coincidencias parciales de texto literal, pero no identifica preguntas semánticamente equivalentes formuladas con palabras distintas; por tanto no cumple el requisito de reconocer consultas semánticamente similares.

Opción B: Amazon MemoryDB for Valkey sí admite el almacenamiento de embeddings vectoriales (por ejemplo mediante hashes), pero una consulta de tipo RANGE está diseñada para recuperar elementos ordenados dentro de un rango de valores (como puntuaciones o marcas de tiempo), no para realizar búsquedas de similitud vectorial (k-NN) entre embeddings. Para hacer búsqueda semántica por proximidad de vectores en Valkey/Redis se necesitarían comandos de búsqueda vectorial dedicados, no un RANGE, por lo que esta opción no resuelve el caso de uso tal como está descrita.

Opción C (Correcta): Amazon OpenSearch Service ofrece capacidades nativas de k-nearest neighbor (k-NN) para indexar y buscar vectores de alta dimensionalidad. La solución consiste en generar embeddings de las preguntas de los clientes (por ejemplo con un modelo de Amazon Bedrock), almacenar los pares pregunta-respuesta junto con sus embeddings en un índice de OpenSearch, y usar una técnica k-NN aproximada (algoritmos como HNSW o IVF) para encontrar, con baja latencia, las consultas previas semánticamente más parecidas a una nueva pregunta, devolviendo la respuesta ya cacheada en lugar de invocar de nuevo al modelo. Esto es exactamente el patrón de "caché semántica" que AWS documenta como buena práctica para reducir invocaciones redundantes a modelos: generar un embedding de cada solicitud entrante y consultar un índice de OpenSearch (incluida la variante Serverless) en busca de solicitudes previas similares por encima de un umbral de similitud configurable (por ejemplo, similitud coseno > 0.95), sirviendo la respuesta cacheada cuando se supera ese umbral. Esto garantiza respuestas consistentes para preguntas equivalentes y reduce tanto la latencia como el número de llamadas al modelo.

Opción D: Un índice secundario global (GSI) de DynamoDB sobre el texto normalizado, incluso aplicando stemming, sigue basándose en coincidencias léxicas (mismas raíces de palabras o texto normalizado idéntico), no en similitud semántica real basada en el significado. Dos preguntas con el mismo significado pero vocabulario totalmente distinto no coincidirían tras el stemming, por lo que esta solución no detecta de forma fiable la reformulación semántica de preguntas.

Referencias: <https://docs.aws.amazon.com/wellarchitected/latest/agent-ai-lens/agentcost02-bp03.html> ; <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/semantic-search.html> ; <https://docs.aws.amazon.com/prescriptive-guidance/latest/migration-solr-opensearch/operational-architecture.html>

A company uses AWS Lambda functions to build an AI agent solution. A GenAI developer must set up a Model Context Protocol (MCP) server that accesses user information. The GenAI developer must also configure the AI agent to use the new MCP server. The GenAI developer must ensure that only authorized users can access the MCP server. Which solution will meet these requirements?

- A Use a Lambda function to host the MCP server. Grant the AI agent Lambda functions permission to invoke the Lambda function that hosts the MCP server. Configure the AI agent's MCP client to invoke the MCP server asynchronously.
- B Use a Lambda function to host the MCP server. Grant the AI agent Lambda functions permission to invoke the Lambda function that hosts the MCP server. Configure the AI agent to use the STDIO transport with the MCP server.
- C Use a Lambda function to host the MCP server. Create an Amazon API Gateway HTTP API that proxies requests to the Lambda function. Configure the AI agent solution to use the Streamable HTTP transport to make requests through the HTTP API. Use Amazon Cognito to enforce OAuth 2.1.
- D Use a Lambda layer to host the MCP server. Add the Lambda layer to the AI agent Lambda functions. Configure the agentic AI solution to use the STDIO transport to send requests to the MCP server. In the AI agent's MCP configuration, specify the Lambda layer ARN as the command. Specify the user credentials as environment variables.

Respuesta correcta: C

- A Use a Lambda function to host the MCP server. Grant the AI agent Lambda functions permission to invoke the Lambda function that hosts the MCP server. Configure the AI agent's MCP client to invoke the MCP server asynchronously.
- B Use a Lambda function to host the MCP server. Grant the AI agent Lambda functions permission to invoke the Lambda function that hosts the MCP server. Configure the AI agent to use the STDIO transport with the MCP server.
- C Use a Lambda function to host the MCP server. Create an Amazon API Gateway HTTP API that proxies requests to the Lambda function. Configure the AI agent solution to use the Streamable HTTP transport to make requests through the HTTP API. Use Amazon Cognito to enforce OAuth 2.1.**
- D Use a Lambda layer to host the MCP server. Add the Lambda layer to the AI agent Lambda functions. Configure the agentic AI solution to use the STDIO transport to send requests to the MCP server. In the AI agent's MCP configuration, specify the Lambda layer ARN as the command. Specify the user credentials as environment variables.

Explicación

Opción A: invocar la función Lambda que aloja el servidor MCP de forma asíncrona rompe el modelo síncrono de petición-respuesta que exige el protocolo MCP (el cliente MCP del agente necesita recibir una respuesta JSON-RPC inmediata a cada solicitud de herramienta, no un acuse de recibo diferido). Además, conceder a las funciones Lambda del agente permiso para invocar la función que aloja el servidor MCP solo controla qué recurso de AWS puede llamar a otro; no resuelve en absoluto la autorización de usuarios finales humanos, que es un requisito explícito del enunciado.

Opción B: según la especificación de MCP, el transporte STDIO está pensado para la comunicación local entre procesos que se ejecutan en la misma máquina (entrada/salida estándar), no para invocar de forma remota una función Lambda a través de la red. La guía prescriptiva de AWS sobre estrategias de hospedaje de MCP distingue explícitamente "Local hosting" (STDIO, sin necesidad de autenticación entre cliente y servidor porque todo ocurre en el mismo proceso/máquina) de "Remote hosting" (HTTP/HTTPS, con control de acceso centralizado). Usar STDIO contra una Lambda no es un patrón soportado ni ofrece ningún control de acceso de usuarios.

Opción C (Correcta): esta opción coincide con el "Deployment Pattern 2 – AWS Lambda + Amazon API Gateway" documentado por AWS para servidores MCP: la lógica del servidor MCP corre en Lambda y se expone como servidor MCP remoto a través de una API HTTP de Amazon API Gateway. Para servidores remotos, el protocolo MCP usa el transporte Streamable HTTP (JSON-RPC 2.0 sobre solicitudes POST HTTP), en contraposición a STDIO, reservado para procesos locales. Colocar Amazon Cognito delante para emitir y validar tokens OAuth 2.1 implementa la autorización de usuarios de forma estándar: el cliente MCP del agente obtiene un token de acceso de un grupo de usuarios (user pool) de Cognito y lo presenta como bearer token en cada solicitud a través de API Gateway, de modo que solo los usuarios autenticados y autorizados por Cognito pueden invocar el servidor MCP. La documentación de AWS para AgentCore Runtime con MCP también usa este mismo patrón de Cognito + token de acceso para proteger servidores MCP remotos con transporte Streamable HTTP.

Opción D: una capa de Lambda (Lambda Layer) es simplemente un paquete de código o dependencias añadido al entorno de ejecución de una función; no es un proceso ni un servicio de red independiente que pueda actuar como servidor MCP, ni algo que se pueda "invocar" mediante STDIO desde otra función. Especificar la ARN de una capa como "comando" en la configuración de un cliente MCP no es una configuración válida del protocolo, y colocar credenciales de usuario en variables de entorno de Lambda tampoco constituye un mecanismo de autorización por usuario: las variables de entorno son estáticas y compartidas por todas las invocaciones de la función, por lo que no permiten identificar, diferenciar ni revocar de forma segura el acceso de usuarios individuales.

Referencias:

- <https://docs.aws.amazon.com/prescriptive-guidance/latest/mcp-deployment-patterns-on-aws/deployment-pattern-2-aws-lambda-amazon-api-gateway.html>
- <https://docs.aws.amazon.com/prescriptive-guidance/latest/mcp-strategies/mcp-hosting-strategy.html>
- <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-mcp.html>

A company wants to create an annual rewards program for its customers. The rewards that customers earn vary based on different parameters such as the categories of the items ordered and the customers' purchase history. The company needs a generative AI (GenAI) solution that uses three Amazon Bedrock agents to help customers during online catalog browsing. The agents must use knowledge bases and action groups to handle the search, recommendation, and order modules. The modules must operate sequentially. An AWS Lambda function must calculate estimated rewards for each recommended item. The solution must provide graceful degradation during service disruptions. Which solution will meet these requirements with the MOST operational efficiency?

- A Define an Amazon API Gateway REST API behind each agent. Create a second Lambda function to orchestrate the calls to the agents and the rewards Lambda function. Configure the second Lambda function with a retry/fallback mechanism.
- B Create an AWS Step Functions state machine with four tasks that run the agents and the rewards Lambda function. Set up retry and catch branches for each of the task steps.
- C Configure each agent with a separate retry/fallback mechanism. Create a second Lambda function to orchestrate the calls to the agents and the rewards Lambda function. Define an Amazon API Gateway REST API behind the second Lambda function.
- D Create a second Lambda function to orchestrate the calls to the agents and the rewards Lambda function. Create an AWS Step Functions state machine with one task that runs the second Lambda function. Set up retry and catch branches for the task step.

Respuesta correcta: **B**

- A Define an Amazon API Gateway REST API behind each agent. Create a second Lambda function to orchestrate the calls to the agents and the rewards Lambda function. Configure the second Lambda function with a retry/fallback mechanism.
- B Create an AWS Step Functions state machine with four tasks that run the agents and the rewards Lambda function. Set up retry and catch branches for each of the task steps. ✓**
- C Configure each agent with a separate retry/fallback mechanism. Create a second Lambda function to orchestrate the calls to the agents and the rewards Lambda function. Define an Amazon API Gateway REST API behind the second Lambda function.
- D Create a second Lambda function to orchestrate the calls to the agents and the rewards Lambda function. Create an AWS Step Functions state machine with one task that runs the second Lambda function. Set up retry and catch branches for the task step.

Explicación

Opción A: crear una segunda función Lambda para orquestar las llamadas a los tres agentes de Bedrock y a la función Lambda de recompensas, colocando además una API Gateway REST detrás de cada agente, obliga a programar y mantener manualmente toda la lógica de secuenciación y de reintento/fallback que AWS Step Functions ya ofrece de forma declarativa (campos Retry y Catch en cada estado). Además, exponer cada agente detrás de su propia API Gateway añade componentes y superficie de mantenimiento innecesarios para un flujo puramente interno y secuencial.

Opción B (Correcta): una máquina de estados de AWS Step Functions con cuatro tareas (una por cada uno de los tres agentes de Bedrock —búsqueda, recomendación y pedido— y una para la función Lambda que calcula las recompensas), con ramas Retry y Catch configuradas en cada paso, resuelve el requisito de "operación secuencial" de forma nativa mediante el propio orden de los estados Task, y satisface la "degradación agraciada ante interrupciones del servicio" usando los mecanismos incorporados de Step Functions: Retry (reintentos con backoff exponencial configurable ante errores transitorios, incluidos States.ALL o errores específicos de servicio como Lambda.ServiceException) y Catch (enrutamiento a un estado de fallback cuando los reintentos se agotan), documentados en la guía oficial "Handling errors in Step Functions workflows". Esto se logra sin escribir ni mantener código de orquestación personalizado, por lo que es la solución de mayor eficiencia operativa. AWS documenta explícitamente este patrón de orquestar agentes de Amazon Bedrock mediante Step Functions en el escenario "Build and orchestrate generative AI applications with Amazon Bedrock and Step Functions".

Opción C: configurar un mecanismo de retry/fallback independiente en cada uno de los tres agentes y, adicionalmente, una Lambda orquestadora con una API Gateway detrás, duplica la responsabilidad de manejo de errores (una vez en cada agente y otra vez implícitamente en el orquestador) y multiplica los componentes a desplegar y mantener, en comparación con una única máquina de estados declarativa que centraliza tanto la secuenciación como el manejo de fallos por paso.

Opción D: envolver toda la orquestación (llamadas a los tres agentes más la Lambda de recompensas) dentro de una sola función Lambda, y ejecutar esa función como una única tarea de Step Functions con un solo bloque Retry/Catch, reduce la granularidad del manejo de errores: un fallo en cualquiera de los cuatro componentes internos se trata de forma indiferenciada a nivel de toda la tarea, en lugar de aplicarse políticas de reintento y captura específicas por agente o por la Lambda de recompensas. Esto disminuye la resiliencia y la trazabilidad de fallos puntuales, y sigue requiriendo mantener código de orquestación personalizado dentro de la Lambda, frente al enfoque totalmente declarativo de la opción B.

Referencias:

- <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
- https://docs.aws.amazon.com/bedrock/latest/userguide/bedrock-agent_example_cross_ServerlessPromptChaining_section.html
- <https://docs.aws.amazon.com/step-functions/latest/dg/sfn-best-practices.html>

A company is creating a workflow to review customer-facing communications before the company sends the communications. The company uses a pre-defined message template to generate the communications and stores the communications in an Amazon S3 bucket. The workflow needs to capture a specific portion from the template and send it to an Amazon Bedrock model. The workflow must store model responses back to the original S3 bucket. Which solution will meet these requirements?

- A Create a flow in Amazon Bedrock Flows. Configure S3 action nodes at the beginning and end of the flow to retrieve and store the communications and the model responses. In the middle of the flow, configure an expression to parse each communication. Configure an agent step to send the parsed input to the model for review.
- B Create an AWS Step Functions Express workflow state machine. Use an Amazon S3 integration GetObject step to retrieve the original communications. Use an intrinsic function Pass step to parse the communications and to pass the results to an Amazon Bedrock InvokeModel step. Configure an Amazon S3 integration PutObject step to store the model responses back to the S3 bucket.
- C Create an Amazon Bedrock agent that has an action group. Configure instructions to define how the agent should parse the communications. Configure the action group to retrieve the communications from the S3 bucket, invoke the Amazon Bedrock model, and store the model responses back to the S3 bucket.
- D Create an Amazon Bedrock agent that has a single action group. Configure three AWS Lambda functions in the action group. Configure the functions to retrieve the communications from the S3 bucket, parse the communications and invoke the Amazon Bedrock model, and store the model responses back to the S3 bucket.

Respuesta correcta: **B**

- A Create a flow in Amazon Bedrock Flows. Configure S3 action nodes at the beginning and end of the flow to retrieve and store the communications and the model responses. In the middle of the flow, configure an expression to parse each communication. Configure an agent step to send the parsed input to the model for review.
- B Create an AWS Step Functions Express workflow state machine. Use an Amazon S3 integration GetObject step to retrieve the original communications. Use an intrinsic function Pass step to parse the communications and to pass the results to an Amazon Bedrock InvokeModel step. Configure an Amazon S3 integration PutObject step to store the model responses back to the S3 bucket.** ✓
- C Create an Amazon Bedrock agent that has an action group. Configure instructions to define how the agent should parse the communications. Configure the action group to retrieve the communications from the S3 bucket, invoke the Amazon Bedrock model, and store the model responses back to the S3 bucket.
- D Create an Amazon Bedrock agent that has a single action group. Configure three AWS Lambda functions in the action group. Configure the functions to retrieve the communications from the S3 bucket, parse the communications and invoke the Amazon Bedrock model, and store the model responses back to the S3 bucket.

Explicación

Opción A: Amazon Bedrock Flows sí incluye nodos de almacenamiento en S3 ("S3 storage node" para escribir y "S3 retrieval node" para leer) y un nodo "Prompt" dedicado a invocar un modelo de Bedrock con una expresión de entrada para transformar los datos. Sin embargo, la opción propone usar un "agent step" (nodo de agente) para enviar el contenido parseado al modelo; el nodo de agente en Bedrock Flows está pensado para invocar un Agente de Bedrock completo, con su propia orquestación, memoria y herramientas, no para realizar una simple inferencia de un modelo (para eso existe el nodo Prompt). Usar el nodo equivocado introduce complejidad y comportamiento innecesarios frente al requisito de simplemente enviar una porción del contenido al modelo.

Opción B (Correcta): Un flujo de trabajo Express de AWS Step Functions puede orquestar todo el proceso mediante integraciones de servicio nativas, sin necesidad de código adicional. Un paso Task con la integración del SDK de AWS para "s3:getObject" recupera la comunicación original desde el bucket; un estado Pass puede usar funciones intrínsecas (por ejemplo, `States.StringToJson`) para parsear y transformar el contenido y preparar la entrada del modelo; a continuación, un paso Task con la integración optimizada de Amazon Bedrock ("bedrock:InvokeModel") envía ese fragmento al modelo — esta integración optimizada admite incluso los parámetros opcionales `Input.S3Uri` y `Output.S3Uri` para leer o escribir directamente en Amazon S3 si el payload es grande; finalmente, otro paso Task con la integración del SDK de AWS para "s3:putObject" guarda la respuesta del modelo de vuelta en el bucket original. Todo esto se define de forma declarativa en la máquina de estados, sin funciones Lambda intermedias ni la sobrecarga de un agente, lo que la convierte en la solución más simple, económica y determinista para un flujo lineal de leer-transformar-invocar-escribir.

Opción C: Un agente de Amazon Bedrock con un grupo de acciones está diseñado para razonar de forma autónoma sobre qué acciones ejecutar a partir de instrucciones en lenguaje natural. Usar un agente para orquestar la recuperación desde S3, la invocación del modelo y el guardado del resultado añade una capa de razonamiento no determinista e imprevisible a un proceso que, en esencia, es una secuencia fija de pasos (leer, parsear, invocar, escribir), resultando menos adecuado, más lento y más costoso que una integración de servicio directa como la de Step Functions.

Opción D: Definir tres funciones Lambda dentro de un único grupo de acciones de un agente de Bedrock para recuperar el archivo, parsear e invocar el modelo, y almacenar el resultado obliga a desarrollar y mantener código Lambda personalizado además de asumir la infraestructura y el comportamiento no determinista de un agente, cuando Step Functions ya ofrece integraciones nativas tanto con Amazon S3 (GetObject/PutObject) como con Amazon Bedrock (InvokeModel) que cubren exactamente estas necesidades sin escribir código adicional.

Referencias: <https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html> ; <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-gather-s3-info.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/flows-nodes.html>

A financial technology company is using Amazon Bedrock to build an assessment system for the company's customer service AI assistant. The AI assistant must provide financial recommendations that are factually accurate, compliant with financial regulations, and conversationally appropriate. The company needs to combine automated quality evaluations at scale with targeted human reviews of critical interactions. What solution will meet these requirements?

- A Configure a pipeline in which financial experts manually score all responses for accuracy, compliance, and conversational quality. Use Amazon SageMaker notebooks to analyze results to identify improvement areas.
- B Configure Amazon Bedrock evaluations that use Anthropic Claude Sonnet as a judge model to assess response accuracy and appropriateness. Configure custom Amazon Bedrock guardrails to check responses for compliance with financial policies. Add Amazon Augmented AI (Amazon A2I) human reviews for flagged critical interactions.
- C Create an Amazon Lex bot to manage the customer service interactions. Configure AWS Lambda functions to check responses against a static compliance database. Configure intents in the bot that call the Lambda functions to check the responses. Add an additional intent to collect end-user reviews.
- D Configure Amazon CloudWatch to monitor response patterns from the AI assistant. Configure CloudWatch alerts for potential compliance violations. Establish a team of human evaluators to review flagged interactions.

Respuesta correcta: **B**

- A Configure a pipeline in which financial experts manually score all responses for accuracy, compliance, and conversational quality. Use Amazon SageMaker notebooks to analyze results to identify improvement areas.
- B Configure Amazon Bedrock evaluations that use Anthropic Claude Sonnet as a judge model to assess response accuracy and appropriateness. Configure custom Amazon Bedrock guardrails to check responses for compliance with financial policies. Add Amazon Augmented AI (Amazon A2I) human reviews for flagged critical interactions.**
- C Create an Amazon Lex bot to manage the customer service interactions. Configure AWS Lambda functions to check responses against a static compliance database. Configure intents in the bot that call the Lambda functions to check the responses. Add an additional intent to collect end-user reviews.
- D Configure Amazon CloudWatch to monitor response patterns from the AI assistant. Configure CloudWatch alerts for potential compliance violations. Establish a team of human evaluators to review flagged interactions.

Explicación

Opción A: hacer que expertos financieros puntúen manualmente todas las respuestas no escala frente al volumen de interacciones de un asistente de atención al cliente y no combina evaluación automatizada a gran escala con revisión humana selectiva; usar notebooks de Amazon SageMaker para analizar los resultados es un paso de análisis posterior, no una solución de evaluación en sí, por lo que incumple el requisito del enunciado.

Opción B (Correcta): Amazon Bedrock ofrece trabajos de evaluación de modelos que usan un "LLM como juez" (evaluator model), donde un modelo como Anthropic Claude (Claude 3.5/3.7 Sonnet, Claude Sonnet 4, etc., todos soportados como modelos evaluadores) puntúa automáticamente y a escala las respuestas del modelo generador según métricas integradas o personalizadas (precisión, relevancia, utilidad, tono, etc.), devolviendo tanto la puntuación como una explicación. De forma complementaria, los guardrails personalizados de Amazon Bedrock (mediante políticas como temas denegados, filtros de contenido, filtros de palabras y comprobaciones de fundamentación contextual) permiten verificar que las respuestas cumplen las políticas de cumplimiento financiero de la empresa. Finalmente, Amazon Augmented AI (Amazon A2I) permite configurar flujos de revisión humana ("human review workflows"/"human loops") que se activan solo para las interacciones marcadas como críticas, aportando la revisión humana dirigida sin necesidad de revisar manualmente el 100% del tráfico. La combinación de estos tres servicios nativos de AWS cubre exactamente los tres requisitos del enunciado (precisión factual evaluada a escala, cumplimiento normativo verificado automáticamente, y revisión humana selectiva de casos críticos) con el mínimo esfuerzo de implementación.

Opción C: un bot de Amazon Lex con funciones AWS Lambda que comprueban las respuestas contra una base de datos estática de cumplimiento no ofrece ninguna evaluación de calidad conversacional ni de precisión factual generada por IA, y depender de que los propios usuarios finales dejen reseñas no equivale a un mecanismo de revisión humana dirigido específicamente a interacciones críticas; es un enfoque rígido, basado en reglas estáticas, que no escala ni se adapta bien a un caso de uso generativo.

Opción D: monitorizar patrones de respuesta con Amazon CloudWatch y generar alarmas ante posibles incumplimientos no constituye una evaluación automatizada de la precisión y el cumplimiento del contenido generado por el modelo; CloudWatch está pensado para métricas operativas y de sistema, no para analizar semánticamente la calidad o veracidad de una respuesta de IA generativa. Además, el enunciado indica que un equipo humano revisaría todas las interacciones marcadas, lo cual no sustituye a una evaluación automatizada real a escala.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-judge.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-denied-topics.html>

<https://docs.aws.amazon.com/sagemaker/latest/dg/a2i-use-augmented-ai-a2i-human-review-loops.html>

A healthcare company is using Amazon Bedrock to develop a real-time patient care AI assistant to respond to queries for separate departments that handle clinical inquiries, insurance verification, appointment scheduling, and insurance claims. The company wants to use a multi-agent architecture. The company must ensure that the AI assistant is scalable and can onboard new features for patients. The AI assistant must be able to handle thousands of parallel patient interactions. The company must ensure that patients receive appropriate domain-specific responses to queries. Which solution will meet these requirements?

- A Isolate data for each agent by using separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a supervisor agent to perform natural language intent classification on patient inquiries. Configure the supervisor agent to route queries to specialized collaborator agents to respond to department-specific queries. Configure each specialized collaborator agent to use Retrieval Augmented Generation (RAG) with the agent's department-specific knowledge base.
- B Create a separate supervisor agent for each department. Configure individual collaborator agents to perform natural language intent classification for each specialty domain within each department. Integrate each collaborator agent with department-specific knowledge bases only. Implement manual handoff processes between the supervisor agents.
- C Isolate data for each department in separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a single general-purpose agent. Configure multiple action groups within the general-purpose agent to perform specific department functions. Implement rule-based routing logic within the general-purpose agent instructions.
- D Implement multiple independent supervisor agents that run in parallel to respond to patient inquiries for each department. Configure multiple collaborator agents for each supervisor agent. Integrate all agents with the same knowledge base. Use external routing logic to merge responses from multiple supervisor agents.

Respuesta correcta: A

- A** Isolate data for each agent by using separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a supervisor agent to perform natural language intent classification on patient inquiries. Configure the supervisor agent to route queries to specialized collaborator agents to respond to department-specific queries. Configure each specialized collaborator agent to use Retrieval Augmented Generation (RAG) with the agent's department-specific knowledge base. ✓
- B** Create a separate supervisor agent for each department. Configure individual collaborator agents to perform natural language intent classification for each specialty domain within each department. Integrate each collaborator agent with department-specific knowledge bases only. Implement manual handoff processes between the supervisor agents.
- C** Isolate data for each department in separate knowledge bases. Use IAM filtering to control access to each knowledge base. Deploy a single general-purpose agent. Configure multiple action groups within the general-purpose agent to perform specific department functions. Implement rule-based routing logic within the general-purpose agent instructions.
- D** Implement multiple independent supervisor agents that run in parallel to respond to patient inquiries for each department. Configure multiple collaborator agents for each supervisor agent. Integrate all agents with the same knowledge base. Use external routing logic to merge responses from multiple supervisor agents.

Explicación

Opción A (Correcta): Según la documentación oficial de Amazon Bedrock Agents sobre "multi-agent collaboration", el patrón soportado nativamente es jerárquico: se designa un agente supervisor y se le asocian uno o más agentes colaboradores especializados por dominio. El agente supervisor recibe la consulta del usuario, determina de qué departamento o especialidad se trata (clasificación de intención en lenguaje natural) y enruta la solicitud al agente colaborador correspondiente. Cada agente del equipo, incluido el supervisor, puede tener sus propias herramientas, grupos de acciones, bases de conocimiento (Knowledge Bases) y guardrails, lo que permite aislar los datos de cada departamento (clínico, verificación de seguros, citas, reclamaciones) en bases de conocimiento independientes, controlar el acceso mediante políticas de IAM asociadas a cada agente/base de conocimiento, y aplicar RAG específico de cada dominio. Esta arquitectura es la que AWS recomienda para escalar y añadir nuevos dominios de forma independiente sin rediseñar todo el sistema, y soporta el volumen de interacciones paralelas requerido.

Opción B: crear un supervisor separado por cada departamento e invertir los roles (haciendo que los "colaboradores" sean quienes clasifiquen la intención dentro de cada especialidad) contradice el modelo jerárquico documentado, en el que es el supervisor quien clasifica y enruta, no los colaboradores. Además, los traspasos manuales entre múltiples supervisores introducen fricción operativa y no escalan a miles de interacciones paralelas en tiempo real.

Opción C: desplegar un único agente de propósito general con múltiples grupos de acciones (action groups) y lógica de enrutamiento basada en reglas dentro de las instrucciones del propio agente no aprovecha el patrón de colaboración multiagente de Bedrock; a medida que se añaden nuevos dominios, las instrucciones y la lógica de enrutamiento del agente único se vuelven cada vez más complejas y difíciles de mantener, y no se benefician del aislamiento ni de la especialización por dominio que ofrece tener agentes colaboradores independientes.

Opción D: ejecutar varios agentes supervisores independientes en paralelo que comparten la misma base de conocimiento y fusionar sus respuestas mediante lógica externa añade complejidad innecesaria, riesgo de respuestas inconsistentes entre supervisores, y no aísla los datos por departamento como exige el enunciado (que requiere bases de conocimiento separadas y control de acceso específico por dominio).

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/create-multi-agent-collaboration.html>

A company uses an AI assistant application to summarize the company's website content and provide information to customers. The company plans to use Amazon Bedrock to give the application access to a foundation model (FM). The company needs to deploy the AI assistant application to a development environment and a production environment. The solution must integrate the environments with the FM. The company wants to test the effectiveness of various FMs in each environment. The solution must provide product owners with the ability to easily switch between FMs for testing purposes in each environment. Which solution will meet these requirements?

- A Create one AWS CDK application. Create multiple pipelines in AWS CodePipeline. Configure each pipeline to have its own settings for each FM. Configure the application to invoke the Amazon Bedrock FMs by using the `aws_bedrock.ProvisionedModel.fromProvisionedModelArn()` method.
- B Create a separate AWS CDK application for each environment. Configure the applications to invoke the Amazon Bedrock FMs by using the `aws_bedrock.FoundationModel.fromFoundationModelId()` method. Create a separate pipeline in AWS CodePipeline for each environment.
- C Create one AWS CDK application. Configure the application to invoke the Amazon Bedrock FMs by using the `aws_bedrock.FoundationModel.fromFoundationModelId()` method. Create a pipeline in AWS CodePipeline pipeline that has a deployment stage for each environment that uses AWS CodeBuild deploy actions.
- D Create one AWS CDK application for the production environment. Configure the application to invoke the Amazon Bedrock FMs by using the `aws_bedrock.ProvisionedModel.fromProvisionedModelArn()` method. Create a pipeline in AWS CodePipeline. Configure the pipeline to deploy to the production environment by using an AWS CodeBuild deploy action. For the development environment, manually recreate the resources by referring to the production application code.

Respuesta correcta: C

- A Create one AWS CDK application. Create multiple pipelines in AWS CodePipeline. Configure each pipeline to have its own settings for each FM. Configure the application to invoke the Amazon Bedrock FMs by using the `aws_bedrock.ProvisionedModel.fromProvisionedModelArn()` method.
- B Create a separate AWS CDK application for each environment. Configure the applications to invoke the Amazon Bedrock FMs by using the `aws_bedrock.FoundationModel.fromFoundationModelId()` method. Create a separate pipeline in AWS CodePipeline for each environment.
- C Create one AWS CDK application. Configure the application to invoke the Amazon Bedrock FMs by using the `aws_bedrock.FoundationModel.fromFoundationModelId()` method. Create a pipeline in AWS CodePipeline pipeline that has a deployment stage for each environment that uses AWS CodeBuild deploy actions.**
- D Create one AWS CDK application for the production environment. Configure the application to invoke the Amazon Bedrock FMs by using the `aws_bedrock.ProvisionedModel.fromProvisionedModelArn()` method. Create a pipeline in AWS CodePipeline. Configure the pipeline to deploy to the production environment by using an AWS CodeBuild deploy action. For the development environment, manually recreate the resources by referring to the production application code.

Explicación

Opción A: usar `aws_bedrock.ProvisionedModel.fromProvisionedModelArn()` ata la aplicación a un ARN de Provisioned Throughput específico y fijo, lo que exige aprovisionar capacidad dedicada por cada modelo que se quiera probar y complica el cambio rápido entre FMs. Además, crear una pipeline distinta en AWS CodePipeline por cada modelo multiplica el número de pipelines a mantener sin necesidad real, ya que el objetivo es poder cambiar de modelo fácilmente, no aislar infraestructura por modelo.

Opción B: mantener una aplicación de AWS CDK independiente y una pipeline de AWS CodePipeline independiente para cada entorno (desarrollo y producción) duplica la definición de infraestructura como código. Cualquier cambio (por ejemplo, añadir soporte para un nuevo FM o ajustar la lógica de invocación) debe replicarse manualmente en ambas aplicaciones y pipelines, lo que aumenta el riesgo de desincronización entre entornos, en contra de las prácticas recomendadas de AWS CDK de reutilizar una misma definición de stacks para múltiples entornos mediante "stages".

Opción C (Correcta): una única aplicación de AWS CDK que invoque los modelos de Amazon Bedrock mediante `aws_bedrock.FoundationModel.fromFoundationModelId()` referencia modelos bajo demanda a partir de su identificador de modelo, un valor que puede parametrizarse fácilmente (por variable de entorno, contexto de CDK o parámetro de configuración) para cada etapa de despliegue, permitiendo a los product owners cambiar el FM usado en cada entorno sin modificar la lógica de la aplicación ni re-desplegar infraestructura de Provisioned Throughput. Esta única aplicación se despliega mediante una sola pipeline de AWS CodePipeline que define una etapa de despliegue por entorno (desarrollo y producción) usando acciones de AWS CodeBuild, lo que centraliza la gestión del ciclo de vida de CI/CD, evita la duplicación de pipelines y sigue el patrón recomendado por AWS CDK de usar "stages" para desplegar el mismo conjunto de stacks en múltiples entornos de forma consistente.

Opción D: construir la aplicación de AWS CDK únicamente para producción y "recrear manualmente" los recursos del entorno de desarrollo copiando a mano el código de producción elimina la reproducibilidad y la trazabilidad que proporciona la infraestructura como código. Esto introduce alto riesgo de desviación de configuración (configuration drift) entre entornos y errores humanos, exactamente lo contrario de una solución mantenible y fácil de sincronizar entre desarrollo y producción.

Referencias: <https://docs.aws.amazon.com/cdk/v2/guide/stages.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html>

A hotel company wants to enhance a legacy Java-based property management system (PMS) by adding AI capabilities. The company wants to use Amazon Bedrock Knowledge Bases to provide staff with room availability information and hotel-specific details. The solution must maintain separate access controls for each hotel that the company manages. The solution must provide room availability information in near real time and must maintain consistent performance during peak usage periods. Which solution will meet these requirements?

- A Deploy a single Amazon Bedrock knowledge base that contains combined data for all hotels. Configure AWS Lambda functions to synchronize data from each hotel's PMS database through direct API connections. Implement AWS CloudTrail logging with hotel-specific filters to audit access logs for each hotel's data.
- B Create an Amazon EventBridge rule for each hotel that is invoked by changes to the PMS database for each hotel. Configure the rule to send updates to a centralized Amazon Bedrock knowledge base in a management AWS account. Configure resource-based policies to enforce hotel-specific access controls for hotel staff.
- C Implement one Amazon Bedrock knowledge base for each hotel in a multi-account structure. Use direct data ingestion to provide real-time room availability information. Schedule regular synchronization for less critical information.
- D Build a centralized Amazon Bedrock agent that uses multiple knowledge bases. Implement AWS IAM Identity Center with hotel-specific permission sets to control hotel staff data access.

Respuesta correcta: C

- A Deploy a single Amazon Bedrock knowledge base that contains combined data for all hotels. Configure AWS Lambda functions to synchronize data from each hotel's PMS database through direct API connections. Implement AWS CloudTrail logging with hotel-specific filters to audit access logs for each hotel's data.
- B Create an Amazon EventBridge rule for each hotel that is invoked by changes to the PMS database for each hotel. Configure the rule to send updates to a centralized Amazon Bedrock knowledge base in a management AWS account. Configure resource-based policies to enforce hotel-specific access controls for hotel staff.
- C Implement one Amazon Bedrock knowledge base for each hotel in a multi-account structure. Use direct data ingestion to provide real-time room availability information. Schedule regular synchronization for less critical information.**
- D Build a centralized Amazon Bedrock agent that uses multiple knowledge bases. Implement AWS IAM Identity Center with hotel-specific permission sets to control hotel staff data access.

Explicación

Opción A: una única base de conocimiento de Amazon Bedrock con los datos combinados de todos los hoteles no ofrece controles de acceso independientes por hotel a nivel de recurso; todos los documentos comparten el mismo almacén vectorial y las mismas cuotas de servicio de la cuenta. Usar AWS CloudTrail únicamente para auditar accesos mediante filtros no restringe realmente qué personal puede consultar los datos de qué hotel, solo registra la actividad después de que ocurre; por tanto no cumple el requisito de controles de acceso separados por hotel.

Opción B: centralizar todas las actualizaciones en una única base de conocimiento alojada en una cuenta de gestión, aunque use EventBridge para disparar la sincronización y políticas basadas en recursos para restringir el acceso, sigue concentrando todos los hoteles en la misma base de conocimiento y en la misma cuenta de AWS. Esto implica que todos los hoteles comparten las mismas cuotas de servicio de Amazon Bedrock (por ejemplo, límites de ingesta y de consultas por segundo), de modo que un pico de uso en un hotel puede degradar el rendimiento del resto, incumpliendo el requisito de rendimiento consistente durante picos de uso.

Opción C (Correcta): Amazon Bedrock Knowledge Bases admite políticas de recursos (resource-based policies) para conceder acceso entre cuentas a las acciones `bedrock:Retrieve` y `bedrock:GetDocumentContent` sobre una base de conocimiento gestionada (managed), lo que permite que cada hotel tenga su propia base de conocimiento en su propia cuenta de AWS dentro de una estructura multicuenta, logrando aislamiento real de datos y controles de acceso independientes por hotel. Además, distribuir cada hotel en su propia cuenta sigue la práctica recomendada por AWS de usar una estrategia multicuenta para aislar las cuotas de servicio y reducir el "blast radius", de forma que el pico de actividad de un hotel no consuma la cuota ni degrade el rendimiento de otro. Para la disponibilidad de habitaciones en tiempo casi real, Amazon Bedrock Knowledge Bases permite la ingesta directa de documentos (a través de un data source de tipo "Custom" y las operaciones de la API `KnowledgeBaseDocuments`, como `IngestKnowledgeBaseDocuments`) sin necesidad de ejecutar un trabajo de sincronización (`StartIngestionJob`), lo que reduce la latencia de actualización. Para la información menos crítica y menos cambiante (por ejemplo, políticas del hotel o descripciones de servicios), se puede seguir usando sincronización programada mediante trabajos de ingesta convencionales. Esta combinación satisface simultáneamente el aislamiento por hotel, el rendimiento consistente en picos y la actualización casi en tiempo real de la disponibilidad.

Opción D: un agente centralizado de Amazon Bedrock que orquesta múltiples bases de conocimiento, combinado con AWS IAM Identity Center y permission sets específicos por hotel, puede controlar razonablemente qué personal accede a qué base de conocimiento. Sin embargo, todas las bases de conocimiento seguirían residiendo en la misma cuenta (o estructura de cuenta única), por lo que comparten las mismas cuotas de servicio de Amazon Bedrock; un pico de solicitudes de un hotel podría afectar el rendimiento de las consultas de otros hoteles, sin el aislamiento de rendimiento y escalado independiente que proporciona una arquitectura multicuenta real.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-cross-account.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-direct-ingestion.html>
- <https://docs.aws.amazon.com/lambda/latest/dg/concepts-application-design.html>

A company is implementing a serverless inference API by using AWS Lambda. The API will dynamically invoke multiple AI models hosted on Amazon Bedrock. The company needs to design a solution that can switch between model providers without modifying or redeploying Lambda code in real time. The design must include safe rollout of configuration changes and validation and rollback capabilities. Which solution will meet these requirements?

- A Store the active model provider in AWS Systems Manager Parameter Store. Configure a Lambda function to read the parameter at runtime to determine which model to invoke.
- B Store the active model provider in AWS AppConfig. Configure a Lambda function to read the configuration at runtime to determine which model to invoke.
- C Configure an Amazon API Gateway REST API to route requests to separate Lambda functions. Hardcode each Lambda function to a specific model provider. Switch the integration target manually.
- D Store the active model provider in a JSON file hosted on Amazon S3. Use AWS AppConfig to reference the S3 file as a hosted configuration source. Configure a Lambda function to read the file through AppConfig at runtime to determine which model to invoke.

Respuesta correcta: **B**

- A Store the active model provider in AWS Systems Manager Parameter Store. Configure a Lambda function to read the parameter at runtime to determine which model to invoke.
- B Store the active model provider in AWS AppConfig. Configure a Lambda function to read the configuration at runtime to determine which model to invoke.** ✓
- C Configure an Amazon API Gateway REST API to route requests to separate Lambda functions. Hardcode each Lambda function to a specific model provider. Switch the integration target manually.
- D Store the active model provider in a JSON file hosted on Amazon S3. Use AWS AppConfig to reference the S3 file as a hosted configuration source. Configure a Lambda function to read the file through AppConfig at runtime to determine which model to invoke.

Explicación

Opción A: AWS Systems Manager Parameter Store permite almacenar el proveedor de modelo activo como un valor de configuración que Lambda puede leer en tiempo de ejecución, evitando así modificar o redesplegar el código. Sin embargo, Parameter Store no ofrece de forma nativa validadores de esquema, estrategias de despliegue gradual (por porcentaje o de forma exponencial) ni rollback automático basado en alarmas de CloudWatch; toda esa lógica de validación y reversión segura tendría que construirse manualmente. Por tanto, no cumple los requisitos de "safe rollout" y "validation and rollback" que pide el enunciado.

Opción B (Correcta): AWS AppConfig está diseñado específicamente para gestionar configuración de aplicaciones en producción sin necesidad de redesplegar código, incluyendo tres capacidades clave que exige el enunciado: (1) Validadores, que comprueban que los datos de configuración sean sintáctica y semánticamente correctos antes de desplegarlos; (2) Estrategias de despliegue (deployment strategies), que permiten un rollout gradual y controlado del cambio de configuración (por ejemplo, estrategias predefinidas como AppConfig.Linear20PercentEvery6Minutes o AppConfig.AllAtOnce con monitorización de 10 minutos); y (3) Rollback automático, integrado con Amazon CloudWatch, que revierte automáticamente la configuración a la versión anterior si una alarma de CloudWatch entra en estado ALARM durante el despliegue. Un Lambda puede leer la configuración activa (por ejemplo, el proveedor de modelo de Bedrock a invocar) en tiempo de ejecución a través de AppConfig (directamente vía SDK o mediante la AppConfig Lambda extension/agent), cambiando el comportamiento sin tocar ni redesplegar el código de la función. Esto satisface de forma directa y nativa todos los requisitos del enunciado.

Opción C: enrutar mediante Amazon API Gateway a funciones Lambda distintas, cada una con el proveedor de modelo fijado (hardcoded) en el código, obliga a modificar y redesplegar código cada vez que se quiera cambiar de proveedor, e implica cambios manuales de la integración del API Gateway. Esto incumple directamente el requisito de "switch between model providers without modifying or redeploying Lambda code in real time", y tampoco ofrece mecanismos de validación ni rollback.

Opción D: usar un archivo JSON alojado en Amazon S3 como fuente de configuración (hosted configuration source) referenciada por AWS AppConfig es una configuración válida y soportada (AppConfig puede obtener datos desde S3, Secrets Manager, Parameter Store o su propio almacén hospedado), y en teoría también se beneficiaría de los validadores, las estrategias de despliegue y el rollback automático de AppConfig. Sin embargo, añade una capa adicional de indirección y complejidad operativa (gestionar y versionar el archivo en S3, permisos de acceso, sincronización) sin aportar ninguna ventaja adicional frente a usar directamente el almacén de configuración nativo (hosted configuration store) de AppConfig, que es más simple y igualmente cumple todos los requisitos. Por eso, entre B y D, la opción B es la solución más directa y eficiente.

Referencias: <https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html> ; <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-creating-deployment-strategy.html> ; <https://docs.aws.amazon.com/appconfig/latest/userguide/monitoring-deployments.html>

A company is building a generative AI (GenAI) application that uses Amazon Bedrock APIs to process complex customer inquiries. During peak usage periods, the application experiences intermittent API timeouts that cause issues such as broken response chunks and delayed data delivery. The application struggles to ensure that prompts remain within token limits when handling complex customer inquiries of varying lengths. Users have reported truncated inputs and incomplete responses. The company has also observed foundation model (FM) invocation failures. The company needs a retry strategy that automatically handles transient service errors and prevents overwhelming Amazon Bedrock during peak usage periods. The strategy must also adapt to changing service availability and support response streaming and token-aware request handling. Which solution will meet these requirements?

- A Implement a standard retry strategy that uses a 1-second fixed delay between attempts and a 3-retry maximum for all errors. Handle streaming response timeouts by restarting streams. Cap token usage for each session.
- B Implement an adaptive retry strategy that uses exponential backoff with jitter and a circuit breaker pattern that temporarily disables retries when error rates exceed a predefined threshold. Implement a streaming response handler that monitors for chunk delivery timeouts. Configure the handler to buffer successfully received chunks and intelligently resume streaming from the last received chunk when connections are re-established.
- C Use the AWS SDK to configure a retry strategy in standard mode. Wrap Amazon Bedrock API calls in try-catch blocks that handle timeout exceptions. Return cached completions for failed streaming requests. Enforce a global token limit for all users. Add jitter-based retry logic and lightweight token trimming for each request. Resume broken streams by requesting only the missing chunks from the point of failure. Maintain a small in-memory buffer of the most recent chunks to minimize redundant data transfer.
- D Set Amazon Bedrock client request timeouts to 30 seconds. Implement client-side load shedding. Buffer partial results and stop new requests when the application performance begins to degrade. Set static token usage caps for all requests. Configure exponential backoff retries, dynamic chunk sizing, and context-aware token limits.

Respuesta correcta: C

- A Implement a standard retry strategy that uses a 1-second fixed delay between attempts and a 3-retry maximum for all errors. Handle streaming response timeouts by restarting streams. Cap token usage for each session.
- B Implement an adaptive retry strategy that uses exponential backoff with jitter and a circuit breaker pattern that temporarily disables retries when error rates exceed a predefined threshold. Implement a streaming response handler that monitors for chunk delivery timeouts. Configure the handler to buffer successfully received chunks and intelligently resume streaming from the last received chunk when connections are re-established.
- C Use the AWS SDK to configure a retry strategy in standard mode. Wrap Amazon Bedrock API calls in try-catch blocks that handle timeout exceptions. Return cached completions for failed streaming requests. Enforce a global token limit for all users. Add jitter-based retry logic and lightweight token trimming for each request. Resume broken streams by requesting only the missing chunks from the point of failure. Maintain a small in-memory buffer of the most recent chunks to minimize redundant data transfer.**
- D Set Amazon Bedrock client request timeouts to 30 seconds. Implement client-side load shedding. Buffer partial results and stop new requests when the application performance begins to degrade. Set static token usage caps for all requests. Configure exponential backoff retries, dynamic chunk sizing, and context-aware token limits.

Explicación

Opción A: un retraso fijo de 1 segundo con un máximo de 3 reintentos aplicado a todos los tipos de error no distingue errores transitorios (timeouts de red) de errores de throttling, cuando la documentación de AWS indica que el modo estándar de reintento de los SDK ya aplica retrasos más cortos para errores transitorios y más largos para errores de limitación ("error-type-specific backoff"). Reiniciar el stream completo ante un corte descarta todo el progreso ya recibido, y un backoff fijo sin jitter ni ajuste dinámico no se adapta a la disponibilidad cambiante del servicio durante los picos de uso. Por tanto, no cumple los requisitos del enunciado.

Opción B: combina una estrategia de reintento adaptativa (backoff exponencial con jitter) con un patrón de circuit breaker que desactiva temporalmente los reintentos cuando la tasa de error supera un umbral, además de un manejador de streaming que almacena en búfer los fragmentos recibidos y reanuda desde el último fragmento correctamente entregado. Este diseño coincide de forma muy cercana con la documentación oficial de AWS: la guía de referencia de los SDK describe el modo "adaptativo" como aquel que añade un limitador de tasa del lado del cliente que ralentiza automáticamente el envío de solicitudes cuando el servicio señala throttling, recomendado explícitamente para cargas de trabajo de IA que invocan una única operación de API a alto volumen (el caso descrito en el enunciado). El AWS Well-Architected Framework (Agentic AI Lens, AGENTPERF06-BP02) también recomienda backoff exponencial con jitter combinado con un "automatic cutoff" (circuit breaker) que abre el circuito al superar un umbral de fallos. Por esta cercanía con la documentación, existen dudas fundadas de que esta sea, en realidad, la opción más correcta (ver campo de duda/concern).

Opción C (marcada como correcta en el material original): configurar el SDK de AWS en modo de reintento estándar (que ya incorpora backoff exponencial con jitter aleatorizado, sin necesidad de "añadir" jitter por separado) junto con manejo de excepciones de timeout, recorte de tokens por solicitud y un búfer en memoria de los fragmentos más recientes es una combinación razonable, pero incluye elementos que no están claramente respaldados por la documentación de Bedrock o que resultan internamente contradictorios. "Devolver completions cacheadas para solicitudes de streaming fallidas" no tiene sentido si cada consulta de cliente es distinta, salvo que se use como respaldo tras abrir explícitamente un circuit breaker (mecanismo que esta opción no menciona). "Imponer un límite de tokens global para todos los usuarios" contradice el requisito de manejo de tokens adaptado a la longitud/complejidad de cada consulta, y es inconsistente con el "recorte ligero de tokens por solicitud" que la misma opción menciona a continuación. Además, "solicitar solo los fragmentos faltantes desde el punto de fallo" sugiere una capacidad de reanudación de streaming por rangos que las API de streaming de Amazon Bedrock no exponen de forma nativa: ante un corte, un cliente típicamente debe reemitir la solicitud completa y fusionar/descartar lo ya almacenado en búfer, no solicitar al servicio "solo lo que falta". Estos matices generan dudas razonables sobre si esta opción es realmente superior a la B frente a la documentación oficial.

Opción D: fijar tiempos de espera estáticos de 30 segundos y topes de tokens estáticos para todas las solicitudes, aun complementado con backoff exponencial, dimensionamiento dinámico de fragmentos y límites de tokens "conscientes del contexto", mezcla de forma inconsistente elementos estáticos y dinámicos, y no ofrece un mecanismo estructurado de tipo circuit breaker o limitador de tasa adaptativo equivalente a los que documenta AWS para picos de uso y throttling cambiante.

Aviso de verificación: aunque la Opción C es la respuesta marcada como correcta (y se mantiene sin cambios), la documentación oficial de AWS respalda con bastante literalidad partes del diseño de la Opción B. La guía de referencia de los SDK de AWS ([feature-retry-behavior.html](https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html)) describe el modo de reintento "adaptativo" recomendándolo explícitamente para "AI workloads that call a single API operation at high volume" y para casos en que se necesita que el SDK frene automáticamente ante señales de throttling, lo cual coincide casi palabra por palabra con los requisitos del enunciado. Además, el AWS Well-Architected Framework (Agentic AI Lens, práctica AGENTREL07-BP02) documenta explícitamente un patrón de "circuit-breaker" para invocaciones de agentes/modelos, igual que describe la Opción B. En cambio, no se encontró respaldo documental para el mecanismo específico de la Opción C de "reanudar streams solicitando solo los chunks faltantes desde el punto de fallo"; las APIs de streaming de Bedrock (`InvokeModelWithResponseStream/ConverseStream`) no exponen esa capacidad de reanudación por rangos. Se recomienda revisión humana para confirmar si la respuesta correcta debería ser B en lugar de C.

Referencias: <https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html> ; <https://docs.aws.amazon.com/wellarchitected/latest/agentic-ai-lens/agentperf06-bp02.html> ; <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>

A bank is developing a generative AI (GenAI)-powered AI assistant that uses Amazon Bedrock to assist the bank's website users with account inquiries and financial guidance. The bank must ensure that the AI assistant does not reveal any personally identifiable information (PII) in customer interactions. The AI assistant must not send PII in prompts to the GenAI model. The AI assistant must not respond to customer requests to provide investment advice. The bank must collect audit logs of all customer interactions, including any images or documents that are transmitted during customer interactions. Which solution will meet these requirements with the LEAST operational effort?

- A Use Amazon Macie to detect and redact PII in user inputs and in the model responses. Apply prompt engineering techniques to force the model to avoid investment advice topics. Use AWS CloudTrail to capture conversation logs.
- B Use an AWS Lambda function and Amazon Comprehend to detect and redact PII. Use Amazon Comprehend topic modeling to prevent the AI assistant from discussing investment advice topics. Set up custom metrics in Amazon CloudWatch to capture customer conversations.
- C Configure Amazon Bedrock guardrails to apply a sensitive information policy to detect and filter PII. Set up a topic policy to ensure that the AI assistant avoids investment advice topics. Use the Converse API to log model invocations. Enable delivery and image logging to Amazon S3.
- D Use regex controls to match patterns for PII. Apply prompt engineering techniques to avoid returning PII or investment advice topics to customers. Enable model invocation logging, delivery logging, and image logging to Amazon S3.

Respuesta correcta: C

- A Use Amazon Macie to detect and redact PII in user inputs and in the model responses. Apply prompt engineering techniques to force the model to avoid investment advice topics. Use AWS CloudTrail to capture conversation logs.
- B Use an AWS Lambda function and Amazon Comprehend to detect and redact PII. Use Amazon Comprehend topic modeling to prevent the AI assistant from discussing investment advice topics. Set up custom metrics in Amazon CloudWatch to capture customer conversations.
- C** Configure Amazon Bedrock guardrails to apply a sensitive information policy to detect and filter PII. Set up a topic policy to ensure that the AI assistant avoids investment advice topics. Use the Converse API to log model invocations. Enable delivery and image logging to Amazon S3. ✓
- D Use regex controls to match patterns for PII. Apply prompt engineering techniques to avoid returning PII or investment advice topics to customers. Enable model invocation logging, delivery logging, and image logging to Amazon S3.

Explicación

Opción A: Amazon Macie está diseñado para descubrir y clasificar datos sensibles en reposo en Amazon S3 (por ejemplo, escaneando buckets), no para inspeccionar ni redactar PII en tiempo real dentro de los prompts enviados al modelo o de las respuestas generadas por este. Además, AWS CloudTrail registra llamadas a la API de gestión (quién invocó qué operación y cuándo), pero no captura el contenido conversacional completo (texto, imágenes o documentos) intercambiado con el asistente, por lo que no cumple el requisito de auditoría de las interacciones.

Opción B: esta solución exige desarrollo a medida (una función Lambda que orqueste Amazon Comprehend para detectar y redactar PII), lo que aumenta considerablemente el esfuerzo operativo frente a una función nativa de Bedrock. Además, el "topic modeling" de Comprehend es una técnica de descubrimiento no supervisado de temas latentes en un corpus de texto, no un mecanismo pensado para bloquear en tiempo real un tema concreto y predefinido como el asesoramiento de inversión durante una conversación. Por último, las métricas personalizadas de Amazon CloudWatch están pensadas para valores numéricos agregados, no para almacenar el contenido íntegro de las conversaciones, incluidas imágenes o documentos.

Opción C (Correcta): Amazon Bedrock Guardrails permite configurar una "sensitive information policy" (política de información sensible) que detecta y enmascara o bloquea PII, tanto mediante entidades predefinidas como mediante filtros de expresiones regulares personalizados, aplicándose tanto a las entradas (prompts) enviadas al modelo como a las respuestas devueltas por este. De forma complementaria, se puede definir una "topic policy" con "denied topics" (temas denegados) para impedir que el asistente responda a solicitudes sobre un tema concreto, como el asesoramiento de inversión. En cuanto a la auditoría, la documentación oficial de Amazon Bedrock confirma que, al usar la Converse API (u otras operaciones de inferencia compatibles), se puede habilitar el "model invocation logging" con un destino en Amazon S3, y que cualquier dato de imagen o documento que se transmita durante la conversación se registra en Amazon S3 si se ha habilitado la entrega ("delivery logging") y el registro de imágenes ("image logging") en S3. Toda esta solución se configura de forma nativa y declarativa en Bedrock, sin necesidad de desarrollar componentes adicionales, lo que la convierte en la opción de menor esfuerzo operativo.

Opción D: los controles basados únicamente en expresiones regulares son frágiles, requieren mantenimiento manual continuo y son propensos a falsos negativos frente a la variedad de formatos en que puede aparecer la PII (a diferencia de la política de información sensible de Guardrails, que combina entidades PII predefinidas con regex personalizados). Además, la ingeniería de prompts no es un mecanismo de cumplimiento fiable ni determinista, ya que el modelo puede ignorarla o ser manipulado para evadirla, a diferencia de un guardrail que actúa como una capa de control externa al propio modelo. Si bien esta opción sí menciona correctamente habilitar el registro de invocación de modelos, la entrega ("delivery logging") y el registro de imágenes en S3 (aspecto correcto y alineado con la documentación), el mecanismo elegido para PII y temas denegados no es el más robusto ni el de menor esfuerzo operativo en comparación con Guardrails.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>
<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-denied-topics.html>
<https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html>

A financial services company is developing a customer service AI assistant application that uses a foundation model (FM) in Amazon Bedrock. The application must provide transparent responses by documenting reasoning and by citing sources that are used for Retrieval Augmented Generation (RAG). The application must capture comprehensive audit trails for all responses to users. The application must be able to serve up to 10,000 concurrent users and must respond to each customer inquiry within 2 seconds. Which solution will meet these requirements with the LEAST operational overhead?

- A Enable tracing for Amazon Bedrock agents. Configure structured prompts that direct the FM to provide evidence presentations. Integrate Amazon Bedrock knowledge bases with data sources to enable RAG. Configure the application to reference and cite authoritative content. Deploy the application in a Multi-AZ architecture. Use Amazon API Gateway and AWS Lambda functions to scale the application. Use Amazon CloudFront to provide low-latency delivery.
- B Enable tracing for Amazon Bedrock agents. Integrate a custom RAG pipeline with Amazon OpenSearch Service to retrieve and cite sources. Configure structured prompts to present retrieved evidence. Deploy the application behind an Amazon API Gateway REST API. Use AWS Lambda functions and Amazon CloudFront to scale the application and to provide low latency. Store logs in Amazon S3 and use AWS CloudTrail to capture audit trails.
- C Use Amazon CloudWatch to monitor latency and error rates. Embed model prompts directly in the application backend to cite sources. Store application interactions with users in Amazon RDS for audits.
- D Store generated responses and supporting evidence in an Amazon S3 bucket. Enable versioning on the bucket for audits. Use AWS Glue to catalog retrieved documents. Process the retrieved documents in Amazon Athena to generate periodic compliance reports.

Respuesta correcta: A

- A** Enable tracing for Amazon Bedrock agents. Configure structured prompts that direct the FM to provide evidence presentations. Integrate Amazon Bedrock knowledge bases with data sources to enable RAG. Configure the application to reference and cite authoritative content. Deploy the application in a Multi-AZ architecture. Use Amazon API Gateway and AWS Lambda functions to scale the application. Use Amazon CloudFront to provide low-latency delivery. ✓
- B** Enable tracing for Amazon Bedrock agents. Integrate a custom RAG pipeline with Amazon OpenSearch Service to retrieve and cite sources. Configure structured prompts to present retrieved evidence. Deploy the application behind an Amazon API Gateway REST API. Use AWS Lambda functions and Amazon CloudFront to scale the application and to provide low latency. Store logs in Amazon S3 and use AWS CloudTrail to capture audit trails.
- C** Use Amazon CloudWatch to monitor latency and error rates. Embed model prompts directly in the application backend to cite sources. Store application interactions with users in Amazon RDS for audits.
- D** Store generated responses and supporting evidence in an Amazon S3 bucket. Enable versioning on the bucket for audits. Use AWS Glue to catalog retrieved documents. Process the retrieved documents in Amazon Athena to generate periodic compliance reports.

Explicación

Opción A (Correcta): habilitar el tracing de Amazon Bedrock Agents proporciona, en cada respuesta, un registro detallado ("trace") del proceso de razonamiento paso a paso del agente (orquestración, invocación de action groups y consultas a la base de conocimiento), lo que constituye una pista de auditoría técnica sin necesidad de instrumentación adicional. Las Amazon Bedrock Knowledge Bases, mediante la API RetrieveAndGenerate, realizan de forma nativa la recuperación y generación en una sola llamada y devuelven la respuesta junto con citas (objetos Citation) que referencian los fragmentos de los documentos fuente usados, cubriendo el requisito de transparencia y evidencia sin construir esa lógica a mano. Combinando esto con una arquitectura Multi-AZ y servicios totalmente gestionados y sin servidor (Amazon API Gateway, AWS Lambda, Amazon CloudFront), la solución escala automáticamente a picos de 10.000 usuarios concurrentes y ofrece baja latencia mediante el edge caching de CloudFront, todo ello con el mínimo esfuerzo operativo porque no hay servidores ni pipelines de RAG que gestionar.

Opción B: sustituir la Knowledge Base nativa de Bedrock por un pipeline de RAG a medida sobre Amazon OpenSearch Service obliga al equipo a implementar y mantener manualmente la lógica de recuperación semántica, el formateo de las citas y la sincronización de los datos, lo que incrementa notablemente la carga operativa frente a usar RetrieveAndGenerate, que ya resuelve esto de forma gestionada. Aunque el resto de la arquitectura (API Gateway, Lambda, CloudFront, CloudTrail) es razonable, no cumple el criterio de "menor esfuerzo operativo".

Opción C: no incorpora ningún mecanismo de RAG ni una base de conocimiento gestionada; embeber los prompts del modelo directamente en el backend no garantiza citas verificables ni trazabilidad real del razonamiento. Además, Amazon RDS es una base de datos relacional pensada para cargas transaccionales y no está diseñada ni dimensionada de forma nativa para absorber, con bajo esfuerzo operativo, el volumen de registros de auditoría de una aplicación con picos de 10.000 usuarios concurrentes.

Opción D: almacenar respuestas y evidencias en Amazon S3, catalogarlas con AWS Glue y consultarlas con Amazon Athena es un patrón de análisis por lotes orientado a generar informes de cumplimiento periódicos, no a servir citas y trazabilidad en el momento en que se genera la respuesta al usuario. Esta arquitectura tampoco aborda el requisito de responder en tiempo real en menos de 2 segundos a 10.000 usuarios concurrentes, ya que Athena y Glue no están pensados para servir respuestas interactivas de baja latencia.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-retrieve-generate.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/trace-events.html>

<https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/genops03-bp02.html>

A healthcare company is developing a document management system that stores medical research papers in an Amazon S3 bucket. The company needs to build a comprehensive metadata framework that will improve search precision for a generative AI (GenAI) application that analyzes the research papers. The metadata framework must include document timestamps, author information, and research domain classifications. The solution must maintain a consistent metadata structure across all uploaded documents. The solution must give foundation models (FMs) the ability to understand document context without accessing the full content. Which solution will meet these requirements?

- A Store document timestamps in Amazon S3 system metadata. Use S3 object tags to implement domain classification. Implement custom user-defined metadata to store author information.
- B Set up S3 Object Lock with legal holds to track document timestamps. Use S3 object tags to store author information. Implement S3 access points for domain classification.
- C Use S3 Inventory reports to track document timestamps. Create S3 access points to implement domain classification. Store author information in S3 Storage Lens dashboards.
- D Use custom user-defined metadata to store author information. Use S3 Object Lock retention periods to track document timestamps. Use S3 Event Notifications to implement domain classification.

Respuesta correcta: A

- A** Store document timestamps in Amazon S3 system metadata. Use S3 object tags to implement domain classification. Implement custom user-defined metadata to store author information. ✓
- B Set up S3 Object Lock with legal holds to track document timestamps. Use S3 object tags to store author information. Implement S3 access points for domain classification.
- C Use S3 Inventory reports to track document timestamps. Create S3 access points to implement domain classification. Store author information in S3 Storage Lens dashboards.
- D Use custom user-defined metadata to store author information. Use S3 Object Lock retention periods to track document timestamps. Use S3 Event Notifications to implement domain classification.

Explicación

Opción A (Correcta): Amazon S3 mantiene automáticamente un conjunto de metadatos de sistema (system-defined metadata) para cada objeto almacenado, incluyendo la fecha de creación del objeto (Last-Modified/Date), que S3 gestiona sin intervención del usuario; esto cubre el requisito de marcas de tiempo del documento. Los object tags son pares clave-valor (hasta 10 por objeto) pensados específicamente para categorizar y organizar objetos, y son ideales para clasificar el dominio de investigación (por ejemplo, "domain=oncología" o "domain=cardiología"), pudiendo usarse además en políticas de IAM, reglas de ciclo de vida o informes de asignación de costes. Los metadatos definidos por el usuario (user-defined metadata), enviados con el prefijo x-amz-meta- en una solicitud PUT/POST, permiten almacenar información arbitraria como el nombre del autor. La combinación de estos tres mecanismos nativos de S3 proporciona una estructura de metadatos consistente y consultable (incluso mediante S3 Metadata/HeadObject) sin necesidad de abrir o leer el contenido completo del documento, que es exactamente lo que necesita el modelo generativo para entender el contexto del documento antes de analizarlo.

Opción B: S3 Object Lock con legal holds es un mecanismo de retención WORM (Write Once Read Many) orientado a cumplimiento normativo y prevención de eliminación/sobrescritura de objetos, no un medio para registrar ni consultar marcas de tiempo del documento. Los access points de S3 son puntos de acceso de red alternativos al bucket, cada uno con su propia política de acceso; no ofrecen ninguna capacidad de clasificación por dominio de investigación.

Opción C: los informes de S3 Inventory generan de forma periódica (diaria o semanal) un listado en CSV/ORC/Parquet con el estado de los objetos de un bucket; no están diseñados para "rastrear" o consultar timestamps de forma dinámica por objeto en el momento de una búsqueda. Los access points, de nuevo, no sirven para clasificar por dominio. Y S3 Storage Lens ofrece métricas agregadas de uso y actividad de almacenamiento a nivel de organización/cuenta/bucket; no es un lugar para almacenar metadatos como el nombre de un autor.

Opción D: S3 Object Lock con periodos de retención es, igual que en la opción B, un control de cumplimiento/WORM que impide borrar o sobrescribir un objeto durante un plazo determinado, no un registro de la fecha del documento. Las S3 Event Notifications disparan eventos (por ejemplo, hacia Lambda, SQS o SNS) ante acciones sobre el bucket, como una subida de objeto, pero no almacenan ni implementan por sí mismas una clasificación de dominio de investigación.

Referencias:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingMetadata.html>

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-tagging.html>

Example Corp provides a personalized video generation service that millions of enterprise customers use. Customers generate marketing videos by submitting prompts to the company's proprietary generative AI (GenAI) model. To improve output relevance and personalization, Example Corp wants to enhance the prompts by using customer-specific context such as product preferences, customer attributes, and business history. The customers have strict data governance requirements. The customers must retain full ownership and control over their own data. The customers do not require real-time access. However, semantic accuracy must be high and retrieval latency must remain low to support customer experience use cases. Example Corp wants to minimize architectural complexity in its integration pattern. Example Corp does not want to deploy and manage services in each customer's environment unless necessary. Which solution will meet these requirements?

- A Ensure that each customer sets up an Amazon Q Business index that includes the customer's internal data. Ensure that each customer designates Example Corp as a data accessor to allow Example Corp to retrieve relevant content by using a secure API to enrich prompts at runtime.
- B Use federated search with Model Context Protocol (MCP) by deploying real-time MCP servers for each customer. Retrieve data in real time during prompt generation.
- C Ensure that each customer configures an Amazon Bedrock knowledge base. Allow cross-account querying so Example Corp can retrieve structured data for prompt augmentation.
- D Configure Amazon Kendra to crawl customer data sources. Share the resulting indexes across accounts so Example Corp can query each customer's Amazon Kendra index to retrieve augmentation data.

Respuesta correcta: A

- A** Ensure that each customer sets up an Amazon Q Business index that includes the customer's internal data. Ensure that each customer designates Example Corp as a data accessor to allow Example Corp to retrieve relevant content by using a secure API to enrich prompts at runtime. ✓
- B** Use federated search with Model Context Protocol (MCP) by deploying real-time MCP servers for each customer. Retrieve data in real time during prompt generation.
- C** Ensure that each customer configures an Amazon Bedrock knowledge base. Allow cross-account querying so Example Corp can retrieve structured data for prompt augmentation.
- D** Configure Amazon Kendra to crawl customer data sources. Share the resulting indexes across accounts so Example Corp can query each customer's Amazon Kendra index to retrieve augmentation data.

Explicación

Opción A (Correcta): la funcionalidad de "data accessor" de Amazon Q Business está diseñada exactamente para este patrón de integración ISV. Cada cliente empresarial configura su propia aplicación de Amazon Q Business (que indexa sus datos internos en el "Amazon Q index") y, desde la consola, añade a Example Corp como data accessor verificado, otorgándole permiso para consultar su índice. Con los parámetros de configuración compartidos (ID de la aplicación, región, ID del retriever, ARN del data accessor, etc.), Example Corp puede invocar la API SearchRelevantContent para recuperar contenido relevante y enriquecer los prompts en el momento de la generación. Esta API respeta los controles de acceso del cliente (solo devuelve contenido al que el usuario final tendría acceso), por lo que el cliente conserva la propiedad y el control total de sus datos. Al no requerirse acceso en tiempo real, y dado que el objetivo es minimizar la complejidad, este mecanismo evita que Example Corp tenga que desplegar o gestionar infraestructura dentro del entorno de cada cliente, cumpliendo todos los requisitos del enunciado (baja complejidad, alta precisión semántica, baja latencia de recuperación y gobernanza de datos).

Opción B: desplegar servidores MCP en tiempo real dentro del entorno de cada cliente exige justamente lo que Example Corp quiere evitar: desplegar y gestionar infraestructura en cada cuenta cliente, multiplicada por "millones" de clientes empresariales. Además, el enunciado indica explícitamente que no se requiere acceso en tiempo real, por lo que un patrón de federación en tiempo real vía MCP añade complejidad operativa innecesaria sin aportar ningún beneficio adicional frente a un patrón de recuperación asíncrona o bajo demanda como el de Q Business.

Opción C: es cierto que Amazon Bedrock Knowledge Bases sí admite consultas entre cuentas mediante políticas de recursos (resource-based policies) que autorizan a un rol de otra cuenta a invocar acciones como Retrieve o GetDocumentContent sobre una base de conocimiento gestionada. Sin embargo, esta opción exige que cada uno de los "millones" de clientes construya y mantenga su propia infraestructura de base de conocimiento (fuentes de datos, ingesta, embeddings, almacén vectorial) desde cero, y que además configure y administre manualmente la política de recursos entre cuentas para cada cliente. Esto añade una complejidad arquitectónica y operativa mucho mayor que el mecanismo nativo y ya gobernado de data accessor de Amazon Q Business, que fue diseñado específicamente para el caso de uso de ISVs que necesitan enriquecer prompts con datos empresariales de terceros sin construir su propia canalización de RAG por cliente.

Opción D: Amazon Kendra no ofrece un mecanismo nativo equivalente a "compartir índices entre cuentas" como el que existe para otros recursos de AWS mediante AWS Resource Access Manager. Para lograr algo similar, cada cliente tendría que aprovisionar y mantener su propio índice de Kendra (con el coste y la gestión operativa que ello implica, incluso durante los periodos en los que no hay consultas) y configurar manualmente roles y políticas de acceso entre cuentas para que Example Corp pueda consultarlo, lo cual introduce una complejidad arquitectónica y de gobernanza notablemente mayor que el patrón de data accessor de Q Business, que resuelve el mismo problema de forma nativa y con controles de acceso ya integrados.

Referencias:

<https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/isv.html>

<https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/concepts-terms.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-cross-account.html>

A company is building a legal research AI assistant that uses Amazon Bedrock with an Anthropic Claude foundation model (FM). The AI assistant must retrieve highly relevant case law documents to augment the FM's responses. The AI assistant must identify semantic relationships between legal concepts, specific legal terminology, and citations. The AI assistant must perform quickly and return precise results. Which solution will meet these requirements?

- A Configure an Amazon Bedrock knowledge base to use a default vector search configuration. Use Amazon Bedrock to expand queries to improve retrieval for legal documents based on specific terminology and citations.
- B Use Amazon OpenSearch service to deploy a hybrid search architecture that combines vector search with keyword search. Apply an Amazon Bedrock reranker model to optimize result relevance.
- C Enable the Amazon Kendra query suggestion feature for end users. Use Amazon Bedrock to perform post-processing of search results to identify semantic similarity in the documents and to produce precise results.
- D Use Amazon OpenSearch Service with vector search and Amazon Bedrock Titan embeddings to index and search legal documents. Use custom AWS Lambda functions to merge results with keyword-based filters that are stored in an Amazon RDS database.

Respuesta correcta: **B**

- A Configure an Amazon Bedrock knowledge base to use a default vector search configuration. Use Amazon Bedrock to expand queries to improve retrieval for legal documents based on specific terminology and citations.
- B Use Amazon OpenSearch service to deploy a hybrid search architecture that combines vector search with keyword search. Apply an Amazon Bedrock reranker model to optimize result relevance. ✓**
- C Enable the Amazon Kendra query suggestion feature for end users. Use Amazon Bedrock to perform post-processing of search results to identify semantic similarity in the documents and to produce precise results.
- D Use Amazon OpenSearch Service with vector search and Amazon Bedrock Titan embeddings to index and search legal documents. Use custom AWS Lambda functions to merge results with keyword-based filters that are stored in an Amazon RDS database.

Explicación

Opción A: configurar una Knowledge Base de Amazon Bedrock con la configuración de búsqueda vectorial por defecto, incluso combinada con expansión de consultas, se apoya únicamente en similitud semántica de embeddings. Este enfoque tiende a perder coincidencias exactas de citas legales, números de expediente y terminología jurídica muy específica, que la búsqueda semántica por sí sola no captura bien porque prioriza el significado sobre la coincidencia literal.

Opción B (Correcta): Amazon OpenSearch Service admite de forma nativa una arquitectura de búsqueda híbrida (Hybrid Search) mediante un "search pipeline" que combina resultados de búsqueda léxica/por palabras clave (algoritmo Okapi BM25, ideal para citas exactas y terminología jurídica precisa) con resultados de búsqueda vectorial/neural (ideal para capturar relaciones semánticas entre conceptos legales), normalizando y combinando las puntuaciones de ambos métodos (por ejemplo con `min_max/l2` y `arithmetic_mean/geometric_mean/harmonic_mean`). Sobre el conjunto de resultados combinado se puede aplicar después la API Rerank de Amazon Bedrock (`bedrock-agent-runtime`), que usa un modelo de reranking dedicado (por ejemplo Cohere Rerank) para reordenar los documentos por relevancia real frente a la consulta. Esta combinación de búsqueda híbrida más reranking maximiza tanto el recall (gracias al componente semántico) como la precisión (gracias al componente léxico y al reranker), con un rendimiento adecuado para un caso de uso que exige rapidez y precisión.

Opción C: la función de sugerencia de consultas (query suggestions) de Amazon Kendra ayuda a los usuarios a completar o refinar sus búsquedas mientras escriben, pero no mejora la relevancia de la recuperación de documentos en sí misma. Además, aplicar un post-procesado adicional con Amazon Bedrock para detectar similitud semántica sobre los resultados ya obtenidos añade llamadas extra al modelo, lo que incrementa la latencia sin ofrecer la calidad de reordenación que proporciona un modelo de reranking dedicado como el de la opción B.

Opción D: usar Amazon OpenSearch Service con búsqueda vectorial y embeddings de Amazon Titan es un buen punto de partida, pero fusionar esos resultados con filtros por palabras clave almacenados en Amazon RDS mediante funciones AWS Lambda personalizadas es una solución compleja, con mayor esfuerzo de desarrollo, mantenimiento y latencia adicional por las llamadas entre servicios, y no ofrece la calidad de reordenación por relevancia que aporta un modelo de reranking dedicado integrado de forma nativa como el de Bedrock.

Referencias: <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/serverless-configure-neural-search.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/rerank.html> ; https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Rerank.html

A company deploys multiple Amazon Bedrock based generative AI (GenAI) applications across multiple business units for customer service, content generation, and document analysis. Some applications show unpredictable token consumption patterns. The company requires a comprehensive observability solution that provides real-time visibility into token usage patterns across multiple models. The observability solution must support custom dashboards for multiple stakeholder groups and provide alerting capabilities for token consumption across all the foundational models that the company's applications use. Which combination of solutions will meet these requirements with the LEAST operational overhead? (Choose two.)

Selecciona 2 respuestas

- A Use Amazon CloudWatch metrics as data sources to create custom Amazon QuickSight dashboards that show token usage trends and usage patterns across FMs.
- B Use Amazon CloudWatch Logs Insights to analyze Amazon Bedrock invocation logs for token consumption patterns and usage attribution by application. Create custom queries to identify high-usage scenarios. Add log widgets to dashboards to enable continuous monitoring.
- C Create custom Amazon CloudWatch dashboards that combine native Amazon Bedrock token and invocation CloudWatch metrics. Set up CloudWatch alarms to monitor token usage thresholds.
- D Create dashboards that show token usage trends and patterns across the company's FMs by using an Amazon Bedrock zero-ETL integration with Amazon Managed Grafana.
- E Implement Amazon EventBridge rules to capture Amazon Bedrock model invocation events. Route token usage data to an Amazon Data Firehose delivery stream that targets Amazon OpenSearch Serverless. Use OpenSearch dashboards to analyze usage patterns.

Respuesta correcta: **B, C**

- A Use Amazon CloudWatch metrics as data sources to create custom Amazon QuickSight dashboards that show token usage trends and usage patterns across FMs.
- B** Use Amazon CloudWatch Logs Insights to analyze Amazon Bedrock invocation logs for token consumption patterns and usage attribution by application. Create custom queries to identify high-usage scenarios. Add log widgets to dashboards to enable continuous monitoring. ✓
- C** Create custom Amazon CloudWatch dashboards that combine native Amazon Bedrock token and invocation CloudWatch metrics. Set up CloudWatch alarms to monitor token usage thresholds. ✓
- D Create dashboards that show token usage trends and patterns across the company's FMs by using an Amazon Bedrock zero-ETL integration with Amazon Managed Grafana.
- E Implement Amazon EventBridge rules to capture Amazon Bedrock model invocation events. Route token usage data to an Amazon Data Firehose delivery stream that targets Amazon OpenSearch Serverless. Use OpenSearch dashboards to analyze usage patterns.

Explicación

Opción A: usar Amazon QuickSight como capa de dashboards implica aprovisionar y mantener un servicio de BI adicional (conjuntos de datos, ingestión desde CloudWatch, licencias/usuarios de QuickSight), lo cual añade complejidad operativa innecesaria. Además, QuickSight está pensado para business intelligence y no ofrece de forma nativa capacidades de alerta en tiempo real sobre umbrales operativos, a diferencia de las alarmas de CloudWatch, por lo que no cubre completamente el requisito de alerting.

Opción B (Correcta): Amazon Bedrock puede entregar el "model invocation logging" (registro de invocación de modelos) a Amazon CloudWatch Logs, capturando metadatos de cada invocación incluyendo conteos de tokens de entrada y salida. Sobre esos logs, CloudWatch Logs Insights permite escribir consultas personalizadas (por ejemplo, agrupando por usuario/IAM principal o por modelo) para analizar patrones de consumo de tokens y atribuir el uso a cada aplicación o equipo, identificando escenarios de alto consumo. Los resultados de esas consultas se pueden añadir como widgets de log a un dashboard de CloudWatch para monitorización continua. Esta es una capacidad nativa documentada por AWS, sin necesidad de construir pipelines adicionales.

Opción C (Correcta): Amazon Bedrock publica de forma nativa en CloudWatch, bajo el namespace AWS/Bedrock, métricas de runtime como Invocations, InputTokenCount y OutputTokenCount (entre otras de latencia y errores). Sobre estas métricas se pueden construir dashboards personalizados de CloudWatch que combinen visualizaciones de token e invocación, y configurar alarmas de CloudWatch (CloudWatch Alarms) para notificar cuando el consumo de tokens supere umbrales definidos. Al ser métricas nativas del servicio, esta solución tiene el menor esfuerzo operativo posible, sin ETL ni infraestructura adicional.

Opción D: no existe una integración "zero-ETL" de Amazon Bedrock con Amazon Managed Grafana. Amazon Managed Grafana puede usar CloudWatch como fuente de datos mediante el conector estándar de Grafana, pero esto no constituye una integración zero-ETL nativa de Bedrock; la opción describe una capacidad que el servicio no ofrece tal como se plantea.

Opción E: construir una tubería con reglas de EventBridge, Amazon Data Firehose y Amazon OpenSearch Serverless es una arquitectura funcional, pero requiere diseñar, desplegar y mantener múltiples componentes adicionales (reglas de eventos, streams de entrega, un dominio/colección de OpenSearch), lo cual implica mucho más esfuerzo operativo que reutilizar las métricas y logs nativos de Bedrock ya integrados con CloudWatch.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/cost-mgmt-request-metadata.html>

A company is designing a solution that uses foundation models (FMs) to support multiple AI workloads. Some FMs must be invoked on demand and in real time. Other FMs require consistent high-throughput access for batch processing. The solution must support hybrid deployment patterns and run workloads across cloud infrastructure and on-premises infrastructure to comply with data residency and compliance requirements. Which combination of steps will meet these requirements? (Choose two.)

Selecciona 2 respuestas

- A Use AWS Lambda to orchestrate low-latency FM inference by invoking FMs hosted on Amazon SageMaker AI asynchronous endpoints.
- B Configure provisioned throughput in Amazon Bedrock to ensure consistent performance for high-volume workloads.
- C Deploy FMs to Amazon SageMaker AI endpoints with support for edge deployment by using Amazon SageMaker Neo. Orchestrate the FMs by using AWS Lambda to support hybrid deployment.
- D Use Amazon Bedrock with auto-scaling to handle unpredictable traffic surges.
- E Use Amazon SageMaker JumpStart to host and invoke the FMs.

Respuesta correcta: B, C

- A Use AWS Lambda to orchestrate low-latency FM inference by invoking FMs hosted on Amazon SageMaker AI asynchronous endpoints.
- B Configure provisioned throughput in Amazon Bedrock to ensure consistent performance for high-volume workloads. ✓**
- C Deploy FMs to Amazon SageMaker AI endpoints with support for edge deployment by using Amazon SageMaker Neo. Orchestrate the FMs by using AWS Lambda to support hybrid deployment. ✓**
- D Use Amazon Bedrock with auto-scaling to handle unpredictable traffic surges.
- E Use Amazon SageMaker JumpStart to host and invoke the FMs.

Explicación

Opción A: los endpoints asíncronos (Asynchronous Inference) de SageMaker AI están diseñados para solicitudes con payloads grandes (hasta 1 GB) o tiempos de procesamiento largos (hasta 1 hora), que se encolan y se procesan en segundo plano devolviendo el resultado a un bucket de Amazon S3. Este mecanismo es lo opuesto a la inferencia en tiempo real de baja latencia bajo demanda que requiere el escenario, por lo que no encaja con ese requisito.

Opción B (Correcta): el Provisioned Throughput de Amazon Bedrock permite reservar capacidad dedicada (medida en Model Units) para un modelo base o personalizado, garantizando un rendimiento constante y predecible a lo largo del tiempo. Esto es exactamente lo que necesitan las cargas de trabajo de alto volumen y procesamiento por lotes, a diferencia del throughput bajo demanda (on-demand), cuya capacidad se escala a nivel regional sin capacidad reservada y no ofrece esas garantías de rendimiento constante.

Opción C (Correcta): Amazon SageMaker Neo compila y optimiza modelos entrenados (Gluon, Keras, MXNet, PyTorch, TensorFlow, ONNX, etc.) para ejecutarse en una plataforma de hardware específica, incluyendo tanto instancias en la nube como dispositivos edge (a través de AWS IoT Greengrass). Esto permite desplegar el modelo optimizado fuera de la infraestructura pública de AWS, en ubicaciones on-premises o edge, ayudando a cumplir requisitos de residencia de datos. Usar AWS Lambda para orquestar la invocación de esos endpoints (tanto los alojados en SageMaker AI en la nube como los desplegados en el edge) habilita el patrón de despliegue híbrido solicitado.

Opción D: Amazon Bedrock es un servicio totalmente gestionado que se ejecuta únicamente en la infraestructura de AWS; su capacidad de auto-scaling (en el modelo on-demand) ayuda a absorber picos de tráfico impredecibles en la nube, pero no ofrece ningún mecanismo para ejecutar cargas de trabajo on-premises, por lo que no resuelve el requisito híbrido del escenario.

Opción E: Amazon SageMaker JumpStart facilita el descubrimiento, ajuste fino (fine-tuning) y despliegue rápido de modelos preentrenados directamente en la nube mediante endpoints de SageMaker AI, pero no aporta por sí mismo ningún mecanismo de despliegue híbrido cloud/on-premises ni de optimización para hardware edge.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>

<https://docs.aws.amazon.com/sagemaker/latest/dg/neo.html>

<https://docs.aws.amazon.com/sagemaker/latest/dg/async-inference-create-endpoint-create-endpoint.html>

A company is planning to deploy multiple generative AI (GenAI) applications to five independent business units that operate in multiple countries in Europe and the Americas. Each application uses Amazon Bedrock Retrieval Augmented Generation (RAG) patterns with business unit-specific knowledge bases that store terabytes of unstructured data. The company must establish well-architected, standardized components for security controls, observability practices, and deployment patterns across all the GenAI applications. The components must be reusable, versioned, and governed consistently. Which solution will meet these requirements?

- A Configure Amazon API Gateway REST API endpoints for the GenAI applications. Deploy common security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens in standardized AWS CloudFormation templates. Use CloudFormation Guard after the deployment to validate policy compliance in each business unit.
- B Create standardized AWS CloudFormation templates to implement security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens. Establish a centralized repository that performs version control. Integrate a CI/CD pipeline with CloudFormation Guard to enforce consistent and repeatable deployments across business units.
- C Use AWS Service Catalog to define standardized portfolios and versioned products for each business unit. Use the portfolios to enforce security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens. Require the business units to use the Service Catalog console to deploy resources.
- D Document security controls, observability requirements, and RAG patterns based on the AWS Well-Architected Generative AI Lens in a shared design document. Use Amazon Macie to enforce deployment. Delegate implementation responsibility to each business unit.

Respuesta correcta: **B**

- A Configure Amazon API Gateway REST API endpoints for the GenAI applications. Deploy common security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens in standardized AWS CloudFormation templates. Use CloudFormation Guard after the deployment to validate policy compliance in each business unit.
- B Create standardized AWS CloudFormation templates to implement security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens. Establish a centralized repository that performs version control. Integrate a CI/CD pipeline with CloudFormation Guard to enforce consistent and repeatable deployments across business units.**
- C Use AWS Service Catalog to define standardized portfolios and versioned products for each business unit. Use the portfolios to enforce security, observability, and RAG patterns based on the AWS Well-Architected Generative AI Lens. Require the business units to use the Service Catalog console to deploy resources.
- D Document security controls, observability requirements, and RAG patterns based on the AWS Well-Architected Generative AI Lens in a shared design document. Use Amazon Macie to enforce deployment. Delegate implementation responsibility to each business unit.

Explicación

Opción A: en este planteamiento, AWS CloudFormation Guard (cfn-guard) se ejecuta después del despliegue, es decir, de forma reactiva. Según la documentación oficial de AWS CloudFormation sobre mejores prácticas ("Integrate cfn-guard into CI/CD pipelines"), el valor de cfn-guard está precisamente en integrarlo dentro del pipeline de CI/CD para validar las plantillas ANTES del despliegue, lo que "ayuda a prevenir que se desplieguen recursos no conformes en el entorno y ofrece retroalimentación temprana a los desarrolladores sobre las violaciones de políticas". Validar después del despliegue significa que los recursos no conformes ya se habrían creado antes de detectarse el incumplimiento, lo cual es menos eficaz y contradice el enfoque de "policy as code" preventivo que promueve AWS.

Opción B (Correcta): crear plantillas estandarizadas de AWS CloudFormation que implementen los patrones de seguridad, observabilidad y RAG recomendados por el AWS Well-Architected Generative AI Lens, mantenerlas versionadas en un repositorio centralizado, e integrar AWS CloudFormation Guard dentro del pipeline de CI/CD es exactamente el patrón que documenta AWS para lograr despliegues consistentes y repetibles: cfn-guard actúa como una puerta de control (policy-as-code) que valida el cumplimiento de las políticas antes de que los recursos se aprovisionen. Esto satisface los tres requisitos del enunciado (componentes reutilizables, versionados y gobernados de forma consistente) para las cinco unidades de negocio.

Opción C: AWS Service Catalog es una herramienta válida de gobernanza para publicar productos versionados y estandarizados, pero la opción exige que las unidades de negocio desplieguen manualmente desde la consola de Service Catalog. Esto introduce un paso manual que reduce la automatización, la repetibilidad y la trazabilidad del proceso frente a un pipeline de CI/CD versionado con validación automática de políticas, que es el enfoque que AWS recomienda para despliegues consistentes a escala.

Opción D: documentar los controles en un documento de diseño compartido y delegar la implementación a cada unidad de negocio no garantiza consistencia ni gobernanza real, ya que no existe ningún mecanismo técnico de aplicación (enforcement). Además, Amazon Macie es un servicio de descubrimiento y clasificación de datos sensibles (por ejemplo, PII en Amazon S3), no una herramienta para "forzar" el cumplimiento de patrones de despliegue de infraestructura, seguridad u observabilidad; su uso aquí es conceptualmente incorrecto.

Referencias:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/best-practices.html>

<https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-ai-lens.html>

<https://docs.aws.amazon.com/cfn-guard/latest/ug/what-is-guard.html>

A company upgraded its Amazon Bedrock powered foundation model (FM) that supports a multilingual customer service assistant. After the upgrade, the assistant exhibited inconsistent behavior across languages. The assistant began generating different responses in some languages when presented with identical questions. The company needs a solution to detect and address similar problems for future updates. The evaluation must be completed within 45 minutes for all supported languages. The evaluation must process at least 15,000 test conversations in parallel. The evaluation process must be fully automated and integrated into the CI/CD pipeline. The solution must block deployment if quality thresholds are not met. Which solution will meet these requirements?

- A Create a distributed traffic simulation framework that sends translation-heavy workloads to the assistant in multiple languages simultaneously. Use Amazon CloudWatch metrics to monitor latency, concurrency, and throughput. Run simulations before production releases to identify infrastructure bottlenecks.
- B Deploy the assistant in multiple AWS Regions with Amazon Route 53 latency-based routing and AWS Global Accelerator to improve global performance. Store multilingual conversation logs in Amazon S3. Perform weekly post-deployment audits to review consistency.
- C Create a pre-processing pipeline that normalizes all incoming messages into a consistent format before sending the messages to the assistant. Apply rule-based checks to flag potential hallucinations in the outputs. Focus the evaluation on the normalized text to simplify testing across languages.
- D Set up standardized multilingual test conversations with identical meaning. Run the test conversations in parallel by using Amazon Bedrock model evaluation jobs. Apply similarity and hallucination thresholds. Integrate the process into the CI/CD pipeline to block releases that fail.

Respuesta correcta: **D**

- A Create a distributed traffic simulation framework that sends translation-heavy workloads to the assistant in multiple languages simultaneously. Use Amazon CloudWatch metrics to monitor latency, concurrency, and throughput. Run simulations before production releases to identify infrastructure bottlenecks.
- B Deploy the assistant in multiple AWS Regions with Amazon Route 53 latency-based routing and AWS Global Accelerator to improve global performance. Store multilingual conversation logs in Amazon S3. Perform weekly post-deployment audits to review consistency.
- C Create a pre-processing pipeline that normalizes all incoming messages into a consistent format before sending the messages to the assistant. Apply rule-based checks to flag potential hallucinations in the outputs. Focus the evaluation on the normalized text to simplify testing across languages.
- D** Set up standardized multilingual test conversations with identical meaning. Run the test conversations in parallel by using Amazon Bedrock model evaluation jobs. Apply similarity and hallucination thresholds. Integrate the process into the CI/CD pipeline to block releases that fail. ✓

Explicación

Opción A: un framework de simulación de tráfico centrado en monitorizar latencia, concurrencia y rendimiento con Amazon CloudWatch evalúa el comportamiento de la infraestructura y del rendimiento del sistema, pero no mide la consistencia semántica ni el significado de las respuestas del asistente entre distintos idiomas. No detecta el problema de que el modelo genere respuestas diferentes ante preguntas equivalentes en distintos idiomas, por lo que no cumple el objetivo.

Opción B: desplegar el asistente en múltiples Regiones de AWS con enrutamiento por latencia de Amazon Route 53 y AWS Global Accelerator mejora el rendimiento y la disponibilidad global, pero no evalúa la calidad ni la consistencia de las respuestas del modelo. Además, las auditorías semanales posteriores al despliegue son un proceso manual y reactivo, no un control automatizado que pueda bloquear un despliegue defectuoso antes de que llegue a producción, incumpliendo el requisito de bloqueo automático en el pipeline de CI/CD.

Opción C: normalizar los mensajes de entrada a un formato consistente y aplicar comprobaciones basadas en reglas para detectar alucinaciones es un enfoque frágil y poco escalable: las reglas manuales no capturan bien las diferencias semánticas sutiles entre idiomas ni logran la cobertura y velocidad de evaluación en paralelo que exige el escenario (15.000 conversaciones en menos de 45 minutos). Tampoco aprovecha las capacidades nativas de evaluación de modelos de Amazon Bedrock, que están diseñadas específicamente para este tipo de análisis.

Opción D (Correcta): Amazon Bedrock ofrece "model evaluation jobs" (trabajos de evaluación de modelos) que permiten crear conjuntos de datos de prueba (prompt datasets) con conversaciones estandarizadas de significado idéntico en varios idiomas, y ejecutarlos como un trabajo automático (automatic model evaluation job) usando un modelo evaluador ("judge model") o métricas basadas en referencia. Bedrock incluye métricas integradas relevantes para este caso, como Builtin.Faithfulness (que identifica contenido no respaldado por el contexto, es decir, alucinaciones) y Builtin.Correctness/Builtin.Relevance (que comparan la respuesta generada con una respuesta de referencia, es decir, similitud semántica). Estos trabajos se pueden crear y consultar mediante la API/CLI de Bedrock (por ejemplo, el comando "aws bedrock create-evaluation-job" y las acciones CreateEvaluationJob/GetEvaluationJob), lo que permite automatizarlos completamente e integrarlos en un pipeline de CI/CD, de forma que el pipeline consulte los resultados y bloquee el despliegue si las métricas de similitud o alucinación no superan los umbrales definidos. Esto es exactamente lo que exige el enunciado: evaluación estandarizada multilingüe, ejecución en paralelo para un alto volumen de casos en una ventana de tiempo corta, métricas de similitud/alucinación, e integración automatizada con bloqueo de despliegue en CI/CD.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-built-in-metrics.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-type-automatic.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-metrics.html>

A company is developing a generative AI (GenAI) application that uses Amazon Bedrock foundation models (FMs). The application has several custom tool integrations. The application has experienced unexpected token consumption surges despite consistent user traffic. The company needs a solution that uses Amazon Bedrock model invocation logging to monitor InputTokenCount metrics and OutputTokenCount metrics. The solution must detect unusual patterns in tool usage and identify which specific tool integrations cause abnormal token consumption. The solution must also automatically adjust thresholds as traffic patterns change. Which solution will meet these requirements?

- A Use Amazon CloudWatch Logs to capture model invocation logs. Create CloudWatch dashboards based on InputTokenCount metrics and OutputTokenCount metrics. Configure static CloudWatch alarms with fixed thresholds for each tool integration.
- B Store model invocation logs in an Amazon S3 bucket. Use AWS Glue to catalog the logs. Analyze token consumption patterns by using scheduled Amazon Athena queries that generate reports on tool usage trends.
- C Use Amazon CloudWatch Logs to capture model invocation logs. Create CloudWatch metric filters to extract tool-specific invocation patterns. Apply CloudWatch anomaly detection alarms that adjust baselines for each tool's metrics.
- D Store model invocation logs in an Amazon S3 bucket. Create an AWS Lambda function to process logs in real time. Manually update Amazon CloudWatch alarm thresholds based on token consumption trends that the Lambda function identifies.

Respuesta correcta: C

- A Use Amazon CloudWatch Logs to capture model invocation logs. Create CloudWatch dashboards based on InputTokenCount metrics and OutputTokenCount metrics. Configure static CloudWatch alarms with fixed thresholds for each tool integration.
- B Store model invocation logs in an Amazon S3 bucket. Use AWS Glue to catalog the logs. Analyze token consumption patterns by using scheduled Amazon Athena queries that generate reports on tool usage trends.
- C Use Amazon CloudWatch Logs to capture model invocation logs. Create CloudWatch metric filters to extract tool-specific invocation patterns. Apply CloudWatch anomaly detection alarms that adjust baselines for each tool's metrics.**
- D Store model invocation logs in an Amazon S3 bucket. Create an AWS Lambda function to process logs in real time. Manually update Amazon CloudWatch alarm thresholds based on token consumption trends that the Lambda function identifies.

Explicación

Opción A: usar Amazon CloudWatch Logs para capturar los logs de invocación de modelos y crear dashboards a partir de las métricas InputTokenCount y OutputTokenCount es un buen punto de partida, pero configurar alarmas de CloudWatch con umbrales fijos ("static") por cada integración de herramienta no cumple el requisito de que el sistema "ajuste automáticamente los umbrales a medida que cambian los patrones de tráfico". Un umbral estático generaría falsos positivos o falsos negativos en cuanto el volumen normal de tráfico cambie, ya que habría que revisarlo y modificarlo manualmente.

Opción B: almacenar los logs de invocación en un bucket de Amazon S3, catalogarlos con AWS Glue y analizarlos mediante consultas programadas (scheduled) de Amazon Athena es un enfoque de análisis por lotes (batch) con retardo inherente al ciclo de programación de las consultas. No ofrece detección de anomalías en tiempo casi real ni ajusta umbrales de forma automática; simplemente genera informes periódicos sobre tendencias de uso, lo que no satisface el requisito de detectar rápidamente patrones inusuales por herramienta.

Opción C (Correcta): Amazon Bedrock puede enviar los logs de invocación de modelos (model invocation logging) a Amazon CloudWatch Logs, donde quedan registrados el texto de entrada/salida y metadatos como InputTokenCount y OutputTokenCount para cada invocación, incluidas las que involucran herramientas (tool use). A partir de esos logs se pueden definir metric filters de CloudWatch que extraigan patrones específicos de cada integración de herramienta (por ejemplo, filtrando por el nombre de la tool invocada) y publiquen esos valores como métricas personalizadas de CloudWatch por cada tool. Sobre esas métricas se puede habilitar CloudWatch Anomaly Detection, una funcionalidad que aplica algoritmos estadísticos y de machine learning para construir un modelo de "banda" de valores esperados (baseline) para cada métrica, teniendo en cuenta tendencias y estacionalidad (por hora, día o semana). Las alarmas basadas en este modelo no usan un umbral fijo, sino que comparan el valor real con la banda de valores esperada y se disparan cuando el valor se sale de esa banda; el modelo se recalcula continuamente a partir de los datos históricos (hasta dos semanas), por lo que el "umbral" se adapta automáticamente a medida que cambian los patrones de tráfico, sin intervención manual. Esta combinación (model invocation logging + metric filters por herramienta + anomaly detection alarms) es exactamente la solución nativa de AWS que cumple los tres requisitos del enunciado: monitorizar InputTokenCount/OutputTokenCount, aislar qué integración de herramienta concreta causa el consumo anómalo, y ajustar los umbrales automáticamente.

Opción D: almacenar los logs en S3 y procesarlos en tiempo real con una función AWS Lambda para después actualizar manualmente los umbrales de las alarmas de CloudWatch introduce trabajo operativo manual y un punto de fallo humano; no es una solución que "ajuste automáticamente" los umbrales como exige el enunciado, a diferencia del ajuste nativo y continuo que ofrece CloudWatch Anomaly Detection.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html>
- https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Anomaly_Detection.html
- <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>

A company is using Amazon Bedrock to develop an AI-powered application that uses a foundation model (FM) that supports cross-Region inference and provisioned throughput. The application must serve users in Europe and North America with consistently low latency. The application must comply with data residency regulations that require European user data to remain within Europe-based AWS Regions. During testing, the application experiences service degradation when Regional traffic spikes reach service quotas. The company needs a solution that maintains application resilience and minimizes operational complexity. Which solution will meet these requirements?

- A Deploy separate Amazon Bedrock instances in North American and European Regions. Use a custom routing layer that directs traffic based on user location. Configure Amazon CloudWatch alarms to monitor Regional service usage. Use Amazon SNS to send email alerts to the company when usage approaches specified thresholds.
- B Use Amazon Bedrock cross-Region inference profiles by specifying geographical codes in profile IDs when the application calls the InvokeModel API. Configure separate Amazon API Gateway HTTP APIs to direct European and North American users to the appropriate Regional endpoints.
- C Deploy a multi-Region Amazon API Gateway HTTP API and AWS Lambda functions that implement retry logic to handle throttling. Configure the Lambda functions to call the FM in the nearest secondary Region when the application reaches service quotas in the primary Region. Use intelligent routing to ensure compliance with data residency requirements.
- D Configure provisioned throughput for Amazon Bedrock in multiple Regions. Implement failover logic in the application code to switch between Regions when throttling occurs. Use AWS Global Accelerator to route traffic to the appropriate endpoints based on user location.

Respuesta correcta: **B**

- A Deploy separate Amazon Bedrock instances in North American and European Regions. Use a custom routing layer that directs traffic based on user location. Configure Amazon CloudWatch alarms to monitor Regional service usage. Use Amazon SNS to send email alerts to the company when usage approaches specified thresholds.
- B Use Amazon Bedrock cross-Region inference profiles by specifying geographical codes in profile IDs when the application calls the InvokeModel API. Configure separate Amazon API Gateway HTTP APIs to direct European and North American users to the appropriate Regional endpoints.** ✓
- C Deploy a multi-Region Amazon API Gateway HTTP API and AWS Lambda functions that implement retry logic to handle throttling. Configure the Lambda functions to call the FM in the nearest secondary Region when the application reaches service quotas in the primary Region. Use intelligent routing to ensure compliance with data residency requirements.
- D Configure provisioned throughput for Amazon Bedrock in multiple Regions. Implement failover logic in the application code to switch between Regions when throttling occurs. Use AWS Global Accelerator to route traffic to the appropriate endpoints based on user location.

Explicación

Opción A: Amazon Bedrock no se despliega como "instancias" regionales que la propia empresa deba aprovisionar y gestionar manualmente; es un servicio totalmente gestionado al que se accede vía API en cada Región donde está disponible. Construir una capa de enrutamiento propia, sumada a alarmas de CloudWatch y notificaciones por correo vía SNS, es un mecanismo puramente reactivo: avisa a un humano de que se está acercando a la cuota, pero no evita la degradación del servicio ni resuelve el pico de tráfico de forma automática, además de añadir una complejidad operativa considerable que la pregunta pide minimizar.

Opción B (Correcta): Amazon Bedrock ofrece perfiles de inferencia entre Regiones (cross-Region inference profiles) de dos tipos: geográficos (Geographic) y globales (Global). Los perfiles geográficos se identifican con un prefijo geográfico en el ID del perfil (por ejemplo "us.", "eu." o "apac.") y, según la documentación oficial, cuando se invoca un perfil ligado a una geografía concreta, Amazon Bedrock selecciona automáticamente una Región comercial de AWS dentro de esa misma geografía para procesar la solicitud, manteniendo el procesamiento de datos dentro de esos límites geográficos (US, EU, APAC). Esto es precisamente lo que se recomienda para organizaciones con requisitos de residencia de datos y cumplimiento normativo. Además, al repartir automáticamente la carga entre varias Regiones de una misma geografía, este mecanismo absorbe picos de tráfico y ayuda a mitigar el impacto de alcanzar las cuotas de servicio de una única Región, sin necesidad de construir lógica de failover propia. Combinar esto con endpoints de Amazon API Gateway separados por Región (uno para tráfico europeo y otro para tráfico norteamericano) dirige a cada usuario al perfil de inferencia geográfico correcto (eu. o us.) desde el origen, cumpliendo así el requisito de residencia de datos con la mínima complejidad operativa, ya que es Bedrock quien gestiona internamente el enrutamiento y el reparto de capacidad entre Regiones.

Opción C: aunque usar Lambda con lógica de reintentos y conmutación a una "Región secundaria más cercana" cuando se agota la cuota en la Región primaria puede aportar resiliencia técnica, existe un riesgo real de que las solicitudes de usuarios europeos terminen procesándose en una Región fuera de Europa (por ejemplo, en Norteamérica) si esa es la Región secundaria más cercana disponible, lo que incumpliría el requisito estricto de que los datos de usuarios europeos permanezcan en Regiones europeas. Además, esta solución obliga a diseñar, implementar y mantener manualmente toda la lógica de conmutación y reintentos, en lugar de aprovechar una funcionalidad nativa de Bedrock que ya resuelve este problema.

Opción D: combinar Provisioned Throughput en varias Regiones con lógica de failover implementada en el código de la aplicación y AWS Global Accelerator para el enrutamiento por ubicación es técnicamente viable, pero exige aprovisionar y pagar capacidad dedicada en cada Región, además de construir y mantener manualmente toda la lógica de detección de throttling y conmutación entre Regiones. Esto añade una complejidad operativa significativamente mayor que la de usar los perfiles de inferencia entre Regiones ya gestionados por Bedrock, que resuelven de forma nativa tanto el reparto de tráfico como el cumplimiento de la residencia de datos.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-support.html>

An international company is building an AI assistant that uses RAG. The company wants the AI assistant to have near real-time, low-latency performance. The AI assistant must provide service to several geographic areas. The company's customers will use proprietary data with the AI assistant. The proprietary data must not leave the company's immediate geographic area. Which solution will meet these requirements?

- A Deploy an Amazon Bedrock model with a cross-Region model inference profile. Create Amazon S3 buckets in each AWS Region the company operates in. Store a knowledge base in each respective S3 bucket. In each Region, configure Amazon Kendra to interact with the respective knowledge base. In each Region, configure an AWS Lambda function that uses Kendra and Amazon Bedrock to process AI assistant prompts.
- B Deploy an Amazon Bedrock model in each AWS Region the company operates in. Configure an Amazon Bedrock cross-Region model inference profile. Configure a vector database that uses Amazon Bedrock Knowledge Bases. Store the knowledge bases in Amazon S3 in each Region the company operates in.
- C Use AWS Outposts to deploy an outpost in each AWS Region the company operates in. Create Amazon S3 buckets to store knowledge bases in each corresponding Region. Deploy Amazon RDS configured as a vector database to each outpost. Deploy an Amazon Bedrock model with a cross-Region inference profile in a central Region.
- D Configure a knowledge base stored in the Amazon S3 Express One Zone storage class in each AWS Local Zone the company operates in. Use Amazon RDS to deploy a vector database in each Local Zone the company operates in. Deploy a large language model (LLM) to Amazon EC2 instances in each Local Zone. Configure the AI assistant to route prompts to the model in the respective Local Zone.

Respuesta correcta: **B**

- A Deploy an Amazon Bedrock model with a cross-Region model inference profile. Create Amazon S3 buckets in each AWS Region the company operates in. Store a knowledge base in each respective S3 bucket. In each Region, configure Amazon Kendra to interact with the respective knowledge base. In each Region, configure an AWS Lambda function that uses Kendra and Amazon Bedrock to process AI assistant prompts.
- B** Deploy an Amazon Bedrock model in each AWS Region the company operates in. Configure an Amazon Bedrock cross-Region model inference profile. Configure a vector database that uses Amazon Bedrock Knowledge Bases. Store the knowledge bases in Amazon S3 in each Region the company operates in. ✓
- C Use AWS Outposts to deploy an outpost in each AWS Region the company operates in. Create Amazon S3 buckets to store knowledge bases in each corresponding Region. Deploy Amazon RDS configured as a vector database to each outpost. Deploy an Amazon Bedrock model with a cross-Region inference profile in a central Region.
- D Configure a knowledge base stored in the Amazon S3 Express One Zone storage class in each AWS Local Zone the company operates in. Use Amazon RDS to deploy a vector database in each Local Zone the company operates in. Deploy a large language model (LLM) to Amazon EC2 instances in each Local Zone. Configure the AI assistant to route prompts to the model in the respective Local Zone.

Explicación

Opción A: usar Amazon Kendra como motor de recuperación obliga a construir manualmente la integración entre Kendra y Amazon Bedrock mediante funciones Lambda propias en cada Región, lo que añade complejidad de desarrollo e integración innecesaria frente a usar una Knowledge Base nativa y totalmente gestionada de Bedrock. Además, la opción no explica cómo se logra la baja latencia entre Regiones, ya que el perfil de inferencia entre Regiones ("cross-Region inference profile") ahí mencionado solo cubre el modelo, no la recuperación de la base de conocimiento.

Opción B (Correcta): esta solución combina dos elementos clave. Primero, un perfil de inferencia entre Regiones (cross-Region inference profile) de tipo geográfico en Amazon Bedrock, que enruta las solicitudes de inferencia únicamente entre Regiones de AWS que pertenecen a la misma área geográfica (por ejemplo, US, EU o APAC). Según la documentación oficial de AWS, este mecanismo "mantiene el procesamiento de datos dentro de los límites geográficos especificados" y está diseñado específicamente para organizaciones con requisitos de residencia de datos y cumplimiento normativo, a la vez que aumenta el rendimiento (throughput) y reduce la latencia respecto a usar una única Región, absorbiendo picos de demanda al repartir la carga entre varias Regiones cercanas. Segundo, mantener una Amazon Bedrock Knowledge Base (con su almacén vectorial gestionado y los documentos fuente en un bucket de Amazon S3) en cada Región donde opera la empresa asegura que los datos propietarios de los clientes nunca salgan de su Región/área geográfica de origen, ya que la recuperación (retrieval) se realiza localmente contra la base de conocimiento de esa misma Región. La combinación de ambos elementos satisface simultáneamente el requisito de baja latencia casi en tiempo real, la cobertura multi-geográfica y la restricción de que los datos propietarios no salgan del área geográfica inmediata de la empresa.

Opción C: desplegar AWS Outposts en cada Región, junto con Amazon RDS autogestionado como base de datos vectorial, es una solución de infraestructura dedicada (hardware físico de AWS en las instalaciones del cliente) mucho más compleja, costosa y con mayor carga operativa de la necesaria para este caso de uso, cuando el mismo objetivo de residencia de datos se puede lograr con servicios totalmente gestionados como Bedrock Knowledge Bases. Además, centralizar el modelo con un perfil de inferencia entre Regiones en una única Región central no garantiza por sí sola la latencia baja que se busca para todas las áreas geográficas.

Opción D: alojar un modelo de lenguaje grande (LLM) propio en instancias EC2 en cada AWS Local Zone obliga a la empresa a gestionar por completo el ciclo de vida del modelo (aprovisionamiento, escalado, parcheo, actualización de pesos, etc.) y su infraestructura de inferencia, lo que incrementa drásticamente el esfuerzo operativo y el costo frente a usar un modelo totalmente gestionado a través de Amazon Bedrock, sin aportar ninguna ventaja adicional en materia de residencia de datos o latencia frente a la opción B.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>

A company is building a generative AI (GenAI) application that processes financial reports and provides summaries for analysts. The application must run two compute environments. In one environment, AWS Lambda function must use the Python SDK to analyze reports on demand. In the second environment, Amazon EKS containers must use the JavaScript SDK to batch process multiple reports on a schedule. The application must maintain conversational context throughout multi-turn interactions, use the same foundation model (FM) across environments, and ensure consistent authentication. Which solution will meet these requirements?

- A Use the Amazon Bedrock InvokeModel API with a separate authentication method for each environment. Store conversation states in Amazon DynamoDB. Use custom I/O formatting logic for each programming language.
- B Use the Amazon Bedrock Converse API directly in both environments with a common authentication mechanism that uses IAM roles. Store conversation states in Amazon ElastiCache. Creating programming language-specific wrappers for model parameters.
- C Create a centralized Amazon API Gateway REST API endpoint that handles all model interactions by using the InvokeModel API. Store interaction history in application process memory in each Lambda function or EKS container. Use environment variables to configure model parameters.
- D Use the Amazon Bedrock Converse API and IAM roles for authentication. Pass previous messages in the request messages array to maintain conversational context. Use programming language-specific SDKs to establish consistent API interfaces.

Respuesta correcta: **D**

- A Use the Amazon Bedrock InvokeModel API with a separate authentication method for each environment. Store conversation states in Amazon DynamoDB. Use custom I/O formatting logic for each programming language.
- B Use the Amazon Bedrock Converse API directly in both environments with a common authentication mechanism that uses IAM roles. Store conversation states in Amazon ElastiCache. Creating programming language-specific wrappers for model parameters.
- C Create a centralized Amazon API Gateway REST API endpoint that handles all model interactions by using the InvokeModel API. Store interaction history in application process memory in each Lambda function or EKS container. Use environment variables to configure model parameters.
- D** Use the Amazon Bedrock Converse API and IAM roles for authentication. Pass previous messages in the request messages array to maintain conversational context. Use programming language-specific SDKs to establish consistent API interfaces. ✓

Explicación

Opción A: la API InvokeModel (y su variante en streaming, InvokeModelWithResponseStream) recibe y devuelve el cuerpo de la solicitud en un formato específico de cada proveedor de modelo, por lo que cambiar de FM o reutilizar el mismo código entre entornos obliga a mantener lógica de formateo de entrada/salida distinta para cada modelo. Además, usar un método de autenticación distinto en cada entorno (Lambda y EKS) contradice el requisito de autenticación consistente; lo correcto es usar roles de IAM con el permiso bedrock:InvokeModel en ambos entornos.

Opción B: aunque sí usa la Converse API con IAM (acertado en ese punto), delegar el estado conversacional en un servicio externo como Amazon ElastiCache y construir wrappers específicos por lenguaje para los parámetros del modelo es innecesario: la propia Converse API ya expone un esquema de solicitud/respuesta unificado (arrays "messages", "system", "inferenceConfig", etc.) igual en todos los SDK de AWS, por lo que no hace falta normalizar parámetros por lenguaje, y el contexto conversacional puede mantenerse reenviando el propio array de mensajes sin infraestructura adicional.

Opción C: centralizar las llamadas en un endpoint REST de Amazon API Gateway que internamente use la API InvokeModel no resuelve el problema del formato heterogéneo por proveedor de modelo (mismo problema que la opción A). Además, guardar el historial de interacción en la memoria de proceso de cada función Lambda o contenedor de EKS no persiste el contexto: las funciones Lambda son efímeras (su entorno de ejecución puede reciclarse entre invocaciones) y los procesos en EKS no comparten memoria entre sí ni con Lambda, por lo que el contexto conversacional se perdería entre turnos.

Opción D (Correcta): la Converse API de Amazon Bedrock (operaciones Converse y ConverseStream, disponibles únicamente en el endpoint bedrock-runtime) proporciona una API consistente e independiente del modelo: permite escribir el código una sola vez y reutilizarlo con distintos FM de Bedrock que admiten mensajes, incluyendo soporte para parámetros específicos de un modelo cuando es necesario. Esta API está disponible de forma uniforme en los distintos SDK de AWS (boto3 para Python, AWS SDK for JavaScript, etc.), de modo que cada entorno —la función Lambda en Python y los contenedores de EKS en JavaScript— puede usar su SDK nativo manteniendo la misma interfaz de solicitud/respuesta. La autenticación se realiza mediante roles de IAM concediendo el permiso bedrock:InvokeModel (o bedrock:InvokeModelWithResponseStream para streaming), logrando un mecanismo de autenticación consistente entre ambos entornos. El contexto conversacional en interacciones multi-turno se mantiene simplemente incluyendo los mensajes previos (con sus roles "user"/"assistant") en el array "messages" de cada nueva solicitud, sin necesidad de infraestructura de almacenamiento externa dedicada solo para ese propósito.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html>

A company is building a serverless application that uses AWS Lambda functions to help students around the world summarize notes. The application uses Anthropic Claude through Amazon Bedrock. The company observed that most of the traffic occurs during evenings in each time zone. Users report experiencing throttling errors during peak usage times in their time zones. The company needs to resolve the throttling issues by ensuring continuous operation of the application. The solution must maintain application performance quality. The company needs a solution that does not require a fixed hourly cost during low traffic periods. Which solution will meet these requirements?

- A Create custom Amazon CloudWatch metrics to monitor model errors. Set provisioned throughput to a value that is safely higher than the peak traffic observed.
- B Create custom Amazon CloudWatch metrics to monitor model errors. Set up a failover mechanism to redirect invocations to a backup AWS Region when the errors exceed a specified threshold.
- C Enable invocation logging in Amazon Bedrock. Monitor key metrics such as Invocations, InputTokenCount, OutputTokenCount, and Invocation Throttles. Distribute traffic across cross-Region inference endpoints.
- D Enable invocation logging in Amazon Bedrock. Monitor InvocationLatency, InvocationClientErrors, and InvocationServerErrors metrics. Distribute traffic across multiple versions of the same model.

Respuesta correcta: **C**

- A Create custom Amazon CloudWatch metrics to monitor model errors. Set provisioned throughput to a value that is safely higher than the peak traffic observed.
- B Create custom Amazon CloudWatch metrics to monitor model errors. Set up a failover mechanism to redirect invocations to a backup AWS Region when the errors exceed a specified threshold.
- C Enable invocation logging in Amazon Bedrock. Monitor key metrics such as Invocations, InputTokenCount, OutputTokenCount, and Invocation Throttles. Distribute traffic across cross-Region inference endpoints.**
- D Enable invocation logging in Amazon Bedrock. Monitor InvocationLatency, InvocationClientErrors, and InvocationServerErrors metrics. Distribute traffic across multiple versions of the same model.

Explicación

Opción A: fijar el Provisioned Throughput a un nivel holgadamente superior al pico observado sí evitaría el throttling, pero el Provisioned Throughput se factura como un coste fijo por hora (o por compromiso de 1/6 meses) independientemente del uso real, por lo que la empresa pagaría esa capacidad reservada también durante las horas de bajo tráfico de cada zona horaria. Esto contradice explícitamente el requisito del enunciado de no incurrir en un coste fijo por hora durante los periodos de baja demanda.

Opción B: configurar un failover hacia una región de respaldo solo cuando los errores superan un umbral es una medida puramente reactiva: actúa después de que el throttling ya se ha producido y los usuarios ya han sido afectados, en lugar de prevenirlo. Además, no aporta visibilidad específica sobre por qué ocurre el throttling (cuotas de tokens/solicitudes por minuto) ni distribuye la carga de forma proactiva entre regiones, por lo que no garantiza mantener la calidad de rendimiento de forma continua.

Opción C (Correcta): habilitar el invocation logging de Amazon Bedrock y vigilar las métricas de CloudWatch del espacio de nombres AWS/Bedrock —Invocations, InputTokenCount, OutputTokenCount e InvocationThrottles (esta última cuenta específicamente las invocaciones limitadas por el sistema, y no se contabiliza como Invocations ni como error)— permite identificar con precisión cuándo y con qué intensidad se producen los throttles de cuota (RPM/TPM). Distribuir el tráfico mediante perfiles de inferencia entre regiones (cross-Region inference) permite que Amazon Bedrock enrute automáticamente las solicitudes hacia otras regiones dentro del perfil, aprovechando capacidad de cómputo adicional y evitando así el throttling causado por picos de tráfico regionales (como los que ocurren en horario nocturno de cada zona horaria) sin necesidad de reservar capacidad. Según la documentación oficial de AWS, cross-Region inference sigue utilizando tarifas on-demand (pago por uso, sin coste de enrutamiento adicional) y está recomendada precisamente para escenarios de "disponibilidad continua" donde la disponibilidad prima sobre el coste, cumpliendo así con el requisito de no tener un coste fijo por hora durante baja demanda.

Opción D: monitorizar InvocationLatency, InvocationClientErrors e InvocationServerErrors no proporciona visibilidad directa sobre los throttles de cuota (esa es precisamente la función de la métrica InvocationThrottles, que aquí se omite). Además, "distribuir el tráfico entre múltiples versiones del mismo modelo" no resuelve el problema de cuota regional, ya que las distintas versiones de un modelo dentro de la misma región comparten los mismos límites de servicio (RPM/TPM) de esa región; no reparten la carga hacia otras regiones como sí hace cross-Region inference.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/capacity-limits-cost-optimization.html>

A company is developing a new AI-powered application that needs to integrate with various specialized tools. These tools currently run as Model Context Protocol (MCP) servers on the local machines of developers and do not maintain states between invocations. The company plans to deploy each MCP server as an AWS Lambda function to support the company's production application. The solution must be accessible to both internal applications and authorized third-party partners. The solution must use strict authentication and authorization controls. Which additional steps will meet these requirements with the LEAST operational overhead?

- A Create a custom Lambda invocation transport by using the Lambda Invoke API. Implement IAM authentication and grant InvokeFunction permissions to authorized users and roles.
- B Expose the Lambda functions through Amazon API Gateway REST API endpoints. Implement API keys for authentication. Configure the applications that need to access the MCP servers to use standard HTTP requests instead of the MCP protocol.
- C Create Lambda function URLs and enable a custom Streamable HTTP transport and SigV4. Implement AWS IAM authentication. Grant InvokeFunctionUrl permissions to authorized users and roles.
- D Expose the Lambda function through Amazon API Gateway HTTP API endpoints with the Streamable HTTP transport. Use Amazon Cognito to implement OAuth authentication. Configure API Gateway to validate OAuth tokens.

Respuesta correcta: C

- A Create a custom Lambda invocation transport by using the Lambda Invoke API. Implement IAM authentication and grant InvokeFunction permissions to authorized users and roles.
- B Expose the Lambda functions through Amazon API Gateway REST API endpoints. Implement API keys for authentication. Configure the applications that need to access the MCP servers to use standard HTTP requests instead of the MCP protocol.
- C Create Lambda function URLs and enable a custom Streamable HTTP transport and SigV4. Implement AWS IAM authentication. Grant InvokeFunctionUrl permissions to authorized users and roles.**
- D Expose the Lambda function through Amazon API Gateway HTTP API endpoints with the Streamable HTTP transport. Use Amazon Cognito to implement OAuth authentication. Configure API Gateway to validate OAuth tokens.

Explicación

Opción A: usar la API Lambda Invoke (RPC de control, no HTTP público) para construir un "transporte personalizado" no proporciona de forma nativa el transporte HTTP en streaming (Streamable HTTP) que exige el protocolo MCP; habría que desarrollar y mantener esa capa de transporte desde cero, además de exponerla de algún modo a terceros externos, lo que aumenta considerablemente el esfuerzo operativo frente a usar una URL HTTP nativa.

Opción B: sustituir el protocolo MCP por peticiones HTTP estándar significa que las aplicaciones cliente dejarían de hablar MCP con esos servidores, lo que incumple el propio objetivo de desplegarlos como servidores MCP. Además, las claves de API de API Gateway son un mecanismo pensado para identificación y throttling, no para autenticación/autorización fuerte, y ofrecen garantías mucho más débiles que la firma SigV4 de IAM que exige el enunciado ("controles estrictos de autenticación y autorización").

Opción C (Correcta): las Lambda function URLs se pueden configurar con "InvokeMode" = RESPONSE_STREAM, lo que permite que la función transmita la respuesta de forma incremental (streaming) y así soportar el transporte Streamable HTTP que usa el Model Context Protocol. Combinando esto con AuthType = AWS_IAM, cada solicitud debe autenticarse con credenciales de AWS firmadas mediante SigV4, y el acceso se controla otorgando permisos IAM (incluyendo lambda:InvokeFunctionUrl, y desde octubre de 2025 también lambda:InvokeFunction) tanto en políticas de identidad como, para el caso de terceros externos, en la política basada en recursos de la función (permitiendo así acceso entre cuentas a socios autorizados). Todo esto se logra sin aprovisionar, configurar ni mantener Amazon API Gateway, que es precisamente el objetivo de mínimo esfuerzo operativo que pide el enunciado; la propia guía de decisión de AWS Lambda posiciona las function URLs como la alternativa más sencilla a API Gateway cuando no se necesitan las funcionalidades adicionales de éste (planes de uso, autorizadores personalizados, dominios personalizados, etc.).

Opción D: usar Amazon API Gateway (HTTP API) junto con Amazon Cognito para OAuth es una solución técnicamente válida y también soporta streaming, pero añade componentes adicionales que hay que aprovisionar, configurar y mantener (grupo de usuarios de Cognito, integración de autorización OAuth en API Gateway, gestión del ciclo de vida de tokens), lo que supone más esfuerzo operativo que usar directamente las Lambda function URLs con autenticación IAM nativa.

Referencias:

<https://docs.aws.amazon.com/lambda/latest/dg/urls-auth.html>

<https://docs.aws.amazon.com/lambda/latest/dg/response-streaming-tutorial.html>

<https://docs.aws.amazon.com/lambda/latest/dg/apig-http-invoke-decision.html>

A company has a generative AI (GenAI) application that uses Amazon Bedrock to provide real-time responses to customer queries. The company has noticed intermittent failures with API calls to foundation models (FMs) during peak traffic periods. The company needs a solution to handle transient errors and provide detailed observability into FM performance. The solution must prevent cascading failures during throttling events and provide distributed tracing across service boundaries to identify latency contributors. The solution must also enable correlation of performance issues with specific FM characteristics. Which solution will meet these requirements?

- A Implement a custom retry mechanism with a fixed delay of 1 second between retries. Configure Amazon CloudWatch alarms to monitor the application's error rates and latency metrics.
- B Configure the AWS SDK with standard retry mode and exponential backoff with jitter. Use AWS X-Ray tracing with annotations to identify and filter service components.
- C Implement client-side caching of all FM responses. Add custom logging statements in the application code to record API call durations.
- D Configure the AWS SDK with adaptive retry mode. Use AWS CloudTrail distributed tracing to monitor throttling events.

Respuesta correcta: **B**

- A Implement a custom retry mechanism with a fixed delay of 1 second between retries. Configure Amazon CloudWatch alarms to monitor the application's error rates and latency metrics.
- B Configure the AWS SDK with standard retry mode and exponential backoff with jitter. Use AWS X-Ray tracing with annotations to identify and filter service components. ✓**
- C Implement client-side caching of all FM responses. Add custom logging statements in the application code to record API call durations.
- D Configure the AWS SDK with adaptive retry mode. Use AWS CloudTrail distributed tracing to monitor throttling events.

Explicación

Opción A: usar un retraso fijo (por ejemplo, 1 segundo) entre reintentos, sin variabilidad aleatoria (jitter), hace que múltiples clientes que fallan al mismo tiempo reintenten sincronizados, lo que puede agravar el throttling durante picos de tráfico en lugar de mitigarlo. Además, las alarmas de Amazon CloudWatch sobre tasas de error y latencia son útiles para alertar, pero no ofrecen trazado distribuido entre los límites de los distintos servicios ni permiten correlacionar la latencia con componentes o llamadas concretas a los modelos.

Opción B (Correcta): configurar el AWS SDK con el modo de reintentos "standard" aplica backoff exponencial con jitter de forma estandarizada entre SDKs, usando retrasos más cortos para errores transitorios (como timeouts de red) y retrasos más largos para errores de throttling (como ThrottlingException). Este modo incluye además una cuota de reintentos (token bucket) que hace que la aplicación falle rápido en lugar de seguir reintentando cuando los fallos son generalizados, lo cual ayuda a prevenir el efecto de fallos en cascada durante eventos de throttling. Por su parte, AWS X-Ray proporciona trazado distribuido real entre los límites de los distintos servicios de la aplicación (segmentos, subsegmentos y el mapa de servicios), permitiendo identificar en qué punto se concentra la latencia. Las anotaciones (annotations) de X-Ray son pares clave-valor indexados que se pueden usar en expresiones de filtro para buscar y agrupar trazas concretas, lo que permite correlacionar los problemas de rendimiento con características específicas del modelo de FM invocado (por ejemplo, el modelo o la versión utilizada).

Opción C: cachear en el cliente todas las respuestas del modelo no soluciona los fallos transitorios de invocación al servicio (el problema que hay que resolver), y además puede servir respuestas obsoletas quitando sentido a la naturaleza "en tiempo real" de la aplicación. Añadir sentencias de logging personalizadas en el código registra duraciones de forma aislada, pero no aporta el contexto de traza correlacionado y propagado entre límites de servicio que sí ofrece X-Ray de forma nativa.

Opción D: el modo de reintentos "adaptive" del SDK de AWS es una opción real y válida (incluye todo lo del modo standard más un limitador de tasa del lado del cliente que puede retrasar incluso la solicitud inicial cuando detecta throttling), pero AWS recomienda usarlo solo en casos concretos —un cliente que apunta a un único recurso con throttling frecuente— y no como opción general, ya que puede ralentizar también solicitudes a recursos no afectados. El problema decisivo de esta opción es la segunda parte: AWS CloudTrail registra la actividad de llamadas a la API (eventos de administración) con fines de auditoría y seguridad, pero no realiza trazado distribuido de solicitudes a través de los componentes de una aplicación; esa capacidad la proporciona AWS X-Ray, no CloudTrail.

Referencias:

- <https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html>
- <https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html>
- <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-management-events-with-cloudtrail.html>

A company is building a video analysis platform on AWS. The platform will analyze a large video archive by using Amazon Rekognition and Amazon Bedrock. The platform must comply with predefined privacy standards. The platform must also use secure model I/O, control foundation model (FM) access patterns, and provide an audit of who accessed what and when. Which solution will meet these requirements?

- A Configure VPC endpoints for Amazon Bedrock model API calls. Implement Amazon Bedrock Guardrails to filter harmful or unauthorized content in prompts and responses. Use Amazon Bedrock trace events to track all agent and model invocations for auditing purposes. Export the traces to Amazon CloudWatch Logs as an audit record of model usage. Store all prompts and outputs in Amazon S3 with server-side encryption with AWS KMS keys (SSE-KMS).
- B Define access control by using IAM with attribute-based controls to map departments to specific permissions. Configure VPC endpoints for Amazon Bedrock model API calls. Use IAM condition keys to enforce specific GuardrailIdentifier and ModelId values. Configure AWS CloudTrail to capture management and data events for S3 objects and KMS key usage activities. Enable S3 server access logging to record detailed file-level interactions with the video archives. Send all CloudTrail logs to AWS CloudTrail Lake. Set up Amazon CloudWatch alarms to detect and alert on unexpected activity from Amazon Bedrock, Amazon Rekognition, and AWS KMS.
- C Restrict access to services by using VPC endpoint policies. Use AWS Config to track resource changes and compliance with security rules. Use server-side encryption with AWS KMS keys (SSE-KMS) to encrypt data at rest. Store the model's I/O in separate Amazon S3 buckets. Enable S3 server access logging to track file-level interactions.
- D Configure AWS CloudTrail Insights to analyze API call patterns across accounts and detect anomalous activity in Amazon Bedrock, Amazon Rekognition, Amazon S3, and AWS KMS. Deploy Amazon Macie to scan and classify the video archive. Use server-side encryption with AWS KMS keys (SSE-KMS) to encrypt all stored data. Configure CloudTrail to capture KMS API usage events for audit purposes. Configure Amazon EventBridge rules to process CloudTrail Insights anomalies and Macie findings. Use CloudWatch alarms to trigger automated notifications and security responses when potential security issues are detected.

Respuesta correcta: **B**

- A** Configure VPC endpoints for Amazon Bedrock model API calls. Implement Amazon Bedrock Guardrails to filter harmful or unauthorized content in prompts and responses. Use Amazon Bedrock trace events to track all agent and model invocations for auditing purposes. Export the traces to Amazon CloudWatch Logs as an audit record of model usage. Store all prompts and outputs in Amazon S3 with server-side encryption with AWS KMS keys (SSE-KMS).
- B** Define access control by using IAM with attribute-based controls to map departments to specific permissions. Configure VPC endpoints for Amazon Bedrock model API calls. Use IAM condition keys to enforce specific GuardrailIdentifier and ModelId values. Configure AWS CloudTrail to capture management and data events for S3 objects and KMS key usage activities. Enable S3 server access logging to record detailed file-level interactions with the video archives. Send all CloudTrail logs to AWS CloudTrail Lake. Set up Amazon CloudWatch alarms to detect and alert on unexpected activity from Amazon Bedrock, Amazon Rekognition, and AWS KMS. ✓
- C** Restrict access to services by using VPC endpoint policies. Use AWS Config to track resource changes and compliance with security rules. Use server-side encryption with AWS KMS keys (SSE-KMS) to encrypt data at rest. Store the model's I/O in separate Amazon S3 buckets. Enable S3 server access logging to track file-level interactions.
- D** Configure AWS CloudTrail Insights to analyze API call patterns across accounts and detect anomalous activity in Amazon Bedrock, Amazon Rekognition, Amazon S3, and AWS KMS. Deploy Amazon Macie to scan and classify the video archive. Use server-side encryption with AWS KMS keys (SSE-KMS) to encrypt all stored data. Configure CloudTrail to capture KMS API usage events for audit purposes. Configure Amazon EventBridge rules to process CloudTrail Insights anomalies and Macie findings. Use CloudWatch alarms to trigger automated notifications and security responses when potential security issues are detected.

Explicación

Opción A: los endpoints de VPC para las llamadas a la API de Bedrock y Amazon Bedrock Guardrails filtran contenido dañino o no autorizado, y los trace events de Bedrock permiten rastrear invocaciones de agentes y modelos, pero esta opción no define ningún control de acceso granular (no hay IAM basado en atributos que mapee departamentos a permisos) ni registra el acceso a nivel de archivo del archivo de vídeo en S3 (no hay eventos de datos de CloudTrail sobre los objetos de S3 ni S3 server access logging). Los trace events ayudan a auditar el uso del modelo, pero no sustituyen una pista de auditoría completa de quién accedió a qué dato y cuándo.

Opción B (Correcta): definir el control de acceso mediante IAM con control de acceso basado en atributos (ABAC), mapeando cada departamento a permisos concretos, junto con endpoints de VPC (AWS PrivateLink) para las llamadas a la API de Amazon Bedrock, proporciona E/S segura del modelo y control granular de quién puede invocarlo. Además, aplicar claves de condición de IAM como `bedrock:GuardrailIdentifier` (documentada oficialmente, permite forzar el uso de un guardrail concreto en las llamadas de inferencia como `InvokeModel`, `InvokeModelWithResponseStream`, `Converse` y `ConverseStream`) junto con una clave de condición asociada al modelo permite controlar de forma precisa qué guardrail y qué modelo fundacional (FM) puede invocar cada principal, cumpliendo el requisito de "controlar los patrones de acceso al FM". Configurar AWS CloudTrail para capturar eventos de gestión y de datos sobre los objetos de S3 y el uso de las claves de KMS, enviarlos a AWS CloudTrail Lake, activar el registro de acceso a nivel de archivo de S3 (S3 server access logging) y establecer alarmas de Amazon CloudWatch sobre Bedrock, Rekognition y KMS proporciona la auditoría detallada de "quién accedió a qué y cuándo" que exige el enunciado, junto con el cumplimiento de estándares de privacidad. Es la única opción que cubre de forma integral los tres requisitos: E/S segura del modelo, control de acceso al FM y auditoría completa.

Opción C: restringir el acceso con políticas de endpoint de VPC y usar AWS Config para rastrear cambios de recursos y cumplimiento de reglas de seguridad no aporta un control de acceso basado en atributos (ABAC) ni condiciones específicas de guardrail o modelo que permitan controlar qué principal puede usar qué FM, que es uno de los requisitos explícitos del enunciado. AWS Config es una herramienta de cumplimiento de configuración, no de auditoría de acceso a datos por usuario.

Opción D: AWS CloudTrail Insights detecta patrones anómalos en el volumen y la forma de las llamadas a la API, pero no constituye por sí solo un registro exhaustivo de cada acceso individual (para eso se necesitan los eventos de gestión y de datos

de CloudTrail estándar). Amazon Macie clasifica la sensibilidad de los datos almacenados en S3, pero no controla el acceso a los FM. Esta opción tampoco incorpora condiciones de IAM que restrinjan qué guardrail o modelo puede usarse, por lo que no controla los patrones de acceso al FM como exige el enunciado.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-permissions-id.html> ; https://docs.aws.amazon.com/service-authorization/latest/reference/list_bedrock.html ; <https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html>

An insurance company uses existing Amazon SageMaker AI infrastructure to support a web-based application that allows customers to predict what their insurance premiums will be. The company stores customer data that is used to train the SageMaker AI model in an Amazon S3 bucket. The dataset is growing rapidly. The company wants a solution to continuously re-train the model. The solution must automatically re-train and re-deploy the model to the application when an employee uploads a new customer data file to the S3 bucket. Which solution will meet these requirements?

- A Use AWS Glue to run an ETL job on each uploaded file. Configure the ETL job to use the AWS SDK to invoke the Sage Maker AI model endpoint. Use real-time inference with the endpoint to re-deploy the model after it is re-trained on the updated customer dataset.
- B Create an AWS Lambda function and webhook handlers to generate an event when an employee uploads a new file. Configure SageMaker Pipelines to re-deploy the model after it is re-trained on the updated customer dataset. Use Amazon EventBridge to create an event bus. Set the Lambda function event as the source and SageMaker Pipelines as the target.
- C Create an AWS Step Functions Express workflow with AWS SDK integrations to retrieve the customer data from the S3 bucket when an employee uploads a new file to the S3 bucket. Use a SageMaker Data Wrangler flow to export the data from the S3 bucket to SageMaker Autopilot. Use SageMaker Autopilot to re-deploy the model after it has been re-trained on the updated customer dataset.
- D Create an AWS Step Functions Standard workflow. Configure the first state to call an AWS Lambda function to respond when an employee uploads a new file to the S3 bucket. Use a pipeline in SageMaker Pipelines to re-deploy the model after it has been re-trained on the updated customer dataset. Use the next state in the workflow to run the pipeline when the first state receives a response.

Respuesta correcta: **D**

- A Use AWS Glue to run an ETL job on each uploaded file. Configure the ETL job to use the AWS SDK to invoke the Sage Maker AI model endpoint. Use real-time inference with the endpoint to re-deploy the model after it is re-trained on the updated customer dataset.
- B Create an AWS Lambda function and webhook handlers to generate an event when an employee uploads a new file. Configure SageMaker Pipelines to re-deploy the model after it is re-trained on the updated customer dataset. Use Amazon EventBridge to create an event bus. Set the Lambda function event as the source and SageMaker Pipelines as the target.
- C Create an AWS Step Functions Express workflow with AWS SDK integrations to retrieve the customer data from the S3 bucket when an employee uploads a new file to the S3 bucket. Use a SageMaker Data Wrangler flow to export the data from the S3 bucket to SageMaker Autopilot. Use SageMaker Autopilot to re-deploy the model after it has been re-trained on the updated customer dataset.
- D** Create an AWS Step Functions Standard workflow. Configure the first state to call an AWS Lambda function to respond when an employee uploads a new file to the S3 bucket. Use a pipeline in SageMaker Pipelines to re-deploy the model after it has been re-trained on the updated customer dataset. Use the next state in the workflow to run the pipeline when the first state receives a response. ✓

Explicación

Opción A: usar AWS Glue para ejecutar un job ETL sobre cada archivo subido y después invocar el endpoint de SageMaker AI mediante el AWS SDK únicamente permite realizar inferencia en tiempo real contra un modelo ya desplegado; no reentrena el modelo con los nuevos datos ni orquesta un ciclo de entrenamiento, por lo que no cumple el requisito central de reentrenamiento automático.

Opción B: aunque la combinación de una función AWS Lambda, un bus de eventos de Amazon EventBridge y SageMaker Pipelines es técnicamente viable para orquestar el reentrenamiento, la propuesta obliga a crear "webhook handlers" personalizados para detectar la subida del archivo. Esto es innecesario porque Amazon S3 ya puede emitir de forma nativa notificaciones de evento (Amazon S3 Event Notifications) o eventos a Amazon EventBridge cuando se crea un objeto, así que añade complejidad de integración que no aporta valor frente a una arquitectura más directa.

Opción C: los flujos de trabajo Express de AWS Step Functions están diseñados para cargas de trabajo de alto volumen y corta duración (un máximo de cinco minutos de ejecución, con semántica "at-most-once" o "at-least-once"), lo que los hace inadecuados para orquestar un proceso de entrenamiento de un modelo de machine learning, que habitualmente tarda mucho más de cinco minutos. Además, encadenar SageMaker Data Wrangler con SageMaker Autopilot introduce un flujo de AutoML pensado para exploración automática de modelos, sin el control fino sobre el pipeline de reentrenamiento que ya existe para el modelo en producción, lo que resulta más pesado de lo necesario para este caso de uso.

Opción D (Correcta): según la documentación de AWS, los flujos de trabajo Standard de Step Functions están pensados para procesos de larga duración, duraderos y auditables (pueden ejecutarse hasta durante un año), soportan reintentos configurables y ofrecen semántica de ejecución "exactamente una vez" (exactly-once) junto con el historial completo de ejecución recuperable vía la API de Step Functions. Estas características encajan perfectamente con la orquestación de un reentrenamiento de modelo, que puede tardar bastante más de los cinco minutos que soporta un flujo Express y que conviene poder auditar. En esta solución, el primer estado del flujo Standard invoca una función AWS Lambda que responde cuando un empleado sube un nuevo archivo de datos de clientes al bucket de Amazon S3; el siguiente estado del flujo ejecuta, a partir de esa respuesta, un pipeline de Amazon SageMaker Pipelines que reentrena el modelo con el dataset actualizado y lo vuelve a desplegar automáticamente en la aplicación.

Referencias:

<https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/step-functions-workflows.html>

<https://docs.aws.amazon.com/sagemaker/latest/dg/automating-sagemaker-with-eventbridge.html>

<https://docs.aws.amazon.com/step-functions/latest/dg/sample-train-model.html>

A company uses Amazon Bedrock to implement a Retrieval Augmented Generation (RAG)-based system to serve medical information to users. The company needs to compare multiple chunking strategies, evaluate the generation quality of two foundation models (FMs), and enforce quality thresholds for deployment. Which Amazon Bedrock evaluation configuration will meet these requirements?

- A Create a retrieve-only evaluation job that uses a supported version of Anthropic Claude Sonnet as the evaluator model. Configure metrics for context relevance and context coverage. Define deployment thresholds in a separate CI/CD pipeline.
- B Create a retrieve-and-generate evaluation job that uses custom precision at k metrics and an LLM-as-a-judge metric that uses a scale of 1-5. Include each chunking strategy in the evaluation dataset. Use a supported version of Anthropic Claude Sonnet to evaluate responses from both FMs.
- C Create a separate evaluation job for each chunking strategy and FM combination. Use Amazon Bedrock built-in metrics for correctness and completeness. Manually review scores before deployment approval.
- D Set up a pipeline that uses multiple retrieve-only evaluation jobs to assess retrieval quality. Create separate evaluation jobs for both FMs that use Amazon Nova Pro as the LLM-as-a-judge model. Evaluate based on faithfulness and citation precision metrics.

Respuesta correcta: **B**

- A Create a retrieve-only evaluation job that uses a supported version of Anthropic Claude Sonnet as the evaluator model. Configure metrics for context relevance and context coverage. Define deployment thresholds in a separate CI/CD pipeline.
- B Create a retrieve-and-generate evaluation job that uses custom precision at k metrics and an LLM-as-a-judge metric that uses a scale of 1-5. Include each chunking strategy in the evaluation dataset. Use a supported version of Anthropic Claude Sonnet to evaluate responses from both FMs.** ✓
- C Create a separate evaluation job for each chunking strategy and FM combination. Use Amazon Bedrock built-in metrics for correctness and completeness. Manually review scores before deployment approval.
- D Set up a pipeline that uses multiple retrieve-only evaluation jobs to assess retrieval quality. Create separate evaluation jobs for both FMs that use Amazon Nova Pro as the LLM-as-a-judge model. Evaluate based on faithfulness and citation precision metrics.

Explicación

Opción A: un trabajo de evaluación "retrieve-only" en Amazon Bedrock solo genera un informe basado en los datos recuperados del sistema RAG (usando métricas integradas como Context relevance y Context coverage), pero no evalúa en absoluto la respuesta generada por el modelo. Como el enunciado exige comparar explícitamente la calidad de generación de dos FMs distintos, este tipo de trabajo es insuficiente por diseño. Además, definir los umbrales de calidad en un pipeline de CI/CD separado desaprovecha el flujo de evaluación nativo de Bedrock en lugar de aprovecharlo.

Opción B (Correcta): un trabajo de evaluación "retrieve-and-generate" (Retrieval and response generation) de Amazon Bedrock es el único tipo que produce un informe basado tanto en los pasajes recuperados de la base de conocimiento como en las respuestas generadas por el modelo, por lo que es el requerido cuando se necesita valorar la calidad de generación de los FMs. Amazon Bedrock permite definir hasta 10 métricas personalizadas por trabajo de evaluación, cada una con un prompt de instrucciones para un modelo "juez" (LLM-as-a-judge) y un esquema de puntuación propio (por ejemplo, una escala de 1 a 5); esto permite crear una métrica personalizada de precision at k para valorar objetivamente la calidad de la recuperación, y usar un modelo evaluador compatible (una versión soportada de Anthropic Claude Sonnet) como juez para puntuar de forma consistente las respuestas generadas por ambos FMs, estableciendo así umbrales de calidad claros antes del despliegue. Entre las cuatro opciones, es la única que combina el tipo de trabajo correcto (evalúa generación, no solo recuperación) con métricas capaces de comparar tanto estrategias de chunking como modelos generadores.

Matiz importante para revisión humana: según la documentación oficial, cada trabajo de evaluación RAG de Bedrock (tanto en la consola como en la API create-evaluation-job) se configura contra una única base de conocimiento/fuente RAG (un solo "knowledgeBaseId" o "ragSourceIdentifier") y un único modelo generador (un solo "modelArn"); textualmente, al traer tus propias respuestas de inferencia "solo puedes evaluar una fuente RAG por trabajo". Esto sugiere que, en la práctica, comparar varias estrategias de chunking y dos FMs distintos normalmente requeriría ejecutar varios trabajos de evaluación (uno por combinación de estrategia de chunking y FM) y comparar después sus resultados, en lugar de un único trabajo que incluya "cada estrategia de chunking" y "ambos FMs" simultáneamente como sugiere el enunciado de la opción B. Aun así, entre las cuatro alternativas dadas, B sigue siendo la más correcta conceptualmente porque es la única que usa el tipo de evaluación (retrieve-and-generate) y las métricas (custom precision@k + LLM-as-a-judge) adecuadas para el objetivo planteado.

Opción C: crear un trabajo de evaluación independiente por cada combinación de estrategia de chunking y FM es operativamente más alineado con la restricción real de "una fuente RAG por trabajo", pero la opción se apoya únicamente en métricas integradas de correctness y completeness (sin métricas personalizadas de recuperación como precision at k) y, sobre todo, delega la aprobación de despliegue a una revisión manual de las puntuaciones, lo cual no constituye un mecanismo automatizado y repetible de cumplimiento de umbrales de calidad.

Opción D: encadenar múltiples trabajos "retrieve-only" para evaluar la recuperación junto con trabajos de generación separados por FM que usan Amazon Nova Pro como juez, basándose en faithfulness y citation precision, fragmenta la evaluación en piezas desconectadas: la fase de recuperación y la de generación se evalúan por separado y nunca de forma conjunta con el mismo trabajo "retrieve-and-generate", por lo que no se mide realmente el efecto combinado de cada estrategia de chunking sobre la respuesta final generada.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-kb.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-evaluation-metrics.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-evaluation-custom-metrics-prompt-formats.html>

A wildlife conservation agency operates zoos globally. The agency uses various sensors, trackers, and audiovisual recorders to monitor animal behavior. The agency wants to launch a generative AI (GenAI) assistant that can ingest multimodal data to study animal behavior. The GenAI assistant must support natural language queries, avoid speculative behavioral interpretations, and maintain audit logs for ethical research audits. Which solution will meet these requirements?

- A Ingest raw videos into Amazon Rekognition to detect animal postures and expressions. Use Amazon Data Firehose to stream sensor and GPS data into an Amazon S3 data lake. Prompt an Amazon Bedrock foundation model (FM) by using basic templates that are stored in AWS Systems Manager Parameter Store. Use IAM policies to control access. Use AWS CloudTrail for audit logging.
- B Use Amazon SageMaker Processing and Amazon Transcribe to pre-process multimodal data. Ingest summaries into an Amazon Bedrock Retrieval Augmented Generation (RAG) knowledge base. Apply Amazon Bedrock guardrails to restrict speculative outputs. Use AWS AppConfig to manage prompt templates. Use AWS CloudTrail to log research activity for audits.
- C Use Amazon OpenSearch Serverless to index behavioral logs and telemetry events. Use Amazon Comprehend to extract entities. Use Amazon Bedrock to build a layer to answer questions. Embed study summaries into OpenSearch Serverless documents. Use IAM to control access. Use AWS CloudTrail to log user interactions with the AI assistant.
- D Configure Amazon Q Business to federate data across Amazon S3, Amazon Kinesis, and Amazon SageMaker Feature Store. Configure Amazon EventBridge to invoke data ingestion jobs. Use custom AWS Lambda functions to filter large language model (LLM) outputs for ethical compliance before returning results to users.

Respuesta correcta: **B**

- A Ingest raw videos into Amazon Rekognition to detect animal postures and expressions. Use Amazon Data Firehose to stream sensor and GPS data into an Amazon S3 data lake. Prompt an Amazon Bedrock foundation model (FM) by using basic templates that are stored in AWS Systems Manager Parameter Store. Use IAM policies to control access. Use AWS CloudTrail for audit logging.
- B Use Amazon SageMaker Processing and Amazon Transcribe to pre-process multimodal data. Ingest summaries into an Amazon Bedrock Retrieval Augmented Generation (RAG) knowledge base. Apply Amazon Bedrock guardrails to restrict speculative outputs. Use AWS AppConfig to manage prompt templates. Use AWS CloudTrail to log research activity for audits.** ✓
- C Use Amazon OpenSearch Serverless to index behavioral logs and telemetry events. Use Amazon Comprehend to extract entities. Use Amazon Bedrock to build a layer to answer questions. Embed study summaries into OpenSearch Serverless documents. Use IAM to control access. Use AWS CloudTrail to log user interactions with the AI assistant.
- D Configure Amazon Q Business to federate data across Amazon S3, Amazon Kinesis, and Amazon SageMaker Feature Store. Configure Amazon EventBridge to invoke data ingestion jobs. Use custom AWS Lambda functions to filter large language model (LLM) outputs for ethical compliance before returning results to users.

Explicación

Opción A: usar Amazon Rekognition para detectar posturas y expresiones animales en los vídeos y Amazon Data Firehose para volcar los datos de sensores y GPS a un data lake en S3 permite ingerir datos multimodales, pero construir los prompts del modelo de Amazon Bedrock a partir de plantillas básicas guardadas en AWS Systems Manager Parameter Store no aporta ningún mecanismo de anclaje (RAG) ni de guardrails que impida al modelo generar interpretaciones especulativas del comportamiento animal, que es un requisito explícito del enunciado. AWS CloudTrail sí cubriría la auditoría, pero falta el control de contenido especulativo.

Opción B (Correcta): usar Amazon SageMaker Processing junto con Amazon Transcribe para preprocesar los datos multimodales (transcribir audio, extraer metadatos de vídeo y normalizar datos de sensores) y convertirlos en resúmenes estructurados es un patrón habitual de preprocesamiento antes de indexar contenido. Ingerir esos resúmenes en una Knowledge Base de Amazon Bedrock implementa Retrieval Augmented Generation (RAG): las respuestas del asistente se generan recuperando primero los fragmentos relevantes de la base de conocimiento y usándolos como contexto factual para el modelo, lo que ancla las respuestas en los registros empíricos reales en lugar de dejar que el modelo "invente" a partir de su conocimiento paramétrico. Aplicar Amazon Bedrock Guardrails permite restringir explícitamente salidas especulativas mediante políticas de "denied topics" (temas denegados), filtros de contenido y otras salvaguardas configurables que se aplican tanto a las entradas del usuario como a las respuestas del modelo. Gestionar las plantillas de prompts en AWS AppConfig permite versionar y actualizar de forma controlada la forma en que se instruye al modelo (por ejemplo, para reforzar la instrucción de "no especular"). Finalmente, AWS CloudTrail registra las llamadas a las API de Amazon Bedrock (incluidas las de generación e invocación de modelos), lo que proporciona el registro de auditoría necesario para las revisiones éticas de la investigación. En conjunto, esta solución cubre las cuatro exigencias del enunciado: consultas en lenguaje natural, ingestión de datos multimodales con base empírica real (RAG), no especulación (guardrails) y auditoría (CloudTrail).

Opción C: indexar logs de comportamiento y telemetría en Amazon OpenSearch Serverless y usar Amazon Comprehend para extraer entidades, junto con una capa de Amazon Bedrock para responder preguntas, permite construir un sistema de búsqueda y respuesta funcional, pero no incorpora ningún control de tipo guardrail (como los "denied topics" o filtros de contenido de Bedrock Guardrails) que evite explícitamente que el modelo genere respuestas especulativas sobre el comportamiento animal, que es uno de los requisitos centrales de la pregunta.

Opción D: Amazon Q Business federando datos de Amazon S3, Amazon Kinesis y Amazon SageMaker Feature Store actúa como un asistente de búsqueda empresarial sobre fuentes de datos estructuradas y documentales, pero no está diseñado como un pipeline especializado para procesar y resumir contenido de audio y vídeo (multimodal) como el que generan los sensores, trackers y grabaciones audiovisuales del enunciado. Además, filtrar las salidas del LLM mediante funciones AWS Lambda personalizadas "para cumplimiento ético" es una solución ad hoc basada en reglas propias del desarrollador, no un guardrail gestionado, versionado y auditable como el que ofrece nativamente Amazon Bedrock Guardrails.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-multimodal.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html>

A company uses an organization in AWS Organizations with all features enabled to manage multiple AWS accounts. Employees use Amazon Bedrock across multiple accounts. The company must prevent specific topics and proprietary information from being included in prompts to Amazon Bedrock models. The company must ensure that employees can use only approved Amazon Bedrock models. The company centrally manages IAM roles for employees. Which combination of solutions will meet these requirements? (Choose two.)

Selecciona 2 respuestas

- A Create an IAM permissions boundary for each employee's IAM role. Configure the permissions boundary to require an approved Amazon Bedrock guardrail identifier to invoke Amazon Bedrock models. Create an SCP that allows employees to use only approved models.
- B Create an SCP that allows employees to use only approved models. Configure the SCP to require employees to specify a guardrail identifier in calls to invoke an approved model.
- C Create an SCP that prevents an employee from invoking a model if a centrally deployed guardrail identifier is not specified in a call to the model. Create a permissions boundary on each employee's IAM role that allows each employee to invoke only approved models.
- D Use AWS CloudFormation to create a custom Amazon Bedrock guardrail that has a block filtering policy. Use stack sets to deploy the guardrail to each account in the organization.
- E Use AWS CloudFormation to create a custom Amazon Bedrock guardrail that has a mask filtering policy. Use stack sets to deploy the guardrail to each account in the organization.

Respuesta correcta: **B, D**

- A Create an IAM permissions boundary for each employee's IAM role. Configure the permissions boundary to require an approved Amazon Bedrock guardrail identifier to invoke Amazon Bedrock models. Create an SCP that allows employees to use only approved models.
- B** Create an SCP that allows employees to use only approved models. Configure the SCP to require employees to specify a guardrail identifier in calls to invoke an approved model. ✓
- C Create an SCP that prevents an employee from invoking a model if a centrally deployed guardrail identifier is not specified in a call to the model. Create a permissions boundary on each employee's IAM role that allows each employee to invoke only approved models.
- D** Use AWS CloudFormation to create a custom Amazon Bedrock guardrail that has a block filtering policy. Use stack sets to deploy the guardrail to each account in the organization. ✓
- E Use AWS CloudFormation to create a custom Amazon Bedrock guardrail that has a mask filtering policy. Use stack sets to deploy the guardrail to each account in the organization.

Explicación

Opción A: un permissions boundary de IAM establece el límite máximo de permisos que puede tener un rol, pero no es el mecanismo documentado por AWS para exigir un guardrail concreto en las llamadas de inferencia; AWS documenta la clave de condición `bedrock:GuardrailIdentifier` para usarse directamente en políticas (incluidas las SCP), por lo que añadir un permissions boundary junto a una SCP que ya restringe los modelos aprobados introduce una capa de gestión redundante sin aportar ninguna capacidad adicional.

Opción B (Correcta): las SCP de AWS Organizations pueden restringir qué IDs de modelo de Amazon Bedrock puede invocar un principal (por ejemplo, limitando el recurso `foundation-model/*` a los ARN de los modelos aprobados o usando la clave `bedrock:modelId`) y, adicionalmente, pueden incluir una condición Deny basada en la clave `bedrock:GuardrailIdentifier` para exigir que toda llamada a `InvokeModel`, `InvokeModelWithResponseStream`, `Converse` o `ConverseStream` especifique el guardrail (y su versión) aprobado centralmente. Al tratarse de una SCP aplicada en el nivel de la organización, la restricción se hereda de forma centralizada por todas las cuentas miembro sin depender de la configuración de cada cuenta individual, lo que encaja con el requisito de gestión centralizada de roles de IAM.

Opción C: combinar una SCP que exija el guardrail con un permissions boundary en cada rol de IAM que restrinja los modelos aprobados duplica la lógica de control (una SCP ya puede cubrir ambos requisitos, el del guardrail y el de los modelos permitidos) y aumenta la complejidad operativa de mantener dos mecanismos distintos sin ningún beneficio de seguridad adicional frente a una SCP bien diseñada.

Opción D (Correcta): el recurso de CloudFormation `AWS::Bedrock::Guardrail` permite definir de forma declarativa las distintas salvaguardas de un guardrail, incluida la política de temas denegados (`TopicPolicyConfig`), que bloquea temas específicos detectados en los prompts o en las respuestas del modelo. Usar `StackSets` de AWS CloudFormation es el mecanismo estándar de AWS para desplegar el mismo conjunto de recursos (en este caso, el guardrail con su política de bloqueo) de manera consistente y repetible en todas las cuentas de la organización, sin tener que configurar manualmente un guardrail en cada cuenta.

Opción E: la política de enmascaramiento (mask) de los filtros de información sensible de Bedrock Guardrails anonimiza o redacta la información sensible detectada sustituyéndola por una etiqueta (por ejemplo, `{NAME}` o `{EMAIL}`), pero permite que la solicitud o la respuesta continúen procesándose; no impide que el prompt que contiene los temas prohibidos o la información propietaria llegue al modelo. Para "prevenir" que dichos temas o datos propietarios se incluyan en los prompts, como exige el enunciado, se necesita una política de bloqueo (block), que corta por completo la solicitud cuando detecta el contenido no permitido, en lugar de una política de enmascaramiento.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-permissions-id.html>
- <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-resource-bedrock-guardrail.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>

A company is designing an API for a generative AI (GenAI) application that uses a foundation model (FM) that is hosted on a managed model service. The API must stream responses to reduce latency, enforce token limits to manage compute resource usage, and implement retry logic to handle model timeouts and partial responses. Which solution will meet these requirements with the LEAST operational overhead?

- A Integrate an Amazon API Gateway HTTP API with an AWS Lambda function to invoke Amazon Bedrock. Use Lambda response streaming to stream responses. Enforce token limits within the Lambda function. Implement retry logic for model timeouts by using Lambda and API Gateway timeout configurations.
- B Connect an Amazon API Gateway HTTP API directly to Amazon Bedrock. Simulate streaming by using client-side polling. Enforce token limits on the frontend. Configure retry behavior by using API Gateway integration settings.
- C Connect an Amazon API Gateway WebSocket API to an Amazon ECS service that hosts a containerized inference server. Stream responses by using the WebSocket protocol. Enforce token limits within Amazon ECS. Handle model timeouts by using ECS task lifecycle hooks and restart policies.
- D Integrate an Amazon API Gateway REST API with an AWS Lambda function that invokes Amazon Bedrock. Use Lambda response streaming to stream responses. Enforce token limits within the Lambda function. Implement retry logic by using Lambda and API Gateway timeout configurations.

Respuesta correcta: A

- A** Integrate an Amazon API Gateway HTTP API with an AWS Lambda function to invoke Amazon Bedrock. Use Lambda response streaming to stream responses. Enforce token limits within the Lambda function. Implement retry logic for model timeouts by using Lambda and API Gateway timeout configurations. ✓
- B** Connect an Amazon API Gateway HTTP API directly to Amazon Bedrock. Simulate streaming by using client-side polling. Enforce token limits on the frontend. Configure retry behavior by using API Gateway integration settings.
- C** Connect an Amazon API Gateway WebSocket API to an Amazon ECS service that hosts a containerized inference server. Stream responses by using the WebSocket protocol. Enforce token limits within Amazon ECS. Handle model timeouts by using ECS task lifecycle hooks and restart policies.
- D** Integrate an Amazon API Gateway REST API with an AWS Lambda function that invokes Amazon Bedrock. Use Lambda response streaming to stream responses. Enforce token limits within the Lambda function. Implement retry logic by using Lambda and API Gateway timeout configurations.

Explicación

Opción A (marcada como correcta en los datos): esta opción propone integrar una HTTP API de Amazon API Gateway con una función de AWS Lambda que invoca Amazon Bedrock, usando Lambda response streaming (mediante la API `InvokeWithResponseStream`) para emitir tokens de forma incremental, aplicando el límite de tokens dentro del código de la función Lambda y configurando la lógica de reintento a través de los tiempos de espera de Lambda y de API Gateway. El razonamiento original argumenta que esta es la solución con menor esfuerzo operativo porque evita gestionar servidores y porque las HTTP API son más simples y económicas que las REST API para integraciones proxy con Lambda. Sin embargo, existe una imprecisión factual importante: según la documentación oficial de API Gateway, el streaming de la respuesta de una integración proxy (incluida la integración con Lambda mediante `InvokeWithResponseStream`) "is only supported for REST APIs" (solo está soportado en REST API), no en HTTP API. Es decir, tal como está descrita, la arquitectura de la Opción A no permitiría realmente transmitir la respuesta en streaming al cliente a través de la HTTP API, ya que esta capacidad de "response transfer mode: STREAM" en integraciones AWS_PROXY/HTTP_PROXY es exclusiva de las REST API. Esto pone en duda que la Opción A sea la respuesta técnicamente correcta.

Opción B: conectar API Gateway directamente a Amazon Bedrock sin una función Lambda intermedia elimina la posibilidad de aplicar lógica personalizada de límite de tokens (Bedrock no ofrece ese control granular por sí mismo desde la integración) y el "streaming simulado" mediante polling del lado del cliente no es streaming real: introduce latencia adicional y consumo innecesario de solicitudes, por lo que no cumple el requisito de baja latencia mediante streaming genuino.

Opción C: desplegar un servidor de inferencia en contenedores sobre Amazon ECS, expuesto mediante una WebSocket API, sí permite streaming real (el protocolo WebSocket soporta mensajes incrementales) y un control total sobre límites de tokens y reintentos, pero exige aprovisionar, escalar y mantener un clúster ECS, definiciones de tareas, políticas de reinicio y ciclo de vida de las tareas: esto implica una carga operativa considerablemente mayor que una arquitectura serverless con Lambda, por lo que no es la opción de menor esfuerzo operativo.

Opción D: describe el mismo patrón funcional que la Opción A —una función Lambda que invoca Bedrock usando Lambda response streaming, con el límite de tokens aplicado en el código de la función y la lógica de reintentos gestionada mediante los tiempos de espera de Lambda y de API Gateway— pero integrada con una REST API de API Gateway en lugar de una HTTP API. Las REST API tienen, en general, una configuración algo más compleja y un coste por llamada mayor que las HTTP API para un simple proxy con Lambda; sin embargo, es precisamente el tipo de API que la documentación de AWS exige para poder usar el streaming de la respuesta de una integración proxy de Lambda (el "response transfer mode" STREAM solo se admite en REST API, no en HTTP API). Esto significa que, de las cuatro opciones, la D es la única cuya combinación de streaming real de baja latencia y ejecución serverless sin gestión de servidores es efectivamente compatible con las capacidades documentadas de API Gateway, lo que genera una duda razonable sobre si debería ser esta la respuesta marcada como correcta en lugar de la A.

Aviso de verificación: aunque la Opción A es la respuesta marcada como correcta (y se mantiene sin cambios), la documentación oficial de Amazon API Gateway ([response-transfer-mode.html](#)) indica de forma explícita: "Response streaming is only supported for REST APIs" — es decir, el modo de transferencia STREAM para integraciones proxy (incluida la integración proxy de Lambda con response streaming que describe la Opción A) NO está soportado en API Gateway HTTP API, solo en REST API. Esto significa que la combinación descrita en la Opción A (HTTP API + Lambda response streaming) no sería

técnicamente viable según la documentación vigente, mientras que la Opción D describe exactamente el mismo patrón mecánico (Lambda response streaming, límites de tokens en la función, reintentos vía timeouts de Lambda/API Gateway) pero usando una REST API, que sí soporta esta función. Se recomienda revisión humana para confirmar si la respuesta correcta debería ser D en lugar de A.

Referencias: <https://docs.aws.amazon.com/apigateway/latest/developerguide/response-transfer-mode.html> ; <https://docs.aws.amazon.com/apigateway/latest/developerguide/response-transfer-mode-lambda.html> ; <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>

A retail company is using Amazon Bedrock to develop a customer service AI assistant. Analysis shows that 70% of customer inquiries are simple product questions that a smaller model can effectively handle. However, 30% of inquiries are complex return policy questions that require advanced reasoning. The company wants to implement a cost-effective model selection framework to automatically route customer inquiries to appropriate models based on inquiry complexity. The framework must maintain high customer satisfaction and minimize response latency. Which solution will meet these requirements with the LEAST implementation effort?

- A Create a multi-stage architecture that uses a small foundation model (FM) to classify the complexity of each inquiry. Route simple inquiries to a smaller, more cost-effective model. Route complex inquiries to a larger, more capable model. Use AWS Lambda functions to handle the routing logic.
- B Use Amazon Bedrock intelligent prompt routing to automatically analyze inquiries. Route simple product inquiries to smaller models, and route complex return policy inquiries to more capable larger models.
- C Implement a single-model solution that uses an Amazon Bedrock mid-sized foundation model (FM) with on-demand pricing. Include special instructions in model prompts to handle both simple and complex inquiries by using the same model.
- D Create separate Amazon Bedrock endpoints for simple and complex inquiries. Implement a rule-based routing system based on keyword detection. Use on-demand pricing for the smaller model and provisioned throughput for the larger model.

Respuesta correcta: **B**

- A Create a multi-stage architecture that uses a small foundation model (FM) to classify the complexity of each inquiry. Route simple inquiries to a smaller, more cost-effective model. Route complex inquiries to a larger, more capable model. Use AWS Lambda functions to handle the routing logic.
- B Use Amazon Bedrock intelligent prompt routing to automatically analyze inquiries. Route simple product inquiries to smaller models, and route complex return policy inquiries to more capable larger models. ✓**
- C Implement a single-model solution that uses an Amazon Bedrock mid-sized foundation model (FM) with on-demand pricing. Include special instructions in model prompts to handle both simple and complex inquiries by using the same model.
- D Create separate Amazon Bedrock endpoints for simple and complex inquiries. Implement a rule-based routing system based on keyword detection. Use on-demand pricing for the smaller model and provisioned throughput for the larger model.

Explicación

Opción A: es viable técnicamente, pero requiere diseñar y mantener una arquitectura multietapa propia (un FM adicional actuando como clasificador de complejidad, más funciones AWS Lambda para orquestar la lógica de enrutamiento). Esto implica desarrollo, pruebas y mantenimiento continuo de código personalizado, por lo que no cumple el criterio de "mínimo esfuerzo de implementación".

Opción B (Correcta): Amazon Bedrock intelligent prompt routing (enrutamiento inteligente de prompts) es una funcionalidad nativa y totalmente gestionada de Bedrock que expone un único endpoint sin servidor capaz de enrutar cada solicitud entre distintos modelos base dentro de la misma familia de modelos. Internamente, predice dinámicamente la calidad de respuesta que obtendría cada modelo candidato para esa solicitud concreta y la dirige al modelo que ofrece el mejor equilibrio entre calidad y costo (por ejemplo, un modelo pequeño y económico como Claude 3 Haiku o Nova Lite para preguntas sencillas, y un modelo más grande y capaz como Claude 3.5 Sonnet o Nova Pro para consultas complejas de política de devoluciones). Se puede usar mediante "default prompt routers" preconfigurados por AWS o configurando un router propio (CreatePromptRouter) sin necesidad de escribir lógica de clasificación ni orquestación personalizada, lo que elimina la complejidad de administración y cumple el requisito de mínimo esfuerzo de implementación, además de optimizar latencia y costo.

Opción C: usar un único modelo de tamaño medio con instrucciones especiales en el prompt para manejar tanto consultas simples como complejas no aprovecha el hecho de que el 70% de las consultas son sencillas y podrían resolverse con un modelo más pequeño y barato; tampoco implementa ningún mecanismo de enrutamiento basado en complejidad, por lo que no optimiza costos ni cumple el objetivo del negocio.

Opción D: crear endpoints separados con un sistema de enrutamiento basado en reglas y detección de palabras clave, combinado con provisioned throughput para el modelo grande, exige diseñar, probar y mantener manualmente las reglas de enrutamiento (con el riesgo de una clasificación poco precisa) y además implica el costo y compromiso de capacidad del provisioned throughput. Esto representa mucho más esfuerzo de implementación y operación que usar el enrutador inteligente nativo de Bedrock.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html>

A specialty coffee company has a mobile app that generates personalized coffee roast profiles by using Amazon Bedrock with a three-stage prompt chain. The prompt chain converts user inputs into structured metadata, retrieves relevant logs for coffee roasts, and generates a personalized roast recommendation for each customer. Users in multiple AWS Regions report inconsistent roast recommendations for identical inputs, slow inference during the retrieval step, and unsafe recommendations such as brewing at excessively high temperatures. The company must improve the stability of outputs for repeated inputs. The company must also improve app performance and the safety of the app's outputs. The updated solution must ensure 99.5% output consistency for identical inputs and achieve inference latency of less than 1 second. The solution must also block unsafe or hallucinated recommendations by using validated safety controls. Which solution will meet these requirements?

- A Deploy Amazon Bedrock with provisioned throughput to stabilize inference latency. Apply Amazon Bedrock guardrails that have semantic denial rules to block unsafe outputs. Use Amazon Bedrock Prompt Management to manage prompts by using approval workflows.
- B Use Amazon Bedrock Agents to manage chaining. Log model inputs and outputs to Amazon CloudWatch Logs. Use logs from Amazon CloudWatch to perform A/B testing for prompt versions.
- C Cache prompt results in Amazon ElastiCache. Use AWS Lambda functions to pre-process metadata and to trace end-to-end latency. Use AWS X-Ray to identify and remediate performance bottlenecks.
- D Use Amazon Kendra to improve roast log retrieval accuracy. Store normalized prompt metadata within Amazon DynamoDB. Use AWS Step Functions to orchestrate multistep prompts.

Respuesta correcta: A

- A** Deploy Amazon Bedrock with provisioned throughput to stabilize inference latency. Apply Amazon Bedrock guardrails that have semantic denial rules to block unsafe outputs. Use Amazon Bedrock Prompt Management to manage prompts by using approval workflows. ✓
- B** Use Amazon Bedrock Agents to manage chaining. Log model inputs and outputs to Amazon CloudWatch Logs. Use logs from Amazon CloudWatch to perform A/B testing for prompt versions.
- C** Cache prompt results in Amazon ElastiCache. Use AWS Lambda functions to pre-process metadata and to trace end-to-end latency. Use AWS X-Ray to identify and remediate performance bottlenecks.
- D** Use Amazon Kendra to improve roast log retrieval accuracy. Store normalized prompt metadata within Amazon DynamoDB. Use AWS Step Functions to orchestrate multistep prompts.

Explicación

Opción A (Correcta): Amazon Bedrock Provisioned Throughput permite reservar capacidad dedicada (Model Units) para un modelo, en lugar de depender de la capacidad "on-demand" compartida por Región; esto elimina la variabilidad de latencia causada por el escalado y limitación de solicitudes de otros clientes, lo que permite mantener la inferencia por debajo de 1 segundo de forma predecible. Amazon Bedrock Guardrails aplica políticas de "denied topics" (temas denegados), que usan coincidencia semántica para detectar y bloquear temas o contenidos no permitidos (como recomendar temperaturas de tostado excesivamente altas), junto con filtros de contenido dañino; para reforzar aún más el bloqueo de alucinaciones, Guardrails también ofrece "contextual grounding checks" (verificación de fundamentación contextual) y "Automated Reasoning checks", pensados específicamente para detectar respuestas no fundamentadas o alucinadas del modelo. Por último, Amazon Bedrock Prompt Management permite crear, versionar, probar variantes y desplegar versiones específicas de un prompt de forma controlada hacia la aplicación, lo que ayuda a mantener la consistencia de las respuestas ante entradas idénticas al fijar una versión validada del prompt en producción (aunque el término "approval workflows" no es una función nativa denominada así por AWS, el flujo de creación, prueba, versionado y despliegue controlado de prompts cumple ese propósito). En conjunto, esta combinación cubre los tres requisitos: latencia estable (<1s), consistencia de salida (99.5%) y controles de seguridad validados.

Opción B: Amazon Bedrock Agents permite orquestar cadenas de prompts multi-paso, y el registro en Amazon CloudWatch Logs junto con pruebas A/B de versiones de prompts son útiles para observabilidad y mejora iterativa, pero no aportan capacidad reservada que garantice una latencia de inferencia estable ni controles de seguridad validados (como guardrails) que bloqueen activamente contenido inseguro o alucinado; son medidas reactivas/de diagnóstico, no preventivas.

Opción C: cachear resultados en Amazon ElastiCache podría reducir la latencia solo para solicitudes repetidas (no ayuda con inputs nuevos o variados), y usar AWS Lambda junto con AWS X-Ray sirve para preprocesar metadatos y trazar/diagnosticar cuellos de botella de rendimiento, pero ninguno de estos servicios proporciona controles de seguridad de contenido ni garantiza consistencia de salida del modelo; X-Ray es una herramienta de trazabilidad, no de prevención de contenido inseguro.

Opción D: Amazon Kendra mejora la precisión de la recuperación de información (relevante para la etapa de retrieval de logs de tueste), y AWS Step Functions puede orquestar los pasos de la cadena de prompts, mientras que Amazon DynamoDB almacenaría metadatos normalizados; sin embargo, esta combinación no resuelve la estabilidad de la latencia de inferencia del propio modelo fundacional (el cuello de botella descrito en la etapa de retrieval podría persistir o trasladarse a la etapa de generación) ni añade controles de seguridad validados sobre las recomendaciones generadas por el modelo.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html>

A company is developing a generative AI (GenAI) application by using Amazon Bedrock. The application will analyze patterns and relationships in the company's data. The application will process millions of new data points daily across AWS Regions in Europe, North America, and Asia before storing the data in Amazon S3. The application must comply with local data protection and storage regulations. Data residency and processing must occur within the same continent. The application must also maintain audit trails of the application's decision-making processes and provide data classification capabilities. Which solution will meet these requirements?

- A Deploy the application in each Region with local IAM policies. Use Amazon Bedrock cross-Region inference to distribute the workload. Use Amazon CloudWatch to log AI decision-making processes and data processing activities. Manually track compliance certifications across Regions.
- B Use SCPs with AWS Organizations to manage location-specific permissions. Use AWS CloudTrail immutable logs to audit the decision-making processes. Import a custom model into Amazon Bedrock and deploy the model to each Region.
- C Use Amazon S3 Object Lock with Region-specific S3 bucket policies. Pre-process the data points within the Region based on geographic origin before sending the data points to Amazon Bedrock. Use Amazon Macie to classify the data. Use AWS CloudTrail immutable logs to audit the decision-making processes.
- D Create separate AWS accounts for each Region with individual compliance frameworks. Use Amazon SageMaker AI with custom monitoring to track model performance and compliance with data residency requirements. Create manual reports for each regulatory jurisdiction.

Respuesta correcta: C

- A Deploy the application in each Region with local IAM policies. Use Amazon Bedrock cross-Region inference to distribute the workload. Use Amazon CloudWatch to log AI decision-making processes and data processing activities. Manually track compliance certifications across Regions.
- B Use SCPs with AWS Organizations to manage location-specific permissions. Use AWS CloudTrail immutable logs to audit the decision-making processes. Import a custom model into Amazon Bedrock and deploy the model to each Region.
- C Use Amazon S3 Object Lock with Region-specific S3 bucket policies. Pre-process the data points within the Region based on geographic origin before sending the data points to Amazon Bedrock. Use Amazon Macie to classify the data. Use AWS CloudTrail immutable logs to audit the decision-making processes.**
- D Create separate AWS accounts for each Region with individual compliance frameworks. Use Amazon SageMaker AI with custom monitoring to track model performance and compliance with data residency requirements. Create manual reports for each regulatory jurisdiction.

Explicación

Opción A: la característica de "cross-Region inference" de Amazon Bedrock (cuando no se restringe explícitamente a un perfil de inferencia geográfico, como los prefijos "us.", "eu." o "apac.") puede enrutar la solicitud hacia otras Regiones dentro de un mismo continente o, según cómo se configure, no ofrece por sí sola garantías estrictas frente a un requisito legal de residencia; además, la opción no incorpora ninguna herramienta de clasificación automática de datos (uno de los requisitos explícitos del enunciado) y el seguimiento manual de certificaciones de cumplimiento no es escalable ni auditable de forma fiable frente a millones de registros diarios.

Opción B: las Service Control Policies (SCP) de AWS Organizations gestionan permisos de IAM a nivel de cuenta/OU, pero no controlan ni garantizan dónde se procesan físicamente los datos ni resuelven la clasificación automática de datos sensibles (Macie no aparece en esta opción). Importar un modelo personalizado a Amazon Bedrock y desplegarlo en cada Región soluciona la ubicación de la inferencia, pero no aporta ni el mecanismo de clasificación de datos ni una vía de preprocesamiento por origen geográfico antes del envío al modelo.

Opción C (Correcta): esta es la única opción que cubre las cuatro exigencias del enunciado. El preprocesamiento de los datos dentro de la misma Región según su origen geográfico, antes de enviarlos a Amazon Bedrock, asegura que el procesamiento ocurra en el continente correspondiente (residencia y procesamiento locales). Amazon S3 Object Lock, combinado con políticas de bucket específicas por Región, aplica un modelo WORM (Write Once Read Many) que impide la eliminación o sobrescritura de los objetos durante el periodo de retención configurado, reforzando la inmutabilidad y el cumplimiento normativo por jurisdicción. Amazon Macie proporciona descubrimiento y clasificación automatizados de datos sensibles (PII, datos financieros, etc.) almacenados en Amazon S3 mediante aprendizaje automático y coincidencia de patrones, cubriendo así el requisito de "capacidades de clasificación de datos". Finalmente, AWS CloudTrail registra de forma inmutable las llamadas a la API y las actividades de la cuenta, proporcionando el rastro de auditoría necesario sobre el procesamiento y las decisiones tomadas por la aplicación.

Opción D: crear cuentas de AWS separadas por Región con marcos de cumplimiento individuales no garantiza por sí mismo ni la residencia de los datos ni su clasificación automática; usar Amazon SageMaker AI con monitorización personalizada permite vigilar el rendimiento del modelo, pero no sustituye a una herramienta de clasificación de datos como Macie, y generar informes de forma manual para cada jurisdicción regulatoria no es una solución escalable ni auditable de manera consistente.

Referencias:

<https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html>

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>

A financial services company is deploying a generative AI (GenAI) application that uses Amazon Bedrock to assist customer service representatives to provide personalized investment advice to customers. The company must implement a comprehensive governance solution that follows responsible AI practices and meets regulatory requirements. The solution must detect and prevent hallucinations in recommendations. The solution must have safety controls for customer interactions. The solution must also monitor model behavior drift in real time and maintain audit trails of all prompt-response pairs for regulatory review. The company must deploy the solution within 60 days. The solution must integrate with the company's existing compliance dashboard and respond to customers within 200 ms. Which solution will meet these requirements with the LEAST operational overhead?

- A Configure Amazon Bedrock guardrails to apply custom content filters and toxicity detection. Use Amazon Bedrock Model Evaluation to detect hallucinations. Store prompt-response pairs in Amazon DynamoDB to capture audit trails and set a TTL. Integrate Amazon CloudWatch custom metrics with the existing compliance dashboard.
- B Deploy Amazon Bedrock and use AWS PrivateLink to access the application securely. Use AWS Lambda functions to implement custom prompt validation. Store prompt-response pairs in an Amazon S3 bucket and configure S3 Lifecycle policies. Create custom Amazon CloudWatch dashboards to monitor model performance metrics.
- C Use Amazon Bedrock Agents and Amazon Bedrock Knowledge Bases to ground responses. Use Amazon Bedrock Guardrails to enforce content safety. Use Amazon OpenSearch Service to store and index prompt-responses pairs. Integrate OpenSearch Service with Amazon QuickSight to create compliance reports and to detect model behavior drift.
- D Use Amazon SageMaker Model Monitor to detect model behavior drift. Use AWS WAF to filter content. Store customer interactions in an encrypted Amazon RDS database. Use Amazon API Gateway to create custom HTTP APIs to integrate with the compliance dashboard.

Respuesta correcta: A

- A** Configure Amazon Bedrock guardrails to apply custom content filters and toxicity detection. Use Amazon Bedrock Model Evaluation to detect hallucinations. Store prompt-response pairs in Amazon DynamoDB to capture audit trails and set a TTL. Integrate Amazon CloudWatch custom metrics with the existing compliance dashboard. ✓
- B** Deploy Amazon Bedrock and use AWS PrivateLink to access the application securely. Use AWS Lambda functions to implement custom prompt validation. Store prompt-response pairs in an Amazon S3 bucket and configure S3 Lifecycle policies. Create custom Amazon CloudWatch dashboards to monitor model performance metrics.
- C** Use Amazon Bedrock Agents and Amazon Bedrock Knowledge Bases to ground responses. Use Amazon Bedrock Guardrails to enforce content safety. Use Amazon OpenSearch Service to store and index prompt-responses pairs. Integrate OpenSearch Service with Amazon QuickSight to create compliance reports and to detect model behavior drift.
- D** Use Amazon SageMaker Model Monitor to detect model behavior drift. Use AWS WAF to filter content. Store customer interactions in an encrypted Amazon RDS database. Use Amazon API Gateway to create custom HTTP APIs to integrate with the compliance dashboard.

Explicación

Opción A (Correcta): Amazon Bedrock Guardrails permite configurar filtros de contenido personalizados (categorías como odio, insultos, contenido sexual, violencia y conducta indebida, que en conjunto actúan como "detección de toxicidad") como control de seguridad nativo para las interacciones con el cliente, sin necesidad de desarrollar lógica propia. Para la detección de alucinaciones, Amazon Bedrock Model Evaluation incluye la métrica "faithfulness" (fidelidad), que mide si la respuesta del modelo contiene información no presente ni inferible del contexto/pasajes proporcionados; esto es útil para comparar y seleccionar modelos antes del despliegue, aunque conviene matizar que el mecanismo de AWS pensado específicamente para detectar y bloquear alucinaciones en tiempo real sobre respuestas en producción es el "contextual grounding check" (verificación de fundamentación contextual) dentro de los propios Guardrails de Bedrock, no Model Evaluation (que es fundamentalmente un proceso de evaluación por lotes/offline, no un control que intercepte y bloquee respuestas en vivo). Amazon DynamoDB, al ser una base de datos serverless de latencia de un solo dígito de milisegundos, permite almacenar los pares prompt-respuesta como pista de auditoría aplicando un TTL para su expiración automática, sin gestionar infraestructura ni clústeres. Las métricas personalizadas de Amazon CloudWatch (por ejemplo, a partir del registro de invocaciones de Bedrock) se integran fácilmente con el dashboard de cumplimiento existente. En conjunto, esta combinación de servicios gestionados/serverless de Bedrock, DynamoDB y CloudWatch es la que exige menos configuración e infraestructura propia, cumpliendo el plazo de 60 días y el requisito de latencia de 200 ms con el menor esfuerzo operativo entre las cuatro opciones.

Opción B: usar AWS PrivateLink para el acceso seguro no aporta nada a los requisitos de gobernanza planteados (seguridad de red, no seguridad de contenido); además, construir validación de prompts personalizada en funciones Lambda y crear dashboards personalizados en CloudWatch implica desarrollar y mantener lógica propia de detección de alucinaciones y de toxicidad, en lugar de usar las capacidades nativas de Guardrails, lo que aumenta considerablemente el esfuerzo operativo. Guardar los pares en S3 con políticas de ciclo de vida es válido para retención de bajo coste, pero no sustituye la necesidad de controles de seguridad y detección nativos que sí ofrece la opción A.

Opción C: usar Amazon Bedrock Agents y Knowledge Bases para fundamentar (RAG) las respuestas y Guardrails para la seguridad de contenido es razonable, pero apoyarse en Amazon OpenSearch Service para indexar y almacenar todos los pares prompt-respuesta, y en la integración de OpenSearch con Amazon QuickSight para generar informes de cumplimiento y detectar la deriva del comportamiento del modelo, añade una carga operativa considerable: hay que dimensionar, gestionar y mantener un clúster de OpenSearch, además de construir manualmente la lógica de detección de deriva, algo que no es una capacidad nativa lista para usar de QuickSight ni de OpenSearch.

Opción D: Amazon SageMaker Model Monitor sí está diseñado explícitamente para detectar la deriva de datos y de calidad del modelo, pero exige configurar trabajos de monitorización, líneas base y programaciones adicionales fuera de Bedrock. Combinado con AWS WAF (pensado para filtrar tráfico HTTP malicioso a nivel de red/aplicación, no contenido generado por el modelo), una base de datos Amazon RDS cifrada que hay que aprovisionar, parchear y escalar manualmente, y APIs personalizadas en Amazon API Gateway para integrar con el dashboard de cumplimiento, esta opción requiere mucha más

configuración, mantenimiento e integración manual que la combinación de servicios nativos y serverless de Bedrock, DynamoDB y CloudWatch de la opción A.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-eval-llm-results.html>

A company uses AWS Lake Formation to set up a data lake that contains databases and tables for multiple business units across multiple AWS Regions. The company wants to use a foundation model (FM) through Amazon Bedrock to perform fraud detection. The FM must ingest sensitive financial data from the data lake. The data includes some customer personally identifiable information (PII). The company must design an access control solution that prevents PII from appearing in a production environment. The FM must access only authorized data subsets that have PII redacted from specific data columns. The company must capture audit trails for all data access. Which solution will meet these requirements?

- A Create a separate dataset in a separate Amazon S3 bucket for each business unit and Region combination. Configure S3 bucket policies to control access based on IAM roles that are assigned to FM training instances. Use S3 access logs to track data access.
- B Configure the FM to authenticate by using IAM roles and Lake Formation permissions based on LF-Tag expressions. Define business units and Regions as LF-Tags that are assigned to databases and tables. Use AWS CloudTrail to collect comprehensive audit trails of data access.
- C Use direct IAM principal grants on specific databases and tables in Lake Formation. Create a custom application layer that logs access requests and further filters sensitive columns before sending data to the FM.
- D Configure the FM to request temporary credentials from AWS STS. Access the data by using presigned S3 URLs that are generated by an API that applies business unit and Regional filters. Use AWS CloudTrail to collect comprehensive audit trails of data access.

Respuesta correcta: **B**

- A Create a separate dataset in a separate Amazon S3 bucket for each business unit and Region combination. Configure S3 bucket policies to control access based on IAM roles that are assigned to FM training instances. Use S3 access logs to track data access.
- B** Configure the FM to authenticate by using IAM roles and Lake Formation permissions based on LF-Tag expressions. Define business units and Regions as LF-Tags that are assigned to databases and tables. Use AWS CloudTrail to collect comprehensive audit trails of data access. ✓
- C Use direct IAM principal grants on specific databases and tables in Lake Formation. Create a custom application layer that logs access requests and further filters sensitive columns before sending data to the FM.
- D Configure the FM to request temporary credentials from AWS STS. Access the data by using presigned S3 URLs that are generated by an API that applies business unit and Regional filters. Use AWS CloudTrail to collect comprehensive audit trails of data access.

Explicación

Opción A: crear un bucket S3 independiente por cada combinación de unidad de negocio y Región no escala bien conforme aumenta el número de unidades de negocio y Regiones (requeriría gestionar $N \times M$ buckets y políticas de bucket dispersas), en lugar de aplicar un control de acceso centralizado a nivel del catálogo de datos de Lake Formation. Además, las políticas de bucket de S3 no ofrecen un mecanismo nativo de redactado o filtrado de columnas sensibles (PII), por lo que no cumple el requisito de exponer al FM solo subconjuntos autorizados con las columnas de PII redactadas.

Opción B (Correcta): AWS Lake Formation permite implementar control de acceso basado en etiquetas (LF-TBAC, Lake Formation tag-based access control) mediante LF-Tags: pares clave-valor (por ejemplo, `unidad_negocio=finanzas`, `region=us-east-1`) que se asignan a bases de datos, tablas e incluso columnas del Data Catalog. En lugar de crear una concesión (grant) por cada combinación de principal, base de datos y tabla —lo que crece de forma combinatoria—, se conceden permisos sobre expresiones de LF-Tags, lo que escala de forma mucho más eficiente cuando existen múltiples unidades de negocio y Regiones, tal como describe la documentación oficial de Lake Formation. Este mecanismo también admite filtrado a nivel de columna (column-level filters), lo que permite excluir o enmascarar las columnas que contienen PII antes de que los datos lleguen al modelo de Amazon Bedrock, evitando así que la PII aparezca en producción. Por otro lado, AWS CloudTrail registra de forma centralizada las llamadas a la API de Lake Formation, incluidas operaciones como `GetDataAccess` (cuando un principal o servicio integrado solicita credenciales temporales para acceder a los datos) y las operaciones de concesión/revocación de permisos, proporcionando así el rastro de auditoría completo que exige el enunciado. La combinación de autenticación mediante roles de IAM, permisos LF-TBAC y registro con CloudTrail satisface simultáneamente los tres requisitos: control de acceso granular y escalable, redactado de columnas sensibles y auditoría completa.

Opción C: los grants directos de IAM/Lake Formation sobre bases de datos y tablas específicas (named resource method) no escalan bien conforme crece el número de combinaciones de unidad de negocio y Región, ya que cada nueva combinación requiere concesiones adicionales explícitas. Añadir una capa de aplicación personalizada para registrar accesos y filtrar columnas sensibles introduce complejidad operativa y un punto adicional de fallo, en lugar de aprovechar el filtrado de columnas nativo y las capacidades de auditoría integradas de Lake Formation.

Opción D: usar AWS STS para credenciales temporales junto con URLs prefiradas de S3 generadas por una API personalizada evita el modelo de permisos centralizado de Lake Formation basado en el Data Catalog. Esta arquitectura obliga a reimplementar manualmente en la API personalizada tanto el filtrado por unidad de negocio/Región como el redactado de columnas con PII, lo que resulta más frágil y difícil de mantener de forma consistente que usar los permisos LF-TBAC y el filtrado de columnas nativos de Lake Formation, aunque CloudTrail sí podría seguir registrando las llamadas a STS.

Referencias:

- <https://docs.aws.amazon.com/lake-formation/latest/dg/tag-based-access-control.html>
- <https://docs.aws.amazon.com/lake-formation/latest/dg/logging-using-cloudtrail.html>
- <https://docs.aws.amazon.com/lake-formation/latest/dg/access-control-metadata.html>

A company runs a Retrieval Augmented Generation (RAG) application that uses Amazon Bedrock Knowledge Bases to perform regulatory compliance queries. The application uses the `RetrieveAndGenerateStream` API. The application retrieves relevant documents from a knowledge base that contains more than 50,000 regulatory documents, legal precedents, and policy updates. The RAG application is producing suboptimal responses because the initial retrieval often returns semantically similar but contextually irrelevant documents. The poor responses are causing model hallucinations and incorrect regulatory guidance. The company needs to improve the performance of the RAG application so it returns more relevant documents. Which solution will meet this requirement with the LEAST operational overhead?

- A Deploy an Amazon SageMaker endpoint to run a fine-tuned ranking model. Use an Amazon API Gateway REST API to route requests. Configure the application to make requests through the REST API to rerank the results.
- B Use Amazon Comprehend to classify documents and apply relevance scores. Integrate the RAG application's reranking process with Amazon Textract to run document analysis. Use Amazon Neptune to perform graph-based relevance calculations.
- C Implement a retrieval pipeline that uses the Amazon Bedrock Knowledge Bases Retrieve API to perform initial document retrieval. Call the Amazon Bedrock Rerank API to rerank the results. Invoke the `InvokeModelWithResponseStream` operation to generate responses.
- D Use the latest Amazon reranker model through the reranking configuration within Amazon Bedrock Knowledge Bases. Use the model to improve document relevance scoring and to reorder results based on contextual assessments.

Respuesta correcta: **D**

- A Deploy an Amazon SageMaker endpoint to run a fine-tuned ranking model. Use an Amazon API Gateway REST API to route requests. Configure the application to make requests through the REST API to rerank the results.
- B Use Amazon Comprehend to classify documents and apply relevance scores. Integrate the RAG application's reranking process with Amazon Textract to run document analysis. Use Amazon Neptune to perform graph-based relevance calculations.
- C Implement a retrieval pipeline that uses the Amazon Bedrock Knowledge Bases Retrieve API to perform initial document retrieval. Call the Amazon Bedrock Rerank API to rerank the results. Invoke the `InvokeModelWithResponseStream` operation to generate responses.
- D Use the latest Amazon reranker model through the reranking configuration within Amazon Bedrock Knowledge Bases. Use the model to improve document relevance scoring and to reorder results based on contextual assessments.**

Explicación

Opción A: desplegar un endpoint de Amazon SageMaker con un modelo de ranking ajustado (fine-tuned), sumado a una API REST propia en Amazon API Gateway para enrutar las peticiones, obliga a la empresa a gestionar infraestructura de cómputo, escalado, actualizaciones del modelo y una capa de integración adicional. Esto añade una sobrecarga operativa considerable frente a usar una capacidad ya integrada en Amazon Bedrock Knowledge Bases.

Opción B: Amazon Comprehend (clasificación de texto/NLP), Amazon Textract (extracción de datos de documentos escaneados) y Amazon Neptune (base de datos de grafos) no son los servicios diseñados para reordenar semánticamente los resultados de una base de conocimiento de Bedrock. Combinarlos para construir una lógica de relevancia basada en grafos es una solución ad-hoc, compleja de mantener y no aprovecha ninguna capacidad nativa de reranking de Bedrock.

Opción C: la Rerank API de Amazon Bedrock (operación Rerank, disponible en el runtime de Agents for Amazon Bedrock) es efectivamente una forma válida de reordenar resultados por relevancia usando un modelo reranker (por ejemplo, de Cohere o Amazon). Sin embargo, en este flujo hay que orquestar explícitamente tres pasos separados: (1) llamar a la Retrieve API de la Knowledge Base, (2) invocar aparte la Rerank API con los documentos recuperados, y (3) llamar a `InvokeModelWithResponseStream` para generar la respuesta. Esto implica más piezas que coordinar, mantener y depurar que si el reranking se activa directamente dentro de la configuración de la Knowledge Base, por lo que no es la opción de menor sobrecarga operativa.

Opción D (Correcta): Amazon Bedrock Knowledge Bases permite habilitar un modelo de reranking directamente en la configuración de la consulta (la sección "Reranking" dentro de las Configurations de la consola, o el campo `rerankingConfiguration` dentro de `vectorSearchConfiguration` en las APIs `Retrieve`/`RetrieveAndGenerate`/`RetrieveAndGenerateStream`). Al seleccionar un modelo reranker soportado (por ejemplo, Amazon Rerank 1.0 o modelos de Cohere), Bedrock reordena automáticamente los fragmentos recuperados según su relevancia contextual real respecto a la consulta, sin necesidad de invocar una API adicional por separado ni de desplegar y mantener infraestructura propia (SageMaker, API Gateway, etc.). Esto resuelve directamente el problema de que la recuperación inicial devuelva documentos semánticamente similares pero contextualmente irrelevantes, reduciendo las alucinaciones del modelo, y lo hace con la mínima sobrecarga operativa al tratarse de una simple configuración dentro del servicio administrado, en lugar de una integración multi-servicio.

Referencias:

- <https://docs.aws.amazon.com/bedrock/latest/userguide/rerank-use.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-supported.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html>

A company is developing a customer communication platform that uses an AI assistant powered by an Amazon Bedrock foundation model (FM). The AI assistant summarizes customer messages and generates initial response drafts. The company wants to use Amazon Comprehend to implement layered content filtering. The layered content filtering must prevent sharing of offensive content, protect customer privacy, and detect potential inappropriate advice solicitation. Inappropriate advice solicitation includes requests for unethical practices, harmful activities, or manipulative behaviors. The solution must maintain acceptable overall response times, so all pre-processing filters must finish before the content reaches the FM. Which solution will meet these requirements?

- A Use parallel processing with asynchronous API calls. Use toxicity detection for offensive content. Use prompt safety classification for inappropriate advice solicitation. Use personally identifiable information (PII) detection without redaction.
- B Use custom classification to build an FM that detects offensive content and inappropriate advice solicitation. Apply personally identifiable information (PII) detection as a secondary filter only when messages pass the custom classifier.
- C Deploy a multi-stage process. Configure the process to use prompt safety classification first, then toxicity detection on safe prompts only, and finally personally identifiable information (PII) detection in streaming mode. Route flagged messages through Amazon EventBridge for human review.
- D Use toxicity detection with thresholds configured to 0.5 for all categories. Use parallel processing for both prompt safety classification and personally identifiable information (PII) detection with entity redaction. Apply Amazon CloudWatch alarms to filter metrics.

Respuesta correcta: **D**

- A Use parallel processing with asynchronous API calls. Use toxicity detection for offensive content. Use prompt safety classification for inappropriate advice solicitation. Use personally identifiable information (PII) detection without redaction.
- B Use custom classification to build an FM that detects offensive content and inappropriate advice solicitation. Apply personally identifiable information (PII) detection as a secondary filter only when messages pass the custom classifier.
- C Deploy a multi-stage process. Configure the process to use prompt safety classification first, then toxicity detection on safe prompts only, and finally personally identifiable information (PII) detection in streaming mode. Route flagged messages through Amazon EventBridge for human review.
- D Use toxicity detection with thresholds configured to 0.5 for all categories. Use parallel processing for both prompt safety classification and personally identifiable information (PII) detection with entity redaction. Apply Amazon CloudWatch alarms to filter metrics.**

Explicación

Opción A: Es incorrecta. Aunque el procesamiento paralelo con llamadas asíncronas ayuda a minimizar la latencia, realizar la detección de PII "sin redacción" no protege realmente la privacidad del cliente: la información sensible (nombres, números de tarjeta, direcciones, etc.) seguiría llegando intacta al modelo fundacional (FM), incumpliendo directamente el requisito de proteger la privacidad del cliente.

Opción B: Es incorrecta. Construir un modelo de clasificación personalizado (custom classification) para detectar contenido ofensivo y solicitudes de consejo inapropiado implica un esfuerzo de entrenamiento, etiquetado y mantenimiento innecesario, cuando Amazon Comprehend ya ofrece de forma nativa, dentro de sus funcionalidades de Trust and Safety, la detección de toxicidad (DetectToxicContent) y la clasificación de seguridad de prompts (prompt safety classification). Además, aplicar la detección de PII solo como filtro secundario "cuando los mensajes pasan el clasificador personalizado" introduce una dependencia secuencial que retrasa el filtrado de privacidad y deja una ventana en la que contenido con PII podría procesarse sin la protección adecuada antes de tiempo.

Opción C: Es incorrecta. El enfoque de proceso en varias etapas (primero prompt safety classification, después toxicity detection solo sobre los prompts marcados como seguros, y finalmente PII "en modo streaming") encadena las comprobaciones de forma secuencial en lugar de en paralelo, lo que acumula la latencia de cada etapa y contradice el requisito de que todo el preprocesamiento termine antes de invocar al FM manteniendo tiempos de respuesta aceptables. Además, Amazon Comprehend no ofrece una modalidad de detección de PII "en streaming": solo ofrece análisis en tiempo real (síncrono, documento a documento, vía ContainsPiiEntities/DetectPiiEntities) o trabajos por lotes asíncronos (StartPiiEntitiesDetectionJob) sobre S3, por lo que ese matiz de la opción es técnicamente inexacto.

Opción D (Correcta): Amazon Comprehend, dentro de su conjunto de funcionalidades de Trust and Safety, ofrece detección de toxicidad (operación DetectToxicContent) con umbrales de confianza configurables por categoría (por ejemplo, 0.5) para categorías como HATE_SPEECH, INSULT, VIOLENCE_OR_THREAT, etc.; clasificación de seguridad de prompts (prompt safety classification, mediante ClassifyDocument) para identificar solicitudes de consejo médico, legal, financiero o de prácticas poco éticas, dañinas o manipuladoras; y detección de entidades PII con redacción. Ejecutar estas comprobaciones en paralelo, en lugar de encadenarlas secuencialmente, es la estrategia que minimiza la latencia acumulada y permite cumplir con el requisito de que el filtrado previo finalice antes de que el contenido llegue al FM. Amazon CloudWatch, por su parte, permite configurar alarmas sobre las métricas resultantes de estas comprobaciones para monitorización operativa.

Nota técnica importante encontrada en la documentación oficial de AWS: la redacción de entidades PII en Amazon Comprehend solo está disponible mediante un trabajo asíncrono por lotes (StartPiiEntitiesDetectionJob) sobre documentos almacenados en Amazon S3; el análisis en tiempo real (ContainsPiiEntities/DetectPiiEntities) únicamente localiza las entidades PII, sin producir el texto redactado de forma síncrona. Esto matiza la afirmación de la opción D de ejecutar "PII con redacción de entidades" en paralelo de baja latencia junto con las demás comprobaciones. Asimismo, la documentación de AWS señala que la función de "prompt safety classification" de Comprehend (empleada en A, C y D) "ya no está disponible para clientes nuevos" y recomienda usar Amazon Bedrock Guardrails para esta capacidad en implementaciones nuevas. Aun con estas matizaciones, entre las cuatro opciones presentadas, D sigue siendo la que mejor representa el patrón de diseño recomendado: uso de las funcionalidades nativas de Comprehend, ejecutadas en paralelo (no secuencialmente), con umbrales configurables y redacción de PII activada.

Referencias:

<https://docs.aws.amazon.com/comprehend/latest/dg/trust-safety.html>

<https://docs.aws.amazon.com/comprehend/latest/dg/how-pii.html>

<https://docs.aws.amazon.com/comprehend/latest/dg/comprehend-availability-change.html>

A software as a service (SaaS) company is building a recommendation model that uses Amazon SageMaker AI to support an application that recommends airline cabin upgrades to customers. The company will host SageMaker AI models on Amazon Bedrock by using Amazon Bedrock Custom Model Import. Airline companies will use the application to send customized offers to customers. The model must examine the travel history of customers to help make more relevant recommendations. The company stores customer travel history data in an Amazon RDS database. The company must ensure that the application delivers consistent, relevant, and accurate results across multiple airlines and customer populations. Which solution will meet these requirements?

- A Use Amazon Bedrock Knowledge Bases to implement a RAG architecture to analyze customer travel history data to give the application semantic search capabilities. Use the semantic search capabilities to retrieve relevant booking patterns, preferences, and loyalty information to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter content. Use AWS Step Functions and AWS Lambda functions to orchestrate validation workflows to reduce hallucinations.
- B Implement text-to-SQL transformations with SQL validations to accurately retrieve relevant booking patterns, preferences, and loyalty information from the RDS database. Use the results to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter content. Use AWS Step Functions and AWS Lambda functions to orchestrate validation workflows to reduce hallucinations.
- C Use Amazon OpenSearch Service to implement vector searches of customer travel history embeddings. Use the vector searches to give the application the ability to perform similarity-based retrieval of booking patterns, preferences, and loyalty information to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter responses. Use confidence scoring and semantic similarity searches to reduce hallucinations.
- D Implement text-to-SQL transformations with SQL validations to accurately retrieve relevant booking patterns, preferences, and loyalty information from the RDS database. Use the results to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter responses. Use confidence scoring and semantic similarity searches to reduce hallucinations.

Respuesta correcta: **B**

- A** Use Amazon Bedrock Knowledge Bases to implement a RAG architecture to analyze customer travel history data to give the application semantic search capabilities. Use the semantic search capabilities to retrieve relevant booking patterns, preferences, and loyalty information to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter content. Use AWS Step Functions and AWS Lambda functions to orchestrate validation workflows to reduce hallucinations.
- B** Implement text-to-SQL transformations with SQL validations to accurately retrieve relevant booking patterns, preferences, and loyalty information from the RDS database. Use the results to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter content. Use AWS Step Functions and AWS Lambda functions to orchestrate validation workflows to reduce hallucinations. ✓
- C** Use Amazon OpenSearch Service to implement vector searches of customer travel history embeddings. Use the vector searches to give the application the ability to perform similarity-based retrieval of booking patterns, preferences, and loyalty information to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter responses. Use confidence scoring and semantic similarity searches to reduce hallucinations.
- D** Implement text-to-SQL transformations with SQL validations to accurately retrieve relevant booking patterns, preferences, and loyalty information from the RDS database. Use the results to generate personalized cabin upgrade recommendations. Apply Amazon Bedrock guardrails to filter responses. Use confidence scoring and semantic similarity searches to reduce hallucinations.

Explicación

Opción A: una arquitectura RAG con Amazon Bedrock Knowledge Bases está diseñada para datos no estructurados o semiestructurados (documentos, PDFs, HTML, contenido multimodal), que se convierten en embeddings vectoriales para hacer búsqueda por similitud semántica. El historial de viajes almacenado en Amazon RDS es un dato estructurado y relacional; según la documentación oficial de AWS, cuando el origen es estructurado no es necesario (ni recomendable) convertirlo en embeddings, porque la búsqueda semántica puede devolver resultados aproximados o imprecisos donde se necesita exactitud transaccional (por ejemplo, sumas de compras, fechas exactas de vuelos o estados de fidelización). Además, la propia funcionalidad de Bedrock Knowledge Bases para "structured data stores" únicamente se conecta a Amazon Redshift o AWS Glue Data Catalog, no directamente a Amazon RDS, por lo que este enfoque tampoco encajaría de forma nativa con el origen de datos descrito.

Opción B (Correcta): dado que el historial de viajes de los clientes reside en una base de datos relacional (Amazon RDS), la documentación de AWS confirma que, para datos estructurados, el propio Amazon Bedrock favorece la conversión de lenguaje natural a SQL (text-to-SQL) en lugar de la búsqueda vectorial, precisamente porque permite recuperar de forma determinista y precisa los registros de reservas, preferencias e información de fidelización. Añadir validaciones de las consultas SQL generadas es explícitamente recomendado por AWS, ya que "la precisión de una consulta SQL generada puede variar según el contexto, el esquema de las tablas y la intención de la consulta del usuario", por lo que conviene evaluar y validar las consultas antes de usarlas. Aplicar guardrails de Amazon Bedrock filtra el contenido generado (contenido dañino, temas denegados, grounding contextual), y orquestar el flujo completo con AWS Step Functions y funciones Lambda añade una capa determinista de validación adicional (por ejemplo, verificar la sintaxis y el resultado de la consulta antes de generar la recomendación final), lo que reduce alucinaciones de manera más fiable que técnicas puramente probabilísticas, garantizando resultados consistentes entre distintas aerolíneas y poblaciones de clientes.

Opción C: la búsqueda vectorial en Amazon OpenSearch Service mediante similitud semántica es adecuada para contenido no estructurado o para casos de recomendación basados en similitud (por ejemplo, "clientes parecidos a este"), pero no ofrece la precisión determinista que exige consultar registros de reservas estructurados y relacionales en RDS; convertir datos tabulares en embeddings para luego "adivinar" por similitud introduce un riesgo de imprecisión innecesario cuando existe una vía directa y exacta (SQL) para obtener el dato.

Opción D: describe una solución casi idéntica a la correcta (text-to-SQL con validaciones SQL sobre RDS, más guardrails de Bedrock) pero sustituye la orquestación con AWS Step Functions y AWS Lambda por "confidence scoring y búsquedas de similitud semántica" para reducir alucinaciones. Aplicar técnicas de similitud semántica sobre un problema de datos estructurados no tiene sentido metodológico (no hay embeddings ni corpus semántico que comparar) y el confidence scoring por sí solo es una

señal probabilística más débil que un flujo de validación explícito y determinista orquestado con Step Functions/Lambda, por lo que esta combinación ofrece menos garantías de consistencia y precisión en un contexto de datos estructurados.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-data.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-generate-query.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>

A company is using Amazon Bedrock to develop a customer support AI assistant. The AI assistant must respond to customer questions about their accounts. The AI assistant must not expose personal information in responses. The company must comply with data residency policies by ensuring that all processing occurs within the same AWS Region where each customer is located. The company wants to evaluate how effective the AI assistant is at preventing the exposure of personal information before the company makes the AI assistant available to customers. Which solution will meet these requirements?

- A Configure a cross-Region Amazon Bedrock guardrail to apply sensitive information filters. Set the guardrail to detect mode during development and testing. Switch to block mode for production deployment.
- B Configure an Amazon Bedrock guardrail to apply sensitive information filters. Set the guardrail to mask mode during development and testing. Switch to block mode for production deployment. Deploy a copy of the guardrail to each Region where the company operates.
- C Configure an Amazon Bedrock guardrail to apply content and topic filters. Set the guardrail to detect mode during development, testing, and production. Disable invocation logging for the Amazon Bedrock model.
- D Configure a cross-Region Amazon Bedrock guardrail to apply a set of content and word filters. Set the guardrail to detect mode during development and testing. Switch to mask mode for production deployment.

Respuesta correcta: **B**

- A Configure a cross-Region Amazon Bedrock guardrail to apply sensitive information filters. Set the guardrail to detect mode during development and testing. Switch to block mode for production deployment.
- B Configure an Amazon Bedrock guardrail to apply sensitive information filters. Set the guardrail to mask mode during development and testing. Switch to block mode for production deployment. Deploy a copy of the guardrail to each Region where the company operates.** ✓
- C Configure an Amazon Bedrock guardrail to apply content and topic filters. Set the guardrail to detect mode during development, testing, and production. Disable invocation logging for the Amazon Bedrock model.
- D Configure a cross-Region Amazon Bedrock guardrail to apply a set of content and word filters. Set the guardrail to detect mode during development and testing. Switch to mask mode for production deployment.

Explicación

Opción A: los guardrails cross-Region de Amazon Bedrock funcionan mediante un "guardrail profile" que enruta las solicitudes de inferencia hacia otras Regiones dentro de la misma geografía (por ejemplo, una solicitud en EE. UU. puede procesarse en cualquier Región de EE. UU.). Según la documentación oficial, aunque la configuración del guardrail se almacena en la Región principal, los prompts de entrada y las respuestas de salida "podrían moverse fuera de tu Región principal" al usar inferencia cross-Region. Esto incumple el requisito de la empresa de que todo el procesamiento ocurra exactamente en la misma Región donde está el cliente, por lo que esta opción no es válida independientemente de que use los filtros correctos (información sensible) y el flujo detect → block adecuado.

Opción B (Correcta): los filtros de información sensible (sensitive information filters) son el mecanismo específico de Amazon Bedrock Guardrails diseñado para detectar y gestionar PII (nombres, direcciones, números de tarjeta, etc.) en las entradas y salidas del modelo, ya sea mediante tipos de PII predefinidos o expresiones regulares personalizadas. Estos filtros admiten dos modos: "mask" (enmascara la información sensible sustituyéndola por una etiqueta como {NAME} o {EMAIL}, sin bloquear toda la respuesta) y "block" (bloquea por completo cualquier contenido en el que se detecte información sensible). Usar el modo mask durante desarrollo y pruebas permite a los desarrolladores ver exactamente qué se está detectando como PII y evaluar la eficacia del guardrail antes de lanzarlo, sin perder la respuesta completa; cambiar a modo block en producción garantiza que ninguna información personal llegue realmente al cliente final. Además, al no usar un guardrail cross-Region sino desplegar una copia independiente del guardrail en cada Región donde opera la empresa, se garantiza que el procesamiento de guardrails permanezca dentro de la misma Región de cada cliente, cumpliendo así la política de residencia de datos. Esta combinación satisface los tres requisitos: filtrado de PII, evaluación previa a producción y cumplimiento de residencia de datos por Región.

Opción C: los filtros de contenido (content filters) y los filtros de tema (denied topics) de Amazon Bedrock Guardrails están diseñados para bloquear contenido dañino (odio, violencia, insultos, etc.) o para restringir que el modelo hable de determinados temas; no son el mecanismo adecuado para detectar y evitar la exposición de información personal identificable, para lo cual existen específicamente los filtros de información sensible. Adicionalmente, deshabilitar el registro de invocaciones (invocation logging) elimina la trazabilidad necesaria para auditar el comportamiento del asistente antes de su lanzamiento, lo cual va en contra de las buenas prácticas de evaluación previa a producción.

Opción D: combina dos problemas. Primero, usa un guardrail cross-Region, que como se explica en la opción A puede mover el procesamiento fuera de la Región exacta del cliente (aunque se mantenga dentro de la misma geografía), incumpliendo el requisito de residencia de datos por Región. Segundo, aplica filtros de contenido y de palabras (word filters) en lugar de filtros de información sensible; los filtros de palabras solo bloquean coincidencias exactas de términos definidos por el usuario y no están diseñados para detectar PII de forma contextual. Por último, mantener el modo mask en producción no impide que la información personal llegue al usuario final (solo la sustituye visualmente en el texto que ya se entregó), a diferencia del modo block, que sí impide la exposición real del contenido.

Referencias: <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html> ; <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-cross-region.html>

A university recently digitized a collection of archival documents, academic journals, and manuscripts. The university stores the digital files in an AWS Lake Formation data lake. The university hires a GenAI developer to build a solution to allow users to search the digital files by using text queries. The solution must return journal abstracts that are semantically similar to a user's query. Users must be able to search the digitized collection based on text and metadata that is associated with the journal abstracts. The metadata of the digitized files does not contain keywords. The solution must match similar abstracts to one another based on the similarity of their text. The data lake contains fewer than 1 million files. Which solution will meet these requirements with the LEAST operational overhead?

- A Use Amazon Titan Embeddings in Amazon Bedrock to create vector representations of the digitized files. Store embeddings in the OpenSearch Neural Plugin for Amazon OpenSearch Service.
- B Use Amazon Comprehend to extract topics from the digitized files. Store the topics and file metadata in an Amazon Aurora PostgreSQL database. Query the abstract metadata against the data in the Aurora database.
- C Use Amazon SageMaker AI to deploy a sentence-transformer model. Use the model to create vector representations of the digitized files. Store embeddings in an Amazon Aurora PostgreSQL database that has the pgvector extension.
- D Use Amazon Titan Embeddings in Amazon Bedrock to create vector representations of the digitized files. Store embeddings in an Amazon Aurora PostgreSQL Serverless database that has the pgvector extension.

Respuesta correcta: **D**

- A Use Amazon Titan Embeddings in Amazon Bedrock to create vector representations of the digitized files. Store embeddings in the OpenSearch Neural Plugin for Amazon OpenSearch Service.
- B Use Amazon Comprehend to extract topics from the digitized files. Store the topics and file metadata in an Amazon Aurora PostgreSQL database. Query the abstract metadata against the data in the Aurora database.
- C Use Amazon SageMaker AI to deploy a sentence-transformer model. Use the model to create vector representations of the digitized files. Store embeddings in an Amazon Aurora PostgreSQL database that has the pgvector extension.
- D Use Amazon Titan Embeddings in Amazon Bedrock to create vector representations of the digitized files. Store embeddings in an Amazon Aurora PostgreSQL Serverless database that has the pgvector extension.** ✓

Explicación

Opción A: el plugin neuronal (Neural Search) de Amazon OpenSearch Service permite realizar búsquedas vectoriales, pero requiere aprovisionar y administrar un dominio o clúster de OpenSearch (dimensionamiento de nodos, parches, escalado), lo que implica una carga operativa considerablemente mayor que usar una base de datos relacional serverless con una extensión de vectores integrada.

Opción B: Amazon Comprehend puede extraer temas (topic modeling) o entidades a partir de texto, pero no genera embeddings semánticos de alta dimensionalidad que permitan calcular similitud de significado entre resúmenes. Los "topics" que produce son agrupaciones estadísticas de palabras clave, no representaciones vectoriales densas; además, el enunciado indica que los metadatos no contienen palabras clave y que se requiere coincidencia por similitud semántica del texto, no por temas discretos, por lo que esta opción no cumple el requisito funcional.

Opción C: desplegar un modelo sentence-transformer en Amazon SageMaker AI sí generaría embeddings de calidad, pero obliga al desarrollador a aprovisionar, escalar, parchear y mantener los endpoints de inferencia (instancias, contenedores, actualizaciones del modelo), lo cual añade una carga operativa notablemente mayor que consumir un modelo de embeddings totalmente gestionado a través de una API como Amazon Bedrock.

Opción D (Correcta): Amazon Titan Embeddings en Amazon Bedrock es un modelo de embeddings totalmente gestionado (sin servidores que aprovisionar ni mantener) que convierte el texto de los archivos digitalizados en vectores numéricos que capturan su significado semántico. Amazon Aurora PostgreSQL Serverless con la extensión pgvector permite almacenar esos vectores junto con los metadatos relacionales en la misma base de datos y realizar búsquedas de similitud (por ejemplo, mediante distancia coseno o L2) para encontrar resúmenes semánticamente parecidos, además de poder filtrar o combinar la consulta con los metadatos estructurados existentes. Al ser Serverless, Aurora ajusta automáticamente la capacidad según la demanda sin que el equipo tenga que gestionar el escalado, lo cual es coherente con un volumen de menos de 1 millón de archivos y con el objetivo de mínimo esfuerzo operativo. Esta combinación (Titan Embeddings + Aurora PostgreSQL Serverless con pgvector) es, de hecho, una de las opciones de almacén vectorial oficialmente soportadas para Knowledge Bases de Amazon Bedrock, lo que confirma que es una arquitectura validada por AWS para este tipo de caso de uso de RAG/búsqueda semántica.

Referencias: <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.VectorDB.html>
<https://docs.aws.amazon.com/rds/latest/auroraextendedcontent/aurora-faq-agentic-ai.html>
<https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>

A financial services company wants to use Amazon Bedrock foundation models (FMs) to analyze call center recordings. When calls end, the call center stores recordings as MP3 files in an Amazon S3 bucket. The company needs to generate summaries and sentiment analysis for the recordings in a structured format as soon as new files are created. The recordings average 20 MB in size. Which combination of solutions will meet these requirements? (Choose two.)

Selecciona 2 respuestas

- A Use AWS Step Functions to orchestrate a workflow to process the recordings. Configure steps to invoke Amazon Transcribe to convert audio to text, validate job completion, and to invoke an AWS Lambda function to process the text by using Amazon Bedrock FMs to generate structured analysis output.
- B Use AWS Step Functions to orchestrate a workflow to process the recordings. Configure steps to invoke Amazon Transcribe to convert audio to text, validate job completion, and to directly invoke Amazon Bedrock FMs to generate summaries and sentiment analysis in JSON format.
- C Use AWS Step Functions to orchestrate a workflow to process the recordings. Configure steps to invoke Amazon Transcribe to convert audio to text, validate job completion, and to invoke an AWS Lambda function to create a prompt to invoke Amazon Bedrock FMs to generate structured analysis output.
- D Configure the source S3 bucket to send events to Amazon EventBridge. Create an EventBridge rule to invoke the Step Functions workflow when an object is created in the bucket.
- E Configure the source S3 bucket to send notifications to the Step Functions workflow when an object is created in the bucket.

Respuesta correcta: **C, D**

- A Use AWS Step Functions to orchestrate a workflow to process the recordings. Configure steps to invoke Amazon Transcribe to convert audio to text, validate job completion, and to invoke an AWS Lambda function to process the text by using Amazon Bedrock FMs to generate structured analysis output.
- B Use AWS Step Functions to orchestrate a workflow to process the recordings. Configure steps to invoke Amazon Transcribe to convert audio to text, validate job completion, and to directly invoke Amazon Bedrock FMs to generate summaries and sentiment analysis in JSON format.
- C** Use AWS Step Functions to orchestrate a workflow to process the recordings. Configure steps to invoke Amazon Transcribe to convert audio to text, validate job completion, and to invoke an AWS Lambda function to create a prompt to invoke Amazon Bedrock FMs to generate structured analysis output. ✓
- D** Configure the source S3 bucket to send events to Amazon EventBridge. Create an EventBridge rule to invoke the Step Functions workflow when an object is created in the bucket. ✓
- E Configure the source S3 bucket to send notifications to the Step Functions workflow when an object is created in the bucket.

Explicación

Opción A: describe el mismo flujo válido de Step Functions (Transcribe → validación → Lambda → Bedrock), pero no incluye ningún mecanismo que dispare automáticamente el flujo cuando se crea un nuevo archivo en S3. Como la pregunta pide una combinación de dos soluciones que cubran tanto el procesamiento como el disparo automático "tan pronto se crean los archivos", esta opción por sí sola es incompleta y no es la mejor pareja frente a C, que sí especifica correctamente el uso de Lambda para dar forma a la salida.

Opción B: invocar directamente los modelos de Amazon Bedrock desde un paso de Step Functions (integración optimizada de servicio) es técnicamente posible, pero no ofrece un lugar natural para construir un prompt personalizado ni para post-procesar/validar la respuesta y garantizar que el resultado tenga el formato JSON estructurado exigido (resumen + análisis de sentimiento). Sin una función Lambda intermedia se pierde la flexibilidad necesaria para ensamblar el prompt con la transcripción y para dar forma/validar la salida estructurada.

Opción C (Correcta): dentro del flujo de trabajo de Step Functions, invocar Amazon Transcribe para convertir el audio (MP3) a texto, validar la finalización del trabajo de transcripción y, a continuación, usar una función AWS Lambda para construir dinámicamente el prompt (incluyendo el texto transcrito) e invocar los modelos de Amazon Bedrock, permite generar el resumen y el análisis de sentimiento en un formato estructurado. Lambda aporta la flexibilidad necesaria para ensamblar el prompt, invocar la API InvokeModel de Bedrock y dar forma/parsear la respuesta en la salida estructurada requerida (por ejemplo JSON), algo que las integraciones directas de servicio en Step Functions no permiten con la misma flexibilidad.

Opción D (Correcta): configurar el bucket S3 de origen para enviar notificaciones de eventos a Amazon EventBridge (activando "Send notifications to Amazon EventBridge for all events in this bucket" en las propiedades del bucket) y crear una regla de EventBridge que use la máquina de estados de Step Functions como destino, es el mecanismo nativo y documentado por AWS para iniciar un flujo de trabajo de Step Functions en respuesta a la creación de un objeto en S3, casi en tiempo real. Este patrón está descrito explícitamente en la documentación oficial de AWS Step Functions.

Opción E: Amazon S3 no tiene la capacidad de enviar notificaciones de eventos directamente a una máquina de estados de Step Functions como destino nativo de las notificaciones de eventos de S3 (que solo soportan destinos como SNS, SQS, Lambda o EventBridge). Para iniciar una ejecución de Step Functions a partir de un evento de S3 es obligatorio pasar por Amazon EventBridge (o por una función Lambda intermedia que llame a StartExecution), tal como confirma la documentación oficial.

Referencias:

- <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-cloudwatch-events-s3.html>
- https://docs.aws.amazon.com/solutions/latest/constructs/aws_s3_stepfunctions.html
- <https://docs.aws.amazon.com/bedrock/latest/userguide/structured-output.html>

A medical device company wants to feed reports of medical procedures that used the company's devices into an AI assistant. To protect patient privacy, the AI assistant must expose patient personally identifiable information (PII) only to surgeons. The AI assistant must redact PII for engineers. The AI assistant must reference only medical reports that are less than 3 years old. The company stores reports in an Amazon S3 bucket as soon as each report is published. The company has already set up an Amazon Bedrock knowledge base. The AI assistant uses Amazon Cognito to authenticate users. Which solution will meet these requirements?

- A Enable Amazon Macie PII detection on the S3 bucket. Use an S3 trigger to invoke an AWS Lambda function that redacts PII from the reports. Configure the Lambda function to delete outdated documents from the bucket and to invoke knowledge base syncing.
- B Invoke an AWS Lambda function to sync the S3 bucket and the knowledge base when a new report is uploaded to the bucket. Use a second Lambda function to invoke Amazon Comprehend to detect and redact PII if a user is part of the engineer Cognito user group. Set up an S3 Lifecycle configuration to remove reports that are older than 3 years from the bucket.
- C Set up an S3 Lifecycle configuration on the bucket to remove reports that are older than 3 years. Schedule an AWS Lambda function to run daily syncs between the bucket and the knowledge base. When users interact with the AI assistant, call the ApplyGuardrail configuration that matches the user's Cognito user group to redact PII from the agent's responses if appropriate.
- D Create a second knowledge base. Set up an S3 Lifecycle configuration on the bucket to remove reports that are older than 3 years. Invoke an AWS Lambda function that syncs the bucket with the original knowledge base when a new report is uploaded to the bucket. Use Amazon Comprehend to detect and redact PII before syncing the bucket with the second knowledge base. When a user interacts with the AI assistant, redirect the model to the appropriate knowledge base depending on the user's Cognito user group.

Respuesta correcta: **C**

- A Enable Amazon Macie PII detection on the S3 bucket. Use an S3 trigger to invoke an AWS Lambda function that redacts PII from the reports. Configure the Lambda function to delete outdated documents from the bucket and to invoke knowledge base syncing.
- B Invoke an AWS Lambda function to sync the S3 bucket and the knowledge base when a new report is uploaded to the bucket. Use a second Lambda function to invoke Amazon Comprehend to detect and redact PII if a user is part of the engineer Cognito user group. Set up an S3 Lifecycle configuration to remove reports that are older than 3 years from the bucket.
- C** Set up an S3 Lifecycle configuration on the bucket to remove reports that are older than 3 years. Schedule an AWS Lambda function to run daily syncs between the bucket and the knowledge base. When users interact with the AI assistant, call the ApplyGuardrail configuration that matches the user's Cognito user group to redact PII from the agent's responses if appropriate. ✓
- D Create a second knowledge base. Set up an S3 Lifecycle configuration on the bucket to remove reports that are older than 3 years. Invoke an AWS Lambda function that syncs the bucket with the original knowledge base when a new report is uploaded to the bucket. Use Amazon Comprehend to detect and redact PII before syncing the bucket with the second knowledge base. When a user interacts with the AI assistant, redirect the model to the appropriate knowledge base depending on the user's Cognito user group.

Explicación

Opción A: Amazon Macie está diseñado para descubrir y clasificar datos sensibles (incluida PII) dentro de un bucket de S3, pero no ofrece un mecanismo para variar qué información se expone según el rol o grupo del usuario que consulta al asistente en tiempo de ejecución; además, redactar los documentos de forma permanente en el bucket destruiría la PII también para los cirujanos, que sí deben poder verla.

Opción B: usar Amazon Comprehend para redactar la PII de forma condicional "si el usuario pertenece al grupo de ingenieros" implicaría modificar o generar una copia del contenido en el momento de la ingesta, no en el momento de la consulta; esto no permite servir dinámicamente la misma base de conocimiento con o sin PII según quién esté preguntando en esa sesión, y añade una lógica de sincronización más compleja y menos auditable que aplicar un guardrail en el flujo de inferencia.

Opción C (Correcta): esta es la solución más eficiente y alineada con el diseño nativo de Amazon Bedrock. Una configuración de ciclo de vida de S3 (S3 Lifecycle) puede expirar/eliminar automáticamente los objetos (informes) con más de 3 años de antigüedad, sin necesidad de lógica personalizada. Una función Lambda programada (p. ej., con Amazon EventBridge Scheduler) ejecuta sincronizaciones periódicas entre el bucket de S3 y la base de conocimiento de Amazon Bedrock (iniciando un ingestion job). Y la API ApplyGuardrail de Amazon Bedrock Guardrails se puede invocar de forma independiente y desacoplada de la invocación del modelo fundacional ("Decoupled from foundation models"), en cualquier punto del flujo de la aplicación, pasándole el identificador del guardrail (guardrailIdentifier) que corresponda. Esto permite que la aplicación, tras autenticar al usuario con Amazon Cognito e identificar su grupo (cirujano o ingeniero), seleccione dinámicamente qué guardrail aplicar a la salida del asistente: uno que enmascare (ANONYMIZED) la PII detectada por la política de información sensible para ingenieros, y otro que la deje pasar para cirujanos. Todo esto se logra sobre una única base de conocimiento, sin duplicar almacenamiento ni datos.

Opción D: mantener dos bases de conocimiento (una con PII intacta y otra redactada mediante Amazon Comprehend) duplica el almacenamiento, la sincronización y el mantenimiento operativo. Además, Amazon Comprehend es un servicio de NLP para detección de entidades/PII independiente de Bedrock Guardrails, por lo que introduce un mecanismo adicional cuando Bedrock Guardrails ya resuelve el enmascaramiento de PII de forma nativa e integrada con ApplyGuardrail.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-use-independent-api.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>

https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ApplyGuardrail.html

A large ecommerce company has deployed a foundation model (FM) to generate product descriptions. The company's engineering team monitors technical metrics such as token usage, latency, and error rates by using Amazon CloudWatch. The company's marketing team tracks business metrics such as conversion rates and revenue impact in its own systems. The company needs a unified observability solution that correlates technical performance with business outcomes. The solution must provide automatic alerts to stakeholders when operational metrics indicate degradation. The solution must provide comprehensive visibility across both technical and business metrics. Which solution will meet these requirements?

- A Create CloudWatch dashboards that include technical metrics and imported business metrics. Configure CloudWatch composite alarms that combine technical data and business data. Use Amazon SNS to set up notifications to stakeholders.
- B Use Amazon Managed Grafana to visualize technical metrics from CloudWatch with business metrics from external sources. Configure Amazon Managed Grafana alerts to invoke AWS Lambda functions. Configure the Lambda functions to remediate issues automatically when metrics exceed predefined thresholds.
- C Stream CloudWatch metrics to Amazon S3 by using CloudWatch metric streams. Create Amazon QuickSight dashboards to visualize the combined technical metrics and business metrics. Set up Amazon EventBridge rules to send notifications to stakeholders when metrics exceed predefined thresholds.
- D Configure CloudWatch custom dashboards that integrate operational metrics with imported business metrics. Set up CloudWatch composite alarms with anomaly detection. Use Amazon SNS to create alarm actions to notify stakeholders when correlated metrics indicate performance issues.

Respuesta correcta: **D**

- A Create CloudWatch dashboards that include technical metrics and imported business metrics. Configure CloudWatch composite alarms that combine technical data and business data. Use Amazon SNS to set up notifications to stakeholders.
- B Use Amazon Managed Grafana to visualize technical metrics from CloudWatch with business metrics from external sources. Configure Amazon Managed Grafana alerts to invoke AWS Lambda functions. Configure the Lambda functions to remediate issues automatically when metrics exceed predefined thresholds.
- C Stream CloudWatch metrics to Amazon S3 by using CloudWatch metric streams. Create Amazon QuickSight dashboards to visualize the combined technical metrics and business metrics. Set up Amazon EventBridge rules to send notifications to stakeholders when metrics exceed predefined thresholds.
- D** Configure CloudWatch custom dashboards that integrate operational metrics with imported business metrics. Set up CloudWatch composite alarms with anomaly detection. Use Amazon SNS to create alarm actions to notify stakeholders when correlated metrics indicate performance issues. ✓

Explicación

Opción A: también se basa en CloudWatch y SNS, pero las alarmas compuestas que propone combinan únicamente estados de alarma con umbrales estáticos (ALARM/OK/INSUFFICIENT_DATA), sin detección de anomalías. Esto significa que los umbrales son fijos y no se adaptan a los patrones normales de tráfico o estacionalidad de las métricas de negocio y técnicas, por lo que la solución es menos sensible a degradaciones sutiles o graduales que no llegan a cruzar un umbral rígido.

Opción B: Amazon Managed Grafana permite efectivamente visualizar en un solo panel métricas de CloudWatch junto con fuentes de datos externas, y sus alertas pueden invocar funciones Lambda. Sin embargo, el requisito del enunciado es de observabilidad unificada y alertas a las partes interesadas, no de remediación automática; añadir Lambda para "remediar automáticamente" introduce una capa de automatización y complejidad operativa (aprovisionar y mantener un servicio adicional, definir lógica de remediación segura) que va más allá de lo que se pide y que puede ser arriesgada si se dispara sin supervisión humana.

Opción C: CloudWatch Metric Streams permite exportar métricas casi en tiempo real hacia un destino (típicamente Kinesis Data Firehose, no directamente S3) para su posterior consumo por servicios de análisis como QuickSight, y EventBridge puede generar notificaciones basadas en reglas. Aun siendo técnicamente viable, esta cadena (metric streams -> S3 -> QuickSight -> EventBridge) es una arquitectura de analítica de datos considerablemente más compleja y con mayor latencia que usar directamente los dashboards y alarmas nativas de CloudWatch, cuando el requisito no exige análisis histórico avanzado ni un data lake.

Opción D (Correcta): los dashboards personalizados de CloudWatch permiten combinar en una sola vista las métricas técnicas nativas (uso de tokens, latencia, tasa de error) con métricas de negocio importadas como métricas personalizadas (por ejemplo, mediante PutMetricData) como conversión e ingresos. Las alarmas compuestas de CloudWatch (composite alarms) usan expresiones de reglas con lógica booleana que evalúan el estado (ALARM, OK, INSUFFICIENT_DATA) de varias alarmas subyacentes; al incorporar detección de anomalías en esas alarmas subyacentes, la solución identifica desviaciones respecto al comportamiento normal aprendido en lugar de depender solo de umbrales fijos, lo que permite correlacionar de forma más precisa la degradación operativa con el impacto en el negocio. Las alarmas compuestas admiten como acción notificar a un topic de Amazon SNS, que distribuye la notificación a las partes interesadas cuando la alarma correlacionada indica un problema de rendimiento. Esta combinación de dashboards nativos, alarmas compuestas con detección de anomalías y SNS satisface el requisito con la arquitectura más simple, totalmente nativa de CloudWatch.

Referencias: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/alarm-combining.html> ; <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/alarm-actions.html> ; https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Anomaly_Detection.html

A GenAI developer is evaluating Amazon Bedrock foundation models (FMs) to enhance a Europe-based company's internal business application. The company has a multi-account landing zone in AWS Control Tower. The company uses SCPs to allow its accounts to use only the eu-north-1 Region and the eu-west-1 Region. All customer data must remain in private networks within the approved AWS Regions. The GenAI developer selects an FM based on analysis and testing and hosts the model in the eu-central-1 Region and the eu-west-3 Region. The GenAI developer must enable access to the FM for the company's employees. The GenAI developer must ensure that requests to the FM are private and remain with the same Regions as the FM. Which solution will meet these requirements?

- A Deploy an AWS Lambda function that is exposed by a private Amazon API Gateway REST API to a VPC in eu-north-1. Create a VPC endpoint for the selected FM in eu-central-1 and eu-west-3. Extend existing SCPs to allow employees to use the FM. Integrate the REST API with the business application.
- B Deploy the FM on Amazon EC2 instances in eu-north-1. Deploy a private Amazon API Gateway REST API in front of the EC2 instances. Configure an Amazon Bedrock VPC endpoint. Integrate the REST API with the business application.
- C Configure the FM to use cross-Region inference through an eu.amazon.* endpoint to ensure that all calls remain within Europe. Configure an Amazon Bedrock VPC endpoint. Extend existing SCPs to allow employees to use the FM through inference profiles in Europe-based Regions where the FM is available. Use an inference profile to integrate Amazon Bedrock with the business application.
- D Deploy the FM in Amazon SageMaker AI in eu-north-1. Configure a SageMaker AI VPC endpoint. Extend existing SCPs to allow employees to use the SageMaker AI endpoint. Integrate the FM in SageMaker AI with the business application.

Respuesta correcta: **C**

- A Deploy an AWS Lambda function that is exposed by a private Amazon API Gateway REST API to a VPC in eu-north-1. Create a VPC endpoint for the selected FM in eu-central-1 and eu-west-3. Extend existing SCPs to allow employees to use the FM. Integrate the REST API with the business application.
- B Deploy the FM on Amazon EC2 instances in eu-north-1. Deploy a private Amazon API Gateway REST API in front of the EC2 instances. Configure an Amazon Bedrock VPC endpoint. Integrate the REST API with the business application.
- C** Configure the FM to use cross-Region inference through an eu.amazon.* endpoint to ensure that all calls remain within Europe. Configure an Amazon Bedrock VPC endpoint. Extend existing SCPs to allow employees to use the FM through inference profiles in Europe-based Regions where the FM is available. Use an inference profile to integrate Amazon Bedrock with the business application. ✓
- D Deploy the FM in Amazon SageMaker AI in eu-north-1. Configure a SageMaker AI VPC endpoint. Extend existing SCPs to allow employees to use the SageMaker AI endpoint. Integrate the FM in SageMaker AI with the business application.

Explicación

Opción A: desplegar una función Lambda expuesta por una API REST privada de API Gateway en eu-north-1 no resuelve el enrutamiento hacia el modelo alojado en eu-central-1 y eu-west-3. Un VPC endpoint del modelo garantiza que el tráfico hacia Bedrock no salga a Internet público, pero no evita que las solicitudes crucen fuera del área geográfica de forma controlada ni sustituye el mecanismo nativo de Bedrock (inference profiles) para mantener las solicitudes dentro de una geografía concreta; además, obliga a construir y mantener una capa de enrutamiento personalizada que Bedrock ya ofrece de forma nativa.

Opción B: los modelos fundacionales de Amazon Bedrock se consumen como un servicio totalmente gestionado a través de la API de Bedrock (InvokeModel, etc.); no es posible ni necesario "desplegar" el FM directamente en instancias EC2, ya que Bedrock no distribuye los pesos del modelo para autoalojamiento de esta forma. Colocar una API Gateway privada delante de EC2 no tiene sentido en este contexto y no aprovecha ninguna capacidad nativa de Bedrock para mantener el tráfico dentro de Europa.

Opción C (Correcta): Amazon Bedrock ofrece "cross-Region inference" mediante perfiles de inferencia (inference profiles), y dentro de esta capacidad existen los perfiles geográficos (Geographic cross-Region inference), identificados por un prefijo como "eu.", "us." o "apac." en el ID del modelo. Según la documentación oficial, las solicitudes enviadas a un inference profile geográfico permanecen dentro de esa geografía (por ejemplo, Europa), de modo que aunque el modelo esté alojado en eu-central-1 y eu-west-3, Bedrock enruta automáticamente la inferencia entre las Regiones europeas soportadas por ese perfil sin salir del continente, y todo el tráfico entre Regiones viaja cifrado por la red interna de AWS sin pasar por Internet público. Configurar un VPC endpoint de interfaz (AWS PrivateLink) para Bedrock asegura que las solicitudes desde la VPC de los empleados hacia el servicio permanezcan dentro de la red privada. Por último, la documentación de AWS indica explícitamente que cuando las SCP bloquean Regiones no utilizadas, es necesario ampliarlas para permitir el acceso a todas las Regiones de destino incluidas en el perfil de inferencia geográfico (en este caso, además de eu-north-1 y eu-west-1, también eu-central-1 y eu-west-3), y usar dicho inference profile para integrar Bedrock con la aplicación de negocio. Esta combinación cubre exactamente los tres requisitos: privacidad de red, permanencia geográfica y acceso habilitado para los empleados.

Opción D: el desarrollador ya seleccionó y alojó el modelo en Amazon Bedrock tras su análisis y pruebas; volver a desplegarlo en Amazon SageMaker AI en eu-north-1 supone duplicar trabajo, no reutiliza el modelo evaluado en Bedrock y renuncia a los inference profiles geográficos de Bedrock, que son el mecanismo diseñado específicamente para mantener las solicitudes de inferencia dentro de una región o geografía concreta.

Referencias:

<https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html>

A healthcare company is developing an application to process medical queries. The application must answer complex queries with high accuracy by reducing semantic dilution. The application must refer to domain-specific terminology in medical documents to reduce ambiguity in medical terminology. The application must be able to respond to 1,000 queries each minute with response times less than 2 seconds. Which solution will meet these requirements with the LEAST operational overhead?

- A Use Amazon API Gateway to route incoming queries to an Amazon Bedrock agent. Configure the agent to use an Anthropic Claude model to decompose queries and an Amazon Titan model to expand queries. Create an Amazon Bedrock knowledge base to store the reference medical documents.
- B Configure an Amazon Bedrock knowledge base to store the reference medical documents. Enable query decomposition in the knowledge base. Configure an Amazon Bedrock flow that uses a foundation model (FM) and the knowledge base to support the application.
- C Use Amazon SageMaker AI to host custom ML models for both query decomposition and query expansion. Configure Amazon Bedrock knowledge bases to store the reference medical documents. Encrypt the documents in the knowledge base.
- D Create an Amazon Bedrock agent to orchestrate multiple AWS Lambda functions to decompose queries. Create an Amazon Bedrock knowledge base to store the reference medical documents. Use the agent's built-in knowledge base capabilities. Add deep research and reasoning capabilities to the agent to reduce ambiguity in the medical terminology.

Respuesta correcta: **B**

- A Use Amazon API Gateway to route incoming queries to an Amazon Bedrock agent. Configure the agent to use an Anthropic Claude model to decompose queries and an Amazon Titan model to expand queries. Create an Amazon Bedrock knowledge base to store the reference medical documents.
- B** Configure an Amazon Bedrock knowledge base to store the reference medical documents. Enable query decomposition in the knowledge base. Configure an Amazon Bedrock flow that uses a foundation model (FM) and the knowledge base to support the application. ✓
- C Use Amazon SageMaker AI to host custom ML models for both query decomposition and query expansion. Configure Amazon Bedrock knowledge bases to store the reference medical documents. Encrypt the documents in the knowledge base.
- D Create an Amazon Bedrock agent to orchestrate multiple AWS Lambda functions to decompose queries. Create an Amazon Bedrock knowledge base to store the reference medical documents. Use the agent's built-in knowledge base capabilities. Add deep research and reasoning capabilities to the agent to reduce ambiguity in the medical terminology.

Explicación

Opción A: construir manualmente un paso de descomposición de consultas con un modelo Anthropic Claude y otro de expansión de consultas con un modelo Amazon Titan, todo orquestado a través de un agente de Amazon Bedrock, requiere diseñar prompts personalizados, gestionar dos modelos distintos y mantener la lógica de orquestación entre ellos. Esto añade complejidad de desarrollo y operación innecesaria cuando Amazon Bedrock Knowledge Bases ya ofrece la descomposición de consultas (query decomposition) como una capacidad nativa configurable.

Opción B (Correcta): Amazon Bedrock Knowledge Bases admite una transformación de consulta de tipo `QUERY_DECOMPOSITION` (configurable mediante `QueryTransformationConfiguration` en la API, o desde la consola) que divide automáticamente una consulta compleja en subconsultas más simples antes de la recuperación, lo que reduce la dilución semántica y mejora la precisión al enfocar cada subconsulta en un concepto o término médico específico. Amazon Bedrock Flows, por su parte, permite construir de forma visual y totalmente gestionada un flujo que conecta un nodo de modelo fundacional (FM) con un nodo de base de conocimiento (Knowledge Base node, respaldado por `KnowledgeBaseOrchestrationConfiguration`), sin necesidad de escribir ni mantener código de orquestación personalizado, funciones Lambda adicionales o agentes complejos. Esta combinación (KB con descomposición de consultas activada + Flow gestionado) satisface los requisitos de reducir la ambigüedad terminológica y de baja latencia/alto rendimiento con el mínimo esfuerzo operativo, ya que ambos componentes son totalmente administrados por AWS.

Opción C: alojar modelos personalizados en Amazon SageMaker AI para realizar tanto la descomposición como la expansión de consultas obliga a entrenar o ajustar esos modelos, desplegarlos en endpoints, gestionar su escalado para soportar 1000 solicitudes por minuto y mantener su ciclo de vida completo (actualizaciones, monitorización, parcheo), lo que representa un esfuerzo operativo mucho mayor que usar la funcionalidad nativa de Bedrock Knowledge Bases. Cifrar los documentos en la base de conocimiento es una buena práctica, pero no resuelve el requisito principal de reducir la dilución semántica con el menor esfuerzo.

Opción D: usar un agente de Amazon Bedrock para orquestar múltiples funciones AWS Lambda personalizadas que descompongan las consultas, en lugar de aprovechar la capacidad nativa de descomposición de consultas de la base de conocimiento, introduce complejidad adicional de desarrollo, pruebas y mantenimiento de código. Añadir capacidades de "deep research" y razonamiento al agente incrementa aún más la latencia y el overhead operativo, lo que dificulta cumplir el requisito de tiempos de respuesta inferiores a 2 segundos con 1000 consultas por minuto.

Referencias:

https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_QueryTransformationConfiguration.html

<https://docs.aws.amazon.com/bedrock/latest/userguide/flows-nodes.html>

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html>